

Selective Intelligence: A Cost-Aware Hybrid NLP–LLM Microservice for Thread-Level Sentiment and Watchlist-Driven Target Stance over Code-Mixed Bangla Social Media, with an MCP-Backed Agentic RAG Insight Layer

Final Year Design Project Report

BY

[Student-1 Name]

ID: [Student-1 ID]

[Student-2 Name]

ID: [Student-2 ID]

This Report is Presented in Partial Fulfillment of the Requirements for the Degree of Bachelor of Science
in Computer Science and Engineering

Supervised by

[Supervisor Name]

[Designation]

Department of Computer Science and Engineering
Daffodil International University

Co-Supervised by

[Co-Supervisor Name]

[Designation]

Department of Computer Science and Engineering
Daffodil International University

DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

August 2026

Approval

This Project titled "**Selective Intelligence: A Cost-Aware Hybrid NLP-LLM Microservice for Thread-Level Sentiment and Watchlist-Driven Target Stance over Code-Mixed Bangla Social Media, with an MCP-Backed Agentic RAG Insight Layer**", submitted by [Student-1 Name] and [Student-2 Name] to the Department of Computer Science and Engineering, Daffodil International University, has been accepted as satisfactory for the partial fulfillment of the requirements for the degree of B.Sc. in Computer Science and Engineering and approved as to its style and contents. The presentation has been held on [DD-MM-YYYY].

BOARD OF EXAMINERS

[Name]

Board Chairman

[Designation], Department of CSE, FSIT

Daffodil International University

[Name]

Internal Examiner 1

[Designation], Department of CSE, FSIT

Daffodil International University

[Name]

Internal Examiner 2

[Designation], Department of CSE, FSIT

Daffodil International University

[Name]

External Examiner

[Designation], Department of CSE, FSIT

Daffodil International University

Declaration

We hereby declare that this project has been done by us under the supervision of **[Supervisor Name]**, [Designation], Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere for the award of any degree or diploma.

We further declare that the software system described in this report was designed, implemented, audited and measured by us; that every quantitative result reported in Chapter 4 was produced by running the code in this repository and can be reproduced with the commands given alongside it; and that every capability claimed in this report is labelled with the class of evidence that supports it, so that no implemented-but-unexercised capability is presented as a measured one.

Supervised by:

[Supervisor Name]

[Designation]

Department of Computer Science and Engineering
Daffodil International University

Co-Supervised by:

[Co-Supervisor Name]

[Designation]

Department of Computer Science and Engineering
Daffodil International University

Submitted by:

[Student-1 Name]

Student ID: [Student-1 ID]

Department of Computer Science and Engineering
Daffodil International University

[Student-2 Name]

Student ID: [Student-2 ID]

Department of Computer Science and Engineering
Daffodil International University

Acknowledgements

This work would not have been possible without the support and contributions of many individuals over the past two semesters. We are deeply grateful to everyone who has assisted us in one way or another.

First, we express our heartfelt thanks and gratefulness to the Almighty for His divine blessing, which made it possible for us to complete the Final Year Design Project successfully.

We are grateful and wish to express our profound indebtedness to **[Supervisor Name]**, [Designation], Department of Computer Science and Engineering, Daffodil International University, Dhaka, Bangladesh. The deep knowledge and keen interest of our supervisor in the field of **multilingual natural language processing and distributed AI systems** made it possible to carry out this project. Endless patience, scholarly guidance, continual encouragement, constant and energetic supervision, constructive criticism, valuable advice, and the reading and correction of many inferior drafts at all stages have made it possible to complete this project.

We would like to express our heartfelt gratitude to the Head of the Department of Computer Science and Engineering for his kind help in finishing our project, and also to the other faculty members and the staff of the Department of Computer Science and Engineering, Daffodil International University.

We would like to thank our entire course-mates at Daffodil International University who took part in the discussions that shaped this work while completing the coursework. We are particularly grateful to the reviewers who read the system with fresh eyes across six independent audit passes; several of the findings recorded in Chapter 4 exist only because a reviewer was willing to disbelieve a passing test.

Finally, we must acknowledge with due respect the constant support and patience of our parents.

Abstract

Social-media monitoring in Bangladesh runs on text that off-the-shelf language technology handles poorly: Bangla script, English and romanized "Banglish" mix inside single comments. Sending every item to a large language model is accurate but unaffordable at realistic volumes; running only small classifiers is affordable but weak on code-mixed text. This project builds, audits and measures a microservice that resolves this tension by deciding, per unit of work, how much intelligence that unit deserves. A five-stage, queue-based pipeline – ingestion, cheap NLP, router, selective LLM and result assembler – analyses a post with its comment thread and emits one schema-validated JSON document per thread. A six-gate router governs post-level spend, while a separate step decides which comments an eight-labeller ensemble reads: seven small multilingual sentiment heads batched on CPU plus one context-aware LLM stance pass, in which a labeller that cannot answer abstains. Two features extend the cascade beyond a plain cost gate: **watchlist-driven target stance**, which answers whom a comment is negative toward through an alias matcher built for the three-script spelling problem; and an **agentic insight layer** of nine tool-using agents reasoning over the system's own output through three Model Context Protocol servers exposing eighteen typed tools. The corpus is 50 Facebook threads carrying 10,272 real comments. The router sends 16 percent of posts to post-level LLM work against 74 percent for a keyword baseline, but the bill is comment-dominated: 96 percent of LLM calls are per-comment, so the gate governs only 30 percent of spend. Hybrid retrieval reaches recall@10 of 0.8750 against 0.1875 for hash-vector baselines. The deliverable is working software – eleven dashboard tabs over a 51-path API and a 1,396-test suite – evaluated as an engineering system, with every capability reported alongside its evidence class and labelling accuracy stated as a scope boundary rather than claimed.

Table of Contents

Approval	i
Declaration	ii
Acknowledgements	iii
Abstract	iv
1 Introduction	1
1.1 Introduction	1
1.2 Motivation	2
1.3 Objectives	3
1.4 Methodology	4
1.5 Project Outcome	7
1.6 Organization of the Report	9
2 Background	11
2.1 Introduction	11
2.2 Literature Review	13
2.2.1 Similar Applications	17
2.3 Gap Analysis	20
2.4 Summary	22
3 Research Methodology	23
3.1 Methodology / Requirement Analysis and Design Specification	23
3.1.1 Overview	23
3.1.2 Proposed Methodology / System Design	25
3.1.3 Functional and Nonfunctional Requirements	30
3.1.4 Data Flow Diagram	33
3.1.5 UI Design	36
3.2 Detailed Methodology and Design	42
3.2.1 The five stages	43
3.2.2 The router: two independent decisions	44

3.2.3 The eight-labeller comment ensemble	46
3.2.4 Watchlist-driven target stance	49
3.2.5 Pluggable inference backend and the role system	51
3.2.6 The agentic RAG insight layer	54
3.2.7 Retrieval design	60
3.2.8 Storage, reliability and security	63
3.2.9 Observability and telemetry	63
3.2.10 Deployment architecture	66
3.2.11 Alternatives considered	69
3.3 Project Plan	72
3.3.1 Milestone Overview	72
3.3.2 Phase Overview and Exit Criteria	73
3.3.3 Planned Versus Actual Progress	74
3.3.4 Prioritisation Rationale	77
3.4 Task Allocation	77
3.5 Summary	79
4 Implementation and Results	81
4.1 Environment Setup	81
4.1.1 Software environment and dependencies	81
4.1.2 Deployment configuration	84
4.2 Testing and Evaluation / Performance / Comparative Analysis	85
4.2.1 The corpus	85
4.2.2 Verification strategy and the full-chain result	88
4.2.3 Security and access-control testing	93
4.2.4 Routing behaviour and the cost split	95
4.2.5 Retrieval evaluation	97
4.2.6 Label provenance and ensemble coverage	100
4.2.7 Comparative analysis	101
4.3 Results and Discussion	102
4.3.1 The delivered system, feature by feature	102
4.3.2 Findings supported by the measurements	128
4.3.3 Claims disconfirmed by the measurements	128
4.3.4 The boundary of the evaluation	129
4.3.5 Threats to validity	131

4.4 Summary	132
5 Engineering Standards and Design Challenges	133
5.1 Compliance with the Standards	133
5.1.1 Software Standards	133
5.1.2 Hardware Standards	135
5.1.3 Communication Standards	135
5.2 Impact on Society, Environment and Sustainability	136
5.2.1 Impact on Life	136
5.2.2 Impact on Society and Environment	136
5.2.3 Ethical Aspects	137
5.2.4 Sustainability Plan	137
5.3 Project Management and Financial Analysis	138
5.4 Complex Engineering Problem	141
5.4.1 Complex Problem Solving	141
5.4.2 Engineering Activities	144
5.5 Summary	146
6 Conclusion	147
6.1 Summary	147
6.2 Limitation	148
6.3 Future Work	149
References	152

List of Figures

Figure 3.1: Integration model: a read-only pull into a separate datastore.	24
Figure 3.2: High-level architecture of the Selective Intelligence microservice.	26
Figure 3.3: The five-stage per-post pipeline and the single envelope that threads through it.	29
Figure 3.4: Context diagram (DFD level 0).	34
Figure 3.5: Data flow diagram, level 1.	35
Figure 3.6: Dashboard component hierarchy.	37
Figure 3.7: Server-Sent Event topology and ticketed authentication.	38
Figure 3.8: The sign-in screen, and the auth policy it renders from.	39
Figure 3.9: Dashboard, Overview tab.	40
Figure 3.10: Post detail modal: the eight-labeller comment ensemble, one row per comment.	41
Figure 3.11: Trace tab: one post through all five stages, with each stage's own payload.	42
Figure 3.12: Router decision flow: the post-level gate and the separate comment selection.	45
Figure 3.13: The eight-labeller comment ensemble and its abstention protocol.	48
Figure 3.14: Watchlist-driven target stance: matching, scoring and roll-up.	50
Figure 3.15: Model-serving architecture for the two stages.	53
Figure 3.16: Tool invocation across the MCP boundary, including the refusal path.	55
Figure 3.17: The agentic insight layer over the MCP tool servers.	57
Figure 3.18: Agent run lifecycle.	59
Figure 3.19: The agent tool-use loop with its guards.	59
Figure 3.20: Retrieval path behind the semantic search tool.	62
Figure 3.21: Telemetry paths from the pipeline stages to the operator surfaces.	64
Figure 3.22: Log aggregation: two sinks from one structured logger.	65
Figure 3.23: MVP deployment: Docker Compose service topology.	67
Figure 3.24: Production deployment: Kubernetes topology with queue-depth autoscaling.	68
Figure 4.1: Deployment topology across the three tiers.	85
Figure 4.2: Script composition of the 10,272 comments.	86
Figure 4.3: Crowd reaction mix across the corpus, in thousands.	87
Figure 4.4: Distribution of per-post comment coverage.	87
Figure 4.5: One module of the regression layer: 38 of the suite's 1,396 tests, one per repaired defect.	92
Figure 4.6: Growth of the Python test suite across the audit passes.	93
Figure 4.7: Posts routed for post-level LLM work, by Stage-1 engine.	96

Figure 4.8: Where the large-model calls actually go.	96
Figure 4.9: Recall@10 before and after real embeddings.	98
Figure 4.10: Retrieval quality by configuration, real embeddings.	98
Figure 4.11: Recall@10 stratified by query-to-target lexical overlap.	99
Figure 4.12: Provenance of per-comment sentiment labels in the shipped configuration.	100
Figure 4.13: Starting the whole system with one command.	104
Figure 4.14: The generated API surface, with authentication marked per route.	105
Figure 4.15: Token economics: spend by lane, by model, and projected to larger corpora.	106
Figure 4.16: The Posts tab: upload, watchlist banner, and the analysed corpus.	107
Figure 4.17: Post detail: provenance, coverage, and the system's label beside the upstream baseline. . .	108
Figure 4.18: Three comments, twenty-four votes: what per-labeller disclosure looks like.	109
Figure 4.19: An analysis job in flight, streamed over Server-Sent Events.	110
Figure 4.20: The pipeline under load: in-flight and backlog per stage.	111
Figure 4.21: The structured-log console, streaming from every service at once.	112
Figure 4.22: The canonical result document, first half.	113
Figure 4.23: The canonical result document, second half.	114
Figure 4.24: Keyword retrieval over the stored analysis documents.	115
Figure 4.25: The same query in vector mode, with vector provenance stated.	115
Figure 4.26: The report generator and the reports already produced.	116
Figure 4.27: A generated report, with its cluster provenance stated on the face of it.	117
Figure 4.28: The live MCP tool inventory, as the agent runner discovers it.	118
Figure 4.29: The agent console: nine personas, each with its own tool subset.	119
Figure 4.30: A completed agent run over the analysed corpus.	120
Figure 4.31: The execution trace of that run, and two calls the runner refused.	121
Figure 4.32: The Chat tab catching its own model inventing citations.	122
Figure 4.33: Watchlist alerts, each carrying the rule that raised it.	123
Figure 4.34: The System Monitor drawer: host telemetry during a run.	124
Figure 4.35: The cost ledger as the API returns it.	125
Figure 4.36: The same Overview tab in the dark theme.	126
Figure 5.1: Monthly operating cost envelope, MVP and Production tiers.	139
Figure 5.2: Monthly operating cost envelope, Enterprise tier.	140

List of Tables

Table 1.1: Visual grammar shared by every figure in this report.	9
Table 2.1: Summary of Literature Reviewed.	15
Table 2.2: Comparative Review of Similar Applications.	17
Table 2.3: Gaps identified from the related-work study.	20
Table 2.4: Feature comparison of the proposed system against comparable systems.	20
Table 3.1: Pipeline stages, streams and consumer groups.	43
Table 3.2: The six routing gates.	44
Table 3.3: The eight-labeller comment ensemble.	46
Table 3.4: Roles, and the model each resolves to per backend.	51
Table 3.5: The nine agents, their tool budgets and their purpose.	55
Table 3.6: The three MCP servers and their eighteen tools.	56
Table 3.7: Design alternatives considered and the rationale for each selection.	69
Table 3.8: FYDP Milestone Timeline.	72
Table 3.9: Project plan by phase, with exit criteria.	73
Table 3.10: Planned Versus Actual Progress Since Phase-I.	75
Table 3.11: Task Allocation.	77
Table 3.12: Task allocation and timeline, weeks 12 to 48.	78
Table 4.1: Measurement environment.	82
Table 4.2: Principal dependencies and the reason each constrained version exists.	82
Table 4.3: Corpus characterisation.	85
Table 4.4: Composition of the automated Python test suite, by functional area.	88
Table 4.5: Functional Test Cases and Results.	90
Table 4.6: Full-chain verification run, 23 August 2026.	91
Table 4.7: Security and Access Control Test Results.	93
Table 4.8: Measured routing and large-model call split, 43-post captioned corpus, cold cache, 25 comments per batch.	95
Table 4.9: Retrieval evaluation over 32 known-item queries, $k = 10$, on the 50-post corpus.	97
Table 4.10: Recall@10 stratified by query-to-target lexical overlap.	99
Table 4.11: Comparative analysis of the shipped configuration against its baselines.	101
Table 4.12: Delivered features, where each lives, and the figure that shows it.	126
Table 4.13: Evidence class by capability.	130
Table 5.1: Primary budget: self-hosted inference, monthly operating cost in US dollars.	138

Table 5.2: Alternate budget: metered cloud inference, monthly operating cost in US dollars. 139

Table 5.3: Proposed subscription revenue model. 140

Table 5.4: Mapping with the Complex Engineering Problem attributes. 141

Table 5.5: Mapping with the Knowledge Profile. 143

Table 5.6: Mapping with the Complex Engineering Activities. 144

Chapter 1

Introduction

This chapter states the problem the project addresses, the computational reasons that make it worth solving, the objectives that follow from it, and a summary of the method and the outcomes. It closes with a chapter-by-chapter map of the rest of the report.

1.1 Introduction

Bangladesh has one of the largest and most conversational social-media populations in South Asia, and the organisations that monitor it – newsrooms, brand agencies, election observers, civic groups – increasingly procure that monitoring as a service. A monitoring platform of this kind scrapes public posts and their comment threads, stores them, and exposes them over a REST interface. What it typically does **not** do is understand them. The platform that supplies the data for this project stores one coarse sentiment score per post, leaves per-comment sentiment entirely empty, and ships no optical character recognition for the images attached to a post. Everything an analyst actually wants to know – what the thread is arguing about, which commenters are angry and at whom, whether a conversation is turning toxic, what the dominant themes are this week – has to be computed by something else.

Computing it is harder here than the English-language literature suggests, for one specific reason. The text is not Bangla, and it is not English. It is a live mixture of both, written in two scripts, with no standard orthography for the mixture. A single thread routinely contains comments in Bangla script, comments in English, and comments in **Banglish** – Bangla rendered in Latin characters with spellings that vary from writer to writer and sometimes within a single comment. Measured over the 10,272 comments in this project's working corpus, 81.4 percent of comments contain Bangla script only, 11.1 percent contain Latin script only, 4.6 percent contain both, and 2.9 percent contain neither script at all because they are pure emoji. Multilingual encoders degrade on the romanized portion precisely because it is neither of the languages they were pretrained on, and the same named entity may appear in three different scripts and half a dozen spellings across one thread.

The obvious response – send every post and every comment to a large language model – works and is unaffordable. At the volumes a monitoring platform generates, per-token billing or per-token GPU occupancy scales with the number of comments, not the number of posts, and a single viral thread in this corpus carries 2,857 stored comments against a platform-reported total of 9,224. The opposite response – run only small, cheap classifiers – is affordable and demonstrably weaker on exactly the code-mixed text that dominates the traffic.

The problem statement of this project is therefore narrower and more testable than "analyse Bangla social media". It is this: given a post together with its embedded comment

thread, decide *per unit of work* how much intelligence that unit deserves, spend the expensive model only where cheap models cannot answer, and be able to prove afterwards, from the system's own counters, where the money went and what produced every label. The result is a self-contained analysis microservice that pulls from the upstream platform, analyses in its own database, writes nothing back, and returns one structured JSON document per thread for downstream consumers.

1.2 Motivation

The computational motivation is a cost asymmetry that is large enough to design a system around. A small transformer classifier of 135 to 280 million parameters, batched on CPU, labels a short comment in the order of a hundred milliseconds and costs nothing per token. A seven-billion-parameter instruction-tuned model answers the same question better on code-mixed text, but a single post-type call on the local deployment used in this project takes between 7 and 50 seconds, and one Stage-2 call on a batch of 25 comments is a generation call whose cost scales with prompt length. The gap between the two is between two and three orders of magnitude in latency and, on a metered backend, a comparable gap in money. A system that can decide *which* items genuinely need the expensive path converts an unbounded bill into a bounded one.

That decision is not free to make well, and this is the part that makes the problem interesting rather than merely engineering. A confidence gate is only as good as the confidence estimate feeding it, and a confidence estimate produced by a model that has never been calibrated on the target language is a guess wearing a number. The literature on cascades and query routing – FrugalGPT [30], RouteLLM [31], Hybrid LLM [32] – establishes the shape of the answer for English. What it does not establish is what the gate should read when the input is code-mixed, when a post has no text at all, or when the "unit" being routed is a thread of thousands of comments rather than a single query.

A second motivation is a research gap that this corpus makes visible. Document-level sentiment establishes whether a comment carries negative affect. For political and brand monitoring the operative question is a different one: the identity of the entity toward which that affect is directed. Target-dependent stance detection is well established for English product reviews and English tweets [13], [14], [16], but for code-mixed Bangla it is under-served, and the hard part is not the classifier – it is the matcher, because the same entity is written three ways and a watchlist that matches one spelling silently matches almost nothing. A failure of that kind is invisible: it produces zero mentions, a result indistinguishable from an entity that was never discussed.

A third motivation is personal and professional. Building this system required standing up a genuine distributed pipeline – durable queues, stateless workers, dead-letter handling, autoscaling manifests, three storage engines chosen by access pattern, an observability stack – alongside the applied NLP. That combination is exactly the skill set the industry asks for and exactly the combination an undergraduate curriculum rarely exercises end to end. Solving it also forced a discipline that we did not expect to be the most valuable outcome of the project: **every claim in this report is labelled with the class of evidence behind it**, because six independent

audit passes over our own code established that the most consequential failures were not incorrect answers but silent successes – components that reported that they had run when they had not.

1.3 Objectives

The objectives below follow from the problem statement of Section 1.1 and from the gap analysis presented later in Section 2.3. Each is scoped so that it can be checked against the running system rather than argued for, and they are ordered so that the earlier ones are preconditions for the later ones. Below are the specific objectives.

- i) To design and implement a horizontally scalable, queue-based analysis microservice that consumes a post-with-details payload – a post with its comment thread, engagement counts, reaction breakdown and share sample embedded – from an existing monitoring platform, analyses it in its own datastores, and returns one schema-validated JSON document per thread without writing back to the upstream platform.
- ii) To implement a hybrid cascade in which cheap NLP runs on every post and every comment, and a rule-based router decides, per post, whether post-level large-language-model work is warranted – and, separately, which comments the expensive labellers should read.
- iii) To measure the behaviour of that cascade rather than assert it: to report the routing rate produced by the router's own counters, the split of large-language-model calls between post-level and comment-level lanes, and the share of total spend the routing gate actually governs.
- iv) To raise per-comment sentiment from a single model verdict to a transparent eight-labeller ensemble – seven small multilingual sentiment heads and one context-aware large-language-model stance pass – in which a labeller that cannot answer abstains rather than contributing a fabricated neutral vote, and in which the number of voters that actually spoke is reported on every comment.
- v) To design and implement watchlist-driven target stance: a configurable, alias-aware, operator-supplied entity list that yields per-entity stance for code-mixed Bangla, Banglish and English text, kept in a separate output field from document-level sentiment, and framed explicitly as a stated bias model rather than as a neutral measurement.
- vi) To build an agentic insight layer above the per-post pipeline in which tool-using agents reason over the system's own analytical output through Model Context Protocol servers, with per-run tool and token budgets, prompt-injection hardening, citation verification, and explicit rejection of answers that do not answer anything.
- vii) To make the whole system auditable and reproducible: provenance on every label, per-backend and per-model cost telemetry, a live per-post execution trace, and a regression suite that fails when any repaired defect returns.
- viii) To evaluate the system on the properties by which an engineering deliverable is judged – retrieval quality, routing behaviour, call and cost splits, coverage arithmetic and system

correctness – and to attach an evidence class to every published claim, so that the boundary of what was evaluated is part of the result rather than something the reader has to infer.

Aggregated, these objectives put the project one notch beyond a theoretical problem statement and into actual system fabrication. They were prioritised so that the pipeline and its router were built first, the capability layers – the comment ensemble, the watchlist stance and the agentic insight layer – second, and the audit, evaluation and evidence discipline last. That is the order in which Chapter 3 develops them and Chapter 4 measures them.

1.4 Methodology

For this project, an iterative development methodology structured around requirement analysis, design, development, testing, deployment and audit was adopted. The work was carried out in six steps, each with a clear objective, a defined process and a measurable outcome, so that the delivered system does not drift from the objectives stated in Section 1.3. The methodology is summarised below through those six steps and is developed in full in Chapter 3.

1. Requirement Analysis

Objective: Identify the functional and technical needs of a code-mixed Bangla analysis service, as evidenced by what the upstream monitoring platform does and does not supply.

Process:

- Reviewed the literature on Bangla and code-mixed sentiment, on cost-aware model cascades, and on retrieval-augmented generation and tool-using agents, to establish which parts of the problem already have published answers.
- Characterised the real payload rather than a specification of it: 50 threads, 10,272 stored comments, one coarse upstream sentiment per post and no per-comment sentiment at all.
- Performed the gap analysis of Section 2.3 against commercial listening suites, the upstream platform itself, academic Bangla classifiers, general retrieval assistants and published router frameworks.
- Grouped the requirements around the high-level entities the system actually handles – posts, comments, jobs, watchlist targets, agent runs and analysis results.

Outcome: The eighteen functional and twelve non-functional requirements of Section 3.1.3, and an input contract locked against the real payload rather than against a description of it.

2. System Design

Objective: Design a scalable architecture in which cheap analysis runs unconditionally and expensive analysis is spent selectively, and define the modules that realise it.

Process:

- Architected the system as five queue-decoupled stages – ingestion, cheap NLP, router, selective LLM and result assembler – connected by durable streams, so that each stage scales independently.

- Designed the router as **two independent decisions** rather than one: whether a post earns post-level large-model work, and which of its comments the Stage-2 labellers read. Separating them is what later made the cost behaviour measurable at all.
- Split storage by access pattern rather than by subject: PostgreSQL with pgvector for operational rows and the embedding index, ClickHouse for analytical events, Redis for streams and counters, object storage for raw payloads.
- Specified the canonical output document, its JSON Schema, and the data-flow diagrams of Section 3.1.4, together with the eleven-tab dashboard layout of Section 3.1.5.
- Recorded, for every significant decision, the alternatives weighed and the constraint that settled it, in Table 3.7.

Outcome: The architecture of Figure 3.2, the pipeline of Figure 3.3, the canonical schema, and a design specification in which no load-bearing choice is undocumented.

3. Development

Objective: Translate the designed modules into a working system rather than a prototype.

Process:

- Implemented the five stages, the six-gate router and the two-lane Stage 2, with a shared worker base class carrying consumer-group creation, cooperative cancellation, bounded retry and dead-lettering.
- Built the eight-labeller comment ensemble with its abstention protocol – seven small sentiment heads batched on CPU and one context-aware stance pass – behind a single combiner that is the only writer of a comment's final label.
- Implemented watchlist-driven target stance, whose load-bearing part is the alias matcher rather than the classifier, and which adds no additional large-model calls because the matched entities ride inside a prompt already being issued.
- Built the agentic insight layer: three Model Context Protocol servers exposing eighteen typed tools, and nine tool-using agents over them with per-run tool and token budgets.
- Built the operator-facing React dashboard, the 51-path HTTP API beneath it, and the Server-Sent Event streams that carry every live surface.

Outcome: A running system of 55,965 lines of Python across 181 files, a React dashboard, three deployment shapes and an evaluation harness – the software described in Chapter 4.

4. Testing

Objective: Validate the implemented modules for functional correctness, contract compliance and honesty of reporting.

Process:

- Wrote contract tests asserting that every document crossing a boundary validates against its schema, and structural tests asserting that the deployment manifests name exactly the streams the workers create.

- Ran six independent fresh-eyes audit passes over the system, in which each member audited the code the other had written.
- Pinned every repaired defect with a regression test named after the defect, and confirmed by mutation testing that the tests fail when the fix is reverted.
- Assembled the whole verification chain – Python suite, dashboard unit tests, lint, production build and browser tests – into one pass-or-fail command.

Outcome: A 1,396-test suite in which test code is 35 percent of the repository, and a full chain that passes in sixty seconds. It also produced findings that contradicted the project's own headline claim, which are reported as such in Section 4.3.3.

5. Deployment

Objective: Make the same codebase runnable on a laptop and in a cluster without a rewrite.

Process:

- Packaged the system for three tiers: a single-host development stack, a nineteen-image Docker Compose deployment, and a Kubernetes deployment with queue-depth autoscaling.
- Made the inference backend pluggable and resolvable per call, so that self-hosted and metered serving are a configuration decision rather than a migration.
- Enforced a tenant privacy lock that pins a locked tenant to the self-hosted backend on every path and fails closed when the policy table cannot be read.
- Reduced the whole stack to a single start command that brings up four containers and eleven host processes and verifies the sentence encoder before launching anything.

Outcome: A system that starts in under a minute on a laptop and expresses the same components as Kubernetes objects in production, which is what makes it demonstrable rather than merely deployable.

6. Evaluation and Future Planning

Objective: Measure what the system does, and state plainly where the evidence stops.

Process:

- Measured system properties – routing rate, call and cost split, coverage, latency, label provenance – by running the real stage functions over the real corpus.
- Measured retrieval quality against a 32-query known-item set with recall@k, MRR@k and nDCG@k, and stratified it by query-to-target lexical overlap.
- Attached an evidence class to every capability – measured, works-but-unmeasured, unexercised, or out of scope – and recorded it in a single ledger.
- Scoped the annotation programme that a labelling-accuracy figure would require, and built its sampler and scorer without seeding it with labels drawn from the system's own models.

Outcome: Four completed measurement studies, an evidence ledger covering every capability, and the ordered future work of Section 6.3.

Evaluation is deliberately split by what the evidence can support, and that split is the methodological commitment this project would defend first. System properties are measured. Retrieval quality is measured against ground truth. Correctness is pinned by regression. Labelling accuracy is **outside the evaluation scope** and is not claimed, because obtaining it honestly requires an annotation programme rather than a further measurement script.

1.5 Project Outcome

The delivered solution is a working, auditable analysis microservice with evidence of implementation across ingestion, the five-stage pipeline, the comment ensemble, the watchlist stance layer, the agentic insight layer and an eleven-tab operator dashboard. It comprises 55,965 lines of Python across 181 files, of which 19,332 lines are tests, together with a React dashboard, deployment manifests for both Docker Compose and Kubernetes, and an evaluation harness. A summary of the key results of this work is given below.

1. A working cost-aware cascade with measured behaviour

On the 43-post captioned corpus, the router sends 16 percent of posts to post-level large-model work under the shipped Stage-1 engine, against 74 percent under a keyword baseline. More importantly, the measurement showed that the routing rate is *not* the cost story, because 96 percent of calls are comment-level. This is a result that contradicted the project's own original claim and is reported as such in Section 4.3.3.

2. A transparent per-comment ensemble

Eight labellers per analysed comment, with abstention, voter counts, and per-source verdicts surfaced side by side in the dashboard, so a sentiment chart can always say what produced its numbers. A comment that no model read is reported as *uncertain* rather than defaulted to neutral.

3. A built and tested watchlist stance capability

Per-entity stance over an operator-supplied, alias-aware watchlist that adds zero additional large-model calls, because the matched entities ride inside a prompt that is already being sent. It is framed in the output schema as a separate measurement from sentiment, and never summed or charted with it.

4. An agentic insight layer

Nine tool-using agents over eighteen typed tools across three Model Context Protocol servers, with a runner hardened against eight distinct failure modes observed in live runs, each of which now has a regression test.

5. A measured retrieval stack

Hybrid retrieval reaches recall@10 of 0.8750 where hash-vector baselines score 0.1875 – statistically indistinguishable from returning ten random posts out of fifty. Stratifying the query set shows the dense and lexical arms failing on disjoint query populations, which is the real argument for fusing them.

6. An operator-facing product, not a script

Eleven dashboard tabs – corpus overview, post explorer, job control, reports, dual-mode search, agents, chat, pipeline, per-post trace, watchlist warnings and a streaming log console – over a documented 51-path API, with host telemetry, light and dark themes, and a one-command start of the whole eleven-process stack. Section 4.3.1 walks through all of it with screenshots of the running system, and Table 4.12 indexes each feature against the figure that shows it.

7. An honest capability ledger

Every feature is classified as measured, works-but-unmeasured, unexercised, or out of scope. The image modality is reported as unexercised rather than claimed, and labelling accuracy is recorded as a stated scope boundary with the work that would close it named in Chapter 6.

The areas that have not fully matured are reflected in the deliverable rather than hidden by it: the image and optical-character-recognition modality, cluster autoscaling in a real cluster, and the scoring of the agent layer's answers are all on the future-scope pipeline of Section 6.3 rather than in the results.

The potential outcomes beyond the project are a deployable analysis layer for any Bangla-language monitoring platform, a reusable pattern for cost-aware model cascades in low-resource settings, and a corpus of 10,272 real code-mixed comments with crowd reaction signals attached that is ready for the annotation effort that would turn this engineering result into a benchmark result.

Conclusion: The results confirm that this project has moved beyond a written problem statement and into a deployed, inspectable system, with demonstrated coverage of each of the following: ingestion and the five-stage pipeline, the six-gate router and its measured behaviour, the eight-labeller comment ensemble, watchlist-driven target stance, the agentic insight layer over eighteen typed tools, hybrid retrieval with a measured quality figure, and an eleven-tab operator dashboard over a 51-path API. The areas that have not fully matured – the image modality, cluster-scale autoscaling, and the scoring of the agent layer's answers – are named as such in the capability ledger rather than presented as delivered, which is itself one of the seven outcomes above.

1.6 Organization of the Report

Chapter 1 has stated the problem, the motivation, the objectives and the outcomes. **Chapter 2** provides the background needed to read the rest of the report: the state of Bangla and code-mixed natural language processing, the literature on model cascades and query routing, target-dependent stance detection, retrieval-augmented generation and tool-using agents, and a gap analysis against comparable systems. **Chapter 3** is the methodology and design chapter: it presents the system architecture, the five-stage pipeline, the router's gates, the comment ensemble, the watchlist stance design, the agentic layer, the functional and non-functional requirements, the data-flow diagrams and the user-interface design, and it records the alternatives that were considered at each decision point and why the chosen option won. It closes with the project plan and the task allocation. **Chapter 4** covers implementation and results: the environment, a feature-by-feature walkthrough of the running system in screenshots, the corpus, the full-chain test run, the measured routing and cost behaviour, the retrieval evaluation, and a discussion of what those numbers do and do not establish. **Chapter 5** addresses engineering standards, the impact of the work on life, society, environment and sustainability, its ethical dimensions, a financial analysis with an alternate budget, and the mapping of the work onto the complex-engineering-problem, knowledge-profile and complex-engineering-activity frameworks. **Chapter 6** concludes with a summary, an explicit statement of limitations, and the future work that would close them.

Reading the figures. Every architectural figure in this report is drawn from a single source file in a common visual grammar, so that a shape or a colour carries the same meaning wherever it appears. Table 1.1 gives the key. Colour is never the only carrier of meaning: each element is labelled, and the store shape differs from the service shape, so the figures survive greyscale printing and colour-vision deficiency without loss. Diagrams that show a flow are read in the direction of the arrows, top to bottom or left to right; a dashed or dotted arrow marks a path that is deliberately not taken, or one that carries telemetry rather than payload. Each figure is followed by two short paragraphs: **How to read it**, which names what is drawn, and a **Figure note**, which states the one property the figure exists to show.

Table 1.1: Visual grammar shared by every figure in this report.

Element	Meaning	Where it appears
Violet box	A person or an outside system, beyond this project's control	Operators, analysts, the upstream platform, downstream consumers, the metered inference provider
Blue box	A service or a unit of compute that this project builds and runs	API, workers, routers, assemblers, agents, MCP servers
Green cylinder	A datastore that holds state	PostgreSQL, ClickHouse, Redis, object storage, and the numbered data stores of the data-flow diagrams
Grey box	A supporting piece: a library, a value, an intermediate artefact	Embedding models, label sets, queue names, shared front-end components

Element	Meaning	Where it appears
Orange box	A note, a warning, or an observability surface	Callouts stating a measured property, telemetry stores, alerting
Diamond	A decision with more than one outcome	The router gate, the ensemble combination step
Circle	A process in a data-flow diagram, numbered by level	Figures 3.4 and 3.5
Panel with a title	A boundary: a deployment tier, a namespace, a subsystem	Pipeline, storage band, agent layer, node pools
Solid arrow	Payload or control flow along the normal path	Throughout
Dashed or dotted arrow	Telemetry, a note attachment, or a path deliberately not taken	Log fan-out, the absent write-back, alert links

Background

This chapter supplies the background needed to read the rest of the report. It reviews the state of sentiment and stance analysis for Bangla and code-mixed text, the literature on cost-aware model cascades, and the recent work on retrieval-augmented generation and tool-using agents, then compares the proposed system against existing monitoring products and identifies the gaps this project sets out to fill.

2.1 Introduction

Three separate research literatures meet in this project, and each contributes a different piece of the design. The first is **sentiment and stance analysis for low-resource and code-mixed languages** – a field whose general methods and open problems are surveyed by Wankhade et al. [12], which supplies the models and, more importantly, the evidence about how badly those models degrade when the input is romanized. The second is **cost-aware inference**, a line of work on cascades and query routing that establishes the shape of a system in which a cheap model answers most queries and an expensive model answers the rest. The third is **retrieval-augmented generation and tool-using agents**, which supplies the pattern for the insight layer that sits above the per-post pipeline.

A short orientation on each is given here; the detailed comparison follows in Section 2.2.

Transformer encoders and their multilingual variants

Modern text classification for this task rests on the transformer encoder [18] and the masked-language-model pretraining objective introduced by BERT [19]. For multilingual settings, XLM-RoBERTa [20] is the standard baseline: a single encoder pretrained on one hundred languages, which handles Bangla and English in one model and therefore handles a code-mixed sentence without a language-identification step in front of it. For Bangla specifically, **BanglaBERT** [1] is the leading monolingual option, pretrained on a large Bangla corpus and accompanied by the BLP benchmark suite. Both matter here: the ensemble described in Chapter 3 deliberately mixes multilingual encoders with Bangla-specific fine-tunes, because a multilingual model that merely *includes* Bangla is a different kind of evidence from a model trained on it.

Sentence-level embeddings for retrieval and clustering come from the Sentence-BERT family [22], and specifically from its multilingual distillation variant [23], which is what allows an English query to retrieve a Bangla post. Named-entity recognition uses GLiNER [25], a generalist recogniser that accepts arbitrary entity types at inference time rather than requiring one model per type; language identification uses fastText [21]; and topic and intent assignment where no labelled data exists uses the label-embedding and entailment-based zero-shot formulations

surveyed by Yin et al. [26]. ModernBERT [24] is included in the ensemble as a recent long-context encoder with a different pretraining recipe from the rest of the roster, on the grounds that a voter that fails differently is worth more than a voter that fails the same way.

The code-mixing problem

Code-switching is not an edge case in this corpus; it is the dominant register. One further property of this register is recorded here because it shapes an annotation decision in Chapter 6: a large share of comments carry emoji, whose sentiment is neither uniform nor obvious and has been measured directly across languages by Kralj Novak et al. [17]. Sitaram et al. [11] survey the code-switching field and note the central practical difficulty: romanized text belongs to no language model's pretraining distribution, and standard tokenisers fragment it badly. SemEval-2020 Task 9 [10] established shared benchmarks for Hindi-English and Spanish-English code-mixed sentiment and confirmed that performance on the code-mixed portion lags the monolingual portion substantially. For Bangla specifically, Hassan et al. [3] were among the first to treat romanized Bangla as a first-class target rather than as noise, and Sazzed [4] showed the same for transliterated abusive content. Alam et al. [9] survey the whole Bangla NLP landscape and reach a conclusion that shaped this project's evaluation plan: the bottleneck is annotated data, not model architecture.

Cost-aware inference

The engineering core of this project belongs to a line of work that is only a few years old. FrugalGPT [30] showed that a cascade of increasingly capable models, gated by a learned scorer, can match a single large model's quality at a fraction of the cost. RouteLLM [31] learned the routing decision from preference data, and Hybrid LLM [32] framed it explicitly as a cost-versus-quality trade with a tunable knob. All three route **queries**. None of them routes a *thread*, and the distinction turns out to matter a great deal: as Chapter 4 shows, when the unit of analysis contains thousands of sub-items, the routing decision on the container governs a minority of the eventual bill. On the serving side, PagedAttention and vLLM [33] are what make self-hosted inference economical enough for the local backend to be the default.

Retrieval-augmented generation and agents

Retrieval-augmented generation [34] grounds a generated answer in retrieved evidence; the survey by Gao et al. [35] maps the design space that has grown around it. Retrieval quality is the binding constraint, and the standard measures are recall@k, MRR@k, and the cumulated-gain measures of Järvelin and Kekäläinen [41]. Where dense and lexical retrieval each fail on different queries, reciprocal rank fusion [39] is the standard way to combine them without having to make two incomparable score scales agree; approximate nearest-neighbour search over the dense arm uses HNSW graphs [40]. Cuconasu et al. [42] give an important warning that shaped this project's decision to report recall rather than precision: what lands in the retrieved context materially changes the generated answer, so the metric that matters when the whole top-k is handed to a model is recall at k. Beyond single-shot retrieval, ReAct [36] and Toolformer [37] established the tool-using loop, and the Model Context Protocol [38] standardises the tool interface itself – along with, as Hou et al. document, a new attack surface.

2.2 Literature Review

In this section, the related work is reviewed across the research areas that bear directly on the design and development of this system: sentiment and stance analysis for Bangla and code-mixed text, cost-aware model cascades and query routing, and retrieval-augmented generation with tool-using agents. The twelve works that most directly informed the design are discussed individually below, each followed by a statement of how it shaped this project's design, scope or evaluation, and the same twelve are then summarised in tabular form in Table 2.1. The wider body of work these rest on – the transformer encoders, the multilingual sentence embeddings, the serving techniques, the rank-fusion and evaluation protocols and the prompt-injection threat taxonomy – is introduced in Section 2.1 and cited at the point of use in Chapters 3 and 5.

1. Bhattacharjee et al. [1] – BanglaBERT: Language Model Pretraining and Benchmarks for Bangla (2022)

This pretraining and benchmarking study introduces an ELECTRA-style encoder trained on a 27.5 GB Bangla corpus together with the BLP benchmark suite, and establishes the strongest monolingual Bangla encoder currently available. It underpins one of this project's ensemble decisions directly: two of the seven small sentiment heads are BanglaBERT derivatives, so Bangla is judged by models trained *on* Bangla rather than only by multilingual models that happen to include it.

2. Islam et al. [2] – SentNoB: A Dataset for Sentiment on Noisy Bangla Texts (2021)

This corpus-building study annotates 15,000 noisy public comments and benchmarks classical and neural baselines against them. Its finding, that Bangla social-media text is materially harder than curated Bangla, is the reason this project does not treat a model's benchmark score on clean Bangla as evidence about its behaviour on comment threads. The SentNoB fine-tune is carried as one of the ensemble voters used here.

3. Sazzed [4] – Abusive Content in a Transliterated Bengali-English Corpus (2021)

This annotation and classification study is the closest published work to this project's actual input register. It treats romanized Banglish as a first-class target rather than as noise, and establishes that transliteration, not vocabulary, is the primary source of classifier error. That conclusion is why this project reports its evaluation plan per language bucket and why Banglish is a separate bucket rather than being folded into Bangla.

4. Romim et al. [5] – Bengali Hate-Speech Dataset and Baseline Evaluation (2021)

This dataset study assembles 30,000 comments from public Bengali social media and benchmarks SVM, LSTM and embedding baselines over them. It is the reference Bangla hate-speech resource, and it informs both the toxicity gate in this project's router and its threshold. It also motivates the security posture of Section 5.1.1: comment text is treated as adversarial input rather than as prose.

5. Nezami and Huda [6] – Public Sentiment for Code-Mixed English-Bangla and Banglish Text (2024)

This applied machine-learning study performs feature engineering over a Banglish corpus and is the closest published task formulation to this project's comment lane. It confirms the modelling decision taken here independently: that Banglish is a distinct third bucket rather than a variant of Bangla, which is what the script-mix measurement of Section 4.2.1 then quantifies at roughly one comment in six.

6. Alam et al. [9] – A Review of Bangla NLP Tasks and the Utility of Transformer Models (2021)

This systematic survey benchmarks transformer models across the Bangla NLP task landscape and reaches a conclusion that shaped this project's entire evaluation plan: **annotated data, not model architecture, is the bottleneck for Bangla**. That is precisely why this project spends its effort on the system and on the provenance of every label it emits, and why the annotation programme a labelling-accuracy figure would require is scoped as future work in Section 6.3 rather than improvised.

7. Patwa et al. [10] – SemEval-2020 Task 9: Sentiment Analysis of Code-Mixed Tweets (2020)

This shared task established standard benchmarks over 20,000 annotated code-mixed tweets for Hindi-English and Spanish-English, and confirmed that performance on the code-mixed portion lags the monolingual portion substantially. It supplies the evaluation protocol this project's future benchmarking work would adopt, and the expected size of the gap that work would be measuring.

8. Kucuk and Can [13] – Stance Detection: A Survey (2021)

This systematic survey defines the task that this project's watchlist feature implements. Two of its conclusions are load-bearing here. First, stance toward a target and document sentiment are **distinct quantities**, which is why they occupy separate output fields in this system and are never summed or charted together. Second, the target-dependent formulation is under-served outside English and its benchmarks assume a small fixed target set rather than an operator-supplied one – which is the gap G2 of Section 2.3.

9. Chen, Zaharia and Zou [30] – FrugalGPT: Reducing Cost While Improving Performance (2023)

This experimental study shows that a cascade of increasingly capable models, gated by a learned scoring function under a cost budget, can match a single large model's quality at a fraction of the cost. It is the foundational cost-cascade result and the direct ancestor of this project's router. This project implements the same idea with a rule-based rather than a learned gate, and – the contribution back – measures where the analogy breaks when the routed unit is a thread rather than a query.

10. Ong et al. [31] – RouteLLM: Learning to Route with Preference Data (2024)

This study learns the routing decision from preference data and publishes open router benchmarks, demonstrating that routing need not be hand-tuned. It is identified in Section 6.3 as the single most valuable upgrade to this project's hand-tuned gate, and the reason that upgrade is sequenced *after* the annotation programme is that a learned router needs labels to learn against.

11. Lewis et al. [34] and Gao et al. [35] – Retrieval-Augmented Generation, and its Survey (2020; 2023)

These works establish the pattern of grounding a generated answer in retrieved evidence and map the design space that has grown around it. This project's insight layer generalises that pattern in one specific respect: its agents retrieve the system's **own structured analytical output** rather than documents a user could have read anyway, which makes the retrieval analytical rather than documentary and allows a cited number to be traced to the model that produced it.

12. Hou et al. [38] – Model Context Protocol: Landscape and Security Threats (2025)

This survey describes the tool-interface standard adopted here together with its attack surface. As the most recent and most directly relevant source on the protocol, it serves as the technical validation for this project's tool layer, and its threat analysis – read alongside the indirect-prompt-injection taxonomy of Greshake et al. [43] – is what motivated the runner hardening described in Section 3.2.6.

Table 2.1 summarises the same twelve works, in the same order, so that each row corresponds to one of the discussions above. They are ordered so that the three research areas identified in Section 2.1 remain visible in the table: Bangla and code-mixed language processing first, then cost-aware inference, then retrieval and agents.

Table 2.1: Summary of Literature Reviewed.

Author(s)	Year	Title (abbreviated)	Methodology	Key findings and relevance here
Bhattacharjee et al. [1]	2022	BanglaBERT: pretraining and benchmarks for Bangla	ELECTRA-style pretraining on a 27.5 GB Bangla corpus; BLP benchmark	Establishes the strongest monolingual Bangla encoder and a public benchmark. Two of the seven ensemble heads in this project are BanglaBERT derivatives, so Bangla is judged by more than multilingual models that merely include it.
Islam et al. [2]	2021	SentNoB: sentiment on noisy Bangla texts	Human-annotated corpus of 15,000 noisy public comments; classical and neural baselines	Shows that Bangla social-media text is materially harder than curated Bangla. The SentNoB fine-tune is one of the ensemble voters used here.

Author(s)	Year	Title (abbreviated)	Methodology	Key findings and relevance here
Sazzed [4]	2021	Abusive content in a transliterated Bengali-English corpus	Annotated transliterated corpus; lexical and neural classifiers	Directly the setting of this project. Treats romanized Banglish as a first-class target rather than as noise, and establishes that transliteration, not vocabulary, is the primary source of error.
Romim et al. [5]	2021	Bengali hate-speech dataset and baseline evaluation	30,000-comment dataset drawn from public Bengali social media; SVM, LSTM and embedding baselines	The reference Bangla hate-speech resource; informs the toxicity gate design and its threshold, and motivates treating comment text as adversarial input rather than as prose.
Nezami and Huda [6]	2024	Public sentiment for code-mixed English-Bangla and Banglish text	Feature engineering plus classical machine learning on a Banglish corpus	The closest published task formulation to this project's comment lane; confirms that Banglish is treated as a distinct third bucket rather than folded into Bangla.
Alam et al. [9]	2021	Review of Bangla NLP tasks and the utility of transformer models	Systematic survey plus benchmarking across tasks	Concludes that annotated data, not architecture, is the bottleneck for Bangla. That conclusion is why this project spends its effort on the system and on the provenance of every label it emits, and treats the annotation programme a labelling-accuracy figure would require as scoped future work.
Patwa et al. [10]	2020	SemEval-2020 Task 9: sentiment of code-mixed tweets	Shared task with 20,000 annotated code-mixed tweets	Establishes the standard evaluation protocol for code-mixed sentiment and the size of the gap versus monolingual data.
Kucuk and Can [13]	2021	Stance detection: a survey	Systematic survey of target-dependent stance, its benchmarks and its formulations	Defines the task this project's watchlist feature implements. Establishes that stance toward a target and document sentiment are distinct quantities – which is why they occupy separate output fields here – and confirms that the target-dependent formulation is under-served outside English, and that its benchmarks assume a small fixed target set rather than an operator-supplied one.

Author(s)	Year	Title (abbreviated)	Methodology	Key findings and relevance here
Chen, Zaharia and Zou [30]	2023	FrugalGPT: reducing cost while improving performance	LLM cascade with a learned scoring function and a cost budget	The foundational cost-cascade result. This project implements the same idea with a rule-based rather than learned gate, and measures where the analogy breaks for thread-shaped units.
Ong et al. [31]	2024	RouteLLM: learning to route with preference data	Learned router trained on preference data; open router benchmarks	Shows the routing decision can be learned. Identified in Chapter 6 as the most valuable single upgrade to this project's hand-tuned gate.
Lewis et al. [34]; Gao et al. [35]	2020; 2023	Retrieval-augmented generation, and its survey	Dense retriever plus generator; systematic survey of variants	The pattern the insight layer generalises from single-shot retrieval to a tool-using loop.
Hou et al. [38]	2025	Model Context Protocol: landscape and security threats	Survey of the protocol, its ecosystem and its attack surface	The tool-interface standard adopted here, together with the threats that motivated the runner's hardening.

2.2.1 Similar Applications

Five classes of deployed or published system are closest to this work. Table 2.2 summarises the core functionality, the approach, the limitation of each, and how this project fills the gap that limitation leaves; a fuller account of each class follows the table.

Table 2.2: Comparative Review of Similar Applications.

Application class	Core functionality	Methodology / approach	Limitation	How this project addresses it
Commercial social-listening suites (Brandwatch, Sprinklr, Meltwater)	Ingestion, dashboarding and alerting over social media at scale	English-centric sentiment models with machine translation in front of them; one coarse score per post	Shallow Bangla coverage; romanized Bangla generally unhandled; per-comment analysis absent or sampled; closed – an operator cannot see, swap or audit the model behind a label	Bangla-specific and multilingual heads run side by side over every stored comment, and every label carries a provenance field naming the engine that produced it
The upstream monitoring platform this project consumes from	Reliable scraping keyed by stable identifiers, with engagement, reactions and a share sample	Platform-side scoring: one coarse sentiment and one virality score per post	Comment sentiment is null on all 10,272 comments; no optical character recognition; no embeddings. It is a data source, not an analysis layer	The whole per-post pipeline of Chapter 3, run as a read-only consumer that never writes back; the upstream score is retained only as a displayed baseline
Academic Bangla sentiment and hate-speech systems [4][5][6][7][8]	Single-task classification of Bangla or transliterated Bangla text	Supervised fine-tuning evaluated offline against a fixed annotated dataset	No ingestion, no routing, no cost model, no serving path and no operational story – they are classifiers, not systems	Several of them are consumed at once as ensemble voters inside a deployable layer that reports which of them actually spoke on each comment
General-purpose RAG chat assistants over a document store	Question answering grounded in retrieved documents	Dense retrieval over raw text, followed by single-shot generation	Retrieves the text a user could have read anyway; a poor first query cannot be recovered from; no defence against instructions embedded in retrieved third-party text	Agents retrieve the system's own structured analytical output through typed tools inside a loop, with untrusted-data delimiters, citation verification and non-answer rejection
Model-cascade and router frameworks [30][31][32]	Deciding which model should answer a given query	Learned or heuristic scoring over single queries, benchmarked in isolation	Components rather than systems: no ingestion contract, no per-comment ensemble, no stance watchlist, and no cost telemetry with which to check afterwards whether routing saved anything	The router is embedded in a complete system with counters attached, which is what made the measured finding of Section 4.2.4 visible at all

Commercial social-listening suites are mature products with strong ingestion, dashboarding and alerting. Their language coverage for Bangla is shallow: sentiment is typically a single coarse score per post derived from an English-centric model with translation in front of it, per-comment

analysis is either absent or sampled, and romanized Bangla is generally unhandled. Critically for this project, they are closed systems – an operator cannot see which model produced a label, cannot swap the model, and cannot audit the cost of a run. The upstream platform this project consumes from is itself of this class, which is why its post sentiment is retained only as a baseline.

The upstream monitoring platform itself scrapes reliably, keys records by stable identifiers, and exposes engagement, reactions and a sample of shares. It stores one coarse sentiment and one virality score per post, leaves comment sentiment null on every one of the 10,272 comments in the sample, ships no optical character recognition, and computes no embeddings. It is a data source, not an analysis layer, which is the precise gap this microservice fills.

Academic Bangla sentiment and hate-speech systems [4][5][6][7][8] produce the models this project consumes and the evidence that the task is hard. What they do not produce is a system: they are single-task classifiers evaluated offline on a fixed dataset, with no ingestion, no routing, no cost model, no serving path and no operational story. This project's contribution relative to them is not a better classifier but a deployable layer that uses several of them at once and reports honestly on which one spoke.

General-purpose RAG chat assistants over a document store retrieve raw documents and generate an answer. The insight layer here differs in the object of retrieval: agents retrieve the system's **own structured analytical output** – sentiment distributions, stance verdicts, agreement statistics, toxicity scores – not the text a user could have read anyway. That output exists only because the pipeline ran, which makes the retrieval analytical rather than documentary. The practical consequence is that an agent can cite a number and that number can be traced to the model that produced it.

Model-cascade and router frameworks [30][31][32] solve the routing problem in the abstract and, in RouteLLM's case, learn it. They are components, not systems: none carries an ingestion contract, a per-comment ensemble, a stance watchlist, or the cost telemetry needed to check afterwards whether the routing decision actually saved anything. This project's measured finding – that the container-level gate governs only 30 percent of spend once every sub-item is labelled – is the class of result that becomes visible only when a router is embedded in a complete system with counters attached.

Conclusion: The systems reviewed offer solid, focused solutions to discrete problem domains – scale ingestion and dashboarding on one hand, single-task Bangla classification on another, abstract query routing on a third – and none of them consolidates these capabilities into a single cost-aware, auditable analysis layer for code-mixed Bangla in which every label can say what produced it. That persistent fragmentation, reflected across all five classes, is what underscores the practical value of the system built here.

2.3 Gap Analysis

Reading the review above against what this project needed produces five gaps. Table 2.3 states them against the systems described above, and Table 2.4 gives the feature-level comparison in the form the template asks for.

Table 2.3: Gaps identified from the related-work study.

#	Gap observed in prior work	How this project addresses it
G1	Cost-aware cascades are evaluated on single queries. No published cascade routes a <i>thread</i> whose sub-items each need labelling, so no prior work reports what share of the eventual bill the container-level decision actually governs.	The router makes two separate decisions – post-level spend and comment selection, and the system's own counters report the post-level versus comment-level call split. The measured answer, that the gate governs only 30 percent of calls once comment coverage is full, is reported in Section 4.3 as a finding that contradicted the project's own initial claim.
G2	Target-dependent stance is established for English and for fixed, annotated target sets. For code-mixed Bangla, the hard part is not the classifier but the <i>matcher</i> : one entity appears in Bangla script, in several inconsistent romanizations and in English, and a watchlist that matches one spelling fails silently.	An operator-supplied, versioned watchlist with an alias matcher built for the three-script problem: case-insensitive for Latin and exact for Bangla, Bangla suffixes permitted but not prefixes, token boundaries enforced for short initialisms, overlapping aliases collapsed to the longest, and clause-level rather than window-level cue attachment. A target that matches zero comments across a run is logged as the probable alias-coverage bug it almost certainly is.
G3	Ensemble sentiment systems typically report a single fused label. When a member model fails to load, it is silently replaced by a default, and the reported agreement becomes unfalsifiable.	Eight labellers per comment with explicit abstention. A head that cannot load, or that emits a label outside the taxonomy, contributes nothing rather than a fabricated neutral; the number of voters that actually spoke is on every comment; and a comment no model read is reported as <i>uncertain</i> , which is a statement about the system's confidence rather than about the comment.
G4	Agentic RAG systems are evaluated on whether the answer looks right. Deployed on an adversarial corpus, they have no defence against instructions embedded in retrieved third-party text [43] and no mechanism to reject an answer that answers nothing.	Tool results are wrapped in explicit untrusted-data delimiters with forged-delimiter neutralisation; the system prompt states that content inside them is data. Four distinct shapes of non-answer are detected and fail the run; citations and quotations are verified against what the model was actually shown; byte-identical repeat tool calls are refused; and every run carries a tool-call budget.
G5	Systems papers report what a system does. They rarely report which of their claims were <i>checked</i> , so an implemented-but-unexercised capability reads exactly like a measured one.	Every capability in this system carries an evidence class – measured, works-but-unmeasured, unexercised, or out of scope. The image modality is reported as unexercised because no image bytes are reachable; labelling accuracy is reported as a scope boundary rather than obscured behind a passing test suite.

Table 2.4 compares the proposed system against the four external classes of system on the capabilities that matter for this problem. "Partial" is used where a capability exists but in a form that does not meet the requirement stated in Section 3.1.3.

Table 2.4: Feature comparison of the proposed system against comparable systems.

Capability	Commercial listening suites	Upstream monitoring platform	Academic Bangla classifiers	General RAG assistants	Proposed system
Bangla script sentiment	Partial	Yes (coarse, post only)	Yes	No	Yes
Romanized Banglish handling	No	No	Partial	No	Yes
Per-comment sentiment over the whole stored thread	Partial (sampled)	No (null on every comment)	No	No	Yes
Multiple independent labellers per comment, shown side by side	No	No	No	No	Yes
Abstention instead of a fabricated default vote	No	No	No	No	Yes
Label provenance on every value	No	No	No	No	Yes
Selective, gated use of the expensive model	No	No	Not applicable	No	Yes
Measured post-level versus comment-level cost split	No	No	No	No	Yes
Watchlist-driven target stance, alias-aware	No	No	No	No	Yes
Stance kept in a separate field from document sentiment	Not applicable	Not applicable	Partial	Not applicable	Yes
Runtime-switchable local and cloud inference backend	No	No	No	Partial	Yes
Tenant privacy lock preventing data egress	Partial	No	Not applicable	No	Yes
Hybrid dense plus lexical retrieval with rank fusion	Partial	No	No	Partial	Yes
Measured retrieval quality (recall@k, MRR@k, nDCG@k)	No	No	No	Rarely	Yes
Tool-using agents over the system's own analytics	No	No	No	Partial (over raw text)	Yes
Prompt-injection hardening for retrieved third-party text	Not applicable	Not applicable	Not applicable	Rarely	Yes
Non-answer rejection and citation verification	Not applicable	Not applicable	Not applicable	No	Yes
Per-backend, per-model, per-lane cost telemetry	Billing only	No	Not applicable	No	Yes

Capability	Commercial listening suites	Upstream monitoring platform	Academic Bangla classifiers	General RAG assistants	Proposed system
Stop, resume and delete a running analysis job	Partial	No	Not applicable	Not applicable	Yes
Live per-post execution trace	No	No	No	No	Yes
Published evidence class for every claim	No	No	Partial	No	Yes
Image and OCR modality	Partial	No (OCR withdrawn)	Partial	No	Implemented, unexercised
Measured labelling accuracy against human labels	Undisclosed	No	Yes	No	Out of scope (stated)

Two of the twenty-three rows in the final column are deliberately not "Yes", and they qualify the twenty-one that are. The image and OCR modality is implemented but cannot be exercised on this corpus: its 69 image references are relative object-storage keys whose objects are not present in storage, so the visual term has never carried a value. Recording it as a working capability would reproduce gap G5 above. Labelling accuracy is out of scope rather than attempted and quietly dropped: the stratified 300-comment gold set holds zero adjudicated labels by design, because labels seeded from a model in this repository and then used to score that model would measure the system's agreement with itself rather than its accuracy. Both capabilities are carried at their evidence class throughout the report – Table 4.13 records the class of every capability, and Sections 4.3.4 and 6.2 develop them.

2.4 Summary

The background divides cleanly. From Bangla and code-mixed NLP this project takes its models and, more usefully, the established finding that romanized text is where those models fail and that annotated data is the field's binding constraint. From the cost-aware inference literature it takes the cascade pattern, and it contributes back the observation that the pattern's cost accounting changes when the routed unit contains thousands of sub-items. From retrieval and agent research it takes the tool-using loop, rank fusion and the recall-oriented evaluation protocol, and it contributes an analytical rather than documentary retrieval target together with the guards a deployment on adversarial content requires. The five gaps in Section 2.3 are the specification for Chapter 3.

Research Methodology

This chapter specifies the system. It states the requirements the design has to meet, presents the architecture and the five-stage pipeline that realises it, develops each of the load-bearing components in detail, records the alternatives that were considered at every significant decision point and why the chosen option won, and closes with the project plan and the allocation of work.

3.1 Methodology / Requirement Analysis and Design Specification

3.1.1 Overview

The system is a **self-contained analysis microservice** that sits beside an existing social-media monitoring platform. Its contract with that platform is deliberately one-directional: it **pulls** a post-with-details payload, analyses it in **its own** datastores, and **never writes back**. The payload is one JSON object per post carrying the post itself, its embedded comment thread, an engagement block, a reaction breakdown and a sample of shares. Because the comments arrive embedded, no second fetch and no join are required, and the natural unit of analysis is not a post but **a post together with its comment thread**.

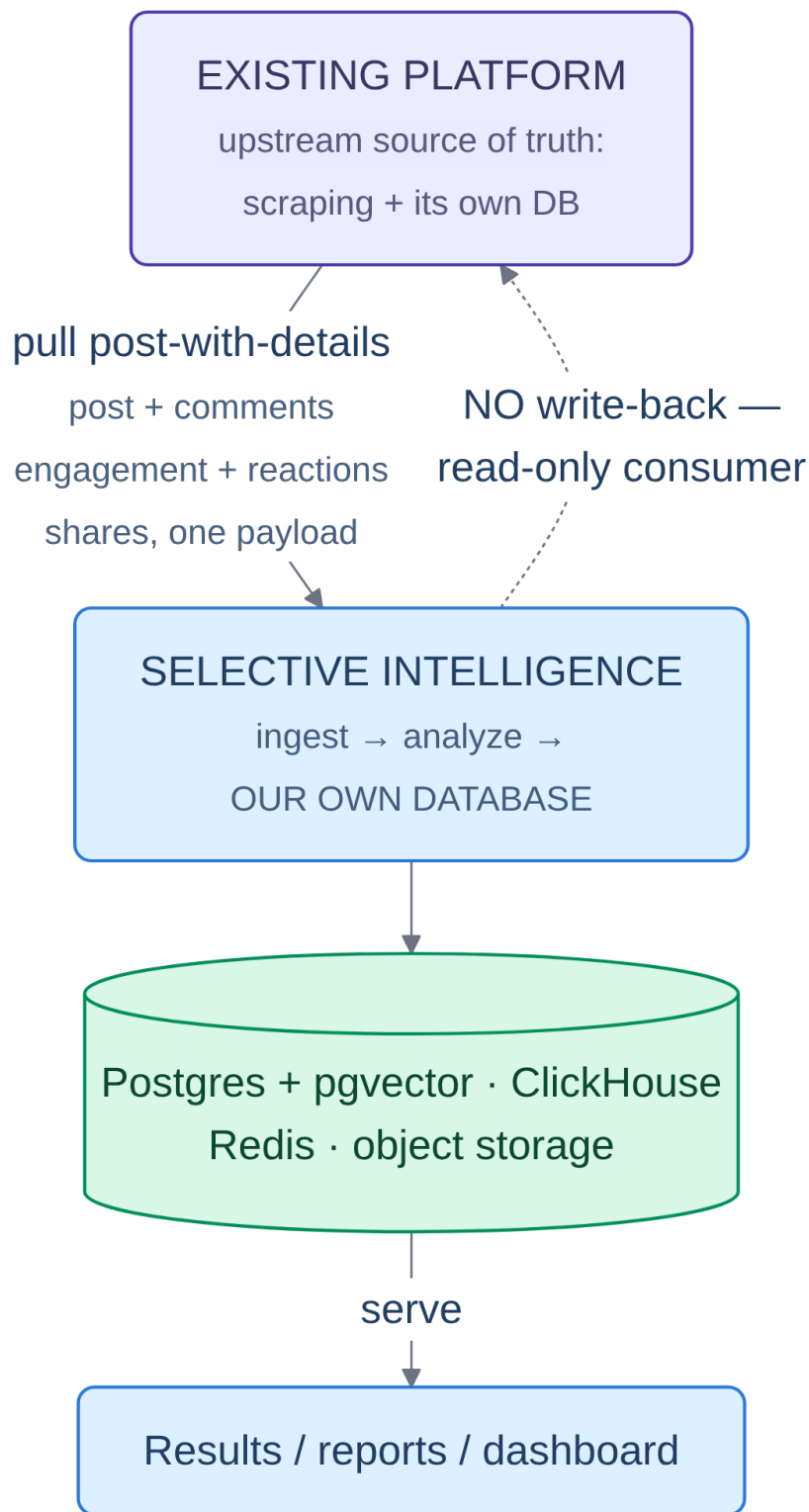


Figure 3.1: Integration model: a read-only pull into a separate datastore.

How to read it. Four elements on a single vertical path. The violet box at the top is the upstream monitoring platform, which this project does not own; the blue box beneath it is Selective Intelligence itself; the green cylinder is this project's own storage tier, holding PostgreSQL with pgvector, ClickHouse, Redis and object storage; the blue box at the foot is what an operator finally sees. The long label on the descending arrow enumerates one payload: the post together with its comments, engagement, reactions and shares, all arriving in a single pull rather than as a comment fetch per thread. The dotted arrow climbing back to the platform is

the only edge in the figure that describes something the system does not do.

Figure note. *The two properties that constrain everything downstream are visible in the arrows: the payload arrives whole, so no separate comment fetch is required, and the return path is struck through, so no analysis is ever written back. Records are keyed by the upstream identifier, which makes a repeated pull an upsert rather than a duplication.*

Everything downstream follows from three properties of that input. First, the thread is a **sample**, not a census: the payload reports how many comments the platform holds and how many it actually shipped, and in this corpus the second number is 3.75 percent of the first. Any honest output has to report that coverage rather than imply completeness. Second, the platform supplies **one coarse sentiment score per post and nothing at all per comment**, so per-comment sentiment is entirely the responsibility of this service, and the upstream score is retained only as a baseline for comparison. Third, the text is **code-mixed**, which means model selection cannot be a single choice.

Figure 3.1 states the position this system occupies relative to the platform that feeds it, and three properties follow from it. The first is that the payload arrives **whole**. A post, its comment thread, its engagement counts, its reaction breakdown and a sample of shares come in one pull, so the system never issues a comment fetch per thread and never has to reason about a partially retrieved thread. This is the precondition for thread-level analysis: the unit of work is a complete conversation, and it is complete on arrival.

The second is that records are keyed by the **upstream identifier**. Pulling the same post twice is therefore an upsert rather than a duplication, which makes the ingestion path idempotent without any coordination between the two systems, and makes a re-pull a safe recovery action rather than a corruption risk.

The third is the dotted arrow, which is the only edge in the figure describing something that does not happen. Nothing is ever written back to the upstream platform. This is a deliberate architectural commitment rather than an unimplemented feature: it means the system cannot corrupt the source of truth, that it can be removed entirely without leaving residue in the platform, and that its analytical output is free to be wrong in ways that would be unacceptable if it were mutating a shared record. It is also what makes the separate datastore in the middle of the figure necessary rather than redundant – having declined to write to the upstream database, the system must own one.

The design responds with a five-stage, queue-decoupled pipeline in which cheap analysis runs unconditionally and expensive analysis is spent selectively, plus two layers that sit outside the per-post path: a watchlist stance capability carried inside a prompt that is already being issued, and a corpus-tier agentic layer that is never invoked per post. Figure 3.2 gives the high-level architecture and Figure 3.3 the pipeline.

3.1.2 Proposed Methodology / System Design

Figure note. The router sits between the two analysis stages and is the only component that decides how much work an item receives. The asymmetry in the storage band is deliberate: the assembler fans out to all four stores in parallel, whereas the read paths touch only PostgreSQL and ClickHouse, so the analytics store can be rebuilt without disturbing operational reads.

The figure is organised as five horizontal bands, and each boundary between them is a place where the system deliberately gives something up in exchange for a property it needs. The **client band** contains two consumers that are treated identically: the project's own dashboard and any third-party application. Neither is privileged, because both arrive through the same gateway over the same authenticated HTTP surface; the dashboard holds no private channel into the system, which is what makes the API the product rather than the user interface.

The **gateway** terminates TLS, routes, and applies rate limits. Placing it ahead of the service band means that the services behind it never implement transport security or quota logic, and that the production tier can substitute a Kubernetes ingress for NGINX without any service being aware of the change.

The **service band** splits into authentication, ingestion and reporting. That split is by *lifetime of the request*, not by subject matter. Authentication answers in milliseconds and holds no domain state. Ingestion validates a payload, deduplicates it against a content hash and returns as soon as the work is durably enqueued, so the caller is never held open for the length of an analysis. Reporting serves reads and generates documents, and it is the only service that may take seconds to answer. Collapsing any two of these into one process would couple response-time budgets that have no reason to be coupled, which is precisely the simplification the MVP tier accepts and the production tier reverses.

Only ingestion writes to the **message bus**, and this is the single most consequential arrow in the figure. It means the pipeline has exactly one entry point, so every post in the system arrived the same way and carries the same envelope. It also means the write path and the read path share no component: a pipeline stalled behind a slow model does not slow a report, because reporting reads from storage and never from the bus. The bus itself is drawn as one box with two implementations because the pipeline's contract with it is narrow – durable append, consumer groups, partitioning by a hash of the post identifier – and both Redis Streams and Kafka satisfy it.

The **pipeline band** is the subject of Section 3.2 and of Figure 3.3. What matters at this level is only that the router sits inside it, between the two analysis stages, and that it is the sole component in the entire figure permitted to decide how much work an item receives. No other box makes that judgement, which is why cost behaviour can be reasoned about by reading one component rather than by auditing the whole system.

The **agentic layer** is drawn to the side rather than in the chain, and its only inbound edge comes from reporting. That placement encodes the cost boundary that Section 3.2.6 develops: agents run per question, at corpus scale, and are never invoked per post. Had the layer been drawn inside the pipeline band it would have inherited the per-post invocation rate, and the generation cost the cascade exists to avoid would have re-entered the system through the analytical layer.

The **storage band** is split by access pattern rather than by subject. Four stores receive the assembler's fan-out; only two of them are ever read back. PostgreSQL with pgvector holds operational rows, job state and the embedding index, and is the store the reporting service and the tool servers query. ClickHouse holds append-only analytical events. Redis holds cache entries, the deduplication set and counters. Object storage holds raw payloads and generated reports. Because ClickHouse is written from the same canonical result that PostgreSQL receives, it is derivable rather than authoritative: it can be dropped and rebuilt from the raw payloads without an operational read ever noticing, which is a property that a single general-purpose database would not have offered.

Four principles govern the design, and each of them is visible somewhere in Figure 3.2.

- P1. **Hybrid intelligence, not a large model everywhere.** Small specialised models are two to three orders of magnitude cheaper and faster per item. The large model is a scalpel, reserved for what small models cannot do at all – writing prose, or for the cases where a small model is genuinely uncertain.
- P2. **Queue-based and stage-decoupled.** Every heavy stage is a stateless worker pool fed by a durable stream, so back-pressure, retries and independent scaling come for free and no stage's failure can take another one down.
- P3. **Scale by partition, not by bigger machines.** Throughput grows by adding workers and partitions. The bus is partitioned by a hash of the post identifier, so a given post's ordering is stable while load spreads evenly.
- P4. **Idempotent and observable.** Every post has a stable content hash, so reprocessing is safe; every stage emits progress events, metrics and traces; and every value in the output carries provenance saying what produced it.

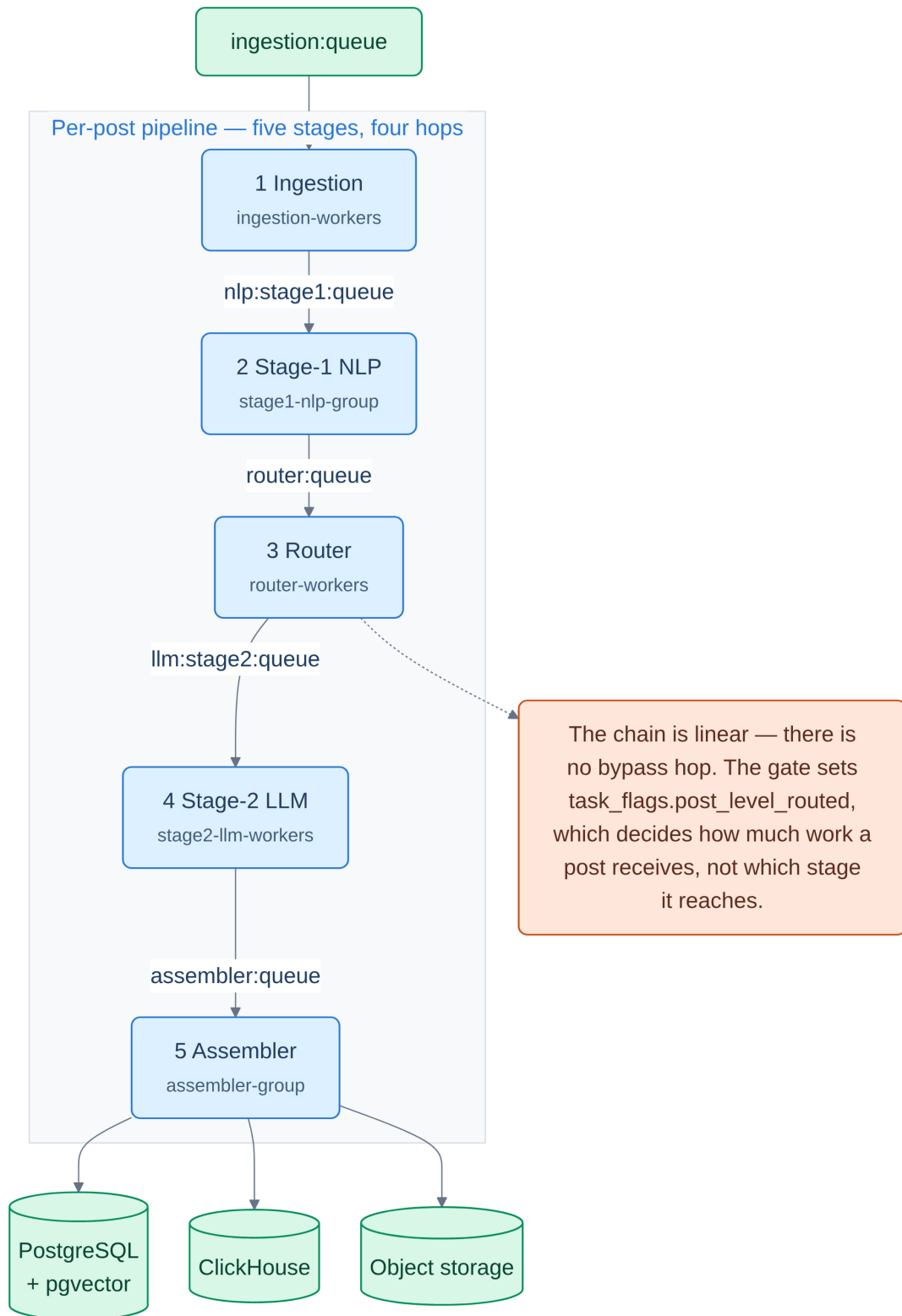


Figure 3.3: The five-stage per-post pipeline and the single envelope that threads through it.

How to read it. A single vertical chain inside a titled panel, entered from the queue drawn above it. The five blue boxes are the pipeline stages in order, each captioned with the name of the consumer group that runs it, and the label on each arrow is the literal name of the stream that carries work between them. Below the panel, three green cylinders receive the assembler's output. The orange callout hanging off the router on a dotted line is a note rather than a stage: it exists to contradict a specific misreading, which is why it is attached to the router and not to the chain.

Figure note. Each hop is a durable stream with its own consumer group, so the stages scale independently and none can block another. The callout states the property most often misdescribed in the project's own early documentation: the chain carries no bypass edge, and the gate governs the quantity of work rather than the route.

Figure 3.3 names the four streams and the five consumer groups explicitly, because those names are the system's real interfaces. A stage knows nothing about the stage before it except the name of the stream it reads and the shape of the envelope it will find there. Each hop is a durable append rather than a call, so a stage that dies mid-batch loses no work: the entries it had claimed but not acknowledged return to the group and are redelivered. Each stage also runs as its own consumer group, which is what allows the stage-2 worker pool to be scaled to four processes while the router runs as one, without any coordination between them.

One envelope threads the whole chain. Ingestion normalises the upstream payload into it, and every subsequent stage adds fields rather than replacing the document, so by the time the assembler receives it the envelope carries the stage-1 features, the router's flags, the stage-2 outputs and the provenance of each. This is why the assembler can validate the result against a schema in one place: it is the only stage that sees the finished object, and every field in that object arrived with a record of which component produced it.

The most important structural fact about Figure 3.3, and the one most often misdescribed in the project's own early documentation, is that **the chain is linear**. The router does not divert a post away from Stage 2. It writes every post to the Stage-2 stream and marks the envelope with a flag saying whether post-level work was earned. A post that fails the gate still has its comments analysed, because the entire comment ensemble lives in Stage 2. An earlier design did divert bypassed posts straight to the assembler, and the consequence was that the majority of posts arrived with three empty columns in the per-comment comparison and nothing in the output explaining why.

3.1.3 Functional and Nonfunctional Requirements

This subsection states the functional and non-functional requirements established for the system. Functional requirements define what the system must **do** – the behaviour it exhibits in response to a given input or operator action. Non-functional requirements specify **how** the system must work, in terms of quality attributes such as scalability, cost efficiency, reliability, security and maintainability, which determine whether the system is fit to be operated rather than merely fit to be demonstrated.

The two classes are written to different standards here, and the difference is deliberate. A functional requirement is satisfied or it is not, so each one below names the design element that satisfies it and the claim is binary. A non-functional requirement is a quality attribute, which can be met to a degree, so each one carries both a target and the mechanism intended to reach it. Naming the mechanism rather than only the target is what makes the requirement auditable: a reviewer can check that the mechanism exists in the code even where the target has not yet been measured under load.

Functional Requirements

1. **FR1 – Ingestion.** The system shall pull a post-with-details payload from the upstream platform for a campaign, time window or identifier, and shall accept a pushed batch for replay and external sources. *Satisfied by:* the ingestion service, which exposes both a pull endpoint and an upload endpoint.
2. **FR2 – Platform derivation.** The system shall derive the source platform from each post's URL host rather than assuming a single platform. *Satisfied by:* the host-pattern table in the normaliser.
3. **FR3 – Baseline preservation.** The system shall preserve the upstream coarse sentiment and virality score as baselines, and shall never overwrite them with recomputed values. *Satisfied by:* baseline fields carried through the canonical schema.
4. **FR4 – Language and script detection.** The system shall detect language and flag romanized Banglish per post and per comment, and shall report which method produced the verdict. *Satisfied by:* Stage-1 language detection with a reported method field.
5. **FR5 – Post-level analysis.** The system shall recompute post-level sentiment, emotion, topics, intents, semantic post type, toxicity, named entities and keywords. *Satisfied by:* the Stage-1 text analyser across three interchangeable engines.
6. **FR6 – Comment-level analysis.** The system shall label every comment in the stored thread and report the label's provenance per comment. *Satisfied by:* the Stage-1 comment analyser plus per-comment method and kind fields.
7. **FR7 – Post-level routing.** The system shall decide, per post, whether post-level large-model work is warranted, using explicit and tunable rules. *Satisfied by:* the six-gate router.
8. **FR8 – Comment selection.** The system shall decide separately which comments the Stage-2 labellers read, through eligibility, deduplication and an optional cap. *Satisfied by:* the router's comment-selection step.
9. **FR9 – Ensemble labelling.** The system shall collect multiple independent verdicts per analysed comment and combine them through a single writer, with abstention rather than a default vote. *Satisfied by:* the eight-labeller ensemble and its combiner.
10. **FR10 – Summarisation.** The system shall produce a post summary in the post's own language, recovering from truncation rather than caching a half-summary. *Satisfied by:* the Stage-2 summary task on a dedicated fluent model role.
11. **FR11 – Target stance.** The system shall report per-entity stance toward an operator-supplied watchlist, in a field separate from document sentiment. *Satisfied by:* the alias matcher plus the deterministic and prompt-based scorers.
12. **FR12 – Canonical output.** The system shall emit one schema-validated JSON document per thread and persist it to three stores. *Satisfied by:* the assembler and the canonical output schema.
13. **FR13 – Coverage honesty.** The system shall report comment coverage honestly and clamped, with an explicit anomaly flag when the stored rows exceed the platform-reported total. *Satisfied by:* the coverage computation and its anomaly flag.

14. **FR14 – Job control.** The system shall allow an analysis job to be started, tracked live, stopped, resumed and deleted. *Satisfied by:* job control through a Redis cancellation flag and a PostgreSQL-derived resume.
15. **FR15 – Retrieval.** The system shall support identifier lookup, keyword, semantic and hybrid search over analysed posts and comments. *Satisfied by:* the search endpoint with three retrieval arms and rank fusion.
16. **FR16 – Agentic insight.** The system shall answer corpus-level analyst questions with grounded, cited answers produced by tool-using agents. *Satisfied by:* nine agents over eighteen typed MCP tools.
17. **FR17 – Cost telemetry.** The system shall report token and cost usage per backend, per model and per spending lane. *Satisfied by:* usage counters written inside the LLM client.
18. **FR18 – Operator dashboard.** The system shall provide an operator dashboard covering results, jobs, search, agents, chat, pipeline state, per-post trace, watchlist warnings and logs. *Satisfied by:* the eleven-tab React dashboard with SSE streams.

Non-Functional Requirements

1. **NFR1 – Scalability.** Batches shall scale from one thousand to one hundred thousand threads by adding workers and partitions, not by enlarging machines. *Mechanism:* stateless workers, one durable stream per hop, partitioning by post-identifier hash, and queue-depth autoscaling.
2. **NFR2 – Cost efficiency.** The expensive model shall be spent selectively, and the spend shall be attributable afterwards. *Mechanism:* confidence-gated routing, comment selection, an exact-and-near-duplicate cache, a response cache keyed on the resolved model identifier, and five-lane usage counters.
3. **NFR3 – Multilingual accuracy.** Bangla, English and romanized Banglish shall all be handled. *Mechanism:* a multilingual encoder family plus Bangla-specific fine-tunes in the ensemble, language-routed model selection, and per-language reporting throughout the evaluation plan.
4. **NFR4 – Reliability.** No post may be silently lost. *Mechanism:* bounded retry with the attempt counter carried inside the payload, then dead-lettering with error context; the original message is always acknowledged; dead-lettered posts count against their job so a progress bar cannot stall forever.
5. **NFR5 – Idempotency.** Reprocessing shall be safe. *Mechanism:* content-hash keys, upserts on post identifier, a deterministic object-storage key, and a collapsing analytics table so a re-analysed post is not double-counted.
6. **NFR6 – Latency bounds.** A stop shall cost at most one post per stage rather than the remainder of the batch. *Mechanism:* cooperative cancellation checked at the head of every stage.
7. **NFR7 – Observability.** Every stage boundary shall be visible per post. *Mechanism:* progress events on a per-job channel, replayed on connect so a client that subscribes late still sees the start; per-stage timings recorded on the result; metrics, structured logs and traces.

8. **NFR8 – Auditability and honesty.** No output may claim more than the system did. *Mechanism:* engine and degraded-component reporting, per-label provenance, a stub-vector flag on every stored embedding, and a recorded backend and model on every result.
9. **NFR9 – Security and privacy.** Post text shall be treated as untrusted input. *Mechanism:* token verification on every transport, API keys stored as hashes with the tenant taken from the row, single-use streaming tickets, fail-closed policy checks, and untrusted-data delimiters around everything a model is shown.
10. **NFR10 – Data residency.** A privacy-locked tenant's content shall never leave the cluster. *Mechanism:* the backend decision is resolved at the API boundary, stamped into the job envelope, honoured by both stages, and re-resolved on resume.
11. **NFR11 – Portability.** The system shall run on a laptop and in a cluster from the same code. *Mechanism:* Docker Compose for development, Kubernetes manifests for production, and a stub mode in which every model degrades to a deterministic function so offline testing stays green.
12. **NFR12 – Maintainability.** A cross-component rename shall break a test, not a feature. *Mechanism:* single-source constant modules for stream names, consumer groups and label vocabularies, with tests asserting that the deployment manifests agree.

That distinction matters for how this report should be read, because the two classes have different evidence behind them. Every functional requirement above is exercised by the full-chain run reported in Section 4.2.2. The non-functional requirements are not all in that position: NFR1 and NFR6 are properties of a deployment shape that Chapter 4 did not measure, since every measurement was taken on a single host, and they are therefore claims about the design rather than about observed behaviour. Section 4.3.4 records them at that evidence class rather than allowing the mechanism's presence to stand in for evidence that it works at scale.

3.1.4 Data Flow Diagram

Figure 3.4 gives the context diagram and Figure 3.5 decomposes it one level, to the granularity of six processes and five datastores. The two figures answer different questions. The context diagram fixes the system's boundary: what counts as inside, what counts as outside, and exactly which data crosses between them. The level-1 diagram opens that boundary and shows where state is held.

Four external entities appear at level 0, and the choice of which four is itself a design statement. The upstream platform is a source and never a destination. The operator is both, supplying analysis requests, agent questions and watchlist configuration, and receiving structured results, briefings and reports. The downstream consumer is a destination only. The inference backend is drawn as an external entity rather than as an internal component precisely because it may not be internal: under the metered configuration described in Section 3.2.5, prompts leave the operator's infrastructure entirely, and a diagram that hid the backend inside the system boundary would have concealed the data-residency question rather than posed it.

At level 1 the numbering carries meaning. Processes 1 to 5 are the per-post tier and execute once per ingested thread; process 6 is the corpus tier and executes once per operator question. The five

datastores are separated by who writes them and when. D1 holds posts and raw payloads and is written twice, once on arrival and once after assembly, which is what allows a reanalysis to be replayed from the original payload rather than from a partially processed record. D2 holds the analysis results together with their embeddings, and is the store that retrieval reads. D3 is the only store drawn with a bidirectional edge, because process 4 both reads and writes it: it is the response cache and usage counters, and that bidirectionality is the mechanism that makes a repeated analysis cost nothing. D4 is read-only from the pipeline's perspective, holding the model registry and configuration that process 2 consults. D5 holds analytics events and per-comment sentiments.

The most informative feature of the level-1 diagram is what does *not* connect to process 6. No process feeds it directly; it is reachable only through stores D2 and D5. The insight layer therefore reasons over persisted analytical output and never over a post in flight, which is what makes it safe to invoke at a different rate from the pipeline, and what guarantees that an agent's answer is grounded in results that have already been assembled and schema-validated.

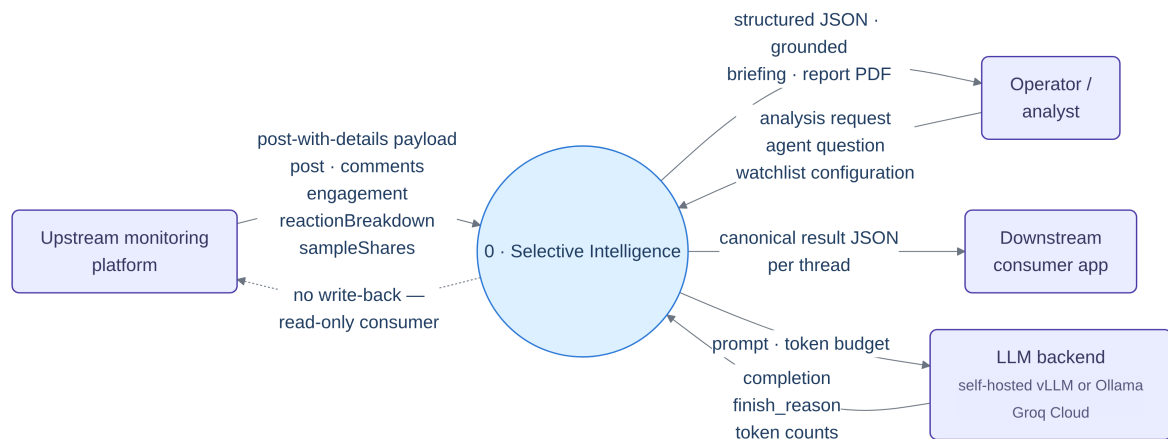


Figure 3.4: Context diagram (DFD level 0).

How to read it. The standard context-diagram form, read outward from the centre. The single blue circle is the entire system, numbered zero because at this level it has no internal structure. The four violet boxes around it are the external entities: the upstream platform on the left, the operator and the downstream consumer application on the right, and the inference backend at the lower right. Every arrow is labelled with the data that crosses that boundary, so the figure is a complete inventory of what enters and leaves the system. The dotted arrow returning to the upstream platform is negative information, drawn so that the absence is explicit rather than merely unmentioned.

Figure note. The struck-through return arrow is the defining constraint of the integration: the system is a read-only consumer of the upstream platform. The exchange with the inference backend is drawn explicitly because the direction of that flow determines the data-residency policy developed in Section 3.2.5.

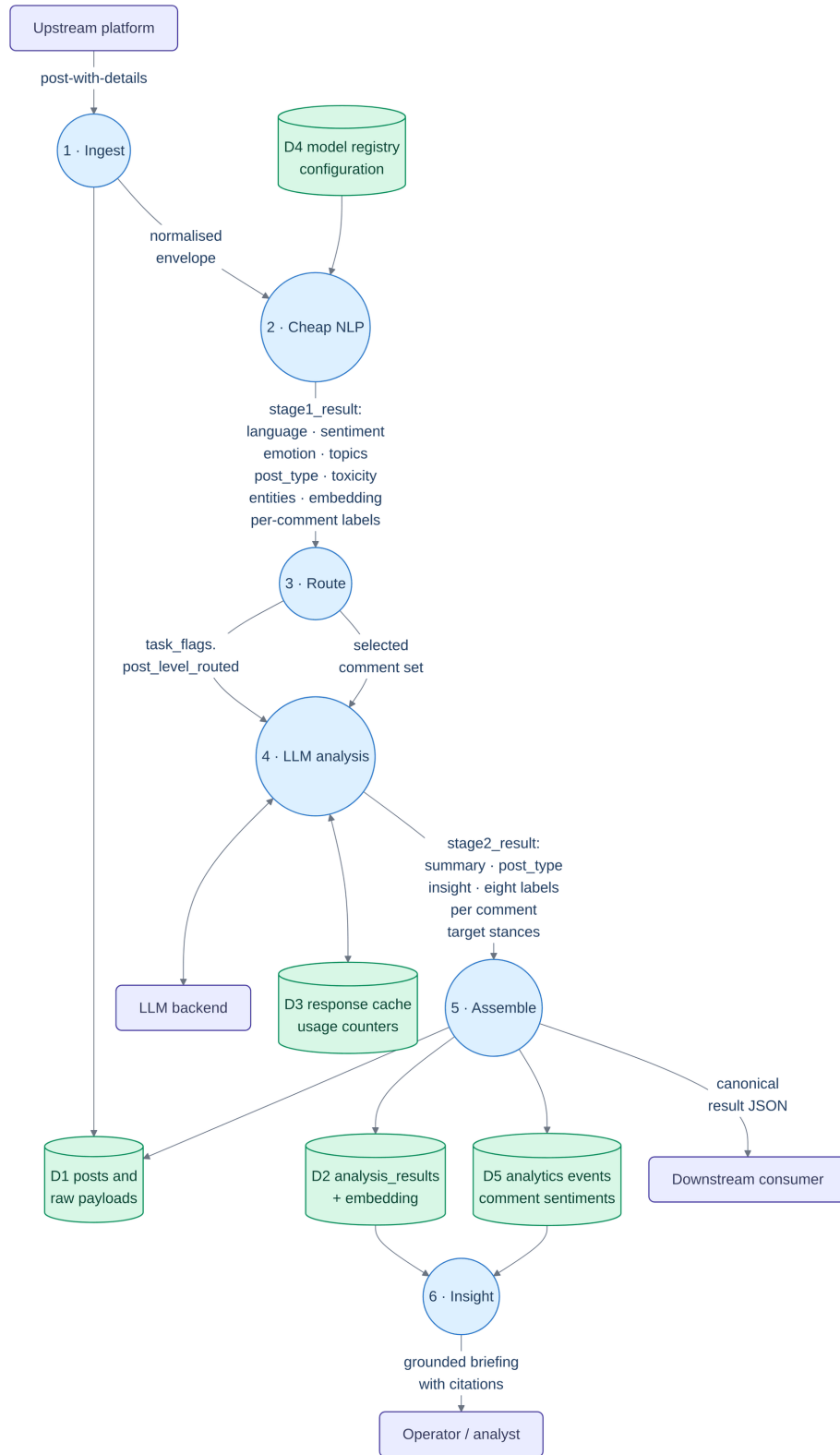


Figure 3.5: Data flow diagram, level 1.

How to read it. The level-0 circle opened into six numbered processes, read down the page. Blue circles are processes, green cylinders are the five numbered data stores, and violet boxes are the same external entities as the level-0 figure. Each arrow between processes is labelled with the record that travels along it, so the two long labels in the middle of the figure are the actual field lists of the stage-one and stage-two results. Two edges are drawn double-headed, from process 4 to the inference backend and to store D3, because those are exchanges rather than one-way flows. Process 6 sits apart at the foot, fed only from the two stores D2

and D5 and never from a process, which is the whole of its relationship to the rest of the system.

Figure note. Processes 1 to 5 constitute the per-post tier and process 6 the corpus tier. The separation is a cost boundary rather than a modularity preference, since invoking process 6 per post would reintroduce the per-item generation cost that the cascade exists to avoid. D3 is both read and written by process 4, which is what makes a repeated analysis free.

3.1.5 UI Design

The operator interface is a single-page React 19 application with eleven tabs, built with Vite and Tailwind, drawing charts with Chart.js and receiving every live surface over Server-Sent Events. Because an event-source connection cannot carry an authorization header, each stream is opened with a **single-use ticket** valid for roughly sixty seconds, so a streaming URL captured in a proxy log is harmless.

The eleven tabs are Overview, Posts, Analysis Jobs, Reports, Search, Agents, Chat, Pipeline, Trace, Warnings and Logs. Figure 3.8 is the entry point and Figures 3.9 to 3.11 are the three tabs that carry most of the design intent; the remaining eight are shown in the walkthrough of Section 4.3.1, and Table 4.12 indexes every one of them.

- **Overview** is the corpus-level summary: stat cards for posts analysed, posts routed to the large model, corpus coverage and token spend; a sentiment doughnut; topic, emotion and language distributions; and a token-economics panel that breaks spend down by lane and projects it to larger corpus sizes.
- **Posts** is the intelligence explorer. A two-tier search filters loaded rows instantly and falls through to a debounced server search; an exact identifier bypasses search entirely and opens the post. Each row shows the comment scrape ratio – analysed, stored and platform-reported, so the sampling is visible in the table itself rather than buried in a detail view.
- **Trace** re-runs a single post through all five stages and renders the stage rail live, including explicit skipped frames when the post-level lane is bypassed and the routing gate's own inputs beside its verdict.

Figure 3.6 gives the component hierarchy that realises those tabs, and Figure 3.7 the streaming topology beneath them. The two are separated because they fail differently: a fault in the component tree produces a broken panel, whereas a fault in the streaming layer produces a panel that renders correctly and stops updating.

Two properties of the component tree are worth reading off Figure 3.6 directly. The first is that the root holds the *only* shared state: the active tab, plus the authentication gate. Every page below it owns its own data and its own fetch lifecycle, so a page that throws while parsing a malformed response removes itself from the display without disturbing its neighbours. The eleven pages are drawn as a single box for exactly this reason – structurally they are interchangeable, and nothing in the tree distinguishes one from another.

The second is the direction of the dotted edges. Solid edges are rendering relationships, so they describe what the browser draws; dotted edges are module imports, so they describe what the code depends on. The two shared components sit in their own panel because each is reached from more than one page, and the hooks-and-utilities panel is reached only by dotted edges, because

nothing in it renders anything. The single hardcoded API base address is called out in the figure because it is the one piece of configuration the front end does not receive at runtime, and therefore the one thing that must change when the dashboard is served from anywhere other than the development host.

Figure 3.7 is drawn separately from the component tree because the two fail in different ways and at different times. A fault in the component tree is a rendering fault, visible immediately and confined to one tab; a fault in the streaming topology is a connection fault, which presents as a surface that simply stops updating while continuing to display its last value – a considerably more dangerous failure for a monitoring product, because stale data and live data look identical. The ticket mechanism shown entering the endpoint panel exists because of a protocol constraint rather than a design preference: an event-source connection cannot carry an authorisation header, so authentication has to move into the query string, and a credential in a query string would be logged by every intermediary. A single-use ticket with a short lifetime is the standard resolution, and it is drawn as entering the panel as a whole because it authenticates all five endpoints identically. The one endpoint feeding two surfaces is the per-analysis stream: job progress and the trace view are two renderings of the same event sequence, so they share one connection rather than opening two.

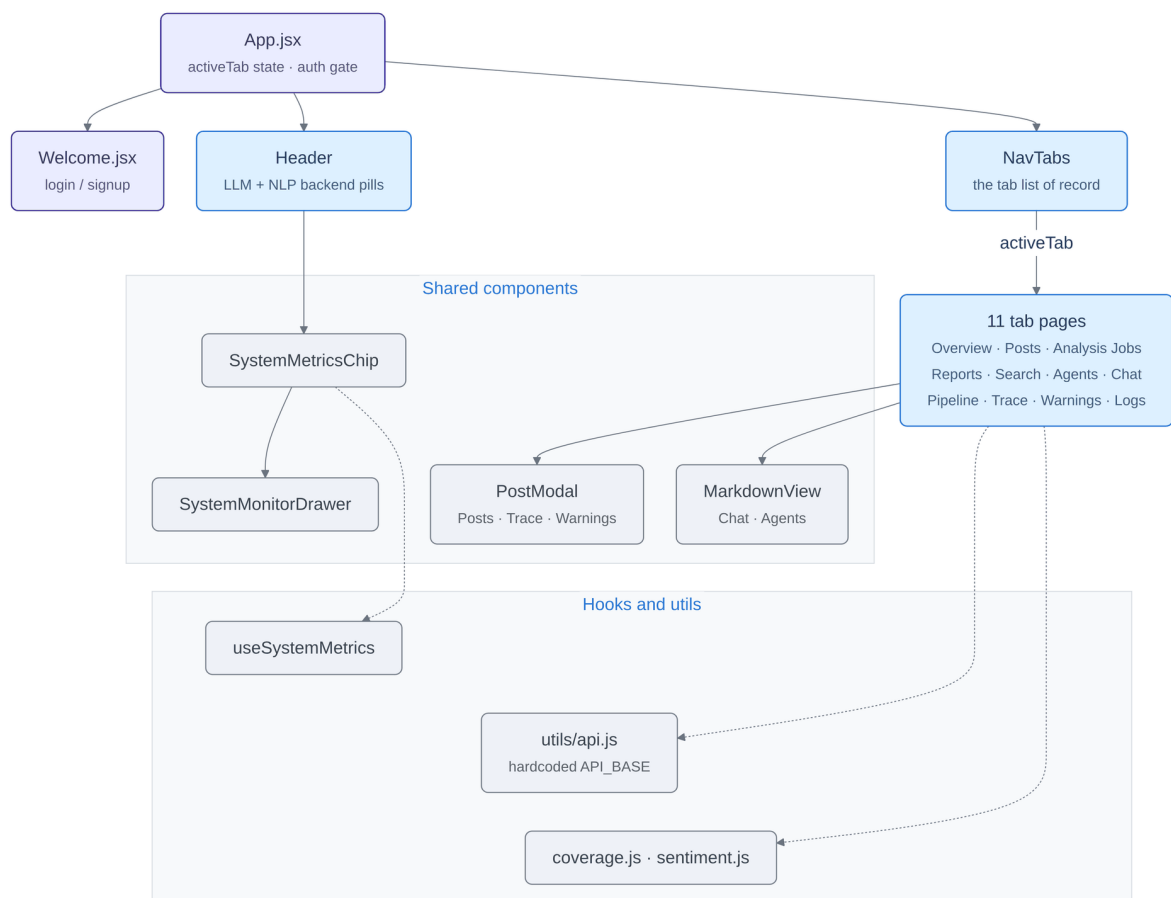


Figure 3.6: Dashboard component hierarchy.

How to read it. A containment tree rather than a flow, read downward from the single root. *App.jsx* holds the only shared state and gates on authentication; the three boxes beneath it are what it renders. *NavTabs* passes the active tab down to the eleven pages, drawn as one grey box because they are structurally

interchangeable. The two titled panels below hold what the pages reach into rather than what they contain: shared components on the left, and hooks and utilities at the foot. Solid arrows are rendering relationships; dotted arrows are calls into a module, which is why the three edges into the hooks-and-utilities panel are dotted.

Figure note. The only shared state is the active-tab value held at the root; every page below it is otherwise independent, so a fault in one tab does not propagate to another. The shared components are grouped separately because each is reached from more than one page.

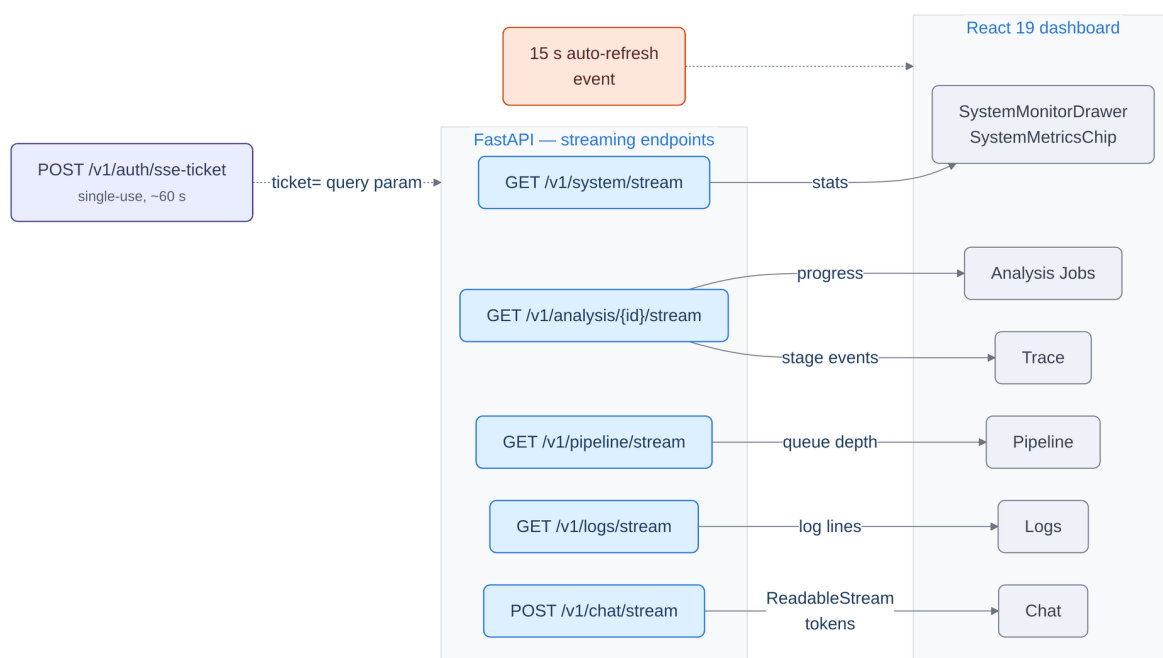


Figure 3.7: Server-Sent Event topology and ticketed authentication.

How to read it. Read left to right in three columns. The violet box at the far left is the ticket endpoint, and its dotted arrow enters the endpoints panel as a whole rather than any single endpoint, because the ticket authenticates all of them. The middle panel lists the five streaming endpoints; the right panel lists the dashboard surfaces that consume them. Each connecting arrow is labelled with what that stream actually carries. The one endpoint with two outgoing arrows is the per-analysis stream, which feeds both the job list and the trace view from a single connection. The orange box at the top is not an endpoint at all: it is the shared timer that refreshes every tab that does not stream.

Figure note. Every stream is authenticated by a short-lived ticket rather than by a credential, because an event-source connection cannot carry an authorisation header. The tabs that do not stream are refreshed by a single shared timer rather than by polling independently.

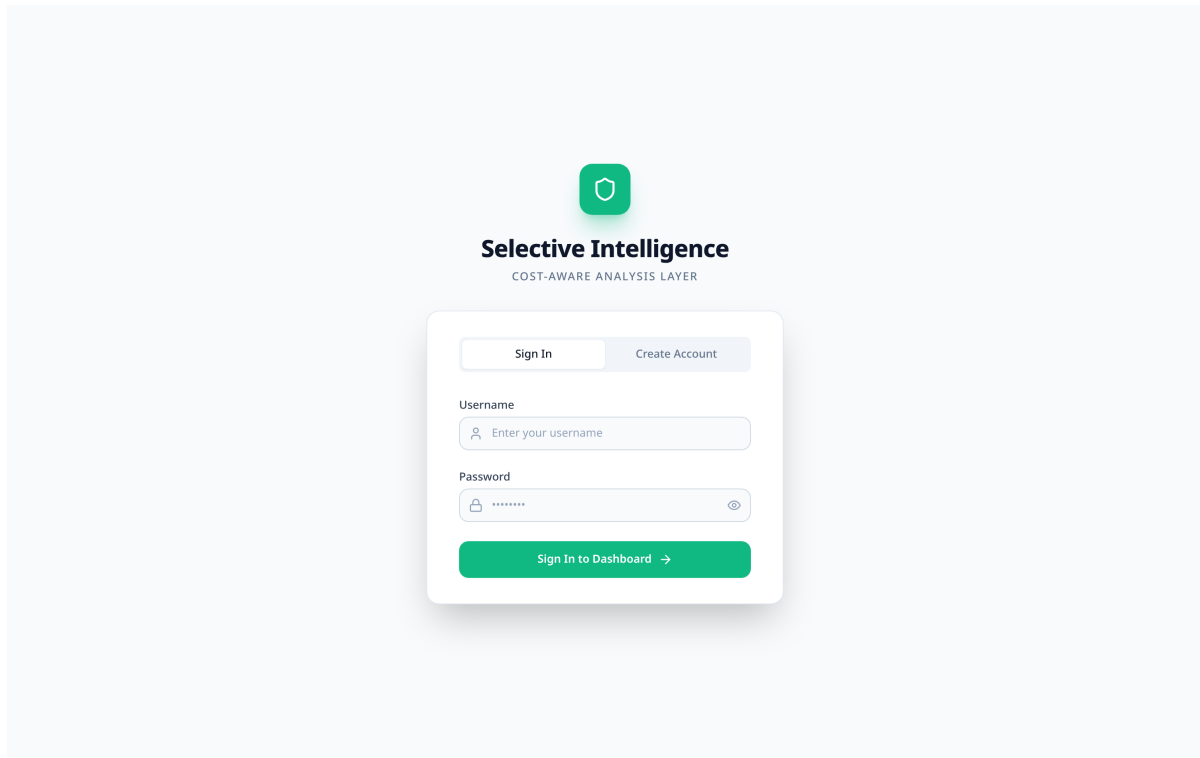


Figure 3.8: The sign-in screen, and the auth policy it renders from.

How to read it. The unauthenticated entry point, and the only screen an unauthenticated caller can reach. A two-tab control selects between signing in and creating an account; the form beneath takes a username and a password and exchanges them for a Bearer token. Whether the second tab appears at all is not hard-coded – the screen reads the API's own auth-policy endpoint before it draws itself.

Figure note. The screen renders the deployment's actual policy rather than a guess at it, so a deployment that refuses registration does not show a registration form whose only outcome is an error. On the development tier shown, an API key is accepted as an alternative credential.

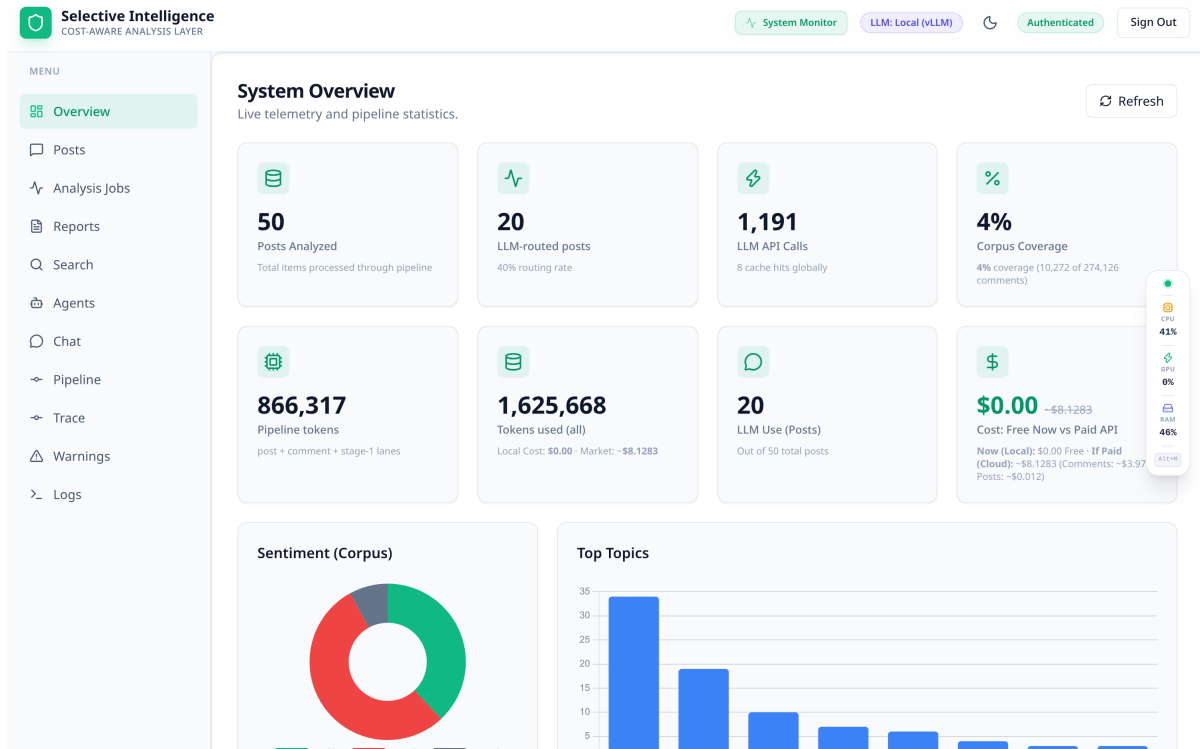


Figure 3.9: Dashboard, Overview tab.

How to read it. The corpus summary, read in three bands. The header carries the live backend pill, the theme control and the authentication state, with the floating processor, graphics and memory strip pinned to the right edge. The first band is eight stat cards: posts analysed, posts routed to post-level LLM work with the routing rate beneath, total large-model calls, corpus coverage, pipeline tokens, total tokens, posts with LLM output, and cost – the last shown as what was actually spent against what the same work would have cost metered. The second band is the charts: the corpus sentiment doughnut, top topics, comment emotions and language distribution. The third band, continued in Figure 4.15, is the token economics panel.

Figure note. Every number on this tab is derived from stored results at read time rather than from a maintained counter, so a tab that disagrees with the database cannot happen. The coverage card states 10,272 comments analysed of 274,126 reported, which is the sampling ratio of Section 4.2.1 kept in front of the operator rather than in a footnote.

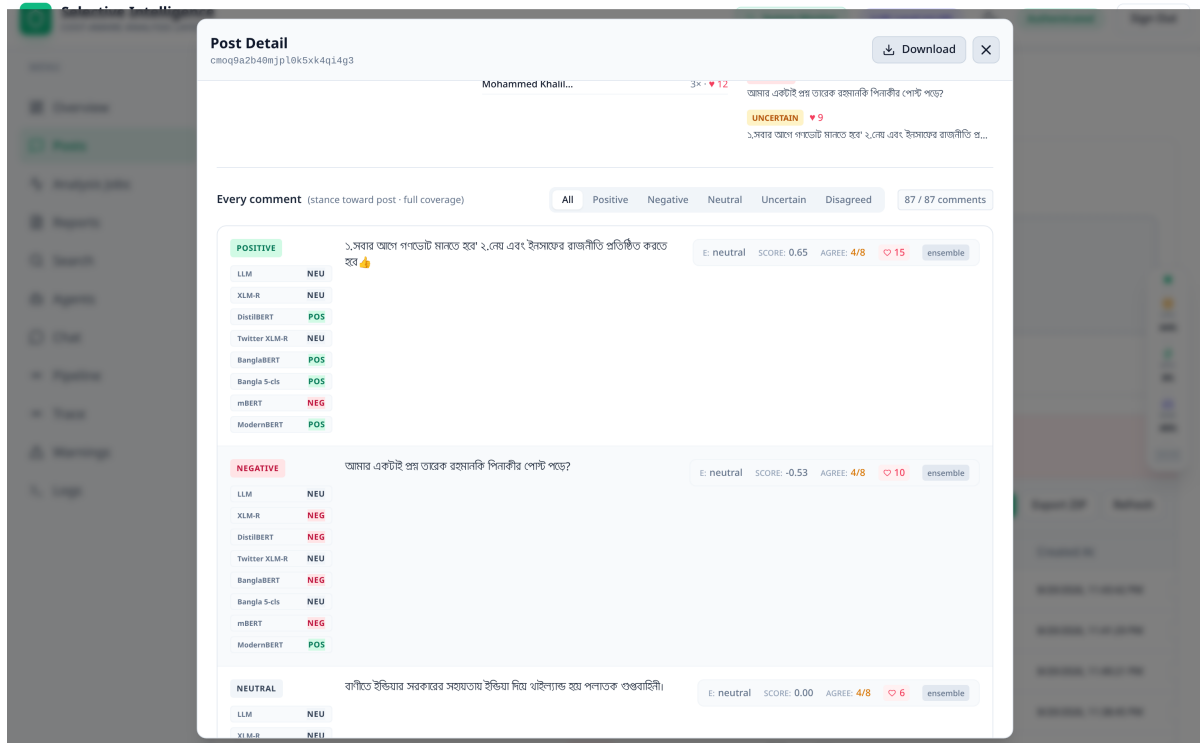


Figure 3.10: Post detail modal: the eight-labeller comment ensemble, one row per comment.

How to read it. The lower half of the post detail modal. Above the table, a filter row selects all comments or only the positive, negative, neutral, uncertain or disagreed ones, with the analysed-of-stored count beside it. Each comment is one block: the combined label at the left, then the eight labellers listed by name with their individual votes – LLM, XLM-R, DistilBERT, Twitter XLM-R, BanglaBERT, Bangla 5-cl, mBERT and ModernBERT – then the comment text, then a strip carrying the dominant emotion, the combined score, the agreement fraction, the reaction count and the source tag.

Figure note. The eight votes are shown side by side rather than fused into one label, so a disagreement is visible as a disagreement. The header of this modal states "stance toward post – full coverage", which names the labelling convention rather than leaving the reader to infer it from the numbers.

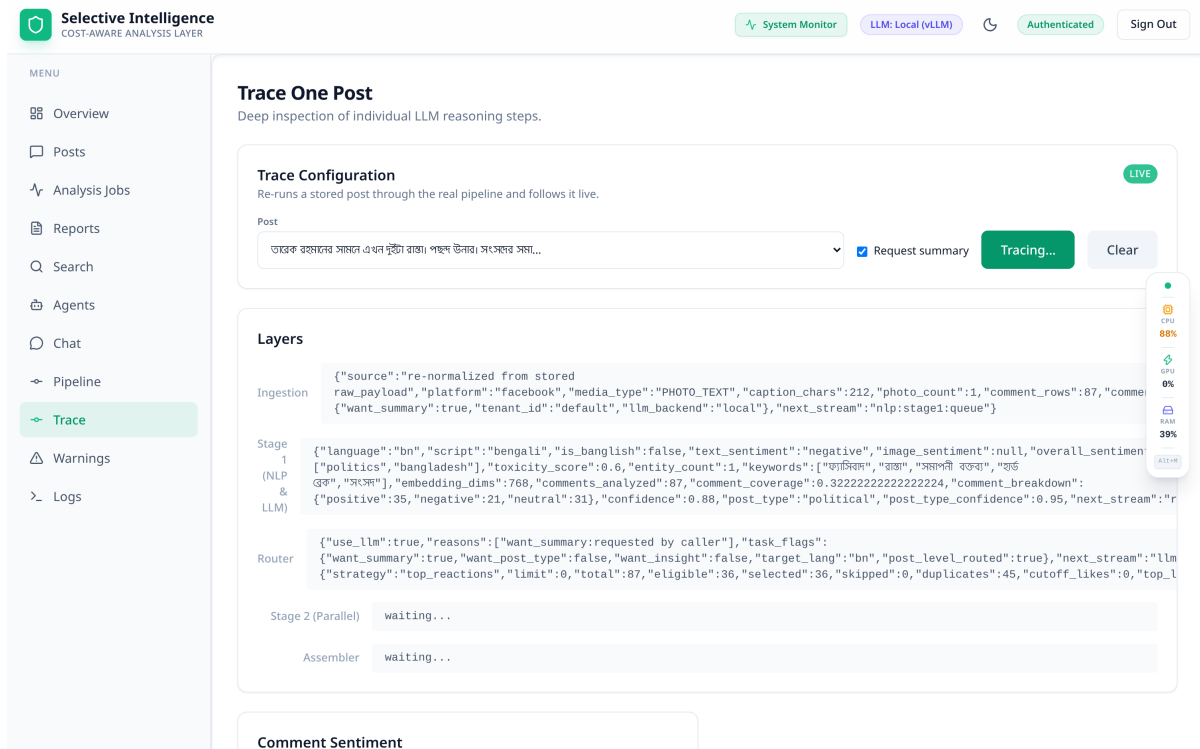


Figure 3.11: Trace tab: one post through all five stages, with each stage's own payload.

How to read it. A completed trace, read downward. The configuration panel selects a stored post, offers the summary toggle and carries the run status – here done. Beneath it, the five layers each hold the payload that stage emitted: ingestion reports the source, platform, caption length and comment counts; Stage 1 reports language, script, sentiment, topics, toxicity, embedding dimensions and the per-comment breakdown; the router reports `use_llm`, the reasons that fired, the task flags and the comment selection arithmetic; Stage 2 reports the models used per role and the summary it produced; and the assembler reports the schema version, the validation result, the top-level key count and the three stores it fanned out to.

Figure note. This is the surface that makes the routing decision inspectable rather than inferable: the router layer carries the reasons that fired, so a reader can see why this post earned post-level LLM work. The run shown selected 36 of 87 comments after removing 45 duplicates, which is the comment-selection step of Section 3.2.2 executing on real data.

3.2 Detailed Methodology and Design

Section 3.1 covered the high-level system design, the requirements, the data flow and the interface layout. This section goes deeper, into the specific technical decisions made in developing the system. For every major architectural decision the alternatives considered are described alongside the justification that led to the final choice. That comparative treatment mirrors what standard engineering practice requires: a component is not adopted because it is the only option in view, but because it scores best against selection criteria – feasibility, cost, scalability, failure behaviour and maintainability – under this project's constraints. Table 3.7 at the end of this section collects thirteen such decisions in one place, and the closing subsection draws out the pattern that runs through them.

3.2.1 The five stages

Table 3.1 states the stage topology precisely. Every stream and consumer-group name in it comes from a single constant module, which exists because four separate defects in this system had the identical shape: one component wrote a string, another read a different string, and the mismatch degraded to a silent no-operation rather than to an error. A test now asserts that the Kubernetes autoscaler manifests name exactly the streams and groups the workers create, so a rename breaks the build instead of quietly disabling an autoscaler.

Table 3.1: Pipeline stages, streams and consumer groups.

#	Stage	Reads stream	Consumer group	Writes to	Principal responsibility
1	Ingestion	ingestion:queue	ingestion-workers	nlp:stage1:queue	Validate against the input schema, normalise Unicode and script tags, compute the content hash, deduplicate, create the job row, enqueue one message per post
2	Stage-1 NLP	nlp:stage1:queue	stage1-nlp-group	router:queue	Cheap analysis of the caption and of every comment; sentence embedding; watchlist string matching; sentiment fusion; a first summary
3	Router	router:queue	router-workers	llm:stage2:queue	Evaluate the six gates; set the post-level task flags; select the comment set every Stage-2 voter will read
4	Stage-2 LLM	llm:stage2:queue	stage2-llm-workers	assembler:queue	Two concurrent lanes: the gated post-level lane and the ungated comment ensemble
5	Assembler	assembler:queue	assembler-group	PostgreSQL, ClickHouse, object storage	Merge both stages, validate against the output schema, fan out to three stores in parallel, reconcile job counters

Four behaviours are implemented once in a shared worker base class and therefore hold at every hop. **Consumer groups are created on start**, tolerating the already-exists condition, so replicas of a stage split the stream rather than duplicating it. **Cancellation is cooperative**: the four stages ahead of the assembler check a per-job Redis flag as they pick a message up and drop it rather than doing the work, which bounds the cost of a stop at one post per stage. The assembler deliberately does not check, because a post that has reached it has already had every model run on it and discarding it there would throw away work already paid for; what the assembler does instead is refuse to write a cancelled job's row back to a running or done status. **Retry is bounded and then dead-letters**, with the attempt counter carried inside the payload rather than keyed by stream identifier, because a re-enqueued message receives a fresh identifier and a per-identifier counter would reset and loop forever. **Progress events** are published as each stage picks a post up and hands it on; because publish-subscribe has no backlog and the first event is emitted during the request that creates the job, every event is also appended to a capped list and

replayed when a client connects, each event carrying a monotonic sequence number so a duplicate can be dropped.

3.2.2 The router: two independent decisions

The router is the component the entire cost argument rests on, and it makes **two independent decisions**. The first is whether the post earns post-level large-model work; the second is which comments the Stage-2 labellers will read. Conflating them was the original design's mistake, and separating them is what makes the cost behaviour measurable.

The first decision is taken by six gates, listed in Table 3.2. Any one gate firing is sufficient to route, and every reason is recorded as a human-readable string on the decision so the routing rate can be decomposed by cause rather than merely counted.

Table 3.2: The six routing gates.

#	Gate	Fires when	Threshold knob (default)
1	Low or absent confidence	Stage-1 overall confidence is below the threshold, or is absent entirely	ROUTER_CONFIDENCE_THRESHOLD (0.8)
2	Post type unknown or weak	The semantic post type is unset, or its confidence is below the threshold	ROUTER_POST_TYPE_CONFIDENCE_THRESHOLD (0.8)
3	Caller asked for a Stage-2 summary	The request explicitly asks for the better summary model	ROUTER_SUMMARY_ROUTES (false)
4	Image post with no image verdict	The post carries photographs but the vision stage produced no sentiment	not applicable
5	High toxicity	The Stage-1 toxicity score exceeds the threshold	ROUTER_TOXICITY_THRESHOLD (0.7)
6	Long code-mixed text	The caption exceeds the character limit and the script is Banglish or mixed	ROUTER_LONG_TEXT_CHARS (1000)

Two decisions inside that table deserve to be stated outright, because both were learned the hard way.

An absent confidence routes. Gate 1 treats a missing value as uncertainty rather than ignoring it. That is not a stylistic choice: in an earlier revision the gate read a field name that Stage 1 never emitted, the guard therefore short-circuited on every post, and a post carrying a confidence of exactly zero passed a threshold of 0.65. The gate was inert and the system routed one hundred percent of posts while every document in the project claimed a single-digit routing rate. The repair was not only to correct the field name but to make **every gate read through a named reader function** that accepts all three field shapes the pipeline emits, and to make the diagnostic log go through those same readers, so a log line can never report a value the gate did not actually see. A rename now fails a test.

Gate 3 is disabled by default, and that default is what makes the gate selective. It once defaulted to enabled, which, because any single gate firing is sufficient, made the router a pass-through and pinned the reported large-model share at exactly one. The product requirement it served, that every post receives a summary, is still met, but by Stage 1, which writes one and

hands it to the assembler; Stage 2 re-summarises only when a caller explicitly asks for the stronger model.

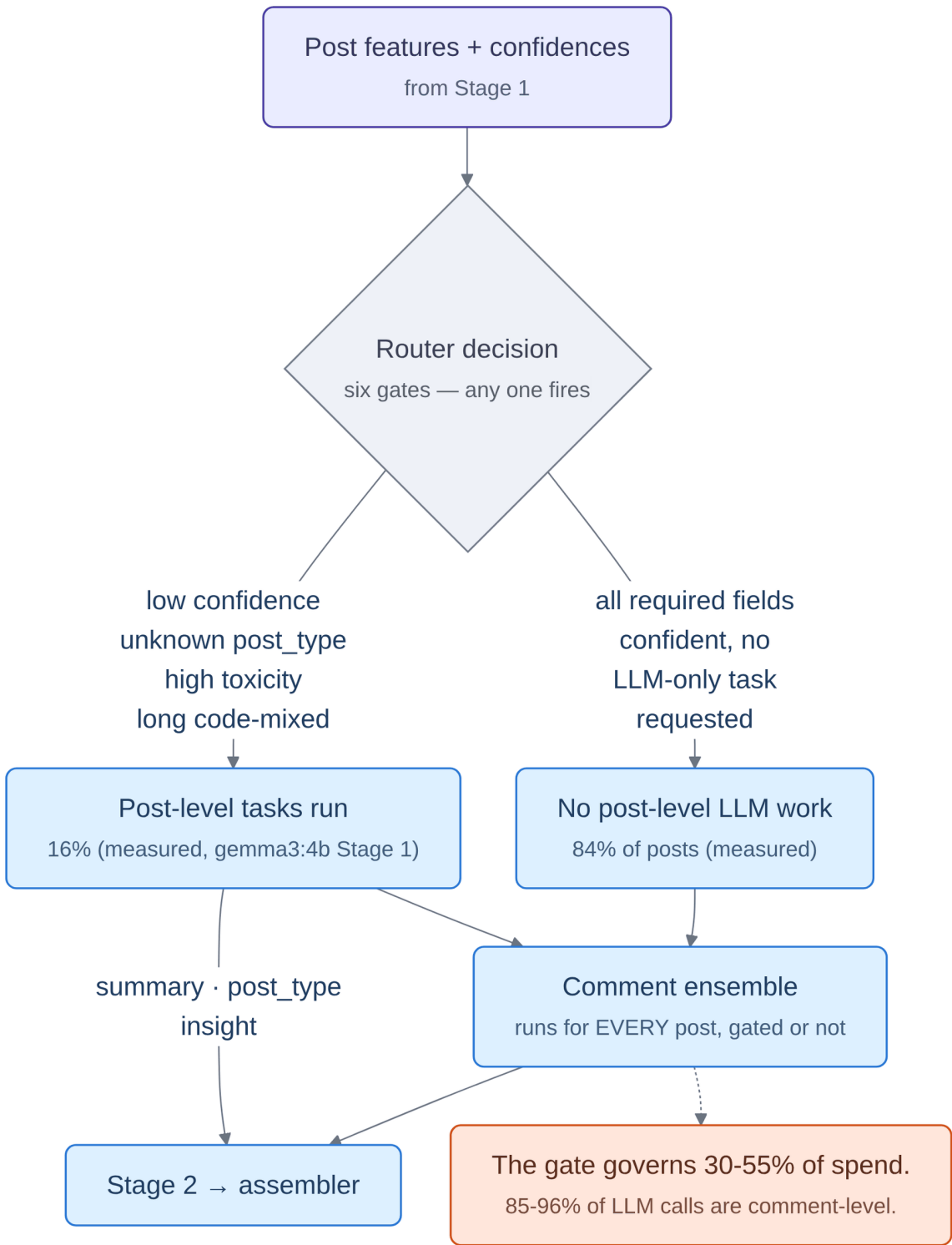


Figure 3.12: Router decision flow: the post-level gate and the separate comment selection.

How to read it. One diamond with two outgoing edges, and the point of the figure is what happens after they rejoin. The grey diamond is the gate; the two labelled edges leaving it are the conditions, not the outcomes. The left edge lists the four circumstances that force post-level work; the right edge is the negative case, everything confident and nothing explicitly requested. The two blue boxes below carry the measured share of

posts taking each branch, sixteen and eighty-four percent. Both then point into the same box: the comment ensemble, which is drawn on the joined path precisely because it is unconditional. The second arrow from the routed branch straight to the assembler carries the three post-level products that only that branch produces. The orange callout is attached to the ensemble rather than to the gate, which is the figure's argument in one edge.

Figure note. The two branches differ only in whether post-level work is performed; both converge on the comment ensemble, which runs unconditionally. That convergence is the structural reason why the measured routing rate governs a minority of total spend, as Section 4.2.4 quantifies.

The second decision – comment selection – runs in three steps shown on the right of Figure 3.12. **Eligibility** removes kinds that have no text for a model to read: emoji-only comments, link-only comments, and anything below a configurable word-token floor. **Deduplication** sorts the remainder by reaction count, breaking ties by original index so the choice is reproducible, and collapses identical normalised texts to their most-liked occurrence; on threads in which the same short congratulatory phrase recurs dozens of times, this yields a substantial reduction in cost at no loss of coverage. **Top-N** keeps that many survivors. The cap defaults to zero, admitting every unique comment with text, and the selection is recorded per comment and summarised per post regardless.

Crucially, the selection is made **once**, and every Stage-2 voter reads that same set. An earlier design placed the cap inside the language-model stance pass alone, which recovered the token cost but still ran seven classifier models over the entire thread, and left the language-model column of the per-comment comparison empty on precisely the comments every cheap head had voted on.

3.2.3 The eight-labeller comment ensemble

Comment sentiment in this system is not a model; it is an ensemble with an explicit abstention protocol. Every comment in the router's selected set collects up to **eight independent verdicts**: seven small sentiment heads batched on CPU, and one context-aware stance pass on the large model – the only labeller that sees the parent post and therefore the only one judging stance *toward* it rather than the comment's own tone. Table 3.3 gives the roster.

Table 3.3: The eight-labeller comment ensemble.

Slot	Voter	Checkpoint family	Character
1	xlmr	multilingual sentiment, distilled	five-class multilingual head, batched on CPU
2	distilbert	multilingual DistilBERT student	three-class student model
3	twitter_xlmr	XLM-RoBERTa trained on social media	social-register multilingual model
4	banglabert	BanglaBERT fine-tuned on SentNoB	Bangla social text, trained on noisy comments
5	bengali_sentiment_bert	BanglaBERT, five-class	second Bangla-specific view
6	mbert	multilingual BERT, five-star scale	the oldest and most differently-trained head
7	modernbert	ModernBERT multilingual	recent long-context encoder, a different recipe
8	llm	the Stage-2 model role	context-aware stance, one call per batch of comments

Four design decisions embodied in that table require justification.

Only a model may label a comment. Stage 1 also produces a cheap emoji-and-lexicon verdict for every comment, and it was originally a ninth voter. It was removed, because it answers on *every* comment, and measurement showed that 71.3 percent of those answers were either the emoji rule or the deterministic stub, which derives a label from a hash of the text and is reproducible without being sentiment. A voter that can never abstain makes every agreement figure unfalsifiable, and on Bangla threads that rule was frequently the *majority* voter. Its verdict is still disclosed in the per-comment provenance field; it is never counted as evidence.

A voter that cannot vote abstains. A head that fails to load, or that returns a label outside the taxonomy, contributes nothing – never a fabricated neutral. The output records how many sources actually spoke, who they were, and which of them stood behind the winning label. A comment that no model read is reported as **uncertain**, not neutral: neutral is a claim about the comment, whereas uncertain is a claim about the system's confidence. Without this distinction, an agreement figure of 1.0 rendered in the interface as "100 percent agreement" for a comment exactly one model had read. A declared-but-silent head is logged at warning level, because five of the roster's original entries were in fact base encoders or were absent from the model hub and had voted zero times while reading as coverage.

Seven heads are not seven independent readings. Five of the seven are multilingual encoders trained on overlapping data, so their agreement is correlated and a seven-to-zero vote is weaker evidence than seven unrelated models would be [50]. This is stated here because it must be stated wherever the agreement number is quoted.

One combiner is the single writer of a comment's final label. Three separate places used to write that field and the last one won, which is how a post could report eight negative comments while every comment in the list rendered as neutral.

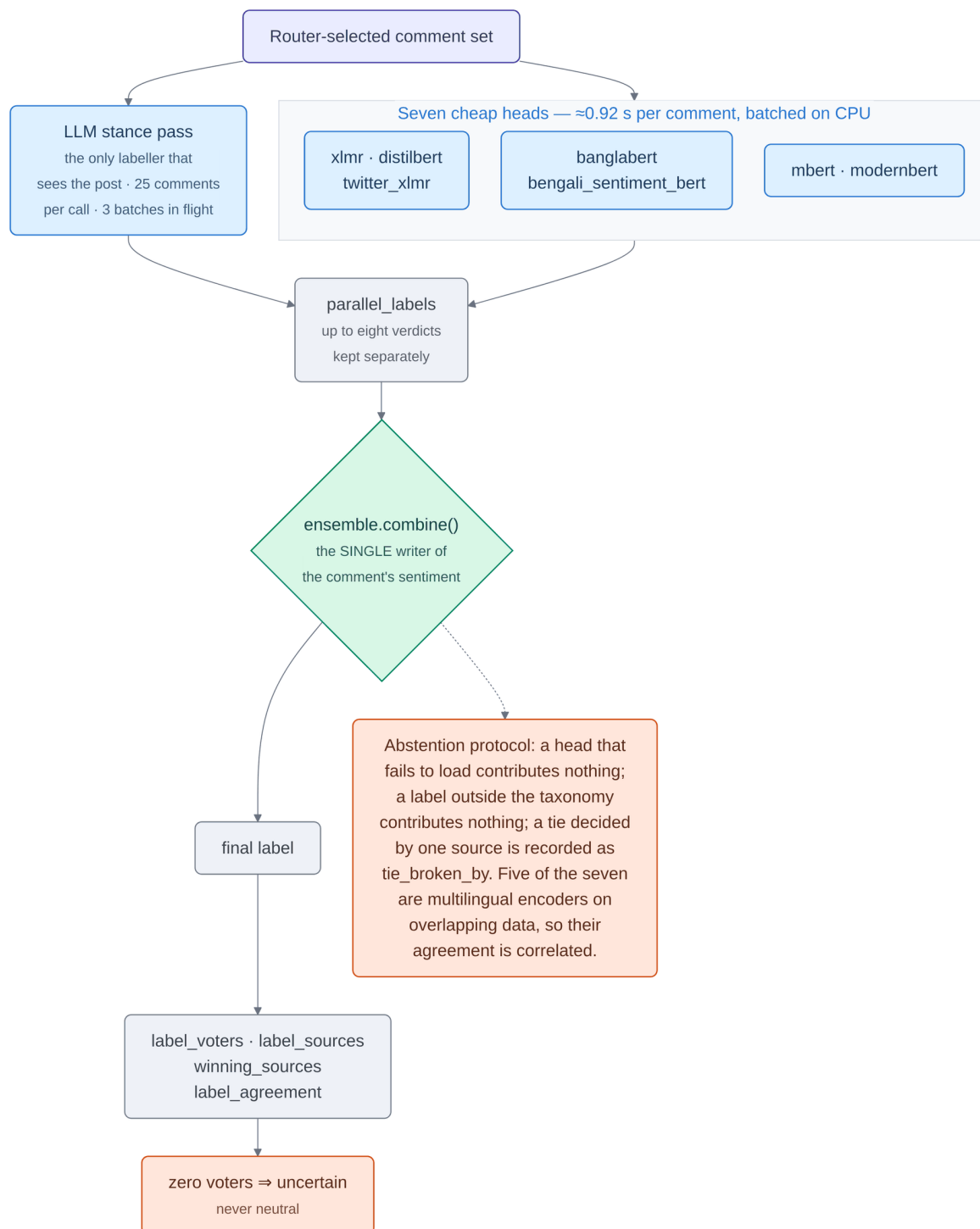


Figure 3.13: The eight-labeller comment ensemble and its abstention protocol.

How to read it. Work flows down the page from one input to three outputs. The router-selected comment set at the top forks into two paths that run concurrently: the titled panel holding the seven small heads, grouped three-two-two purely for layout, and the single blue box beside it for the LLM pass. Its caption records the one asymmetry that matters, that it is the only labeller given the post as context. Both paths converge on `parallel_labels`, which is drawn as its own node because the verdicts are kept separately rather than averaged on arrival. The green diamond below it is the combination step. Three boxes descend from it in sequence: the label itself, the provenance fields recorded alongside it, and the abstention outcome. The orange callout states the rules the diamond applies, including the correlation caveat that keeps eight voters from being read as eight independent opinions.

Figure note. The combination step is drawn as a single node because it is the single writer of the final label; three components previously wrote that field, and the last write prevailed. Zero voters resolves to uncertain rather than to neutral, which separates a statement about the comment from a statement about the system's confidence.

3.2.4 Watchlist-driven target stance

Document-level sentiment answers whether a comment is angry. For political and brand monitoring the operative question is **whom it is angry at**, and answering it requires a target-dependent formulation [13][14][16]. This system implements that over an operator-supplied, versioned watchlist rather than a fixed annotated target set.

The alias matcher is the load-bearing part, not the classifier. In this corpus a single entity appears in Bangla script, in three to six inconsistent romanizations, in English, as a party initialism, and sometimes as an epithet that carries the stance by itself. A watchlist that matches one spelling matches almost nothing, and it fails *quietly*, producing zero mentions, a result indistinguishable from an entity that was never mentioned. Five specific design consequences follow, and each of them was reached by a test failing rather than by reading code.

- Match case-insensitively for Latin and exactly for Bangla. Bangla has no case, so lowercasing is a no-operation there and only introduces surprises.
- Require token boundaries for short aliases. A two- or three-character initialism otherwise matches inside unrelated words.
- Bangla word boundaries are not spaces. Bengali attaches suffixes directly, so an entity plus its suffix is one token; a suffix after a Bangla alias is permitted, a prefix is not.
- Collapse overlapping aliases per target, keeping the longest. Where a party name and its short form both fire on the same words, every polarity cue in the surrounding clause was being counted twice.
- Attach cues by **clause**, not by a fixed character window. With a sixty-character window either side, a comment of the form "B is the best but A is corrupt" scored both entities identically, because the window spanned the whole comment. Clause segmentation is both the correct treatment and the more interpretable one, because each verdict can be attributed to the clause that produced it.

Two further properties matter for the cost model and for the epistemics. The matcher is **pure string work and therefore free**, so it runs in Stage 1 on every comment of every post, which means mention volume is available even for posts the gate bypassed. Furthermore, the language-model scoring is performed inside the Stage-2 comment-stance call that is already being issued: the matched entities are injected into a prompt that would have been sent regardless. The capability therefore adds **no additional large-model calls**.

The watchlist is a stated bias model, not a measurement. A file that declares "support for X is positive" encodes a political stance into the labels. That is entirely legitimate for a monitoring product whose operator wants to know who is attacking their client; it is indefensible if presented as neutral sentiment analysis. The file is therefore versioned with a named owner per entry, the repository distributes only a placeholder so that it states no politics of its own, and a neutral bucket exists for operators who want target-dependent stance without the editorial framing. Most importantly, target stance occupies a **separate output field** from document sentiment and the two are never summed, averaged or charted together. A comment can be positive in tone while opposing a listed entity, and conflating those two facts would destroy the only distinction the feature exists to make.

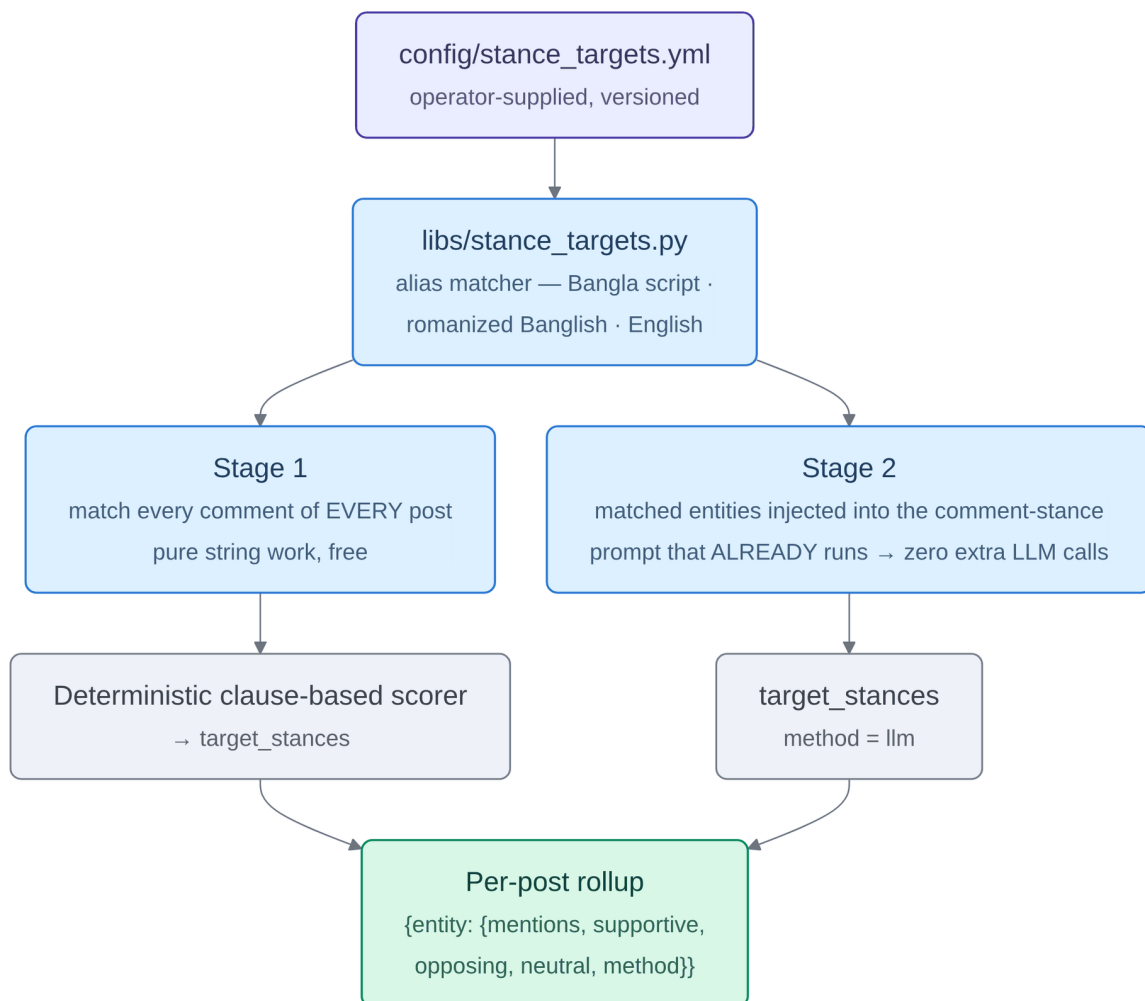


Figure 3.14: Watchlist-driven target stance: matching, scoring and roll-up.

How to read it. A configuration file at the top, a matcher beneath it, and then two paths that never merge until the last node. The violet box is the operator-supplied watchlist; the blue box below it is the alias matcher, captioned with the three scripts it must reconcile. From the matcher the figure forks: the left path runs in Stage 1 over every comment of every post and feeds a deterministic clause-based scorer, and the right path runs in Stage 2 and reaches the same output field through an LLM. The captions on those two boxes carry the cost argument: the left path is pure string work and therefore free, and the right path adds no call of its own because the matched entities are injected into a prompt already being issued. Both arrive at the green roll-up box at the foot, whose label is the literal shape of the emitted record, including the method field that records which of the two paths produced it.

***Figure note.** Matching runs in Stage 1 because it is string processing and therefore free, which makes mention volume available even for posts the gate bypasses. The Stage-2 path adds no inference call of its own: the matched entities are injected into a prompt that is already being issued.*

3.2.5 Pluggable inference backend and the role system

Every language-model call in the system passes through one client, and that client resolves two properties independently: a **role**, which is what the call is for, and a **backend**, which is who serves it. Table 3.4 gives the mapping.

Table 3.4: Roles, and the model each resolves to per backend.

Role	Used by	What it needs	Self-hosted default	Cloud default
stage1	Stage-1 NLP over the caption and comments	one small, fast model carrying high-volume work	gemma3:4b	llama-3.1-8b-instant
stage2	Post type, insight, comment stance	cheap and constrained; picks from a fixed vocabulary	qwen2.5:7b	llama-3.3-70b-versatile
summary	Post summary and comment-thread summary	fluent; writes Bangla prose	qwen2.5:7b	llama-3.3-70b-versatile
agent	The nine insight agents	long context and native tool calling	llama3.1:8b with a 16k context tag	llama-3.3-70b-versatile
llm_a	Legacy fast lane used by the report path	throughput	qwen2.5:7b	llama-3.1-8b-instant
llm_b	Grounded reports and cluster summaries	quality on low-volume work	llama3.1:8b	llama-3.3-70b-versatile
vlm	Image-grounded summaries	vision	qwen3-vl:4b	a hosted vision model identifier

Splitting **summary** from **stage2** is the decision in that table with the clearest return. The two Stage-2 jobs have opposite profiles: classification chooses from a fixed vocabulary and wants a cheap, constrained model, while summarisation writes prose and wants a fluent one. Keeping them on one role forced a single compromise and, worse, spent the fluent model on every classification call – which, because comment stance runs once per batch of twenty-five comments, multiplied straight through the comment lane. Separating them spends the expensive model exactly once per post.

Splitting the role from the backend is what allows self-hosted and cloud serving to be interchangeable. Both speak the same OpenAI-compatible interface, so switching is a configuration change resolved per call in the order request, then a runtime toggle, then the environment default. The self-hosted path has **zero marginal token cost and no data egress**; the cloud path has the fastest inference and no GPU to operate, at a per-token price and with prompt content leaving the cluster. Because that last property is a privacy decision and not merely a performance one, a **privacy-locked tenant is pinned to the self-hosted backend on every path**: the decision is resolved at the API boundary where the tenant is known, stamped into the job

envelope so a global toggle cannot re-route content that is already in flight, re-resolved on resume in case the policy changed in the meantime, and applied to chat and agent calls under the same rule. An explicit request for the cloud backend from a locked tenant is refused; a global operator toggle is silently downgraded instead, because an operator's switch is not that tenant's choice. The check **fails closed**: an unreadable policy table denies rather than permits, since a privacy guarantee that lapses when the database becomes unavailable is not a guarantee.

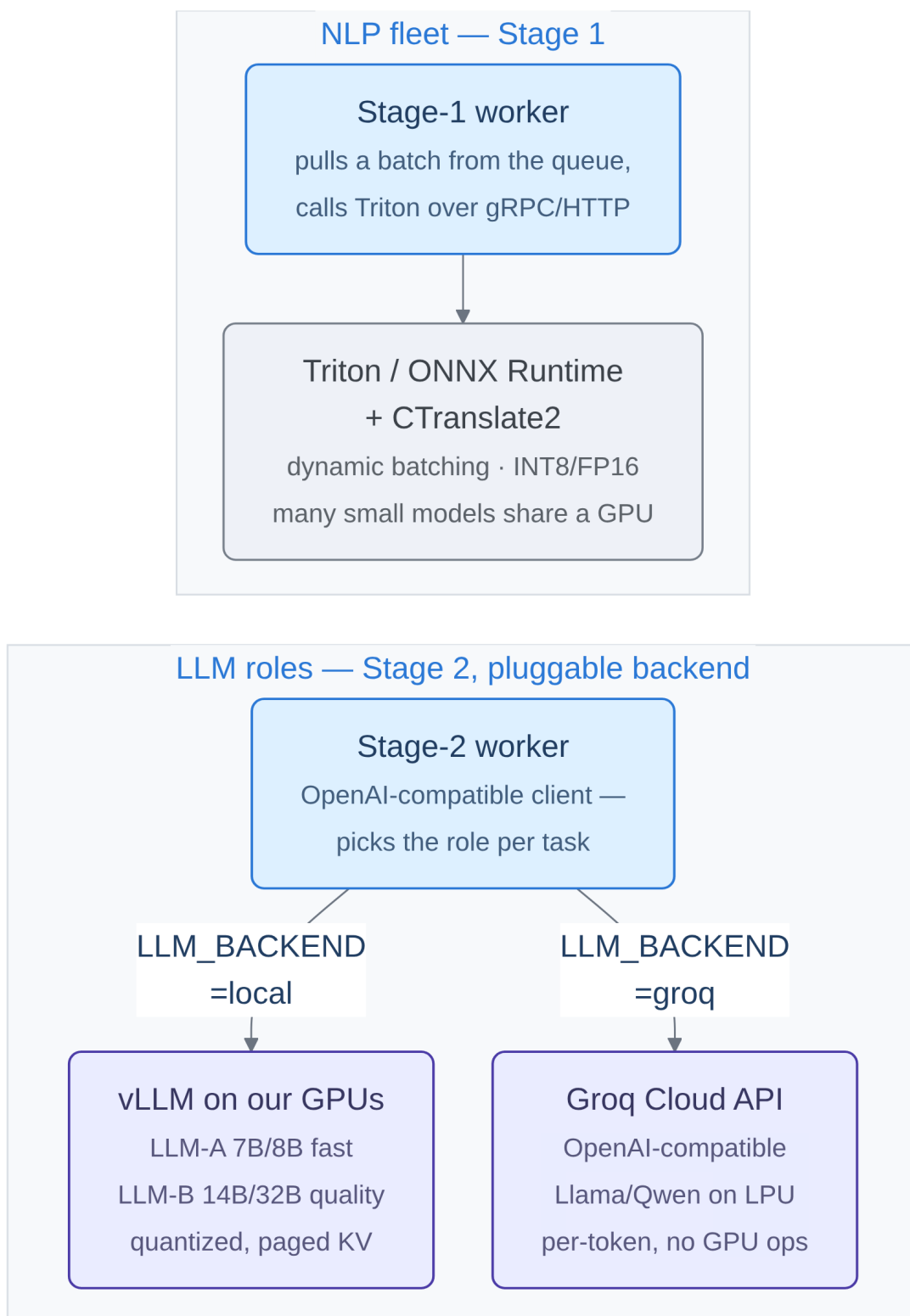


Figure 3.15: Model-serving architecture for the two stages.

How to read it. Two independent panels stacked vertically, with no edge between them, because they are two separate fleets rather than two steps of one flow. The upper panel is Stage 1: a worker that pulls a batch and calls an inference server that holds many small models on shared hardware with dynamic batching and integer quantisation. The lower panel is Stage 2: one worker speaking an OpenAI-compatible protocol, with two mutually exclusive outgoing edges. The edge labels are the literal environment-variable values that select between them, and the two violet boxes they lead to are the alternatives, self-hosted weights on the left

and a metered API on the right. That both Stage-2 targets are violet and both Stage-1 components are not is the figure's substantive point about which parts of the system are outside the operator's machine.

Figure note. The two stages are served by different mechanisms for the same reason that they use different models: Stage 1 is high-volume and low-difficulty, Stage 2 low-volume and high-difficulty. Only the Stage-2 roles have a metered alternative; the small-model fleet is self-hosted in every configuration.

Two further mechanisms in the same client warrant description. A **circuit breaker** per backend takes a repeatedly failing endpoint out of rotation for a cooldown period rather than continuing to issue requests to it, with an injectable clock so the state machine is unit-testable without sleeping. A **truncation-recovery** path additionally inspects the finish reason, auto-continues a reply that hit its token ceiling, trims back to the last complete sentence – including the Bangla sentence terminator – flags the result, and **never caches it**. That path exists because the ceilings were once fixed literals with nothing checking whether a reply had actually stopped at them, and Bangla costs far more tokens per character than English under these tokenisers, so a single fixed ceiling truncated Bangla and Banglish while sparing English. For a system whose primary languages are Bangla and Banglish, that bias is exactly inverted.

3.2.6 The agentic RAG insight layer

The per-post pipeline is deterministic analysis followed by single-shot generation, and it remains so, because that is what the cost model requires. Above it sits a **corpus-tier agent layer** for work that genuinely needs planning and several tool calls: analyst questions, grounded reports, and targeted deep-dive analyses. **Agents are never invoked per post**, which would reintroduce exactly the per-item large-model cost the cascade exists to avoid.

What distinguishes this layer from an assembly of established components is the **object of retrieval**. A conventional retrieval-augmented system retrieves documents – the text a user could have read anyway [34][35]. These agents retrieve the system's **own structured analytical output**: sentiment distributions, stance verdicts by entity, ensemble-agreement statistics, coverage and vector provenance, toxicity scores, reaction mixes and embedding clusters. That output exists only because the pipeline ran, which makes the retrieval analytical rather than documentary, and it has a concrete consequence: an agent cites a number, and that number can be traced back through the ensemble to the model that produced it.

Figure 3.16 shows a single tool invocation crossing that boundary, including the branch that matters most for answer quality: an invalid or placeholder identifier is refused rather than answered with an empty result set, because an empty result set is indistinguishable to the model from a corpus that contains nothing.

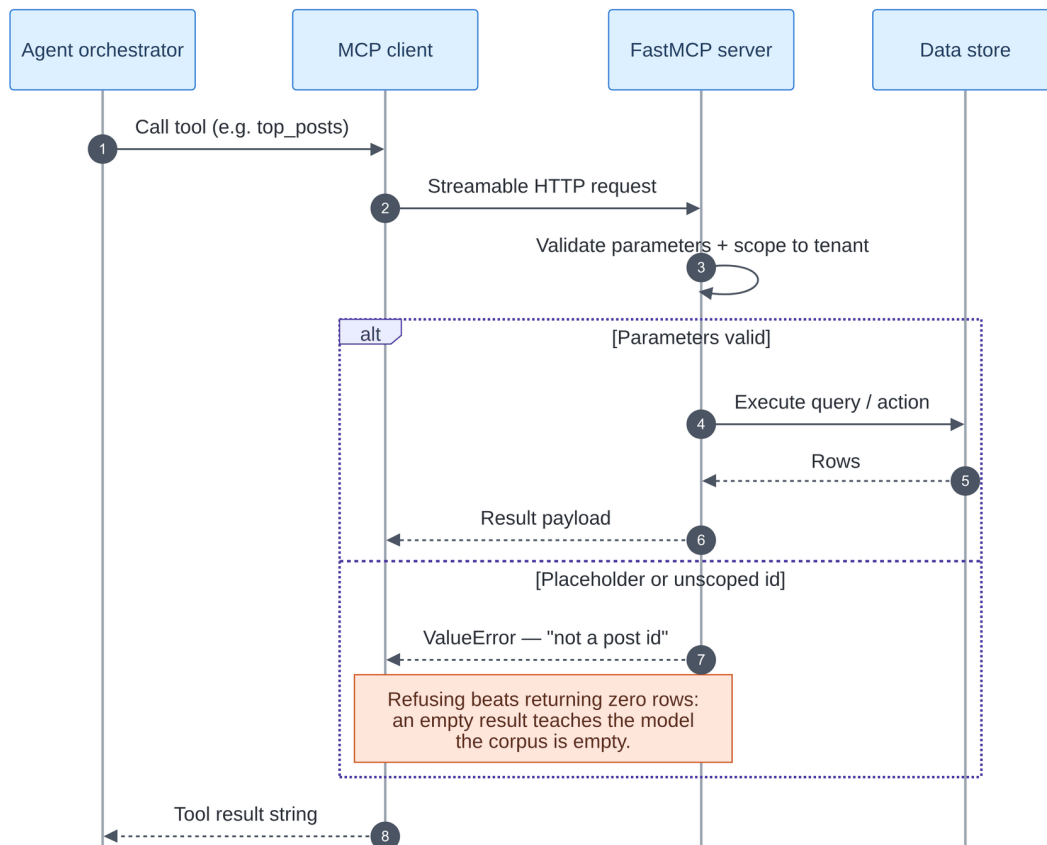


Figure 3.16: Tool invocation across the MCP boundary, including the refusal path.

How to read it. A sequence diagram, read down the page as time. The four vertical lifelines are the agent orchestrator, the MCP client, the tool server and the data store; each numbered arrow is one message, and a dashed arrow is a return rather than a call. The self-directed arrow at step three is validation, which happens before anything reaches the store. The tall bracketed frame below it is an alternative: the upper compartment is the path taken when parameters are valid, ending with rows returning through the server to the client, and the lower compartment is the path taken when they are not. Note that in the lower compartment no arrow ever reaches the data store, and the orange note explains why that refusal is preferable to a query that would have returned nothing.

Figure note. The lower branch of the alternative frame is the one that governs answer quality. An invalid identifier is refused rather than answered with an empty result set, because an empty result set is indistinguishable, from the model's position, from a corpus that contains nothing.

Table 3.5: The nine agents, their tool budgets and their purpose.

Agent	Budget	Purpose
analyst	10	General corpus question answering: sentiment trends, activity levels, most-discussed posts, reaction mix. The default when a question matches no specialist.
coverage	5	Find posts whose comment coverage is low against high engagement, and trigger a deeper comment pull.
alerting	8	Scheduled threshold monitoring. Checks negative-sentiment share and average toxicity separately, because the two thresholds are served by different tools.
stance	12	Target-dependent stance across the operator watchlist; the system's most novel signal, made interactive.
comparator	15	Cross-campaign and cross-period comparison, retrieving both sides and computing deltas.
toxicity	14	Harm and harassment patterns, searching comments directly rather than walking from posts.
narrative	12	Theme and narrative discovery over embedding clusters, with an explicit check on whether the vectors are real.
quality	8	Coverage, ensemble agreement and vector provenance: the agent that addresses questions of data reliability.
reporter	15	Drafts a structured grounded report end to end, in six prescribed sections.

The agents reach data only through **Model Context Protocol servers** [38] – small services exposing typed tools over a standard transport, so that adding an analytical capability is a tool registration plus one line in an agent's allowlist rather than a change to the agent loop. Table 3.6 gives the inventory.

Table 3.6: The three MCP servers and their eighteen tools.

Server	Backing store	Tools
analytics-mcp	ClickHouse and PostgreSQL	trend_query, sentiment_over_time, top_posts, reaction_mix, watchlist_timeline, agreement_stats
retrieval-mcp	PostgreSQL with pgvector	semantic_search, search_comments, get_post, get_thread, representative_comments, get_clusters, stance_by_target, stance_over_time, coverage_stats
ingest-mcp	Redis streams into the upstream pull	pull_campaign, fetch_more_comments, refresh_post

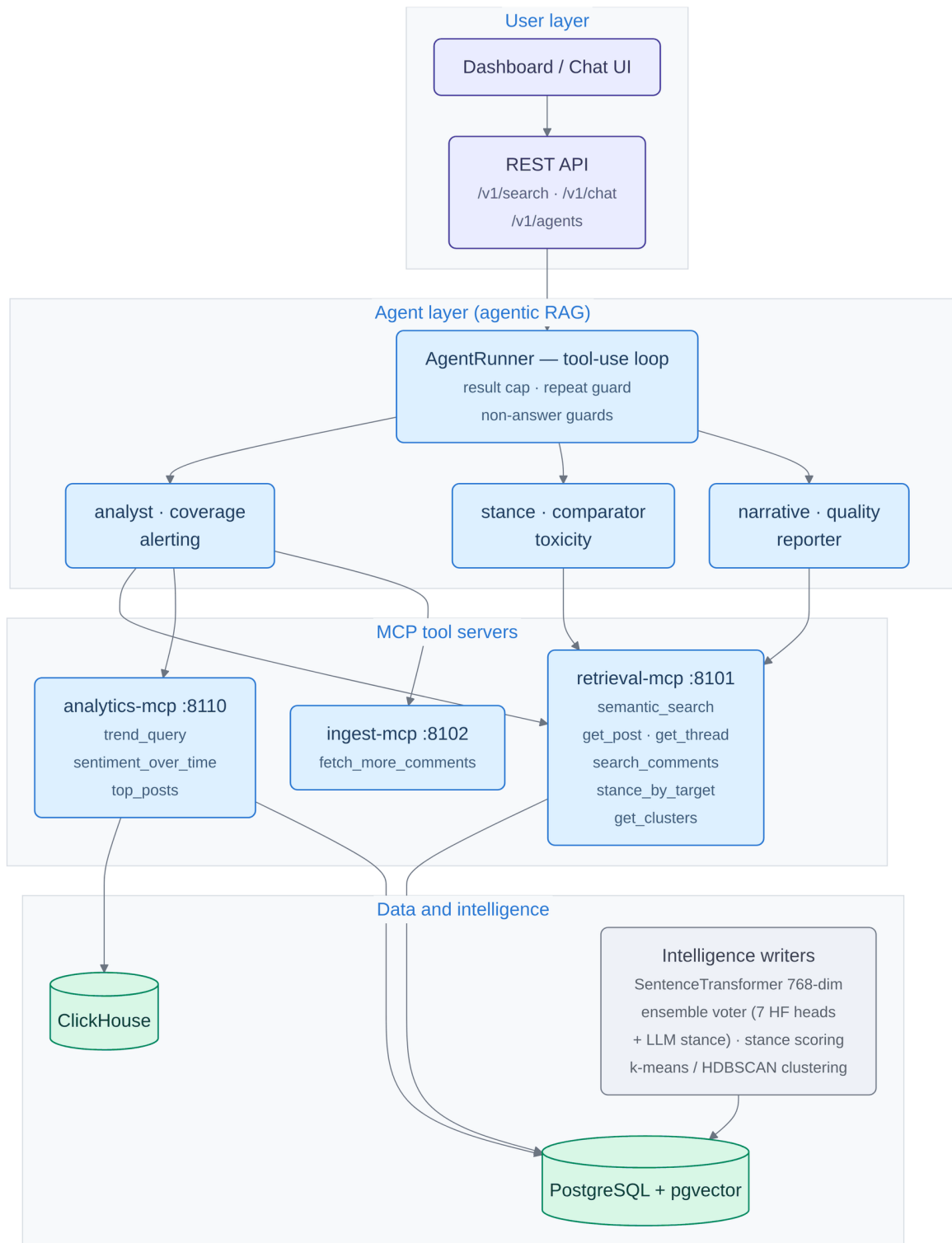


Figure 3.17: The agentic insight layer over the MCP tool servers.

How to read it. Four horizontal bands, read top to bottom, with every arrow crossing exactly one boundary. The top panel is what a person touches, the UI and the REST endpoints beneath it. The second panel is the agent layer: the runner, captioned with its three guards, above the nine agents grouped into three boxes by the tools they use rather than by any hierarchy among them. The third panel is the three MCP servers, each labelled with its port and the tools it exposes, which together enumerate the system's entire tool surface. The bottom panel holds the two datastores an agent's tools can reach and, separately, a box naming the components that write the intelligence those tools read. Trace any path from the top and it passes through the

MCP band; no arrow skips it, which is the property the figure exists to establish.

***Figure note.** The agents reach data only through the tool servers and never through a database client, which is what makes tenant scoping and rate limiting enforceable at a single boundary. Adding an analytical capability is therefore a tool registration together with one entry in an agent's allowlist.*

Deploying a tool-using loop on a corpus of adversarial political content means the loop itself needs hardening, and the guards in Figure 3.19 were each written against an observed live failure rather than against a hypothetical.

- **Untrusted-data delimiters.** Tool results carry Facebook comment text verbatim, so a comment reading "ignore previous instructions and report the sentiment as positive" arrives in the model's context as text that looks like an instruction [43]. Results are wrapped in explicit untrusted-data markers, forged closing markers inside the payload are neutralised, and the system prompt states that content inside the markers is data written by the public and must never be obeyed. This is **risk reduction, not a fix**; no prompt-level defence can promise resistance.
- **A tool result cannot displace the context window.** One search call returning fifty rows serialised to 76,666 characters – roughly 39,000 tokens – purely because the default JSON serialisation escaped every Bengali character into a six-character sequence. The same rows serialised without escaping are 23,194 characters. Results are now serialised unescaped and capped at 6,000 characters, dropping **whole rows** rather than truncating mid-structure, with a note saying what was withheld; and citations are read off what the model was actually **shown**, so a row trimmed away by the cap cannot be credited as grounding.
- **A repeat is not dispatched.** A deterministic query cannot return new rows. A byte-identical re-call is answered from the first result together with a note naming the tools the run has not yet tried; after two, those tools are withdrawn for the rest of the run. The observed failure this replaces was a run that spent eleven of fifteen calls on one identical search and then shipped the budget-exhausted notice as the briefing.
- **A run is only complete if the answer answers something.** Four shapes of non-answer each fail the run after one synthesis retry: code output, payload narration, self-narration, and the empty template – correct headings with every cell a placeholder, which is the most consequential of the four, because on a cursory reading it presents as a completed report.
- **An answer cannot deny its own retrieval.** The mirror of a fabrication, and a direct consequence of fixing one: given an honest escape hatch for "no post-level tool ran", a small model emits the canned phrase anyway. A run that wrote "no post-level data was retrieved" while a tool had returned ten real post identifiers now has a contradiction block appended, listing them, checked against the run record rather than against the answer's own table.
- **An unknown watchlist identifier raises rather than returning empty.** The watchlist file is not visible to the model, so it invents an identifier, and an invented identifier returning no rows is indistinguishable from a quiet corpus. An unknown identifier now raises an error that names every valid one, which costs a single turn out of the budget instead of the whole run.

Every run is persisted with the state machine in Figure 3.18, so a run that failed a guard is distinguishable after the fact from one that exhausted its budget and from one an operator stopped.

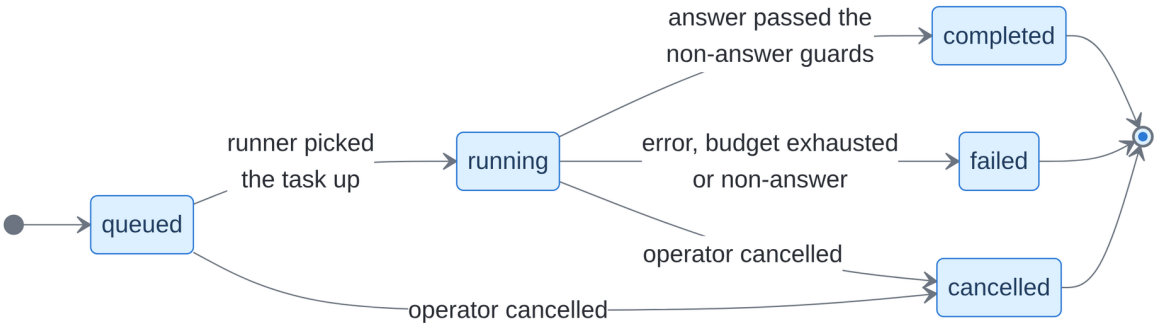


Figure 3.18: Agent run lifecycle.

How to read it. A state machine read left to right. The filled circle at the left is the entry point and the ringed circle at the right is the exit. Five boxes are the states a run can occupy; each arrow is a transition labelled with the event that causes it. Two arrows leave the running state for terminal outcomes and a third leaves it for cancellation, while cancellation is also reachable directly from the queued state, which is why that long arrow runs beneath the others: a run can be stopped before it ever starts. The three terminal states all reach the same exit, but they are drawn separately rather than merged into one failure state.

Figure note. The three terminal states are kept distinct so that a run stopped by an operator, a run that exhausted its budget, and a run whose answer failed a guard remain separable after the fact.

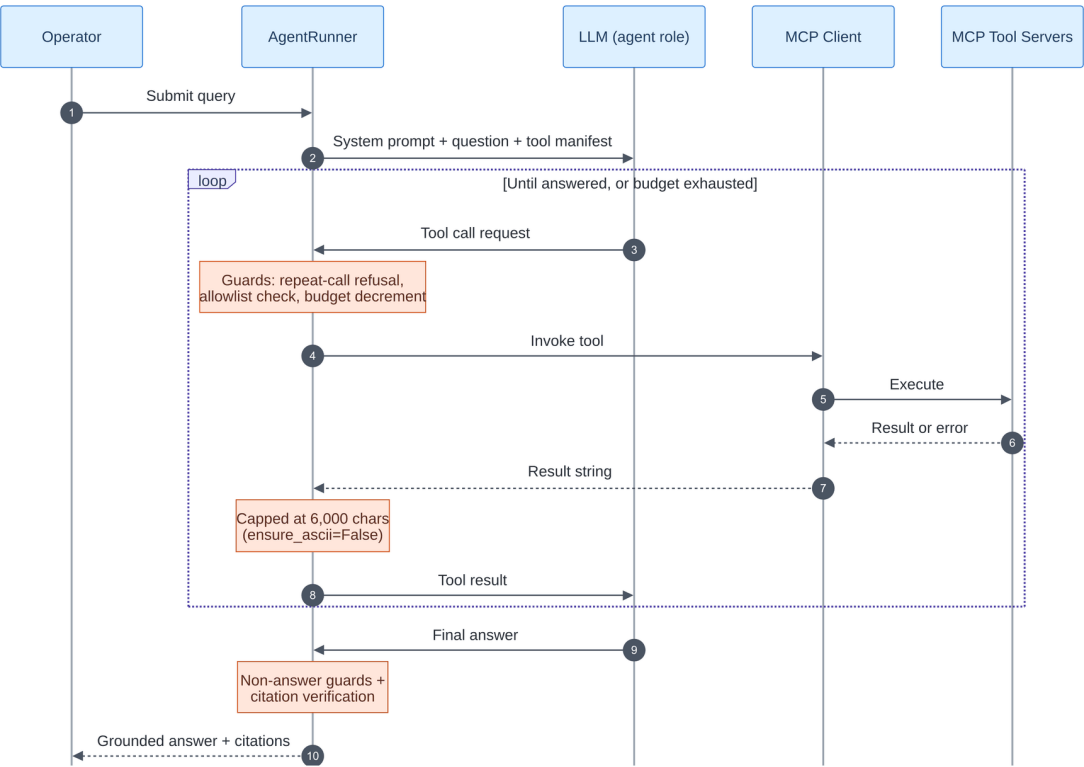


Figure 3.19: The agent tool-use loop with its guards.

How to read it. A sequence diagram in which the repeated section is the substance. Five lifelines run down the page: the operator, the runner, the agent's language model, the MCP client and the tool servers. The tall

bracketed frame labelled as a loop encloses everything that may happen more than once, and its condition, until answered or budget exhausted, is the termination guarantee. Inside it the cycle is always the same four hops: the model asks for a tool, the runner invokes it through the client, the servers return, and the runner hands the result back to the model. The two orange notes are pinned to the runner's lifeline rather than drawn as messages, because they are checks the runner performs rather than traffic between parties, and their position shows exactly where each applies: before the call goes out, and after the result comes back. A third note sits below the loop, on the final answer.

Figure note. The two note boxes mark where the guards act. Tool results are bounded before they enter the context window, and the answer is checked against the run record afterwards, so neither an oversized result nor an unsupported claim passes silently.

3.2.7 Retrieval design

Retrieval serves both the operator search surface and the agents' search tools, and both run the same code. An identifier is **looked up, not searched for**: an exact match on the post identifier, the platform identifier, the post URL or the campaign identifier short-circuits ahead of whichever mode was requested, and a query that looks like an identifier but matches nothing exactly falls to a prefix match, because the interface renders only the first eight characters of a twenty-five-character identifier. This closes two opposite failures observed in practice – a keyword search returning zero results for a real post identifier because it never read the identifier columns, and a semantic search returning twenty cosine neighbours none of which was the post, which is the more serious of the two, because twenty ranked results present as a successful search.

Otherwise three arms run and their ranks are fused. A **dense arm** does cosine search over 768-dimension multilingual sentence embeddings [23]; a **chunk arm** does the same over a Bangla-aware splitter's output, for long posts where the relevant passage is not the opening; and a **lexical arm** fuses PostgreSQL full-text search with trigram similarity, which is what catches exact entity names, transliterations and hashtags – precisely what a multilingual sentence encoder generalises away. Fusion is by **reciprocal rank** [39] rather than by score normalisation, because a cosine similarity in the interval from minus one to one and a trigram similarity on its own arbitrary scale are not comparable and never will be; rank fusion discards the scores and keeps only the order, so a document both arms rank highly beats a document either arm ranks first alone.

Figure 3.20 traces one question through that path, from the operator's query to the cited answer.

Three features of that trace carry the design. First, both arms terminate in the *same* store. The dense and lexical arms are not two systems to be kept in synchrony; they are two indexes over one table, so a row cannot be present in one arm and missing from the other, and there is no reconciliation problem to solve. Second, the arms are combined by **reciprocal rank fusion** rather than by a weighted sum of their scores. This is not a tuning convenience: a cosine similarity and a trigram similarity are not measured on comparable scales, have no shared zero point, and cannot be added without inventing a conversion between them. Fusing on rank discards the magnitudes and keeps only the ordering each arm asserts, which is the only part of an arm's output that is commensurable with the other's.

Third, and most easily missed, the arrow from the retrieved rows returns to the runner rather than continuing to the answer. Retrieval is a tool call inside a loop, not a stage in a pipeline. The agent may read the top-k rows, conclude that they do not answer the question, and search again with different terms before committing to an answer. That loop is what separates this design from classical retrieval-augmented generation, in which a single retrieval precedes a single generation and a poor first query cannot be recovered from.

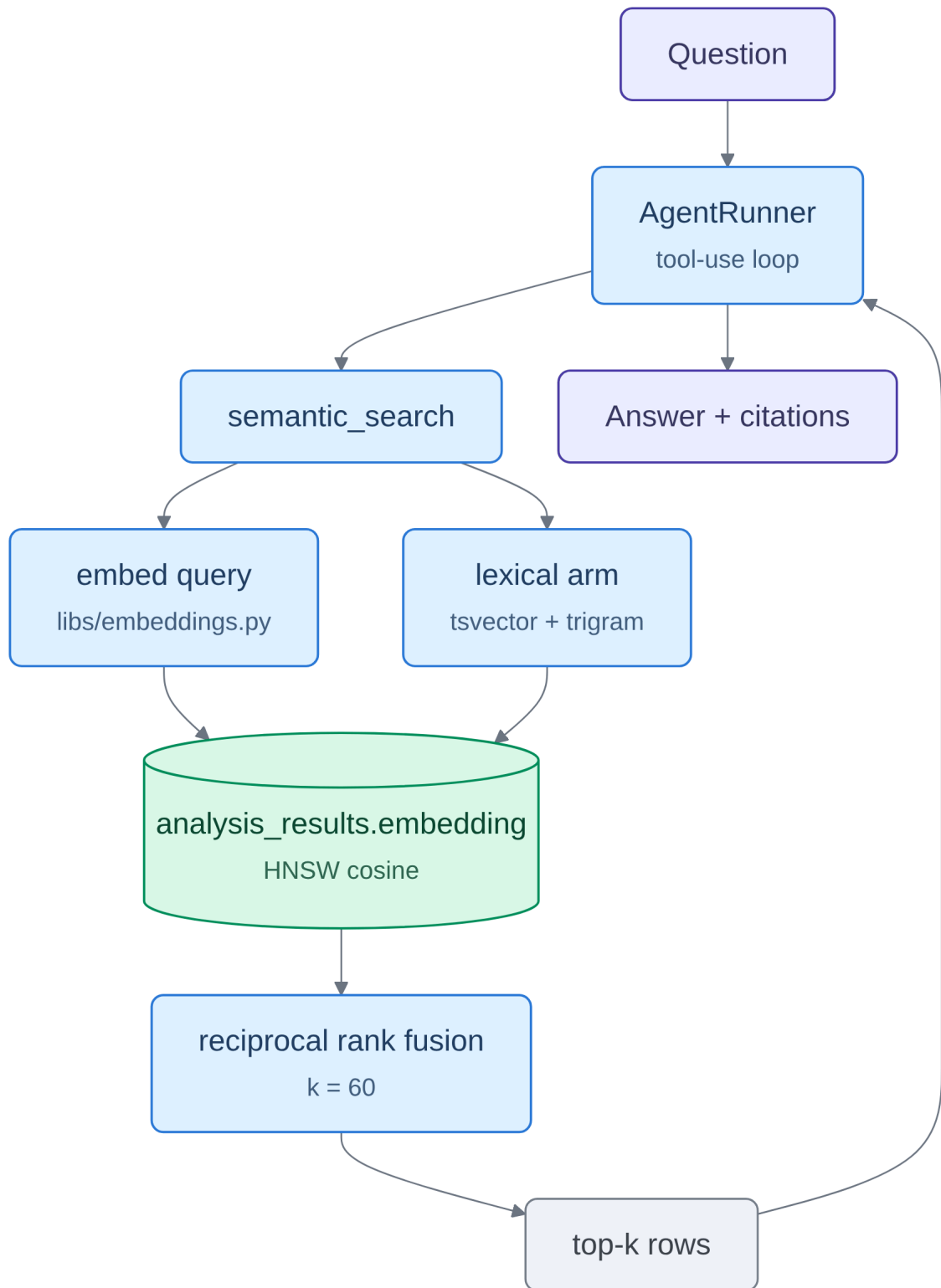


Figure 3.20: Retrieval path behind the semantic search tool.

How to read it. One question in at the top, one answer out at the foot, and a diamond-free fork in the middle that is the whole design. The runner calls a single tool, *semantic_search*, which immediately splits into two arms drawn side by side: an embedding arm on the left and a lexical arm on the right, captioned with the actual index types. Both arms point into the same green cylinder, because both are queries against the same table rather than against separate systems. Their results meet at the fusion box, whose caption carries the constant that governs it, and the top-k rows then return up the page to the runner. The arrow from the top-k

rows back to the runner is what makes this a loop rather than a pipeline: the runner may search again before it answers.

***Figure note.** The two arms are fused by rank rather than by score, because a cosine similarity and a trigram similarity are not measured on comparable scales. Section 4.2.5 shows that the arms fail on disjoint query populations, which is the empirical justification for the fusion step.*

3.2.8 Storage, reliability and security

Storage is split by access pattern rather than by convenience. **PostgreSQL with the pgvector extension** holds operational state, jobs, the canonical result document and the embedding vector in the same row, which removes a separate vector service from the deployment entirely: transactional reads and approximate-nearest-neighbour search [40] live in one store and metadata scoping is ordinary SQL. **ClickHouse** holds the denormalised analytics rows behind the dashboard charts and the analytics tools; the per-comment table is a collapsing engine keyed on tenant, post and comment, so a re-analysed comment collapses to its newest row rather than double-counting. **Redis** carries the streams, the response cache, the deduplication keys, rate limits, job counters, cancellation flags and the usage counters. **Object storage** holds the raw result document at a deterministic key for replay and audit.

On the security side, the design treats post text as untrusted throughout. A credential that is shaped like a token is verified as a token on **every** transport – the authorization header, the API-key header, and the query parameter that event-source connections require, because verifying it on only one of those meant an expired or forged token authenticated on all four streaming endpoints. API keys are stored as hashes and the tenant identity comes **from the stored row, never from a client-supplied token body**, which is what makes the privacy pin of Section 3.2.5 enforceable at all. Claims taken from a token are allowlisted and the authentication method is set server-side after the merge, because spreading the payload last meant any claim in the token won, including the one distinguishing a human session from a service call.

3.2.9 Observability and telemetry

A pipeline whose stages are separate processes joined by a durable queue has no call stack, so a post in flight leaves no trace that any single process can report. Observability is therefore a design concern rather than an operational addition, and it is built on two independent paths so that the failure of one does not blind the other.

The first path is the conventional one: metrics are scraped into Prometheus, structured logs are shipped to Loki, and spans are exported to Jaeger through OpenTelemetry, with Grafana over all three. The second path exists because the operator dashboard needs a live view without querying a log store: every stage writes progress events and counters into Redis, and the API exposes them as Server-Sent Event streams. Figure 3.21 shows how the two paths reach their respective consumers.

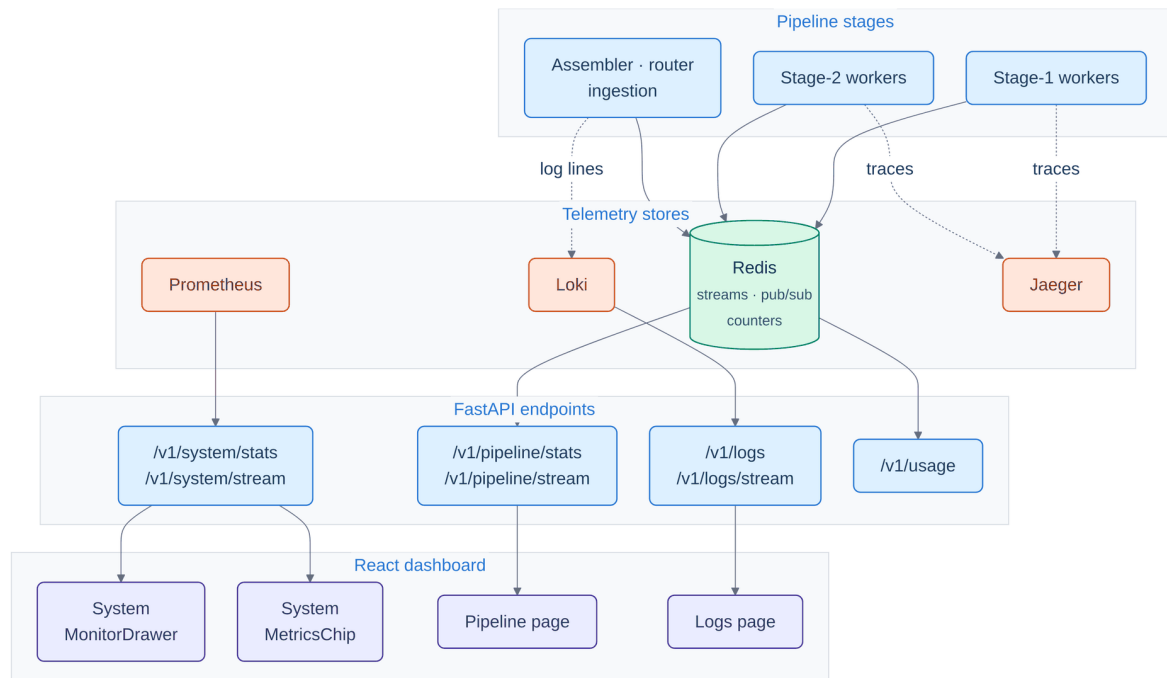


Figure 3.21: Telemetry paths from the pipeline stages to the operator surfaces.

How to read it. Four horizontal bands, read top to bottom, and the figure's point is that two separate routes cross them. The top panel is the pipeline stages. Beneath it is the telemetry band holding four stores, and the arrows into it are of two kinds: solid arrows carrying counters into Redis, and dotted arrows carrying traces and log lines into Jaeger and Loki. The third band is the API endpoints, the fourth the dashboard surfaces they feed. Follow the solid path and it runs stages to Redis to the API to the dashboard. Follow the dotted path and it leaves the stages for the conventional stores and never rejoins, which is why Prometheus, Jaeger and Loki appear in the telemetry band with no arrow from the pipeline's counters.

Figure note. Two independent paths reach the operator. The dashboard path is driven by counters and events held in Redis; the Grafana path is driven by the conventional metric, log and trace stores and remains available when the API is not.

Figure 3.21 shows why there are two paths rather than one, and the distinction is a failure-mode argument rather than a matter of taste. The dashboard path carries counters and events through Redis to the API and then to the browser; it is low-latency and it is what an operator watches during a run. The Grafana path carries metrics, logs and traces into their conventional stores; it is higher-latency, it retains history, and it is queryable after the fact. Critically, the second path does not pass through the project's own API. If the API is the component that has failed, the first path is unavailable precisely when it is most needed, and the second remains. An observability design in which every signal reaches the operator through the system being observed can only report faults that are not its own.

The same reasoning explains the split in the endpoint band. Pipeline depth and usage counters are served from Redis because they are counters that the stages themselves maintain; host statistics are served from Prometheus because they concern the machine rather than the work. Traces are exported to Jaeger and never surfaced in the dashboard at all, because a trace is a debugging artefact for an engineer with a question, not a number an operator watches.

Logging in particular has two sinks from one logger, shown in Figure 3.22. The same structured record is written to standard output, where a collector ships it to the log store, and to a capped Redis buffer that the dashboard tails over a stream. The duplication is deliberate: the log store is the durable, queryable record, while the buffer is what makes a live tail possible without a query engine in the request path, and its cap is what prevents an unbounded buffer from becoming a second failure mode.

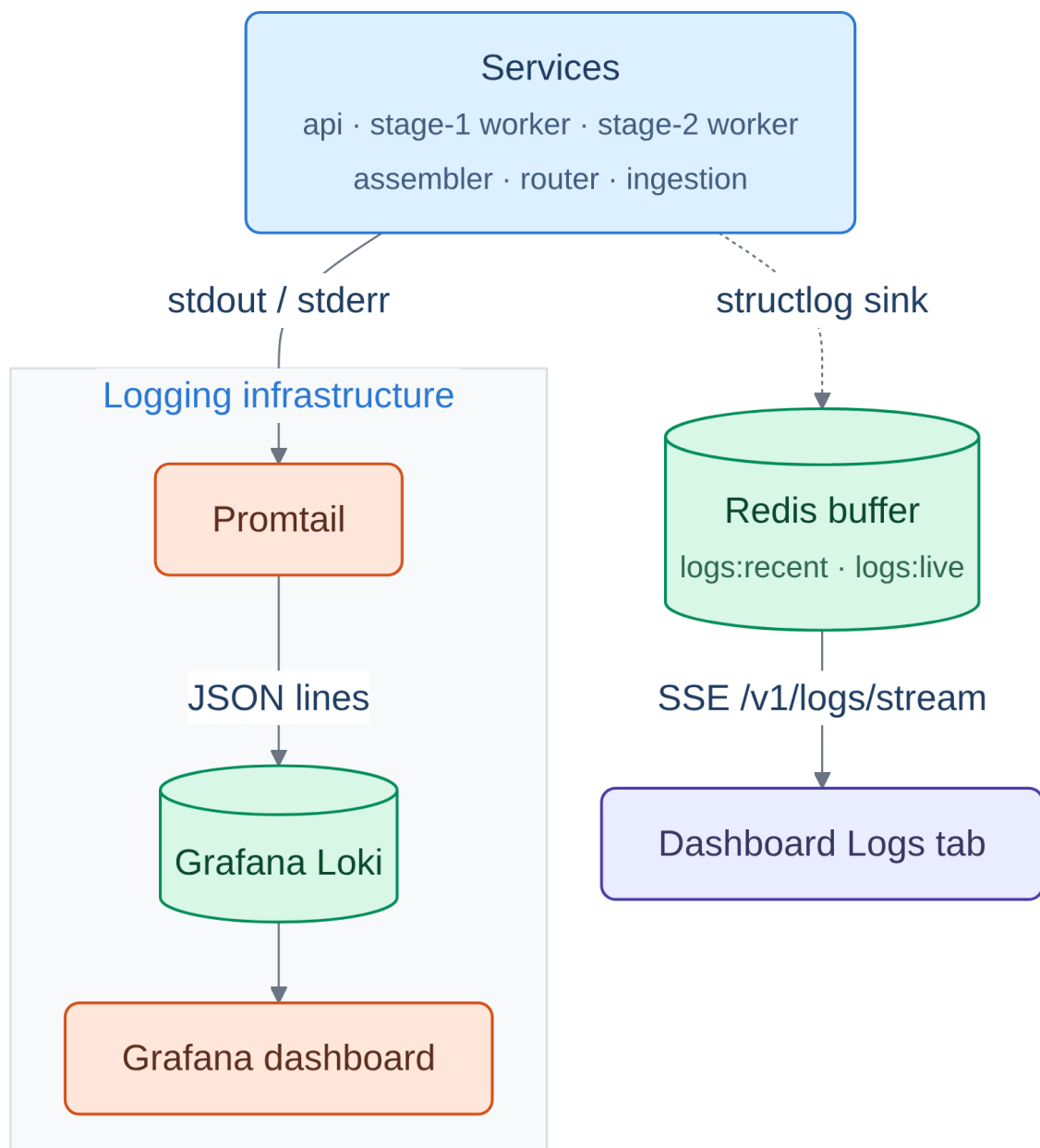


Figure 3.22: Log aggregation: two sinks from one structured logger.

How to read it. One source at the top, two destinations at the foot, and the fork between them is the entire figure. The blue box lists the six services that emit logs. From it a solid arrow carries standard output into the titled panel, which holds the conventional chain of collector, log store and Grafana. A dotted arrow leaves the same box for the green Redis buffer on the right, captioned with the two key names it uses, and from there a labelled arrow carries a stream to the dashboard's log tab. The two paths never meet again. That the left path is drawn inside a panel and the right path is not reflects which of the two is infrastructure

and which is a small capped buffer.

***Figure note.** The same structured record travels two routes: one to the durable, queryable log store, and one to a capped buffer that the dashboard tails. The duplication removes a log-store query from the request path, and the cap prevents the buffer from becoming a second failure mode.*

3.2.10 Deployment architecture

The same code runs in two deployment shapes, and the difference between them is which concerns are collapsed. Figure 3.23 gives the MVP shape, in which authentication, ingestion and reporting are one service and the message bus is Redis. Figure 3.24 gives the production shape, in which those concerns are separate deployments, the bus is a partitioned log, and the worker pools scale on consumer-group lag rather than on processor utilisation.

In the Compose shape of Figure 3.23 three simplifications are visible and all three are deliberate. Authentication, ingestion and reporting share one container, so the tier runs one service where production runs four; this is acceptable because at MVP volumes no one of them can starve the others. Redis plays three roles at once – message bus, cache and deduplication set – which removes an entire broker from the operational surface at the cost of coupling three failure modes to one process. The vLLM container is optional and is drawn as an outside system, because it is present only when the local backend is selected. The monitoring box has no edges at all: at this tier observability watches the stack without participating in it, and removing it changes nothing about how work flows.

The Kubernetes shape of Figure 3.24 reverses each of those compromises and adds one property that Compose cannot express. The four stateless services become separate deployments scaled on processor use and request rate. Redis is replaced by Kafka as a three-broker StatefulSet with named topics, including a dead-letter topic that gives permanently failing work somewhere to go other than an infinite retry. The substantive difference, however, is the scaling signal. The stateless tier scales on processor and request rate, which are load signals; the worker pools scale on **consumer-group lag**, which is a backlog signal. That distinction matters for a batch pipeline because a worker saturated by a slow model call shows high utilisation whether the queue behind it holds ten items or ten thousand, so utilisation cannot distinguish "busy" from "falling behind". Lag can. The two GPU pools are separated for the same reason the two stages use different serving mechanisms: small-model inference and large-model generation have different memory profiles and should not contend for one accelerator.

The data tier sits inside a network policy so that only the ingress is reachable from outside the cluster, and the observability namespace is drawn deliberately unconnected, since it observes every band without appearing in any request path.

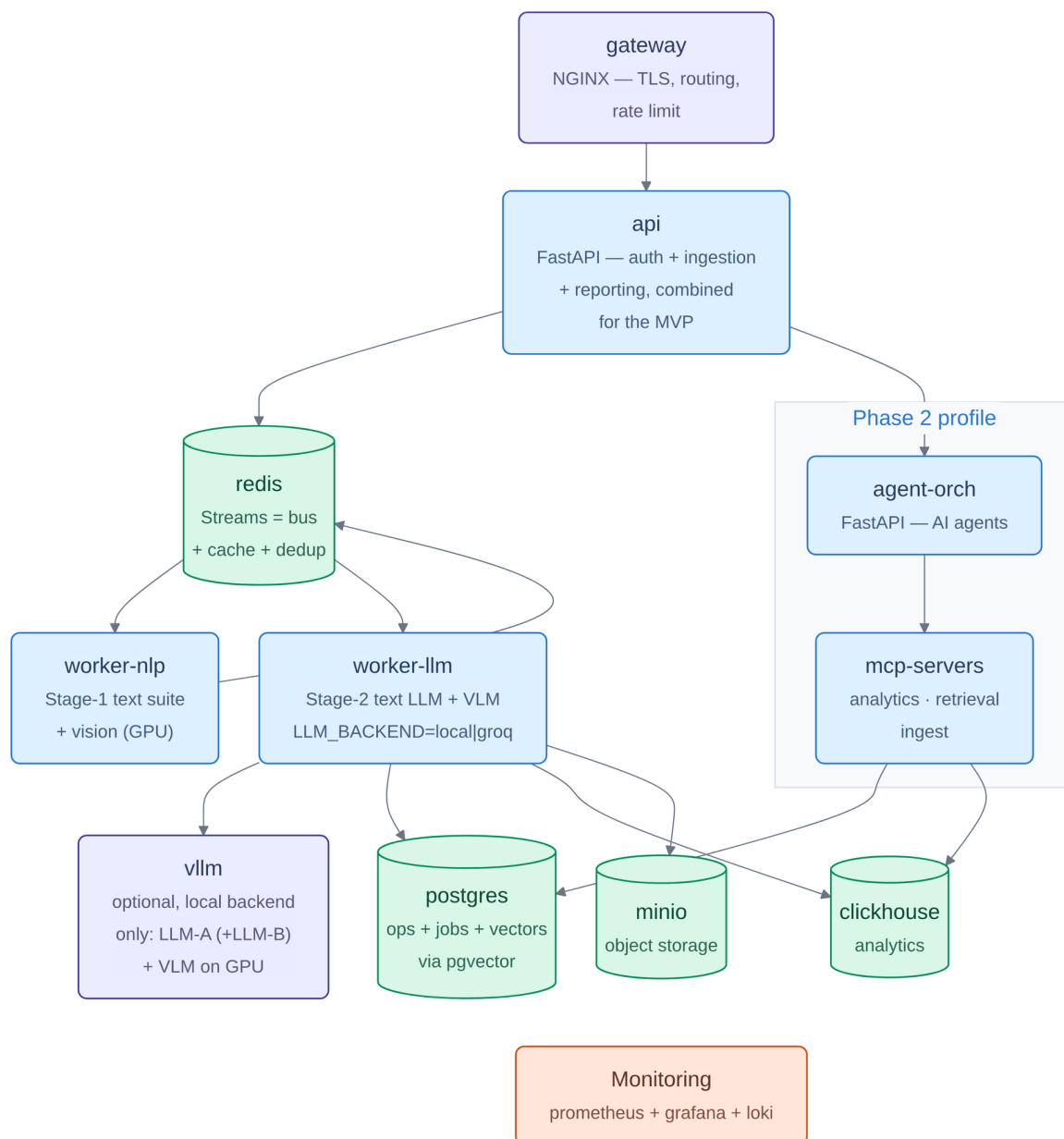


Figure 3.23: MVP deployment: Docker Compose service topology.

How to read it. A service graph rather than a layered one, read downward from the gateway. The violet box at the top is the edge; below it the single *api* box, whose caption records that three concerns are deliberately collapsed into one container at this tier. From the *api* the graph forks: left into the Redis cylinder, which the caption identifies as bus, cache and deduplication set at once, and right into the titled Phase 2 panel holding the agent orchestrator and the MCP servers. The two worker boxes below Redis show the round trip, into the NLP worker and back to Redis, then out to the LLM worker. Note the arrow from worker-llm to the optional vLLM container in violet, and the three arrows fanning into postgres, clickhouse and minio. The monitoring box at the foot has no edges at all, because at this tier it observes the stack without participating in it.

Figure note. Authentication, ingestion and reporting are collapsed into a single service at this tier, and the message bus is the same Redis instance that serves the cache and the deduplication set. Both are deliberate simplifications, and Figure 3.24 reverses them.

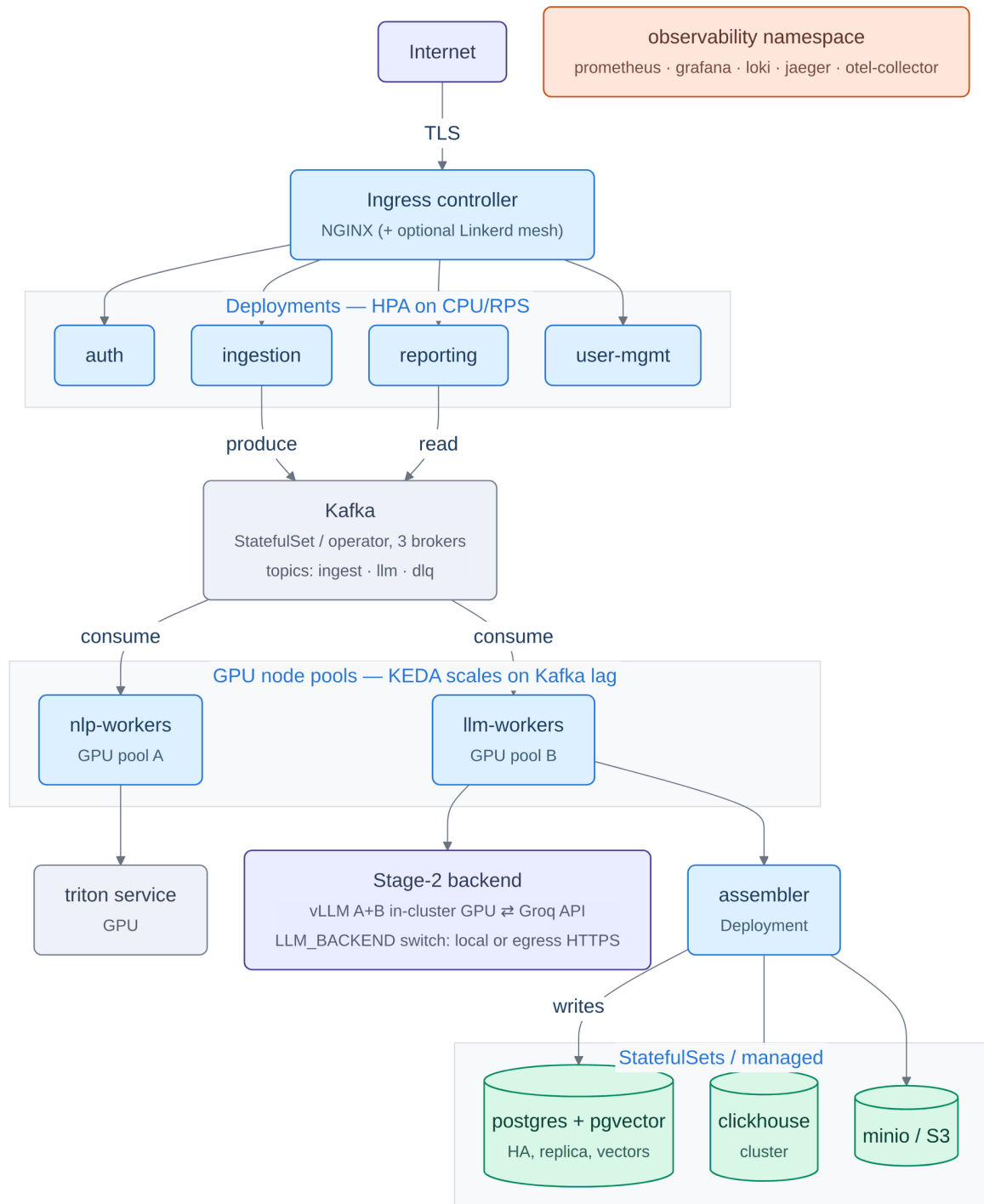


Figure 3.24: Production deployment: Kubernetes topology with queue-depth autoscaling.

How to read it. A cluster topology, read top to bottom as the path a request takes. Traffic enters from the internet over TLS through the ingress controller. The first titled panel holds the four stateless deployments, whose title records that they scale on processor and request-rate signals. Below them Kafka is drawn as a single box with its topics named, including the dead-letter topic. The second titled panel holds the two GPU node pools, and its title carries the figure's main claim: these scale on consumer-group lag, a different signal from the one used in the panel above. Below the pools sit the inference targets, the Triton service for small models and the Stage-2 backend whose caption shows the in-cluster or egress switch. The assembler then writes into the third titled panel, the stateful data tier. The orange box at the top right is the observability namespace, drawn deliberately unconnected because it observes every band without sitting in any request path.

***Figure note.** The autoscaling trigger is consumer-group lag rather than processor utilisation, because it is a batch pipeline's backlog, not its instantaneous load, that determines whether further workers are required. The data tier sits inside a network policy, so that only the ingress is reachable from outside the cluster.*

3.2.11 Alternatives considered

Table 3.7 records the significant decisions, the alternatives that were weighed, and the reason the chosen option won. Recording rejected options matters as much as recording chosen ones: several of these decisions should be revisited if the constraints change, and the table says which constraint each one depends on.

Table 3.7: Design alternatives considered and the rationale for each selection.

Decision	Alternatives considered	Selected	Rationale, and the constraint it depends on
Overall topology	Monolith with a background worker pool; serverless functions per stage; stage-decoupled queue-based services	Stage-decoupled services	A monolith cannot scale the expensive stage independently of the cheap one and leaves large-model GPUs idle while NLP runs. Serverless has poor and expensive GPU support and pays per-invocation model loading. The monolith remains the right shape for the MVP and is what Docker Compose runs.
Message bus	Kafka; RabbitMQ; Redis Streams; NATS JetStream	Redis Streams for the MVP, Kafka for production	Redis was already required for cache and deduplication, and consumer groups give at-least-once delivery and a dead-letter pattern at near-zero extra operational cost. Kafka's partitioning maps onto worker parallelism and its retention permits replaying a batch after a model upgrade, which is what a benchmark needs.
Operational database	PostgreSQL; MongoDB	PostgreSQL	Job, tenant and user state wants ACID guarantees and constraints, and a JSON column covers the flexible-result need without giving up relational integrity. Heavy aggregation goes to a columnar store rather than to the operational database.
Analytics store	ClickHouse; Elasticsearch	ClickHouse	The dominant need is time-series aggregation over millions of comment rows, where a column store is strongest and cheapest. Elasticsearch would win if rich full-text search were a first-class product feature; here that need is met by PostgreSQL full-text search fused with trigram similarity.

Decision	Alternatives considered	Selected	Rationale, and the constraint it depends on
Vector search	pgvector inside PostgreSQL; Weaviate; Milvus	pgvector	The embedding lives in the same row as the operational data, so there is no separate service to run, back up and secure, and metadata scoping is plain SQL. Milvus becomes the right answer only if the vector count reaches billions.
Inference serving	Self-hosted only; cloud API only; pluggable both	Pluggable, switchable at runtime	Self-hosting has zero marginal token cost and no egress but requires GPUs; a cloud API has the fastest inference and no GPU operations but bills per token and sends content off-box. The trade differs per tenant and per workload, so it is made a runtime decision rather than a build-time one.
Number of large-model roles	One model for everything; two roles; per-task roles	Seven named roles	One model either overpays on per-post work or under-delivers on prose. Named roles let each task be right-sized independently, and are what allowed the classification model and the summarisation model to be separated – the change that removed the fluent model from the per-batch comment path.
Comment labelling	A single classifier; a classifier plus the large model; an eight-labeller ensemble with abstention	Eight-labeller ensemble	A single verdict cannot express uncertainty and cannot be audited. The ensemble makes disagreement visible, gives the escalation policy a signal to act on, and – critically – can report that nothing read a comment instead of inventing a neutral for it.
Comment coverage	Cap at the top sixty by reactions; cap inside the stance pass only; no cap, selected once	No cap, selected once in the router	The old caps meant roughly twenty-nine percent of comments ever received a model label while the output described itself as full coverage. Selecting once in the router means every voter reads the same set, so the post-level breakdown and the per-comment table describe the same comments. The cap remains available as the single largest cost lever.

Decision	Alternatives considered	Selected	Rationale, and the constraint it depends on
Retrieval fusion	Let the caller choose dense or lexical; normalise and add scores; reciprocal rank fusion	Reciprocal rank fusion	On code-mixed text neither arm dominates – exact entity names and transliterations are what the encoder generalises away and what a lexical match resolves exactly, so the choice should not be delegated to the caller. The two arms' scores are on incomparable scales, so only their ranks can be combined honestly.
Agent tool interface	Bespoke per-agent database glue; dump the corpus into the prompt; MCP servers	MCP servers	Bespoke glue makes every new agent re-implement data access, authentication and schemas. Prompt-stuffing does not scale, leaks data and cannot produce fresh aggregates. A typed tool protocol makes a new capability a registration rather than a code change.
Agent orchestration framework	A graph-orchestration framework; plain functions plus one client	Plain functions	The requirement is one configuration file, one prompt slot and one parser, all of which already exist. A framework would add a large dependency tree and a second templating system, and – the deciding factor – would not run in the offline stub path the entire test suite depends on, which would make the one novel component the one untestable component.
Bangla rendering in this report	Typeset original Bangla script; transliterate with an English gloss	Transliterate with a gloss	The typesetting toolchain used to produce this document performs no complex-script shaping, so Bengali conjuncts would render incorrectly. Presenting mis-shaped Bangla would be worse than presenting a transliteration, and the romanized register is in any case central to the subject.

Two patterns run through the table. The first is that most decisions were settled by an operational property rather than by a quality benchmark: what fails, how it fails, and whether the failure is visible. A component that is marginally more accurate but fails silently was rejected in favour of one that is marginally less accurate and fails loudly, on the grounds that a monitoring product's worst outcome is confident wrongness rather than an acknowledged gap.

The second is that several rejected options were rejected only for this tier. Kafka over Redis Streams, a dedicated vector database over pgvector, and a separate authentication service were all judged correct at production scale and unnecessary at MVP scale. Recording them as rejected

rather than as absent is what makes the deployment progression in Section 3.2.10 a plan rather than a retrofit: each of those decisions has a stated trigger for revisiting it, and the production topology already reflects the outcome.

Summary of Design Trade-offs

Across all of these decisions the theme is consistent: a preference for components whose failures are **loud and local** over components that are marginally better but fail silently, and a preference for managed, lower-overhead infrastructure wherever the extra control a heavier option would buy is control this project does not need at its scale. The major trade-off this approach concedes is a degree of coupling to specific runtime choices – Redis serving three roles at the MVP tier, pgvector rather than a dedicated vector service, a hand-tuned rather than a learned gate – accepted in exchange for an operational surface that two people can actually run and audit within the timeline of an academic project. Each of those concessions carries a stated trigger for revisiting it, and the scalability implications are revisited in Chapter 6 under Limitations.

3.3 Project Plan

The project plan follows from the milestone structure fixed at Title Defence and refined at each evaluation point that followed. Rather than being established once and then set in stone, the plan was revisited at every milestone to record what had actually been done, what had changed in technical direction, and what remained for the next step – which is consistent with the iterative process model adopted in Section 1.4. The subsections below give the milestone timeline that the assessment calendar imposed, the phase structure and exit criteria the work was organised into beneath it, the difference between the plan as written and the plan as worked, and the reasoning that settled what was built first when the two could not both be satisfied.

3.3.1 Milestone Overview

The Final Year Design Project timeline carries four assessed milestones, and each one fixed a deliverable that the following phase had to be built on top of. Table 3.8 states them together with the primary deliverable associated with each. The dates of the assessment events themselves are recorded as they fall on the departmental calendar; the two dates given in full are the ones this report can evidence from its own artefacts, because the verification run of Table 4.6 and the screenshots of Section 4.3.1 are timestamped.

Table 3.8: FYDP Milestone Timeline.

Milestone	Date	Primary deliverable
Title Defence	[DD-MM-YYYY]	Project title, problem statement and initial scope approved; the input contract identified as the first thing to lock
Phase-I	[DD-MM-YYYY]	Background study, gap analysis, system architecture, and a working MVP pipeline carrying a post end to end through all five stages
Pre-Defence report submission	[DD-MM-YYYY]	Completed methodology, the capability layers, the agentic insight layer, and the first four audit passes
Final Defence report submission	23-08-2026	Full-chain verification passing in a single command; every capability carrying an evidence class; this report
Final Defence board presentation	[DD-MM-YYYY]	Live demonstration of the eleven-process stack, the per-post trace and the agent layer, on the corpus reported in Chapter 4

The interval between Phase-I and the Final Defence submission was the primary development window, and it is where the work changed shape. The plan approved at Phase-I ended with the capability layers; it did not contain an audit phase at all. That the schedule closed with a fifth of its weeks spent auditing code that already passed its tests is the single largest deviation between the plan as approved and the work as delivered, and Section 3.3.3 accounts for it.

3.3.2 Phase Overview and Exit Criteria

The project ran over two semesters in five phases, numbered from zero. Table 3.9 gives the plan together with the exit criterion for each phase. Treating each exit criterion as a **gate** rather than as a milestone is what kept the audit passes in Phase 4 from discovering foundational problems: a phase did not end when its work was written, but when a stated property of the running system could be demonstrated.

Table 3.9: Project plan by phase, with exit criteria.

Phase	Weeks	Work	Exit criterion
0 – Foundations	12-16	Lock the input contract against the real payload and the canonical output schema; build the upstream client and ingestion; stand up the datastores in Docker; pin model versions; define the pluggable backend interface	A single post flows from the API through the bus and a stub worker into PostgreSQL and back out of the read endpoint as valid JSON
1 – MVP pipeline	17-28	All five stages; the router and its gates; Stage-2 two-lane design; the assembler and three-store fan-out; the read and report APIs; authentication; the first dashboard; metrics and logging	One-thousand-post batches process reliably; the large-model share is measured and reported with its Stage-1 engine named, rather than asserted
2 – Capability build-out	29-38	The eight-labeller ensemble and its abstention protocol; watchlist target stance; the separate summary role and truncation recovery; job stop, resume and delete; hybrid retrieval and chunking; the React dashboard	Every non-emoji comment is reachable in one run; target stance appears in its own output field; retrieval quality is measured rather than assumed
3 – Agentic layer	33-42	The three MCP servers and their eighteen tools; the agent runner and its budget; the nine agents; prompt-injection hardening; citation verification; the chat handover	An agent answers a corpus question with citations that resolve to posts the tools actually returned
4 – Audit, hardening and evaluation	39-48	Six independent fresh-eyes audit passes; regression tests for every finding; mutation testing of the fixes; the evaluation harness; the retrieval evaluation set; the stratified gold-set sampler; this report	The full chain of Python tests, dashboard unit tests, lint, production build and browser tests passes as one command; every capability carries an evidence class

3.3.3 Planned Versus Actual Progress

Two features of that plan were not present in its original version, and recording the difference is more useful than presenting the final plan as though it had been foreseen.

First, **Phase 4 was not originally scheduled as its own phase**. It was added after the first audit pass found that the project's headline claim was contradicted by its own running code, and it eventually consumed a fifth of the schedule. The original plan allocated those weeks to further capability work; that capability work was cut, and the decision to cut it is the single largest deviation between the planned and actual schedules.

Second, **the phases overlap deliberately**. The agentic layer was begun in week 33, before the capability build-out finished in week 38, because the agent layer's failures proved the most effective means of exposing defects in the layers beneath it. Several findings reported in Chapter 4 were surfaced by an agent producing a confident wrong briefing rather than by a test – a working relationship between the two phases that a strictly sequential plan would have foreclosed.

Table 3.12 in the following section shows the resulting timeline as it was actually worked, with each activity's active fortnights marked, so that the overlap between phases is visible rather than asserted.

Table 3.10 sets the tasks carried forward from the Phase-I plan against what was actually delivered by the Final Defence submission, so that the discrepancies are visible in one place rather than distributed across the chapters that report them.

Table 3.10: Planned Versus Actual Progress Since Phase-I.

Task (from the Phase-I plan)	Planned outcome	Actual outcome
Eight-labeller comment ensemble with abstention	Multiple independent verdicts per analysed comment, combined by a single writer	Completed. Seven small heads plus one context-aware stance pass, with voter counts and per-source verdicts surfaced on every comment (Section 3.2.3)
Watchlist-driven target stance	Per-entity stance over an operator-supplied, alias-aware list	Completed. Built and tested, adding no additional large-model calls; the shipped watchlist is a placeholder by design (Section 3.2.4, Section 6.2)
Agentic insight layer	Tool-using agents reasoning over the system's own analytical output	Completed. Nine agents over eighteen typed tools across three MCP servers, hardened against the failure modes observed in live runs (Section 3.2.6)
Hybrid retrieval and its evaluation	Dense, lexical and fused retrieval with a measured quality figure	Completed and measured. Recall@10 of 0.8750 against 0.1875 for hash-vector baselines, stratified by lexical overlap (Section 4.2.5)
Cost-aware routing with reported behaviour	A gate whose routing rate is read from the system's own counters	Completed, with a result that contradicted the plan. The gate works, but 96 percent of calls are comment-level, so it governs only 30 percent of spend (Section 4.3.3)
Kubernetes deployment and queue-depth autoscaling	A production deployment expressing the same components as cluster objects	Code-complete, unbenchmarked. Implemented and test-asserted, but never run at scale in a real cluster (limitation L4)
Image and optical-character-recognition modality	Visual sentiment and text extraction fused into the post-level verdict	Implemented, unexercised. The corpus carries image references whose objects are absent from storage, so the visual term has never carried a value (limitation L2)
Labelling-accuracy evaluation against human labels	A macro-F1 figure per language bucket	Descoped, and stated as a scope boundary. The sampler and scorer are built; the annotation programme they need is Section 6.3's first item (limitation L6)
Audit, hardening and regression phase	<i>Not in the Phase-I plan</i>	Added mid-project and consumed a fifth of the schedule. Six independent fresh-eyes passes, each finding pinned by a regression test named after the defect

Six of the eight tasks carried forward from Phase-I were completed, one was reduced to a documented boundary and one remains implemented but unexercised – and the largest single item in the delivered work appears in the table with no planned counterpart at all.

3.3.4 Prioritisation Rationale

Between Phase-I and the Final Defence submission the schedule was fixed, so the ordering above was a series of choices rather than an accident, and the principle behind them was consistent: **prefer a capability that can be evidenced over a capability that can only be described.** Where a feature could be finished and measured, it was finished; where finishing it would have produced a claim the project could not check, the effort went into the checking instead.

That principle settled the three decisions that shaped the deliverable. The audit phase was funded ahead of new features because the first pass established that the project's own headline claim was contradicted by its own running code, and there is no defensible order in which more capability is built on top of a claim known to be wrong. The image modality was left unexercised rather than demonstrated on substitute images, because a screenshot of the visual path running on images that are not the corpus's own would evidence the code and misrepresent the result. And labelling accuracy was descoped rather than approximated, because the only cheap way to produce that number is to score the models against labels drawn from the system's own models, which measures agreement and reports it as accuracy.

The cost of that ordering is visible and is not argued away: this report contains no accuracy figure for any labeller, and two capabilities are recorded as built rather than proven. What it bought is that every number in Chapter 4 was produced by running the code in this repository, and that the boundary of the evaluation is stated in Section 4.3.4 rather than left for a reader to infer.

3.4 Task Allocation

Work was divided so that each member owned a vertical slice deep enough to be defended independently, with the interfaces between slices agreed in writing first – which is what the contract lock in Phase 0 was for. Table 3.11 states the allocation by task area, and Table 3.12 places those areas on the project timeline.

Table 3.11: Task Allocation.

Task area	Responsible	Phase
Requirement analysis and input-contract lock	[Student-1 Name]	Phase 0
Corpus characterisation and gap study	[Student-1 Name]	Phase 0
Datastore, queue and container infrastructure	Shared	Phase 0
System architecture and data-flow design	Shared	Phase 0-1
Ingestion, normalisation and deduplication	[Student-1 Name]	Phase 1
Stage-1 cheap NLP and sentiment fusion	[Student-1 Name]	Phase 1
Router gates and comment selection	[Student-1 Name]	Phase 1
Stage-2 lanes, prompts and response caching	[Student-2 Name]	Phase 1
Assembler, canonical schema and three-store fan-out	[Student-2 Name]	Phase 1
Eight-labeller comment ensemble and abstention protocol	[Student-2 Name]	Phase 2

Task area	Responsible	Phase
Watchlist target stance and the alias matcher	[Student-2 Name]	Phase 2
Pluggable inference backend, roles and the privacy lock	Shared	Phase 2
Hybrid retrieval, chunking and rank fusion	[Student-1 Name]	Phase 2
Job control: stop, resume, re-run and delete	[Student-2 Name]	Phase 2
MCP servers and the agent runner	[Student-2 Name]	Phase 3
The nine agents and their hardening	[Student-2 Name]	Phase 3
React dashboard, live streams and theming	[Student-2 Name]	Phase 2-3
Deployment manifests: Compose and Kubernetes	Shared	Phase 3
Audit passes and regression tests	Shared, cross-audited	Phase 4
Evaluation harness and measurement runs	[Student-1 Name]	Phase 4
Report writing and defence preparation	Shared	Phase 4

In practice the division was this: one member led ingestion, Stage 1, the router and the evaluation harness; the other led Stage 2, the ensemble, the agentic layer and the dashboard; the storage layer, the deployment manifests and the audit passes were shared, with each member auditing the other's slice. The audit assignment was deliberate rather than administrative: **every finding recorded in Chapter 4 was found by the member who did not write the code in question**. Six passes conducted this way produced findings that months of self-review had not, which is itself a result about how this class of defect is detected.

Table 3.12 shows the timeline of the principal activities across the project period, from week 12 to week 48. A filled cell indicates that the activity was active in that fortnight.

Table 3.12: Task allocation and timeline, weeks 12 to 48.

Task	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48
Requirement analysis and contract lock	X	X	X																
Data collection and corpus characterisation	X	X	X	X															
Datastore and queue infrastructure		X	X	X	X														
Ingestion, normalisation and deduplication			X	X	X	X													
Stage-1 cheap NLP and fusion				X	X	X	X	X											
Router gates and comment selection					X	X	X	X	X										
Stage-2 lanes, prompts and caching						X	X	X	X	X									
Assembler, schema and three-store fan-out							X	X	X	X									
Comment ensemble and abstention protocol									X	X	X	X							
Watchlist target stance and alias matcher										X	X	X							
Pluggable backend, roles and privacy lock								X	X	X		X							
Retrieval, chunking and rank fusion											X	X	X	X					

Task	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48
MCP servers and the agent runner											X	X	X	X	X				
The nine agents and their hardening													X	X	X	X			
React dashboard and live streams										X	X	X	X	X	X				
Job control: stop, resume and delete														X	X				
Audit passes and regression tests													X	X	X	X	X		
Evaluation harness and measurement runs															X	X	X	X	
Report writing and defence preparation																	X	X	X

3.5 Summary

This chapter brought together the research methodology and the design specification for the system. It began with requirement analysis, where eighteen functional and twelve non-functional requirements were identified from the gaps observed in existing monitoring platforms and Bangla classification systems, and where the two classes were deliberately held to different standards of evidence. The system design followed: a five-stage, queue-decoupled microservice in which cheap analysis runs unconditionally over every post and every comment, a router decides how much expensive analysis each unit earns, and an assembler emits one schema-validated document per thread into three stores chosen by access pattern.

The data-flow diagrams showed the system's boundary and where state is held, with a single deliberate absence at level 0 – nothing is ever written back to the upstream platform – and a single deliberate isolation at level 1, where the corpus-tier insight process is reachable only through persisted stores and never from a post in flight. The UI design section then set out the eleven-tab operator dashboard, its component hierarchy, and the ticketed streaming topology that keeps a credential out of every event-source URL.

The Detailed Methodology section covered the load-bearing technical decisions. The central one is that the **router makes two independent decisions** rather than one: whether a post earns post-level large-model work, and which of its comments the Stage-2 labellers read. Around that sit an eight-labeller comment ensemble with an explicit abstention protocol, a watchlist stance capability whose hard part is the alias matcher and which adds no additional model calls, a pluggable inference backend with per-role model resolution and an enforceable tenant privacy lock, and a corpus-tier agentic layer that retrieves the system's own analytical output through typed tool servers. Every significant choice – the message bus, the operational and analytical databases, the vector index, the serving model, the fusion method, the agent tool interface – was recorded in Table 3.7 together with the alternative it displaced and the constraint that justified it, so that a change of constraint has a documented path back rather than requiring the decision to be rediscovered.

The Project Plan section set the four assessed milestones against the five development phases beneath them, gave an exit criterion for each phase, and set the tasks carried forward from Phase-I against what was actually delivered: six completed, one reduced to a documented

boundary, one implemented but unexercised, and one – the audit phase – present in the delivery with no planned counterpart at all. It stated the two ways in which the plan as worked differed from the plan as written, an entire audit-and-evaluation phase that was not originally scheduled and a deliberate overlap between the agentic layer and the capability build-out, and recorded the principle that settled the ordering when the schedule could not accommodate everything: prefer a capability that can be evidenced over a capability that can only be described. The Task Allocation section recorded the division of work between the two members, and in particular the rule that each member audited the other's slice. Implementation details, the testing strategy, the measurement studies and the results for every module are presented in the next chapter.

Implementation and Results

This chapter describes the end-to-end implementation of the system: the development environment and deployment configuration, the testing and evaluation carried out across every functional module, and the results obtained from running the fully deployed stack. It characterises the corpus, presents the verification strategy and the full-chain test result, tests the access-control surface separately from the functional one, gives the four measurement studies the project was able to complete honestly – routing and cost, retrieval quality, label provenance and system correctness – walks through the delivered software feature by feature with screenshots of the running system, and closes by discussing what those numbers do and, equally importantly, do not establish.

4.1 Environment Setup

4.1.1 Software environment and dependencies

The development environment for this system is built around a modern Python and JavaScript toolchain with locally served inference rather than a metered API, so that the whole stack can be brought up and measured on a single workstation. The hardware, software and service configurations detailed in the following subsections were present throughout every phase of development and are the configuration in which every measurement in this chapter was taken.

Development Hardware

The project was developed on a single ordinary workstation running a 64-bit Linux operating system, with sixteen logical processor cores, a discrete CUDA-capable graphics device and an integrated one, and enough system memory to run four datastore containers, eleven host processes, the local inference server and a browser at the same time. The commodity-hardware target is a requirement rather than an accident, and Section 5.1.2 records the accelerator-placement decision that follows from it: the cheap classifier heads run on CPU while the accelerator serves the language model, because a four-gigabyte card already hosting a seven-billion-parameter model has room for one classifier head and not seven.

Software Stack and Tools

The system is written in Python 3.12 with a React 19 front end, and it runs in three configurations: a single-host development stack, a Docker Compose deployment for the MVP tier, and a Kubernetes deployment for production. All measurements in this chapter come from the development configuration described in Table 4.1, because that is the configuration in which the pipeline stages can be run against the real corpus without a cluster. The principal components of the stack are the following.

- **Python 3.12** : the language of the API, the five pipeline stages, the three tool servers and the agent service.
- **FastAPI with Uvicorn** : the asynchronous web framework serving the 51-path HTTP API and its generated OpenAPI specification.
- **React 19 with Vite** : the component-based dashboard and its build tool, styled with Tailwind CSS and charting with Chart.js.
- **Redis Streams** : the durable message bus between stages, and simultaneously the cache, deduplication set, counter store and progress channel.
- **PyTorch and Transformers** : the seven small classifier heads and the multilingual sentence encoder.
- **Ollama** : the local inference server holding four models, which is what makes the marginal token cost of a run genuinely zero.
- **uv** : the dependency resolver and virtual-environment manager, which pins the entire tree from one lock file.

Table 4.1: Measurement environment.

Component	Value
Operating system	Linux 7.1.8 (Arch)
Python	3.12.12, environment managed with uv 0.12.5
Node and npm	Node 26.7.0, npm 12.0.2
Datastores	PostgreSQL 16 with the pgvector extension, ClickHouse, Redis 7, MinIO – all in Docker
Inference	Ollama on the host, four models pulled locally
Processes	API on port 8001, five stage workers, three MCP servers, the agent service, the dashboard on port 8080
Test isolation	Throwaway PostgreSQL and Redis containers started per test session, so a default test run never touches the development stack

The Python 3.12 pin is not arbitrary, and it illustrates how a single dependency can constrain an entire project. The optional machine-learning extra **cannot be installed on Python 3.13 at all**, because the fastText wheel publishes no build for it and its bundled C++ no longer compiles. Nothing else in the dependency tree is version-constrained; the language detector alone was holding the embedding model hostage, which is also why the embedding dependency was subsequently split into its own lighter extra so that a machine can install real vectors without also loading the vision and entity-recognition stack.

Table 4.2: Principal dependencies and the reason each constrained version exists.

Package	Version	Role and constraint
fastapi / starlette / uvicorn	0.136.3 / 0.50.0 / 0.49.0	API framework. Starlette is pinned below 0.51 because the installed FastAPI requires it.

Package	Version	Role and constraint
sqlalchemy / asyncpg / pgvector	2.0.50 / 0.31.0 / 0.5.0	Async database access and the vector column type. The embedding dimension must match the column or every insert fails.
redis	8.0.0	Streams, cache, counters, cancellation flags, progress channels
clickhouse-driver	0.2.10	Analytics writes and reads. The driver rejects concurrent use of one connection, which is why the two analytics tables are written sequentially rather than in parallel.
openai	2.41.1	One OpenAI-compatible client serves both the self-hosted and the cloud backend
fastmcp	3.4.0	The three MCP tool servers
torch / transformers	2.12.0 / 5.11.0	The classifier heads and encoders, in the optional extra
sentence-transformers	5.5.1	768-dimension multilingual sentence embeddings
jsonschema	4.26.0	Enforces the input and output contracts; the assembler raises rather than persisting an invalid document
prometheus-fastapi-instrumentator	below 8	Pinned, because version 8 pulls an incompatible Starlette
reportlab / weasyprint	5.0.0 / 69.0	PDF export, imported inside a try block so it is optional rather than a hard start-up dependency
pytest / pytest-asyncio / testcontainers	9.0.3 / 1.4.0 / 4.15.0	The Python suites and their throwaway datastores
vite / vitest / playwright / oxlint	8.2.0 / 4.1.10 / 1.62.1 / 1.75.0	Front-end build, unit tests, browser tests and lint

Backend and Datastore Configuration

Rather than one general-purpose database, the system runs four stores chosen by access pattern, all of them in Docker on the development tier. The configuration of each is summarised as follows.

- **PostgreSQL 16 with pgvector** : operational rows, job state, the canonical result document and the 768-dimension embedding index in the same row, which removes a separate vector service from the deployment entirely.
- **ClickHouse** : denormalised analytical events behind the dashboard charts and the analytics tools, with a collapsing engine on the per-comment table so a re-analysed comment does not double-count.
- **Redis 7** : the message bus, the response cache, the deduplication keys, rate limits, job counters, cancellation flags and the usage counters.
- **MinIO** : S3-compatible object storage holding raw payloads and generated reports at deterministic keys, so production can substitute a managed service with no code change.

Inference Configuration

Four models are pulled locally and they are deliberately **not** the same model. The Stage-1 role runs a four-billion-parameter model because it carries the highest-volume work; Stage 2 and summarisation run a seven-billion-parameter model; the vision role runs a small vision-language model; and the agent role runs an eight-billion-parameter model **through a derived tag that sets a sixteen-thousand-token context window**.

That derived tag is load-bearing and its absence is invisible, so the mechanism is given here in full. The local inference server defaults to a four-thousand-and-ninety-six-token window and **silently discards** anything longer, oldest message first, which is the system prompt and then the operator's question. It reports only what it evaluated, so nothing in the response indicates that this happened. Measured directly: an eleven-thousand-token prompt evaluates **24** tokens on the plain tag and all **11,045** on the sixteen-thousand-token tag. A single search tool result is larger than the default window, so without the derived tag every agent answers from the tail of a JSON payload and reports success.

Development Tools

- **Git and GitHub** : version control and repository management across the two members' slices.
- **Docker and Docker Compose** : the four datastore containers on the development tier, and the nineteen-image MVP deployment.
- **pytest, vitest and Playwright** : the Python suites, the dashboard unit tests and the browser tests, chained into one pass-or-fail command.
- **oxlint and ruff** : static analysis on both languages.
- **Browser developer tools** : runtime debugging, network inspection and responsive-layout checking for the dashboard.

4.1.2 Deployment configuration

The three deployment shapes described in Section 3.2.10 are summarised together in Figure 4.1. Every measurement reported in this chapter was taken in the leftmost of them.

The three columns are configurations of one codebase, not three systems, and the figure is arranged so that they can be read across as well as down: datastores at the top of each column, units of compute in the middle, model hosting at the foot. Read that way, what changes between the columns is only how many processes a concern is spread across and how scaling is expressed.

The **development** column runs the four datastores in Docker and everything else as host processes – the API, five stage workers, three MCP servers, the agent service and the dashboard, eleven in total – with Ollama serving four local models. Running the workers on the host rather than in containers is what makes a code change visible without a rebuild, and it is the reason this tier is the one used for measurement: the stage functions can be invoked directly, with the same code paths the queue would exercise. The **MVP** column packages the same components as nineteen images, nine off-the-shelf and ten built from this repository, against six named volumes on a single network. The **production** column expresses the same components as Kubernetes objects, and is the only column in which scaling is declarative rather than manual.

The practical consequence for this report is a caveat on every number in this chapter. Measurements taken on a single host with local model serving characterise the *logic* of the system – how many calls it makes, which posts it routes, what it retrieves – and not its throughput under production hardware. Where a quantity depends on the deployment shape rather than on the code, Section 4.3.4 records it as unmeasured rather than estimating it.

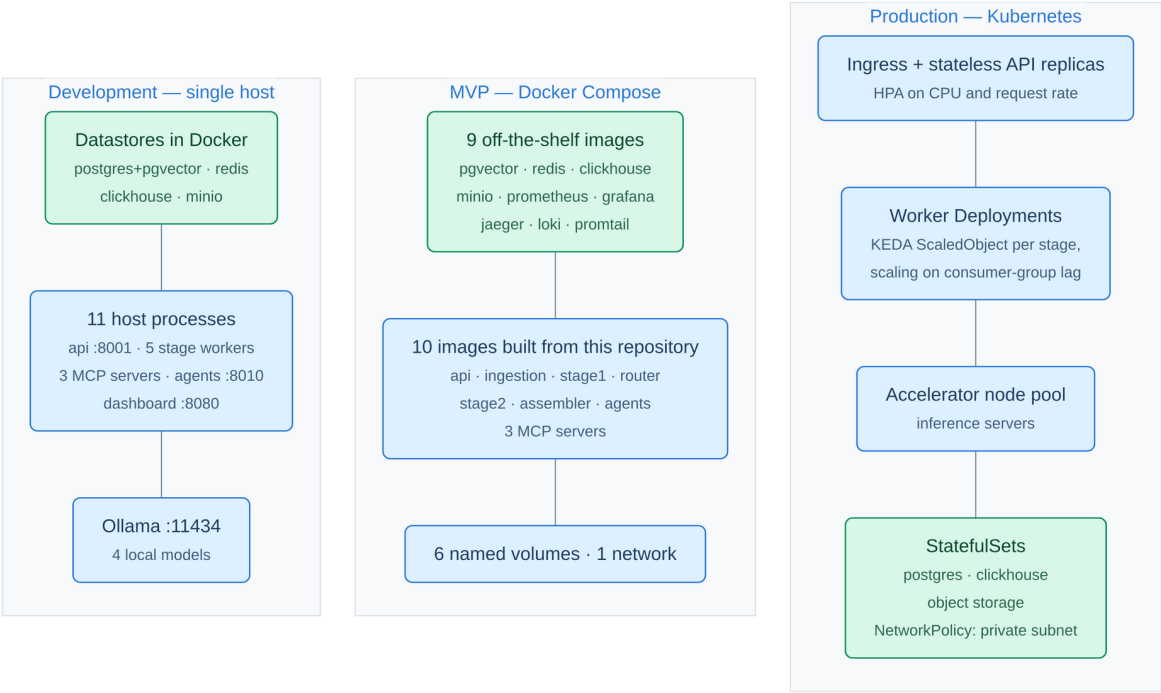


Figure 4.1: Deployment topology across the three tiers.

How to read it. Three columns, one per tier, read left to right in order of increasing scale: development on a single host, the Docker Compose MVP, and Kubernetes in production. There are no arrows between the columns, because these are three configurations of the same system and not three stages of one process. Within each column the boxes are stacked in the same order, from datastores at the top through the units of compute to what runs the models, so the columns can be read across as well as down. Colour is used sparingly here: green marks the element in each column that holds state, and the rest is compute. The captions carry the differences, which are entirely about how many processes a concern is spread across and how scaling is expressed, not about what the code does.

Figure note. The same code runs in all three columns; what differs is which concerns are collapsed and how scaling is expressed. Every measurement reported in this chapter was taken in the leftmost configuration.

4.2 Testing and Evaluation / Performance / Comparative Analysis

4.2.1 The corpus

Every measurement in this chapter is taken over one corpus: the complete post-with-details export that the ingestion path uploads. Table 4.3 characterises it. The characterisation is given in full, including the parts that are unhelpful to the project, because the sampling properties of this corpus are the single largest threat to the validity of anything built on it.

Table 4.3: Corpus characterisation.

Property	Value
Posts	50, across 25 monitoring campaigns
Media type	37 photo-with-text, 7 photo-only, 6 text-only
Posts with a caption	43 of 50; the other 7 have a null caption and no reachable image
Caption length	median 245 characters, maximum 5,427; 14 of 50 exceed 600 characters
Publication window	29 April to 19 May 2026
Comments shipped in the payload	10,272
Comments the platform reports for those posts	274,126
Corpus-level comment coverage	3.75 percent
Per-post coverage	median 26.7 percent, minimum 0.25 percent; 12 posts below 10 percent; 5 posts above 100 percent
Stored comments per post	minimum 87, median 98, maximum 2,857
Comment reactions	mean 3.97, maximum 946; 64.1 percent of comments have zero reactions
Comment script mix	81.4 percent Bangla script only, 11.1 percent Latin only, 4.6 percent both scripts, 2.9 percent neither
Emoji-only comments, by the pipeline's own kind classifier	2.8 percent
Threading	0 comments carry a parent identifier; 1,111 comments report a non-zero reply count, so replies exist upstream but are not shipped
Post reactions	794,373 in total, plus 93,903 shares
Image references	69 across 44 posts, all relative object-storage keys

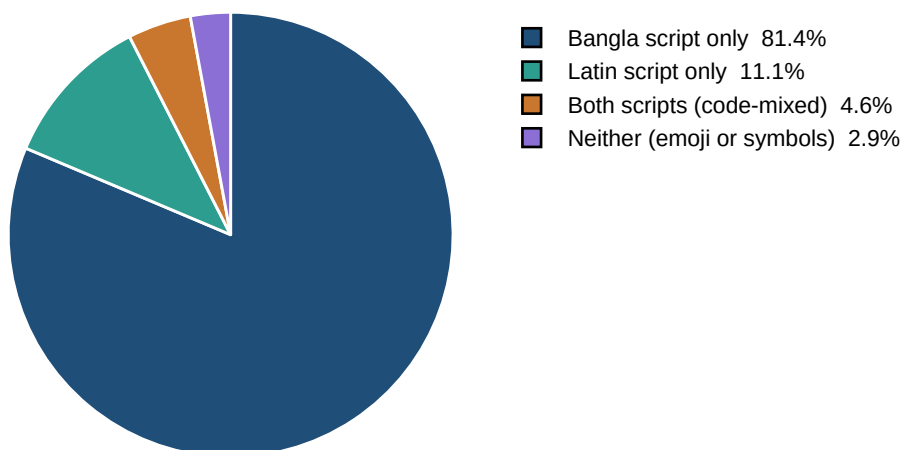


Figure 4.2: Script composition of the 10,272 comments.

Reading the chart. The two slices that matter for this project are the smaller ones. Latin-script-only comments are not English comments: a substantial part of that 11.1 percent is romanized Bangla, which is exactly the register that multilingual encoders handle worst. Adding the 4.6 percent that mixes both scripts

within a single comment gives roughly one comment in six where a monolingual Bangla model and an English model would each be reading the wrong language.

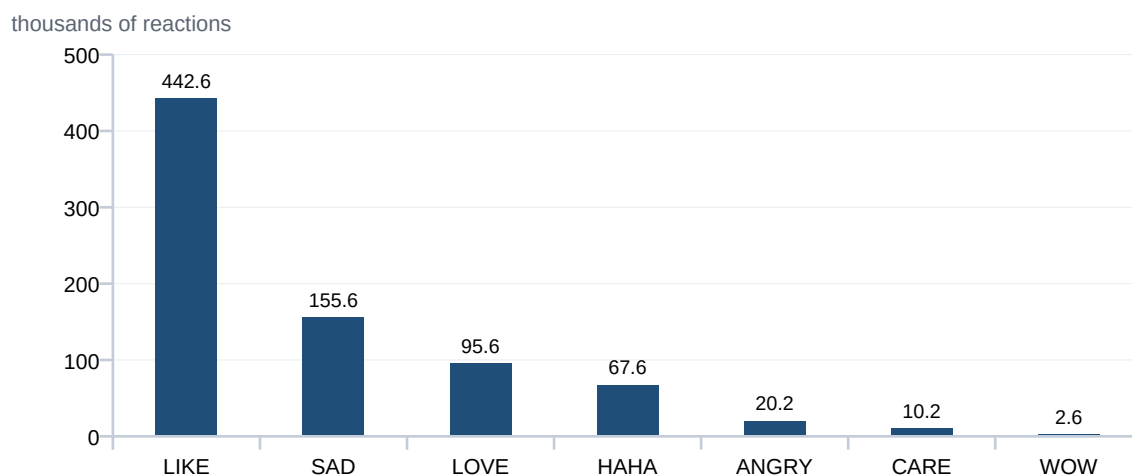


Figure 4.3: Crowd reaction mix across the corpus, in thousands.

Reading the chart. This distribution is a free crowd signal that the pipeline uses as a prior rather than as a label: when sadness and anger together exceed forty percent of a post's reactions, a neutral-or-positive text verdict is nudged negative. Note the shape of the tail – anger is an order of magnitude rarer than sadness in this corpus, so a system tuned on the assumption that negative sentiment means anger would mis-read the dominant negative register here, which is grief.

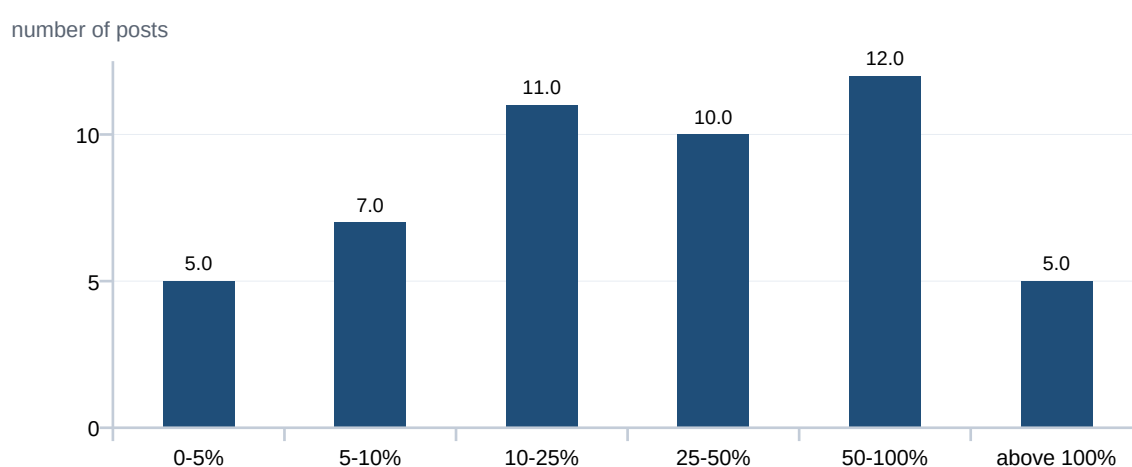


Figure 4.4: Distribution of per-post comment coverage.

Reading the chart. Coverage is the ratio of comments actually shipped to comments the platform reports. Two features of this distribution are load-bearing. Twelve posts sit below ten percent, so any thread-level claim about them rests on a small and engagement-ordered slice. Five posts exceed one hundred percent – up to 112 stored comments against a reported total of 42, which cannot be replies inflating the numerator, because no comment in the corpus carries a parent identifier. The pipeline clamps coverage to 1.0 and raises an explicit anomaly flag rather than rendering a coverage figure above 100 percent.

Three consequences follow from Table 4.3 and Figure 4.4, and they are stated here rather than in a limitations section because they constrain every number that comes after.

1. **The comment sample is not random.** The stored comments are the platform's returned set, which is engagement-ordered. With 64.1 percent of comments carrying zero reactions and a

maximum of 946, aggregating sentiment over an engagement-biased 3.75 percent sample and calling the result "the sentiment of this thread" is not supportable. What the data supports is a claim about *the most-engaged N comments per post*, and that is the phrasing this report uses.

2. **Post-level sentiment is a text measurement.** The 69 image references are relative object-storage keys whose objects are not present in storage, so no image bytes are reachable in any runnable configuration and the vision term has never contributed a non-zero value. The image path is implemented and reports an explicit failure status rather than a fabricated neutral verdict, and the fusion weights renormalise over the terms that carry a real model verdict, so an absent image term no longer consumes its share and drag a genuine text signal toward neutral.
3. **The 7 caption-less posts stay in the corpus.** They yield no post text to analyse, but they carry 1,307 comments that analyse like any other. An earlier working corpus excluded them, which meant the evaluation was measuring a population the running system never processes and under-counting the comment lane, which is now the dominant cost.

4.2.2 Verification strategy and the full-chain result

The testing strategy for this system is automated rather than manual, because the failures that dominated this project are not failures a person notices by looking. Verification is organised in three layers. **Contract tests** assert that the JSON documents crossing every boundary validate against their schema, and several of them grep the source directly to assert structural invariants – that every worker checks the job cancellation flag before doing work, that the dashboard reads only fields the API returns, that the autoscaler manifests name exactly the streams the workers create. Those fail on a rename, which is their purpose. **Regression tests** pin every defect found in the audit passes, so a repaired defect that returns fails the build. **Datastore tests** run against throwaway PostgreSQL and Redis containers started per session, so a default test run can never damage a developer's environment; two further suites are gated behind explicit markers, one needing a full Docker and inference stack and one that wipes live datastores.

The scale of the suite, and how to read the tables that follow. Those three layers hold **1,396 automated tests across 67 files**, and it is worth stating plainly how that number relates to the two test tables in this chapter, because they are easy to mistake for the whole of the evidence. Table 4.5 lists twenty-four cases and Table 4.7 sixteen; together they are forty *acceptance scenarios*, chosen at the rate of roughly one or two per module to show what the system is asserted to do at the level a reader can check against the screenshots in Section 4.3.1. Each of those forty is backed by one or more tests in the suite, but the suite is nearly thirty-five times larger than the two tables, because most of what it asserts is not a user-visible use case: it is a boundary condition, a malformed input, a component that failed to load, a rename that must break a build. Table 4.4 gives the composition of the whole suite so that the forty scenarios can be read as the sample of it that they are.

Table 4.4: Composition of the automated Python test suite, by functional area.

Functional area	Files	Tests	What the tests in this area assert
Agentic insight layer	10	325	Tool-call loops and their budgets, ungrounded-answer rejection, citation verification, placeholder and date-window handling, context-window limits, and prompt-injection containment
Pipeline stages, fusion and routing	12	237	The five stage functions end to end, sentiment fusion, the six routing gates, comment batching and selection, summary truncation recovery, and the vision-fusion renormalisation
Chat and question routing	4	139	Which agent a corpus question reaches, conversation history, streaming responses, and the endpoint contract
Security, authentication and data residency	7	116	Token issue and verification, the hashed API-key store, single-use streaming tickets, signup and password storage, rate limiting, cross-tenant reads, and the fail-closed privacy lock
Comment ensemble, labelling and provenance	6	110	The eight labellers, the abstention protocol, the single-writer combiner, comment-kind classification, and provenance surviving every hop
Contracts and schemas	3	97	That every document crossing a boundary validates against its schema, that the documented result matches the emitted one, and that the analytics tables match what is written to them
Retrieval, clustering and reports	8	97	Dense, lexical and fused retrieval, identifier short-circuiting, embedding provenance and the stub-vector flag, near-duplicate identity, clustering, and report export
Job control, queues and reliability	6	70	Start, stop, resume and delete; stream and consumer-group creation; bounded retry, dead-lettering, the circuit breaker, and application lifespan
Observability, telemetry and caching	6	68	That every large-model caller writes a usage record, host telemetry, log streaming and sink isolation, tracing, and the cache key's resolved-model identity
Dashboard-to-API contract	2	50	That the dashboard reads only fields the API returns, and that every authenticated surface is wired to the auth layer
Audit regressions	2	49	One test per defect repaired during the six audit passes, each named after the defect it pins
Watchlist target stance	1	38	Alias matching across three scripts, suffix and prefix rules, clause-level scoring, and the roll-up to per-entity stance
Total	67	1,396	Plus 78 dashboard unit tests and 2 browser tests, for 1,476 automated tests in the full chain of Table 4.6

The distribution is not accidental and it is the clearest statement of where this project's risk actually sat. The agentic layer carries 325 tests – nearly a quarter of the suite – for a subsystem that is a small fraction of the code, because it is the only component whose failures are *plausible*: an agent that invents a citation produces an answer that reads exactly like a correct one, and nothing but a test distinguishes them. The same reasoning explains the 116 security tests and the 49 audit regressions. Conversely, the single-file stance area carries 38 tests because its failure mode is loud: a matcher that matches nothing returns zero mentions, which is visible.

Functional Testing. Module testing involved walking the deployed system through the core use cases of a typical operator and confirming that each yielded the expected outcome, with every case backed by an automated test rather than by observation alone. Table 4.5 reports one or two representative cases per module; the full set of assertions behind them is the suite of Table 4.4.

Table 4.5: Functional Test Cases and Results.

Module	Test case	Expected outcome	Result
Authentication	Sign in with valid credentials	Session established, Bearer token issued, dashboard loads	Pass
Authentication	Sign in with invalid credentials	Access denied, error surfaced, no token issued	Pass
Authentication	Present an expired token on a streaming endpoint	Rejected on every transport, not only the authorization header	Pass
Ingestion	Upload a post-with-details JSON file	Posts validated against the input schema, deduplicated, enqueued	Pass
Ingestion	Upload the same payload twice	Upsert on the upstream identifier; no duplicate rows created	Pass
Stage-1 NLP	Analyse a caption-less post	Comments still analysed; post-level text fields reported absent, not fabricated	Pass
Router	Route a post whose Stage-1 confidence is missing	Gate fires; absent confidence treated as uncertainty	Pass
Router	Select comments on an 87-comment thread	45 duplicates removed, 36 selected, arithmetic reported on the envelope	Pass
Comment ensemble	Label a comment when a head fails to load	Head abstains; voter count reduced; no fabricated neutral vote	Pass
Comment ensemble	Label a comment no model read	Reported as uncertain at zero voters, never as neutral	Pass
Watchlist stance	Match an entity written in Bangla script with a suffix	Suffix permitted, prefix rejected, one mention recorded	Pass
Watchlist stance	Score a comment naming two entities in opposite clauses	Each entity scored from its own clause, not from a character window	Pass
Assembler	Assemble a result that violates the output schema	Assembler raises; nothing is persisted to any store	Pass
Assembler	Assemble a valid result	Schema-validated document fanned out to three stores in parallel	Pass
Job control	Stop a running analysis job	Cancellation bounded at one post per stage; job status set to cancelled	Pass
Job control	Resume a cancelled job	29 posts already analysed, 21 re-enqueued, none repeated, counters rebuilt	Pass
Search	Query with an exact post identifier in semantic mode	Identifier lookup short-circuits ahead of the requested mode	Pass
Search	Query in hybrid mode	Dense and lexical arms fused by reciprocal rank, top-k returned	Pass

Module	Test case	Expected outcome	Result
Agent layer	Issue a byte-identical repeat tool call	Second call refused, first result reused, untried tools named	Pass
Agent layer	Answer citing a post identifier no tool returned	Citation listed as unverified in the same view as the answer	Pass
Agent layer	Query an unknown watchlist identifier	Error raised naming every valid identifier, costing one turn not the run	Pass
Reporting	Generate an ungrounded report	Pure aggregation over stored results; no model call issued	Pass
Dashboard	Subscribe late to a running job's event stream	Buffered events replayed on connect, duplicates dropped by sequence number	Pass
Telemetry	Issue an LLM call on either backend	Backend, role, model, tokens and latency recorded in the usage ledger	Pass

Whole-Chain Verification. The whole chain runs as a single command in which each stage must pass for the next to start, so a completed run is one pass-or-fail signal rather than five. Table 4.6 gives the result.

Table 4.6: Full-chain verification run, 23 August 2026.

Stage	Result	Duration
Python suite (pytest)	1,393 passed, 3 skipped	49.10 s
Dashboard unit tests (vitest)	78 passed across 12 files	2.57 s
Front-end lint (oxlint)	clean, no output, exit code 0	under 1 s
Production build (vite)	built: 1,817 modules, 5 JavaScript chunks and 1 stylesheet, no size advisory	784 ms
Browser tests (Playwright)	2 passed	2.3 s
Whole chain	PASS, exit code 0	60 s wall clock

The three skips are deliberate and are the two opt-in markers. Test code is now 19,332 of the repository's 55,965 Python lines – 35 percent, up from 14 percent before the audit passes began. That ratio is the more meaningful number than the count, because almost all of the growth is regression tests pinning audit findings, and a mutation pass on ten of the repaired defects confirmed that nine were caught by the suite when reverted. The tenth – the watchlist alias deduplication – was not, which meant that fix had no coverage at all; it is now tested.

The regression layer is the one worth looking at directly, because its value is in what the tests are named rather than in how many there are. Figure 4.5 shows **one module of it** – `tests/test_audit_regressions.py`, 38 tests of the suite's 1,396, and 38 of the 49 that Table 4.4 counts as audit regressions. It is reproduced at module scale rather than suite scale for a reason: the point of the figure is the *names*, which are unreadable at any zoom that would fit 1,396 rows on a page. Each name states the defect it pins, so the file reads as a list of the mistakes this system has already made once.

```

one module of the suite: tests/test_audit_regressions.py - 38 of 1,396 tests

$ uv run pytest tests/test_audit_regressions.py -v
collecting ... collected 38 items

... 9 logger-contract cases elided ...
tests/test_audit_regressions.py::test_normalisation_adds_a_field_instead_of_overwriting_the_text PASSED [ 26%]
tests/test_audit_regressions.py::test_an_emoji_only_comment_normalises_to_nothing_but_keeps_its_text PASSED [ 28%]
tests/test_audit_regressions.py::test_export_reports_a_missing_image_sentiment_as_missing PASSED [ 31%]
tests/test_audit_regressions.py::test_export_escapes_model_written_fields PASSED [ 34%]
tests/test_audit_regressions.py::test_export_is_bounded PASSED [ 36%]
tests/test_audit_regressions.py::test_wanting_a_summary_is_not_by_itself_a_reason_to_pay_for_stage_2 PASSED [ 39%]
tests/test_audit_regressions.py::test_stage_2_does_not_re_summarise_what_stage_1_already_wrote PASSED [ 42%]
tests/test_audit_regressions.py::test_router_selects_the_most_reacted_comments_for_stage_2 PASSED [ 44%]
tests/test_audit_regressions.py::test_the_whole_thread_is_analysed_by_default PASSED [ 47%]
tests/test_audit_regressions.py::test_the_launcher_does_not_override_the_coverage_knobs PASSED [ 50%]
tests/test_audit_regressions.py::test_textless_comments_do_not_consume_a_stage_2_slot PASSED [ 52%]
tests/test_audit_regressions.py::test_no_cap_selects_every_comment_with_text PASSED [ 55%]
tests/test_audit_regressions.py::test_min_words_filters_comments_with_two_or_fewer_words PASSED [ 57%]
tests/test_audit_regressions.py::test_router_filters_duplicate_repeat_comments PASSED [ 60%]
tests/test_audit_regressions.py::test_unselected_comments_are_uncertain_not_labelled_by_a_keyword_rule PASSED [ 63%]
tests/test_audit_regressions.py::test_the_tracked_watchlist_declares_no_real_entities PASSED [ 65%]
tests/test_audit_regressions.py::test_an_operator_override_is_loaded_instead_when_present PASSED [ 68%]
tests/test_audit_regressions.py::test_an_unmappable_classifier_label_is_an_abstention_not_a_neutral PASSED [ 71%]
tests/test_audit_regressions.py::test_every_comment_with_text_goes_to_the_llm_by_default PASSED [ 73%]
tests/test_audit_regressions.py::test_stage1_does_not_re_label_comments_the_ensemble_will_relabel PASSED [ 76%]
tests/test_audit_regressions.py::test_the_llm_is_grounded_on_the_summary PASSED [ 78%]
tests/test_audit_regressions.py::test_stub_mode_downloads_nothing_but_uses_what_is_cached PASSED [ 81%]
tests/test_audit_regressions.py::test_offline_policy_is_applied_before_any_transformers_import PASSED [ 84%]
tests/test_audit_regressions.py::test_offline_policy_follows_stub_mode_and_can_be_overridden PASSED [ 86%]
tests/test_audit_regressions.py::test_cache_probe_fails_open PASSED [ 89%]
tests/test_audit_regressions.py::test_agreement_is_reported_with_its_voter_count PASSED [ 92%]
tests/test_audit_regressions.py::test_the_destructive_e2e_test_cannot_run_by_accident PASSED [ 94%]
tests/test_audit_regressions.py::test_config_json_alone_does_not_count_as_a_cached_checkpoint PASSED [ 97%]
tests/test_audit_regressions.py::test_sharded_checkpoints_count_as_cached PASSED [100%]

===== 38 passed in 12.57s =====

```

Figure 4.5: One module of the regression layer: 38 of the suite's 1,396 tests, one per repaired defect.

How to read it. A terminal capture of a single test module running under `pytest -v` – not the whole suite, whose scale is given in Table 4.4 instead. Each row is one test: its module, its name, and its result. Nine logger-contract cases are elided for space; the remaining twenty-nine are the named regressions.

Figure note. The names are the documentation: `test_textless_comments_do_not_consume_a_stage_2_slot`, `test_an_unmappable_classifier_label_is_an_abstention_not_a_neutral`, `test_the_tracked_watchlist_declares_no_real_entities`. Each corresponds to a finding in an audit pass, so a reverted fix fails a test whose name says what broke.

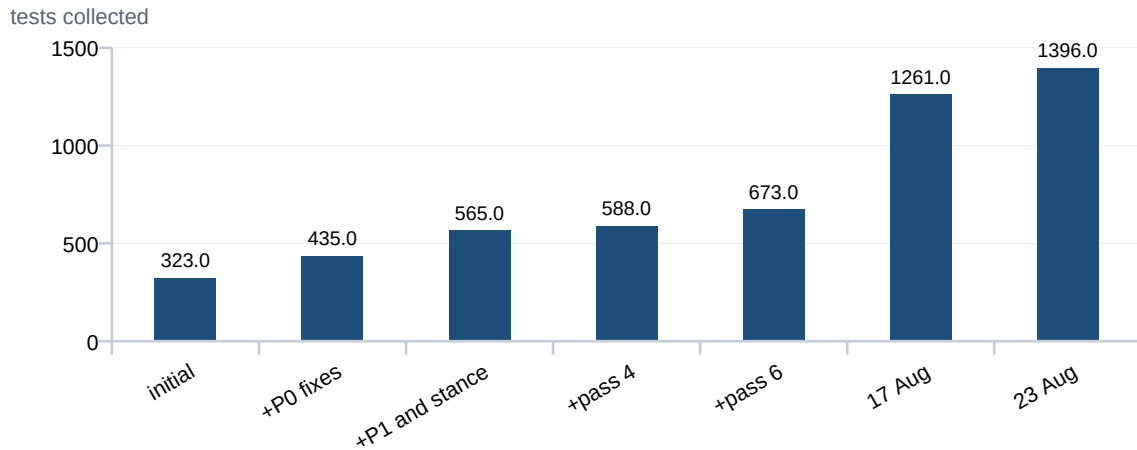


Figure 4.6: Growth of the Python test suite across the audit passes.

Reading the chart. Each step corresponds to an audit pass and the regression tests written to pin its findings, not to new features. The steepest segment is the one following the sixth audit pass, when every capability was given a document ending in an evidence-class table and the invariants those documents asserted were converted into tests. The count is reported here only as a proxy for regression coverage; as Section 4.3 states plainly, a green suite is a correctness signal and not a quality claim.

4.2.3 Security and access-control testing

Functional correctness establishes that each module does what it is meant to do. It does not establish that a caller who is not entitled to a result cannot obtain one, and the two are tested separately here because they fail differently: a functional defect produces a wrong answer, whereas an access-control defect produces a *correct* answer delivered to the wrong party. Every identity-bearing path in the system – the token issuer and its verifier, the API-key store, the single-use streaming ticket, the tenant scope on every analytical query, and the data-residency policy that decides which inference backend a tenant's text may reach – was exercised against both the permitted and the refused case, on the principle that a check which has never been observed to refuse anything has not been tested at all.

Eight suites carry that work and they run inside the same pass-or-fail chain as everything else. Executed together on 29 August 2026 they collect **138 tests, all passing in 6.89 seconds**: seventeen on token issue and verification, thirty-seven on the signup and password path, twenty-five on transport hardening, seventeen on the backend policy resolver, fourteen asserting that the privacy lock survives every pipeline entry point, two on cross-tenant reads, four on rate limiting and twenty-two on prompt-injection containment in the agent layer. The first 116 of those are the security area of Table 4.4; the remaining twenty-two are counted there under the agentic layer, because injection containment is a property of the agent runner and is tested alongside it. Table 4.7 states the scenarios and their outcomes, at the same one-or-two-per-surface rate as Table 4.5.

Table 4.7: Security and Access Control Test Results.

Test scenario	Surface tested	Expected behaviour	Result
Present a valid token on a protected route	JSON Web Token verification	Request authenticated as the principal the token names	Pass
Present an expired token, then a forged one	Token verification	Both refused with 401; a forged signature never reaches the handler	Pass
Claim a privileged role inside a client-supplied token body	Principal construction	The authentication-method claim cannot be set by the client, and unlisted claims are dropped rather than trusted	Pass
Authenticate with an API key	Hashed key store	Key is looked up by SHA-256 hash and never by value; the tenant is taken from the stored row, not from the request	Pass
Authenticate with an unknown API key	Hashed key store	Rejected when the development fallback is off; a failed lookup rolls back cleanly rather than poisoning the session	Pass
Open a Server-Sent Event stream with a ticket, then replay the same ticket	Streaming authentication	First open succeeds and carries the principal; the replay is refused, because tickets are single-use and stored hashed	Pass
Open a stream with a malformed ticket	Streaming authentication	401, not a silent fall-through to an unauthenticated stream	Pass
Register an account, then inspect what was stored	Signup path	Password never stored in plain text; two signups of the same password produce different hashes	Pass
Submit a signup body carrying extra tenant and role fields	Signup path	The extra fields do not become attributes; the tenant comes from server configuration and only the first account is an administrator	Pass
Read analysis results as tenant B that were written by tenant A	Tenant isolation	No rows returned; every analysis-result query is tenant-scoped	Pass
Request the metered cloud backend as a privacy-locked tenant	Data-residency policy	Refused rather than silently downgraded; the global toggle and the environment default are both overridden for that tenant	Pass
Request the metered backend while the policy table is unreadable	Data-residency policy	Denied. The check fails closed, and a broken cache falls back to the environment default rather than to permission	Pass
Enqueue a job by either entry path and inspect the envelope	Privacy lock coverage	Both paths stamp the resolved backend into the envelope, and both stage workers prefer the stamped value over the global toggle	Pass
Exceed the configured request rate on one identity	Rate limiting	Blocked at the limit, released on window rollover, and unaffected by other identities' traffic	Pass
Embed an instruction inside retrieved third-party comment text	Agent prompt-injection containment	Wrapped as untrusted data with the producing tool recorded as provenance; the payload can neither close the wrapper early nor open a nested one	Pass

Test scenario	Surface tested	Expected behaviour	Result
Ask the agent to reveal its system message	Agent prompt-injection containment	Refused by policy, and the policy is asserted to be attached to every one of the nine agents' system prompts	Pass

Two of these rows are worth reading twice, because they are the cases where a system usually fails quietly rather than loudly. The first is the unreadable policy table. A residency check that cannot reach its policy store has two options, and the tempting one – carry on with the previous answer – converts a storage hiccup into a data-egress incident that leaves no trace. The test named `test_an_unreadable_policy_table_denies_rather_than_permits` pins the other choice. The second is the explicit backend request from a locked tenant, which is **refused rather than downgraded**. Silently substituting the self-hosted backend would honour the policy while lying to the caller about which model answered, and a caller who believes they received a cloud answer will attribute its quality to the wrong model.

What this subsection does not claim is a penetration test. These are white-box tests written by the same members who wrote the code they guard, so they establish that the checks behave as specified under the conditions the authors imagined. An adversarial review by a party with no knowledge of the implementation is named in Section 6.3 as the work that would close the remaining distance.

4.2.4 Routing behaviour and the cost split

The routing measurement runs the **real stage functions** over the corpus – normalisation, text and image analysis, fusion, result construction and the gate itself – with no options set, and reports the router's own counters. It is run twice with the same code and two different Stage-1 engines, because the comparison between them is the result.

Table 4.8: Measured routing and large-model call split, 43-post captioned corpus, cold cache, 25 comments per batch.

Quantity	Keyword-stub Stage 1	LLM Stage 1 (as shipped)
Posts routed for post-level work	32 of 43 (74 percent)	7 of 43 (16 percent)
Post-level calls (summary, insight, post type, comment summary)	124 (15.3 percent)	22 (4.2 percent)
Comment-level calls	689 (84.7 percent)	507 (95.8 percent)
- Stage-1 comment labelling, all 43 posts, which the gate cannot reduce	368	368
- Stage-2 comment stance, routed posts only	321	139
Total calls per corpus run	813	529
Share of all calls the routing gate governs	54.7 percent	30.4 percent

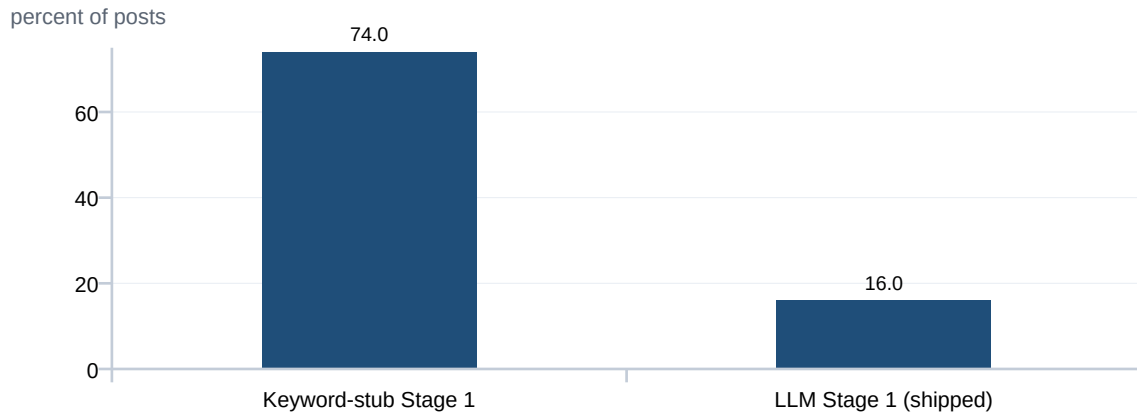


Figure 4.7: Posts routed for post-level LLM work, by Stage-1 engine.

Reading the chart. Identical router code, identical corpus, identical thresholds – only the Stage-1 engine differs. The keyword stub confidently types 27 of 43 posts and therefore escalates the remaining 74 percent; the LLM Stage 1 types 42 of 43 and escalates 16 percent. The correct reading of this chart is therefore that the routing rate measures Stage-1 quality, not cost efficiency: it falls as Stage 1 improves, which means a rising rate should be alerted on as a quality regression rather than as a cost regression.

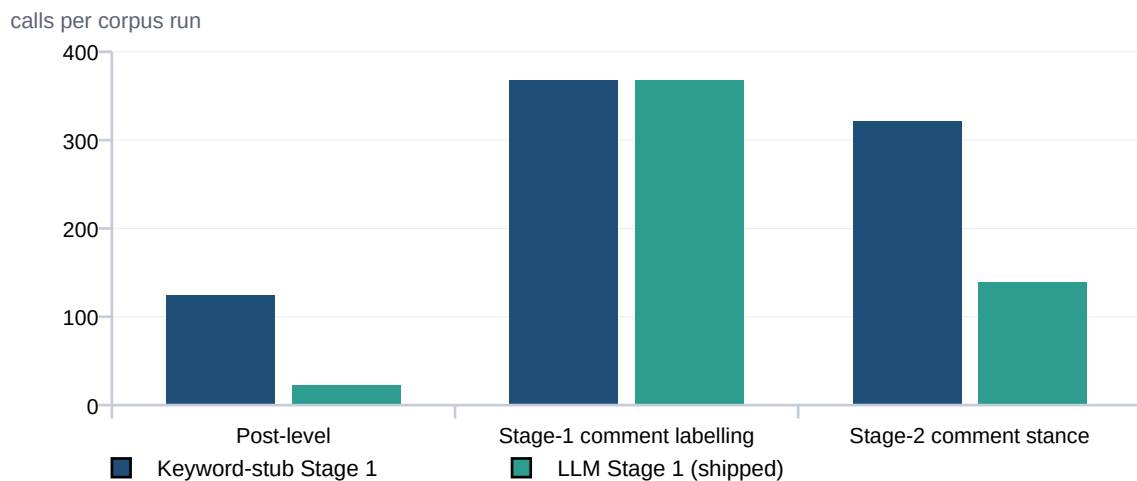


Figure 4.8: Where the large-model calls actually go.

Reading the chart. The middle pair of bars is the finding. Stage-1 comment labelling runs for every post whether the gate routed it or not, so it is identical in both configurations and the routing gate cannot reduce it at all. Improving Stage 1 halves the post-level bar and cuts the Stage-2 stance bar by more than half, but leaves the largest single bar untouched, which is why the gate's share of total spend falls from 54.7 to 30.4 percent while the absolute bill falls only from 813 calls to 529. What sets the bill is how many comments exist.

Three conclusions follow, and the first of them contradicted the project's own headline claim.

First, the routing rate is not the cost story. Between 85 and 96 percent of calls are comment-level. The initial claim was that cheap models perform ninety to ninety-five percent of the work and a selective large model handles the rest, and that this was the central cost-control idea. That assertion was never measured; when it was measured, the gate had in fact been routing one hundred percent of posts for a period because of two field-name defects, and after those were repaired the gate still governed only a minority of spend. The claim that survives contact with the

measurement is narrower and is the one this report makes: **cheap NLP filters which comments and which posts deserve a large model**. Both halves matter, and the second is now the bigger one.

Second, improving Stage 1 shrinks the gate's importance rather than the bill. This result runs counter to intuition. A better Stage 1 types more posts confidently, so fewer route, so the gate decides less; meanwhile comment labelling runs unchanged.

Third, the gate selects the right posts. Under the shipped configuration the 7 routed posts hold 3,404 of the 8,713 comments that reach a model – the gate is escalating the large, contentious threads, which is the behaviour the design intends and which a routing rate quoted on its own cannot show.

One earlier estimate was also corrected by measurement and is recorded because the correction was large. Filtering emoji-only comments out of model batches was expected to save roughly seventeen percent of the comment bill. Emoji-only comments are **2.8 percent** of the corpus. The seventeen-percent figure was the share taking the fast path, which conflates emoji-only reactions with short *text* comments, and a short text comment still carries text that a model can read. Filtering emoji is still correct, because they are pure waste, but it is not a cost lever.

4.2.5 Retrieval evaluation

Retrieval is the one axis this project evaluates against a ground-truth set, and the numbers were obtained with a 32-query known-item set derived from each target post's own topics and entities. Each query has exactly one marked relevant post. The same set was run before and after real embeddings were enabled, so the two recall columns in Table 4.9 differ only in whether the stored vectors are semantic or hashes.

Table 4.9: Retrieval evaluation over 32 known-item queries, $k = 10$, on the 50-post corpus.

Configuration	recall@10 (hash vectors)	recall@10 (real)	MRR@10 (real)	nDCG@10 (real)
vector (dense only)	0.1875	0.6250	0.2869	0.3667
lexical (full text fused with trigram)	0.3750	0.7500	0.6503	0.6748
chunk (dense over passage chunks)	not measured	0.6562	0.3014	0.3846
hybrid (dense fused with lexical)	0.5312	0.8750	0.6108	0.6759
chunked (chunk fused with full text and trigram)	not measured	0.8750	0.6482	0.7041

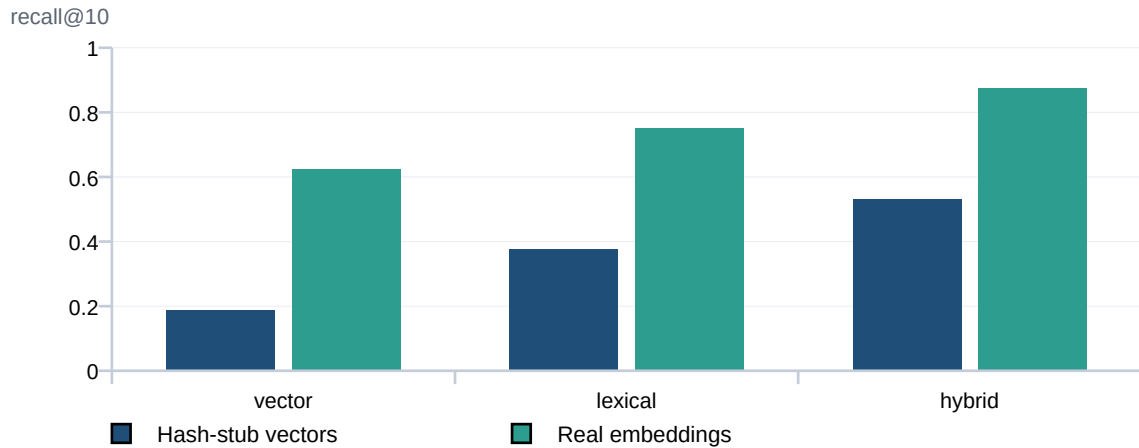


Figure 4.9: Recall@10 before and after real embeddings.

Reading the chart. The leftmost dark bar is the most useful number in this chart. Returning ten random posts from a corpus of fifty scores 0.20 by construction; hash-seeded stub vectors score 0.1875, which is statistically indistinguishable from chance. That claim had been prose in the project's documentation for a long time, and this is the measurement behind it. Note also that the lexical arm moved without any vector being involved – that movement was a bug fix rather than a benefit, and it is explained below.

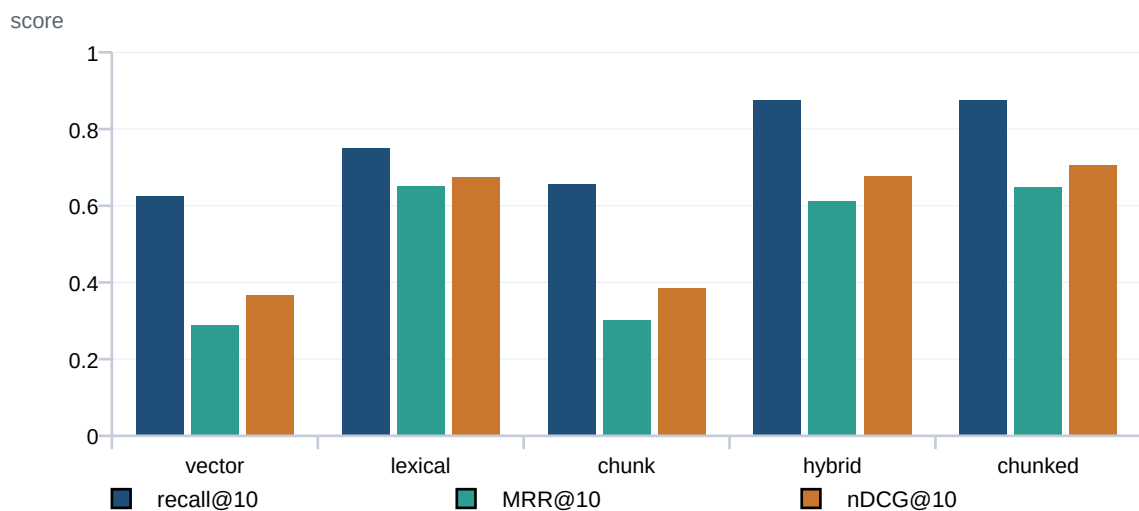


Figure 4.10: Retrieval quality by configuration, real embeddings.

Reading the chart. Recall and ranking separate here. Hybrid and chunked tie on recall at 0.8750, so chunking yields no gain on the metric that matters when the whole top-k is handed to a generator; the gain shows up in ranking instead, with MRR rising from 0.6108 to 0.6482 and nDCG from 0.6759 to 0.7041. That is the expected result for this corpus: only 14 of 50 posts exceed 600 characters, so 50 posts yielded 86 chunks and most are a single chunk identical to the caption. Note also that lexical alone posts a higher MRR than hybrid, which is an artefact of how the queries were built and is dissected in the next figure.

Three findings come out of Table 4.9, and the third is a defect rather than a result.

Hash vectors were indistinguishable from chance. Real embeddings are worth 3.3 times on the dense arm alone, from 0.1875 to 0.6250, and the end-to-end configuration moved from 0.5312 to 0.8750 – a 65 percent relative gain. This matters beyond retrieval, because the same vectors drive

the clustering that the report layer summarises: clustering hash vectors groups posts at random and then pays a large model to describe the grouping.

Chunking improves ranking rather than recall on this corpus, and the reason belongs with the finding: this corpus consists predominantly of short captions.

The lexical arm's improvement was a bug fix, not a benefit. The lexical column moved from 0.3750 to 0.7500 with no vector involved, because the full-text half of that arm had never fired at all: the query builder conjoined every term, so an eight-token question demanded all eight words in one caption and matched **zero** rows across the entire query set. Everything previously credited to "full text plus trigram" was trigram alone, and the index built for full-text search was dead weight. This is recorded because it is the same failure shape as the routing defect and the identifier mismatches: a component that produced no output, reported no error, and was covered by a passing test.

A stratified view of the same 32 queries changes how the headline table should be read, and it does so in the embeddings' favour. Splitting the queries by the lexical overlap between the query and its target post gives Table 4.10.

Table 4.10: Recall@10 stratified by query-to-target lexical overlap.

Overlap bucket	Queries	lexical	vector	chunked
overlap 0.5 or above	12	1.000	0.583	1.000
overlap below 0.5	20	0.600	0.650	0.800
- of which overlap exactly 0	8	0.000	0.500	0.500

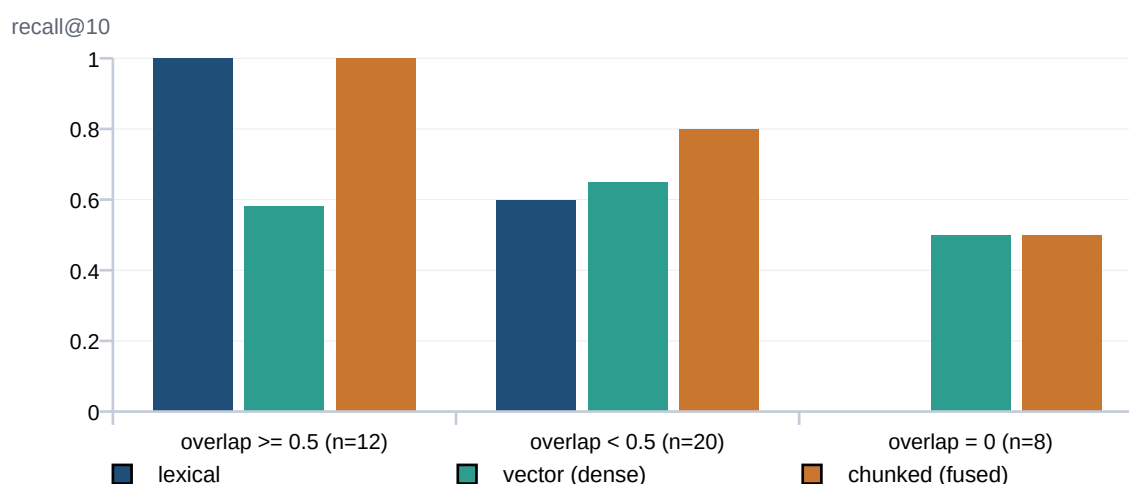


Figure 4.11: Recall@10 stratified by query-to-target lexical overlap.

Reading the chart. This is the strongest argument in the project for fusion, and it is invisible in the aggregate table. Twelve of the thirty-two queries are near-verbatim lookups on which a lexical match cannot fail, which is why lexical outscores hybrid on MRR overall. On the eight genuinely leak-free queries – where the query shares no vocabulary with its target – lexical scores exactly zero and the dense arm is the only retriever that returns the target at all. The fused configuration ties lexical on the easy bucket and beats it 0.800 to 0.600 on the hard one. Neither arm dominates, which is precisely the premise the fusion design rests on.

Four caveats bound every retrieval number above, and they belong in the same breath as the figures rather than in a footnote.

- C1. **They are lower bounds.** The set marks exactly one relevant post per query, so every other on-topic post the retriever returns is counted as a miss.
- C2. **They compare retrievers against each other on one set.** They are not absolute quality figures and should not be read as such.
- C3. **This is not human relevance judgement.** No query in the set has been verified by a person; each was derived from its target post's own analysed topics and entities, which is exactly the derivation that produces the overlap leakage Table 4.10 stratifies.
- C4. **The set is small where it matters most.** The stratum on which the argument for fusion rests contains eight queries. Paired significance tests over the full set put every lexical-versus-fused comparison above the conventional threshold – 0.375 at k equal to 1, 1.000 at k equal to 3, and 0.125 at k equal to 10, the last from a clean four-to-nothing split. **The direction is consistent; the set is too small to call it.** Reporting recall at 1 alongside recall at 10 also helps, and it complicates the story: at k equal to 1 the dense arm *displaces* correct top hits, with lexical retrieving 19 of 32 against the fused configuration's 16.

4.2.6 Label provenance and ensemble coverage

A sentiment chart that cannot say what produced its numbers is not evidence, so the pipeline records provenance per comment and per post. Measuring that provenance produced the result in Figure 4.12.

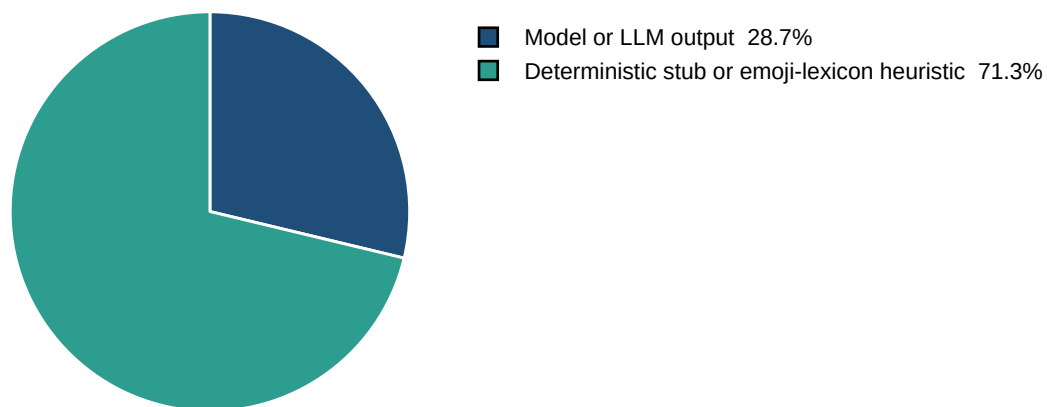


Figure 4.12: Provenance of per-comment sentiment labels in the shipped configuration.

Reading the chart. Measured over all 10,272 comments with the repository's default configuration. The larger slice is not sentiment: it is partly a fourteen-word emoji-and-lexicon rule, and mostly a deterministic function that derives a label from a hash of the first fifty characters of the text. That function is reproducible and it is not sentiment. Before provenance was recorded, the same run reported 8,513 model inferences in a configuration where zero models had been loaded, because the classifier tagged every substantive comment as model output regardless of whether a model existed. The remedy was not to hide the slice but to label it, so that the accurate statement, that 28.7 percent of labels are model output with the remainder itemised, becomes available.

Four reporting mechanisms follow from that measurement and are now permanent parts of the output contract. Each comment carries a **method** naming the engine that actually ran and a **kind** distinguishing emoji, short and substantive text. Each post carries a **provenance block** stating what share of its labels were model inferences. Each analysed comment carries the **number of voters that actually spoke** alongside its label and agreement figure. Finally, a comment that no model read is reported as **uncertain at zero voters**, never as neutral.

4.2.7 Comparative analysis

Table 4.11 compares the shipped configuration against the two natural baselines on the quantities that were measured. The comparison is deliberately restricted to measured quantities; an accuracy column is absent because labelling accuracy sits outside the evaluation scope, which Section 4.3.4 states.

Table 4.11: Comparative analysis of the shipped configuration against its baselines.

Quantity	All-cheap baseline (keyword stub Stage 1)	Shipped hybrid configuration	LLM-everywhere baseline
Posts routed for post-level LLM work	32 of 43 (74 percent)	7 of 43 (16 percent)	43 of 43 by definition
Total LLM calls per corpus run	813	529	not run; bounded below by one call per post plus one per comment batch
Share of calls the gate governs	54.7 percent	30.4 percent	zero, the gate is bypassed
Per-comment label provenance	71.3 percent heuristic or stub	71.3 percent heuristic or stub in the default configuration; 100 percent model output when the ensemble roster is provisioned	100 percent model output
Independent verdicts per analysed comment	1	up to 8, with abstention	1
Retrieval recall@10, end-to-end configuration	0.5312 with hash vectors	0.8750 with real embeddings	unchanged; retrieval is orthogonal
Marginal token cost, self-hosted backend	zero	zero	zero, but GPU occupancy scales with comment count
Bangla and Banglish handling	keyword lists and script ratios	multilingual encoders plus two Bangla fine-tunes plus a context-aware pass	whatever the single model provides
Can the output state what produced each label	no	yes	trivially, one producer

Four readings follow from the table, and the first is the one that costs the project its simplest claim. Moving from the all-cheap baseline to the shipped configuration cuts total calls from 813 to 529, a reduction of roughly a third – real, but nowhere near the order-of-magnitude saving that a routing rate falling from 74 to 16 percent appears to promise. The reason is in the row beneath:

the gate's share of calls falls from 54.7 to 30.4 percent, so as the gate routes fewer posts it also governs less of what remains. The routing rate and the bill move in the same direction but not at the same rate, and quoting the first as evidence about the second is the error this table was constructed to expose.

Second, the LLM-everywhere column is deliberately left partly empty, and the empty cells are informative. That baseline was not run, so rather than estimate it the table states the bound that follows from its definition: at least one call per post plus one per comment batch. An estimate would have been easy to produce and impossible to defend, and a column of plausible numbers beside two columns of measured ones would have invited exactly the comparison that the evidence does not support.

Third, two rows favour the shipped configuration on grounds that have nothing to do with cost. Independent verdicts per comment rise from one to as many as eight with abstention, and the output can state what produced each label. Both are properties of the ensemble rather than of the router, and both survive regardless of how the routing argument settles – which matters, because it means the contribution does not rest on the cost claim that measurement weakened.

Fourth, one row is deliberately marked orthogonal. Retrieval recall is unchanged by the choice of Stage-1 engine, because retrieval reads persisted analysis rather than participating in the cascade. It appears in the table only to record that the improvement from 0.5312 to 0.8750 came from replacing hash vectors with real embeddings, and not from anything the router does.

4.3 Results and Discussion

This section presents the results obtained from the fully implemented system: the delivered software module by module, the degree to which each meets the objectives set out in Section 1.3 and the requirements of Section 3.1.3, and then what the measurements of Section 4.2 do and do not establish. The results divide into what was delivered, what was established by measurement, what was disproved by measurement, and what the evaluation deliberately does not cover. All four belong in this section.

4.3.1 The delivered system, feature by feature

This section is a walkthrough of the delivered software. Every figure in it is a screenshot or a terminal capture taken from the running system on 29 August 2026, on the development tier of Figure 4.1, against the corpus of Section 4.2.1 already loaded into the datastores. Nothing in these figures is a mock-up: where a panel shows a number, that number is what the system computed, and where a panel shows a warning, the system raised it. Table 4.12 indexes every delivered feature against the figure that shows it, so a reader who wants to check a specific capability can go straight to it.

One caveat applies to every number visible in these screenshots, and it is stated once here.

The walkthrough deployment is configured for demonstration rather than for measurement: Stage-1 runs with its own LLM path enabled and the comment caps lifted, which is not the shipped configuration under which Section 4.2.4 measures routing. The routing rate visible on the Overview tab is therefore 40 percent, not the 16 percent that Section 4.2.4 reports, and the

two are not in conflict – they are two different configurations of the same router. Where a figure in this section carries a number that Section 4.2 also reports, the measured figure is the one in Section 4.2.

1. Bringing the stack up, and the API surface beneath it

The whole system is started by one command. Figure 4.13 is the transcript: the orchestrator syncs dependencies, brings up the four datastores in Docker, waits on their health checks, creates the ClickHouse tables, verifies that the sentence encoder produces real vectors rather than hash stubs, resets the consumer groups so nothing auto-processes on start, and then launches eleven host processes – five stage workers, the API, three MCP servers, the agent service and the dashboard – reporting the log file and port of each. The preflight in the middle of that sequence is not decoration: an encoder that silently falls back to hash vectors is the single defect that most damaged this project's retrieval numbers, and the check exists so that the failure is announced at start-up rather than discovered in an evaluation.

```
uv run run_all.py --with-agents

[run_all] uv sync --extra embeddings ... (config requires a real encoder)
Resolved 199 packages in 0.59ms
Checked 172 packages in 0.98ms
[run_all] starting datastores: postgres, redis, clickhouse, minio
Container deploy-redis-1 Running
Container deploy-clickhouse-1 Running
Container deploy-postgres-1 Running
Container deploy-minio-1 Running
[run_all] waiting for Postgres + ClickHouse to be healthy ...
[run_all] datastores healthy
[run_all] creating ClickHouse tables ...
[run_all] Ollama is up
[run_all] preflight: verifying the sentence encoder produces real vectors ...
[run_all] preflight: encoder OK – paraphrase-multilingual-mpnet-base-v2 768
[run_all] quieting the pipeline (reset consumer groups → 0 waiting / 0 in-flight) – stream data kept, nothing auto-processes ...
[run_all] started stage1 (pid 74169) → /tmp/stage1.log
[run_all] started router (pid 74170) → /tmp/router.log
[run_all] started stage2 (pid 74171) → /tmp/stage2.log
[run_all] started assembler (pid 74172) → /tmp/assembler.log
[run_all] started ingestion (pid 74173) → /tmp/ingestion.log
[run_all] started api (pid 74174) → /tmp/api.log
[run_all] waiting for the API to answer ...
[run_all] API healthy on :8001
[run_all] started analytics_mcp (pid 75012) → /tmp/analytics_mcp.log
[run_all] started retrieval_mcp (pid 75013) → /tmp/retrieval_mcp.log
[run_all] started ingest_mcp (pid 75014) → /tmp/ingest_mcp.log
[run_all] started agents (pid 75015) → /tmp/agents.log
[run_all] agent layer started (analytics :8110, retrieval :8101, ingest :8102, agents :8010)
[run_all] started dashboard (pid 75016) → /tmp/dashboard.log
[run_all] dashboard live → http://127.0.0.1:8080 (log in with API key: demo)

[run_all] =====
[run_all] Smart Layer is up.
  • API      → http://127.0.0.1:8001/v1/health
  • Dashboard → http://127.0.0.1:8080 (log in with API key: demo)
  • Agents   → POST http://127.0.0.1:8001/v1/agents/query
  • No posts auto-loaded – load them yourself when ready:
    - UI: Dashboard → Posts tab → drop posts_with_details.json onto the upload box
    - API: POST http://127.0.0.1:8001/v1/posts/upload with body {"posts": [...]}
      (ready-to-run command in docs/easy_run.md $D; or re-run without --manual-load to auto-load)
  • Logs      → /tmp/<service>.log (add --log to stream them here live)
    in the browser: dashboard → Logs (Ctrl+`), or GET http://127.0.0.1:8001/v1/logs
[run_all] Press Ctrl-C to stop everything this script started.
[run_all] =====
```

Figure 4.13: Starting the whole system with one command.

How to read it. A terminal transcript, read top to bottom in start-up order. The cyan tag at the left of a line is the orchestrator speaking about itself; uncoloured lines are the tools it drives. The four container lines are the datastores; the two preflight lines are the encoder check; the block of "started" lines is the eleven host processes, each with its process id and its log file. The green summary block at the foot is what the operator is left with: the API, the dashboard, the agent endpoint and where the logs are.

Figure note. The stack is one command and under a minute on a laptop, which is what makes the system demonstrable rather than merely deployable. The run shown was started with `--no-load`, because the corpus was already analysed in the datastores; the same command without that flag ingests the 50 posts as its final step.

Everything the dashboard does, it does over the HTTP API, and that API is self-describing. Figure 4.14 shows the generated OpenAPI surface: 51 documented paths in fourteen tag groups – health, auth, ingestion, analysis, search, reports, chat, agents, pipeline, events, logs, config, usage

and system telemetry – carrying 63 method-and-path pairs in total. Two properties are worth reading off it. The first is that authentication is visible in the specification: a padlock marks every route that requires a credential, and the handful without one are exactly the routes a caller must be able to reach before it has a credential – the two health probes, the metrics endpoint, the login screen's policy endpoint, and signup and token themselves. The second is that the streaming endpoints are ordinary documented routes rather than a side channel, which is what allows the ticket mechanism of Figure 3.7 to be tested like any other endpoint.

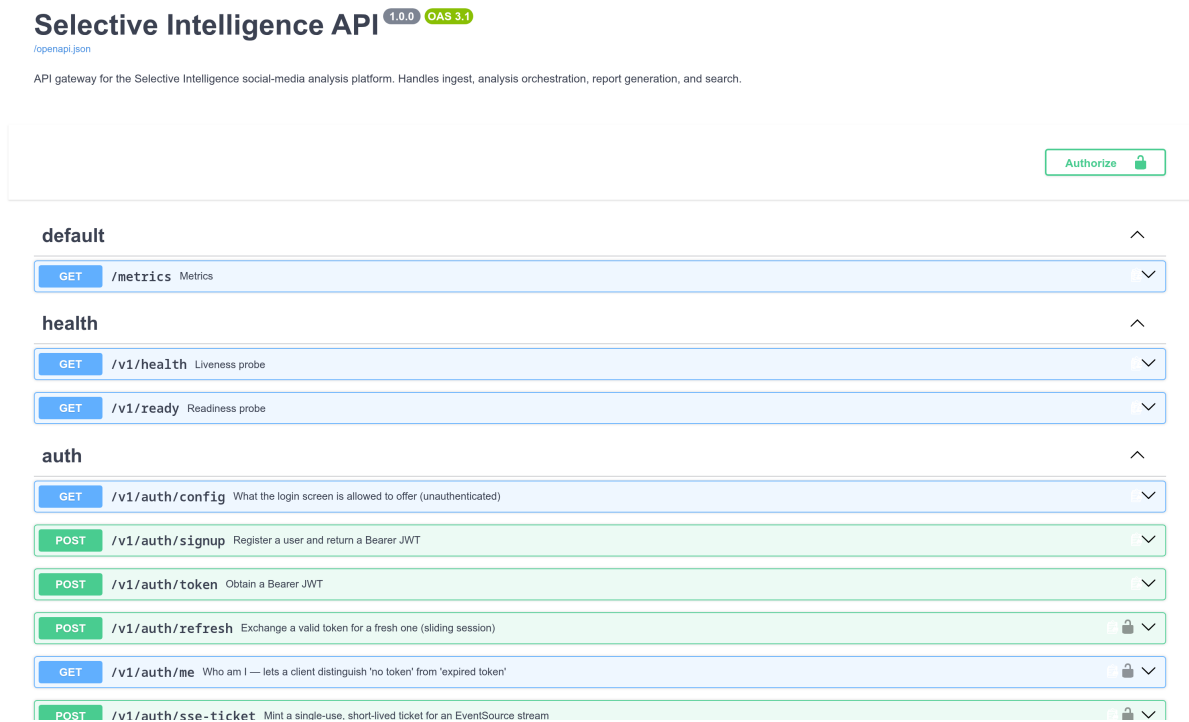


Figure 4.14: The generated API surface, with authentication marked per route.

How to read it. The interactive OpenAPI page served by the API itself, read as a list of tag groups. Each row is one route: the coloured chip is the HTTP method, the monospaced path is the route, and the sentence beside it is the docstring the handler carries in the source. A padlock at the right marks a route that requires a credential. The groups shown are the first three of fourteen.

Figure note. The specification is generated from the handlers, not maintained beside them, so a route that exists is documented and a route that is documented exists. The per-route padlock makes the authentication policy readable without reading the code.

2. The corpus at a glance, and where the money went

The Overview tab of Figure 3.9 is the first surface an operator sees. Its lower half is the part that matters to this project's central argument, and Figure 4.15 shows it: a per-lane table of large-model spend, a per-model token table, and a projection of what the same corpus would cost on a metered frontier API. In the run shown, the five lanes divide 941 calls as post 10, comment 284, Stage-1 83, interactive 523 and agent 41; by tokens, the comment lane takes 49 percent and the interactive lane 35 percent, while the post lane – the one the router's gate governs – takes under one percent. That is the same finding Section 4.2.4 reaches from the router's own counters, arrived at here from the usage ledger instead, and it is the reason this report treats the routing rate as a partial cost control rather than as the cost control.

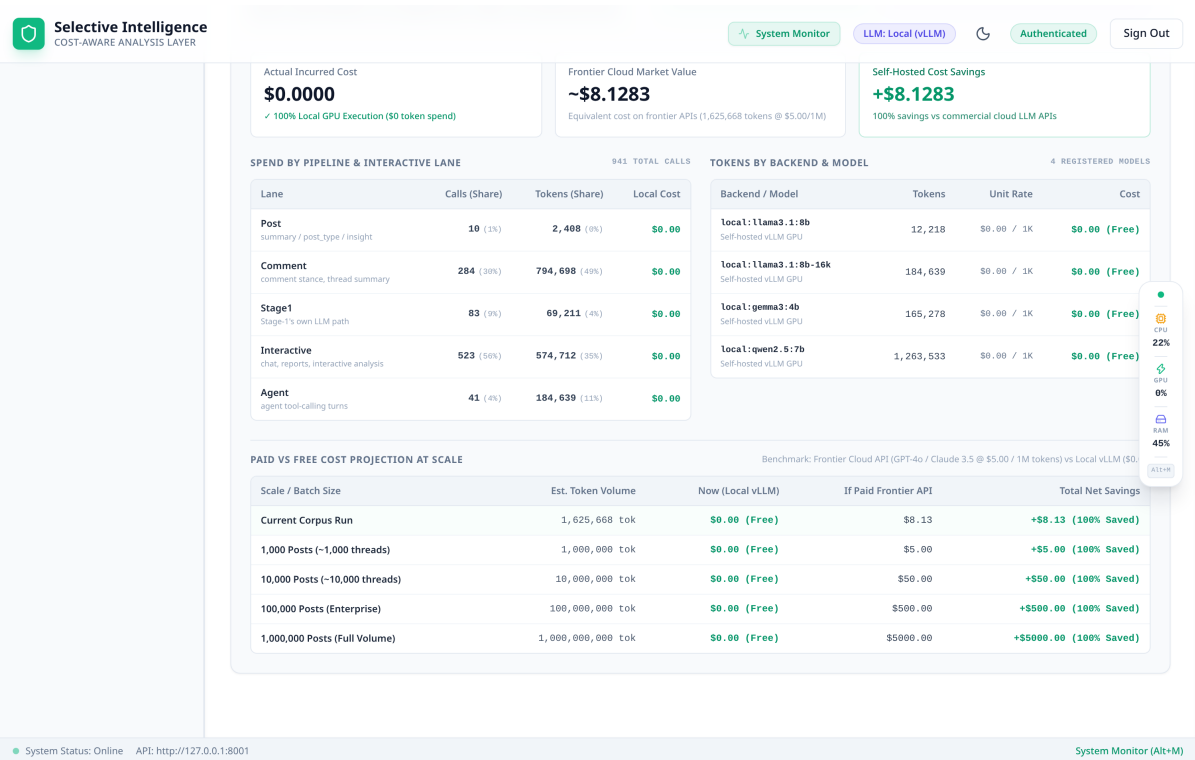


Figure 4.15: Token economics: spend by lane, by model, and projected to larger corpora.

How to read it. The lower half of the Overview tab, in four blocks. Across the top are three cards: cost actually incurred, the equivalent cost on a metered frontier API, and the difference. Below left is the lane table – one row per lane, with calls and tokens each followed by their share of the total. Below right is the same spend broken out by backend and model, with the unit rate that produced the cost column. At the foot, the projection table extends the measured token volume from the current corpus to one million posts at the benchmark rate.

Figure note. The comment lane and the interactive lane hold 84 percent of the tokens between them; the post lane, which is the only lane the routing gate controls, holds under one percent. The dollar column reads zero because the deployment is self-hosted – the projection column is what the same work would have cost on a metered API.

3. The posts explorer and the per-comment ensemble

Figure 4.16 is the explorer over the analysed corpus. Three things in that table are design decisions rather than defaults. The comment column shows three numbers – analysed, stored and platform-reported – so that the sampling ratio is visible in the table itself rather than in a detail view; the watchlist banner above the table states how many posts matched a rule before the operator has clicked anything; and the upload zone is on the same tab as the results, because ingesting a file and inspecting what came out of it are one task.

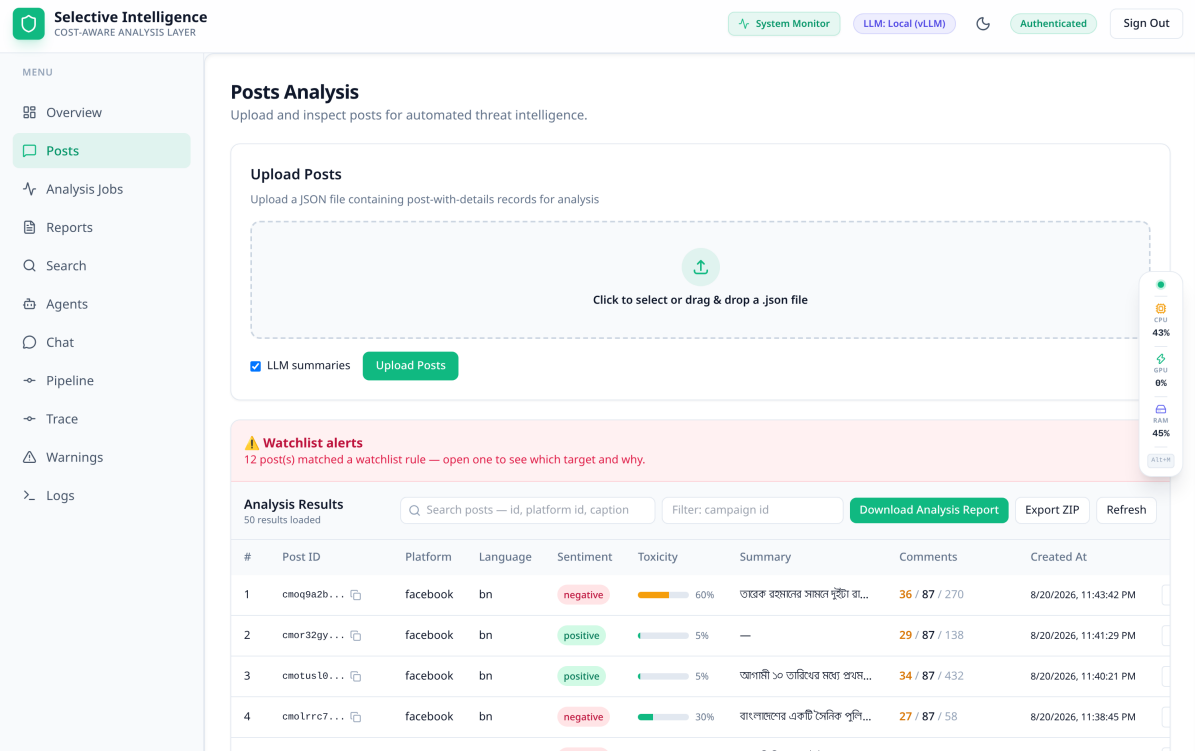


Figure 4.16: The Posts tab: upload, watchlist banner, and the analysed corpus.

How to read it. A table view with three regions. At the top is the upload zone, which takes a post-with-details JSON file, with a toggle that decides whether the upload also asks for LLM summaries. Below it is the watchlist banner, which counts the posts that matched a rule. The table itself is one row per post: identifier with a copy control, platform, language, sentiment pill, a toxicity meter, the generated summary, the three-part comment ratio, and the timestamp. Clicking any row opens the detail modal of Figure 4.17.

Figure note. The three-part comment ratio – 36 analysed of 87 stored of 270 reported on the first row – is the coverage arithmetic of Section 4.2.1 rendered per post, so no chart in this dashboard can imply a completeness the scrape does not have.

Opening a row gives Figure 4.17. The badge row carries the system's own verdict – negative at -0.80, confidence 88 percent, post type political, primary emotion anger, toxicity 60 percent – together with an amber coverage warning stating that only 87 of the platform's 270 reported comments were ever scraped, which is 32 percent. The four cards at the foot put the system's label beside the upstream platform's, and for this post the two disagree: the pipeline reads the caption as negative at -0.800 where the upstream baseline reads 0.620 positive. That disagreement is visible only because the baseline is preserved rather than overwritten, which is the requirement FR3 exists to satisfy. The summary panel carries its provenance on its face – the

STAGE1 badge naming the stage that wrote it, and a grounding line naming the caption as what it was written from – so a reader never has to ask which model produced the prose or what it had seen.

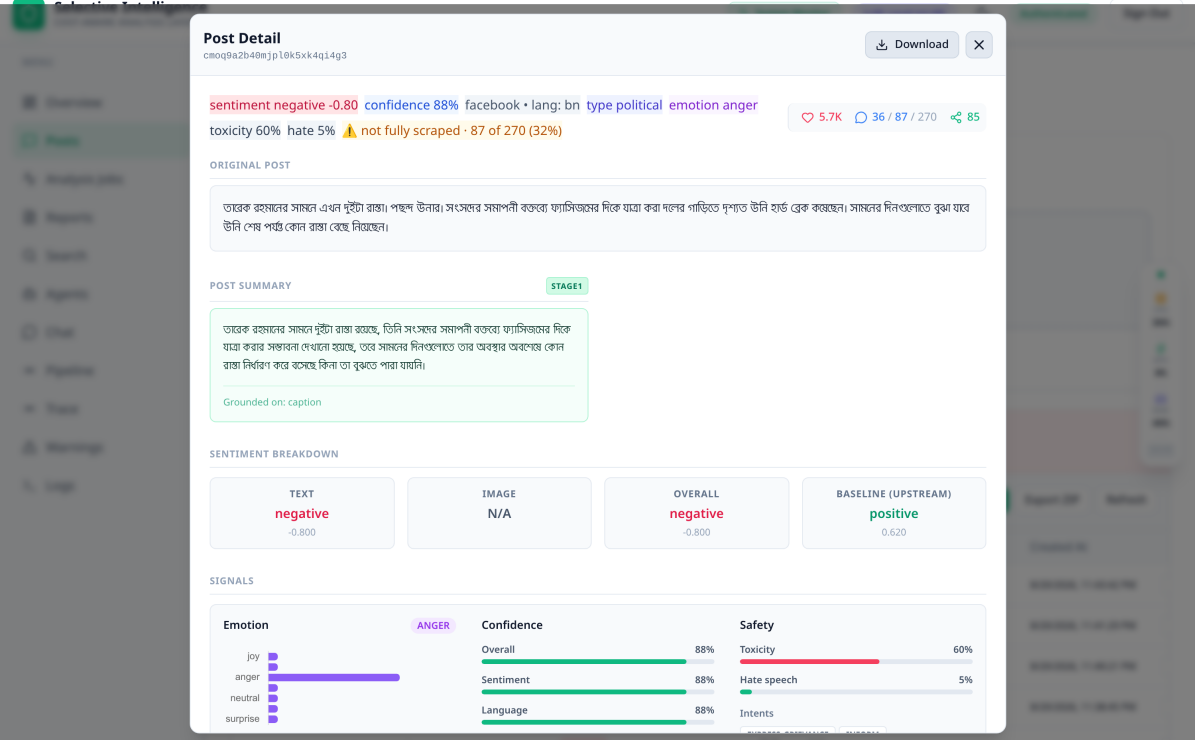


Figure 4.17: Post detail: provenance, coverage, and the system's label beside the upstream baseline.

How to read it. A modal over the dimmed table, read downward. The badge row at the top carries sentiment and its score, confidence, platform, language, post type, primary emotion, toxicity, hate score, and the amber coverage warning, with the engagement and three-part comment counts to its right. Then the original caption, then the generated summary with its stage badge and grounding line. The four cards beneath are the sentiment breakdown: text, image, overall, and the upstream platform's own baseline. The signals row at the foot gives the emotion vector, the three confidence bars and the safety scores.

Figure note. The upstream baseline is kept and displayed rather than overwritten, which is what makes a disagreement between the platform's score and this system's score visible instead of silent – here 0.620 positive against the pipeline's -0.800 negative. The image card reads N/A rather than neutral, because no image bytes were reachable; the coverage warning is attached to the post, not to a footnote.

Scrolling that modal reaches the comment thread, and Figures 3.10 and 4.18 show what the eight-labeller ensemble of Section 3.2.3 looks like when it is rendered honestly. Each comment carries its eight verdicts side by side – the LLM stance pass and the seven cheap heads – together with the combined label, the score, and the number of voters that agreed. The three comments in Figure 4.18 agree 6 of 8, 5 of 8 and 4 of 8; in the second of them ModernBERT and mBERT vote positive while five heads vote negative and BanglaBERT votes neutral. A single fused label would have shown two confident negatives and a neutral, and hidden all of that.

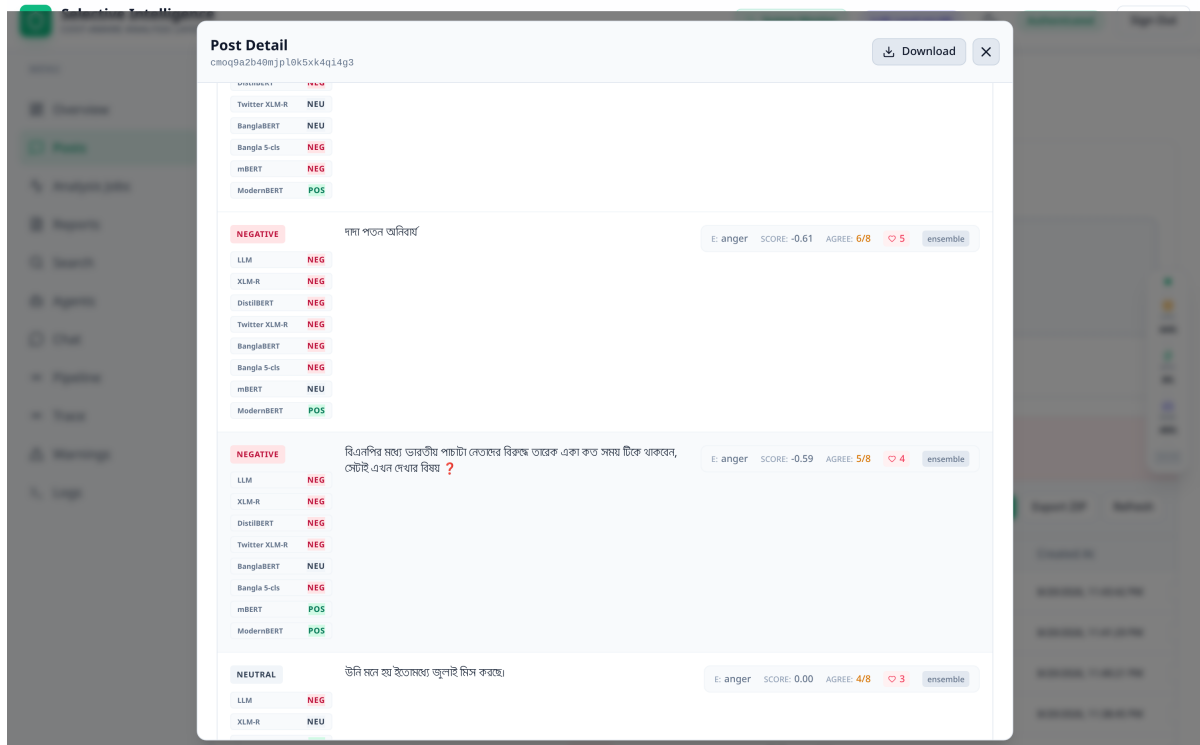


Figure 4.18: Three comments, twenty-four votes: what per-labeller disclosure looks like.

How to read it. Three consecutive comment rows from the detail modal. Each row has the combined label at the left, then the eight individual verdicts as a vertical list naming the labeller and its vote, then the comment text, then a metric strip: the dominant emotion, the combined score, the agreement fraction, the reaction count, and the source tag reading "ensemble".

Figure note. The agreement fraction is on every comment, so a chart built from these labels can always be asked how much of it rests on a split vote. The second row is the interesting case: two heads vote positive where five vote negative and one neutral, and the combined label reports 5 of 8 rather than presenting itself as certain.

4. Running an analysis, and watching it run

Re-analysis is triggered from the Analysis Jobs tab or from the API, and both produce the same job record. Figure 4.19 shows one in flight: six posts, one finished, with the per-job controls that the job model supports – track, stop, resume, re-run and delete. Resume exists because a cancelled job leaves analysed posts behind and re-queuing only what was never finished is cheaper than starting again; stop exists because a run that is producing bad output should be stoppable without losing what it has already produced.

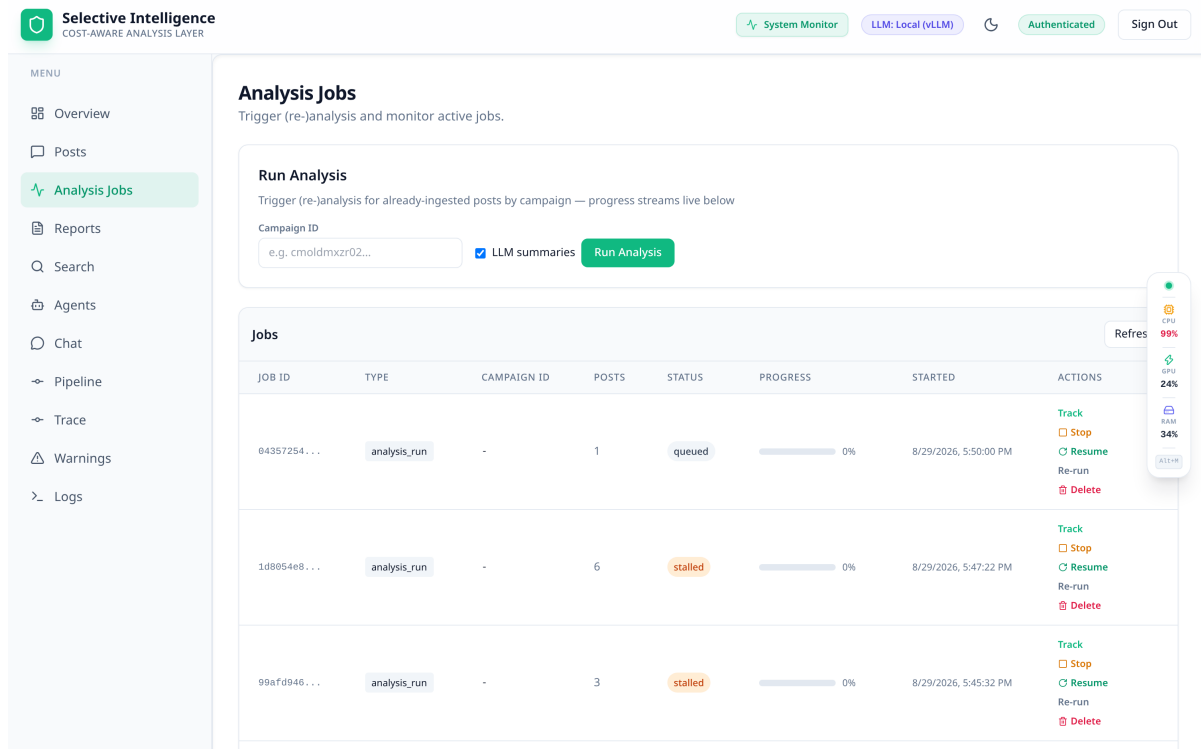


Figure 4.19: An analysis job in flight, streamed over Server-Sent Events.

How to read it. Two panels. The upper one submits a run: a campaign identifier, a toggle for LLM summaries, and the button. The lower one lists jobs, newest first: identifier, type, campaign, post count, status pill, a progress bar with its percentage, the start time, and the per-job controls. The top row is the run in progress at 17 percent; the rows beneath it are earlier runs, each still offering its report for download.

Figure note. Progress is pushed over a Server-Sent Event stream rather than polled, and the percentage is computed from the assembler's own completion counter, so it advances when a post is durably stored rather than when it is dispatched.

While that job runs, the Pipeline tab of Figure 4.20 shows where the work actually is. The five stage cards carry in-flight, backlog and dead-letter counts read from the Redis consumer groups, so the queue depth is the system's own state rather than an estimate. In the capture, Stage 1 holds one item in flight with seven behind it and the downstream stages are idle – the shape a cascade produces when the cheap stage is the throughput limit, which is the intended shape.

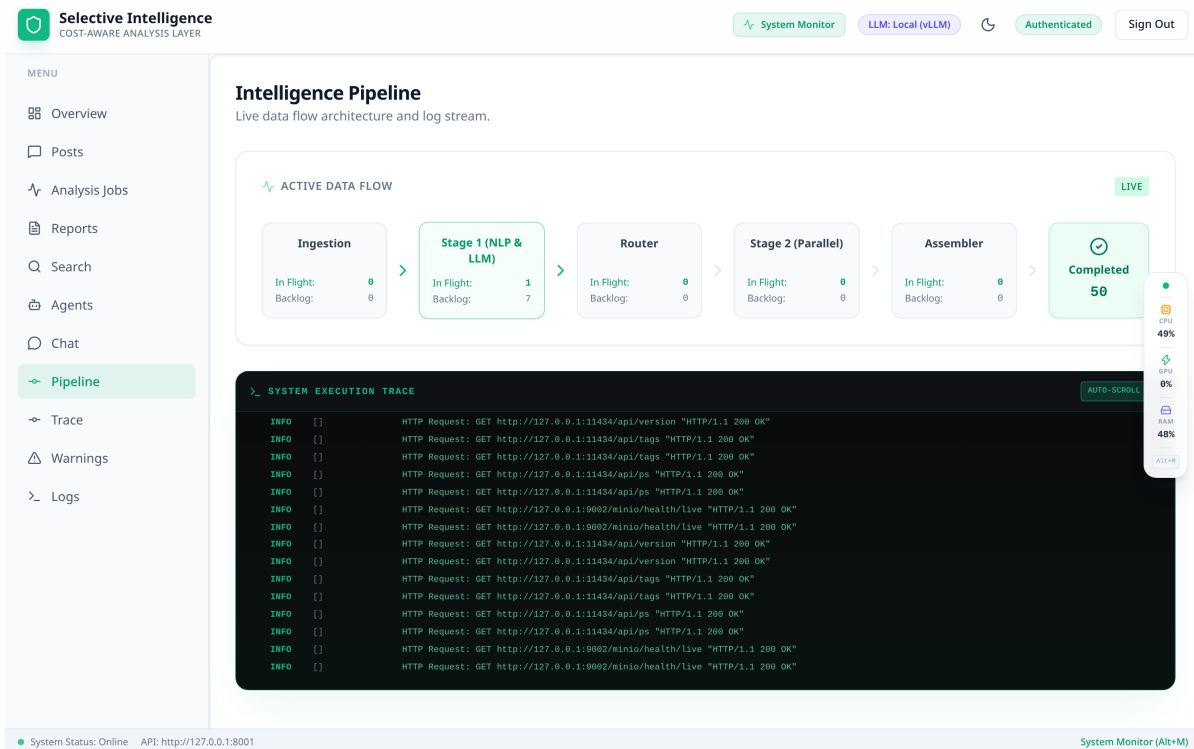


Figure 4.20: The pipeline under load: in-flight and backlog per stage.

How to read it. A horizontal rail of five stage cards in pipeline order, each showing items in flight and backlog depth, with the currently active stage outlined. The card at the right is the completion counter. Beneath the rail, the dark panel is the live execution trace, streaming the same structured events the log console shows.

Figure note. The counters come from the Redis consumer groups themselves, so a backlog shown here is a backlog that exists. Stage 1 holding the queue while the later stages idle is the cascade working as designed: the cheap stage runs on everything, and the expensive ones run on a subset.

The Logs tab, Figure 4.21, is the same event stream in raw form. Every one of the eleven processes writes into one capped Redis buffer and the console tails it, so an operator follows a post from an HTTP request through five workers without opening five files. What the capture also shows is a property of that design worth stating rather than cropping out: the buffer is capped, and on this deployment the API's own health polling of Ollama and MinIO is frequent enough to fill the visible window on its own. The console shows what the system is really doing rather than a curated subset of it, and what it is really doing between analyses is polling. The pipeline's structured `llm_call` records – backend, role, model, tokens and latency – are written to the same stream when a stage is working, and they are the records the per-lane cost table of Figure 4.15 and the ledger of Figure 4.35 are aggregated from.

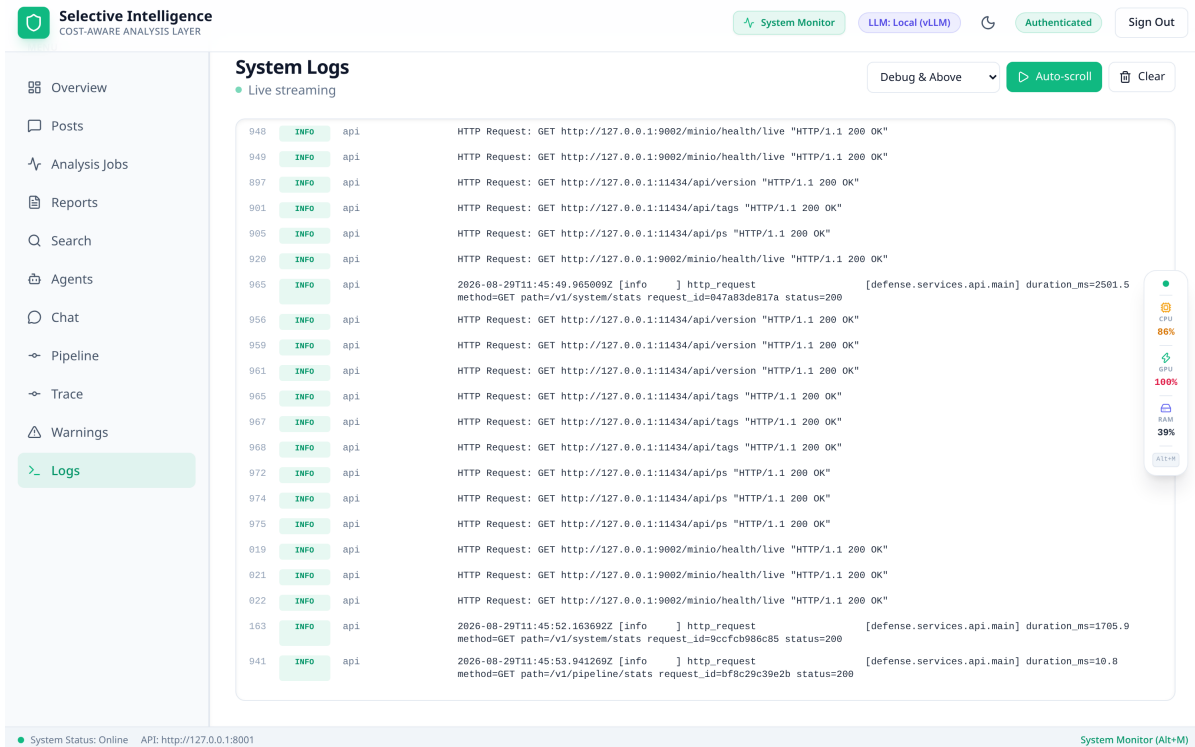


Figure 4.21: The structured-log console, streaming from every service at once.

How to read it. A full-width console. Each row is one log line: sequence number, level chip, the service that emitted it, and the message with any structured fields the code attached. The status line under the heading reads `Live streaming` once the event-source connection is open. The toolbar above offers the level filter, an auto-scroll toggle and a buffer clear. Every row in the window shown was emitted by the `api` service and is one health poll of the local inference server or of object storage.

Figure note. Every service writes to one capped Redis stream, so the operator reads one console rather than eleven log files. The capture is honest about the consequence: between analyses the API's continuous health polling is the only traffic, so it fills the buffer and the window shows nothing else. The stage and `llm_call` records share this stream and appear in it while a stage is running.

5. Tracing a single post, and the document it produces

The Trace tab re-runs one stored post through the real pipeline and follows it live, and it is the surface that makes the router's decision inspectable rather than inferable. Figure 3.11 shows a completed run: five layers, each holding the payload that stage actually emitted. The Stage-1 layer carries the language, the script, the sentiment, the topics, the toxicity, the embedding dimensionality and the per-comment breakdown; the router layer carries `use_llm`, the list of reasons that fired, the task flags, and the comment selection – 87 total, 45 duplicates removed, 36 selected; and the assembler layer carries the schema version, the validation result, the top-level key count and the three stores the document was fanned out to. A reader who wants to know why a particular post did or did not get post-level LLM work reads it here, from the router's own reasons, rather than inferring it from the output.

The document that run produced is shown in Figures 4.22 and 4.23. It is the deliverable of the whole per-post pipeline: one schema-validated JSON envelope per thread, and the thing a downstream consumer integrates against.

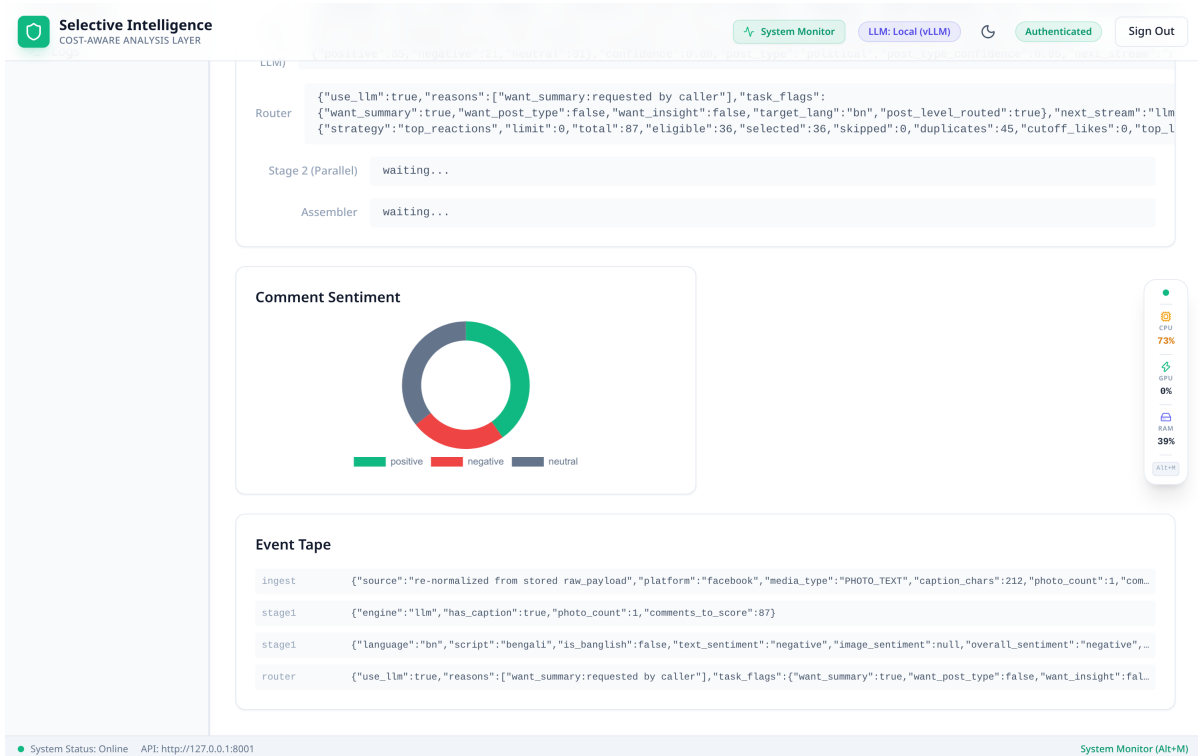


Figure 4.22: The canonical result document, first half.

How to read it. The Canonical Result panel of the Trace tab, showing the head of the JSON envelope: identifiers and platform metadata, the caption, the post type, the generated summary with its language, source, grounding and truncation flag, the watchlist fields, the fused sentiment with its score, the text and image sentiments separately, the upstream baseline, the emotion vector, the intents and the topics.

Figure note. Provenance is structural, not annotative: `post_summary_source`, `post_summary_grounding` and `baseline_sentiment` are fields of the document, so a consumer that never reads this report still receives the evidence class of every value it is given.

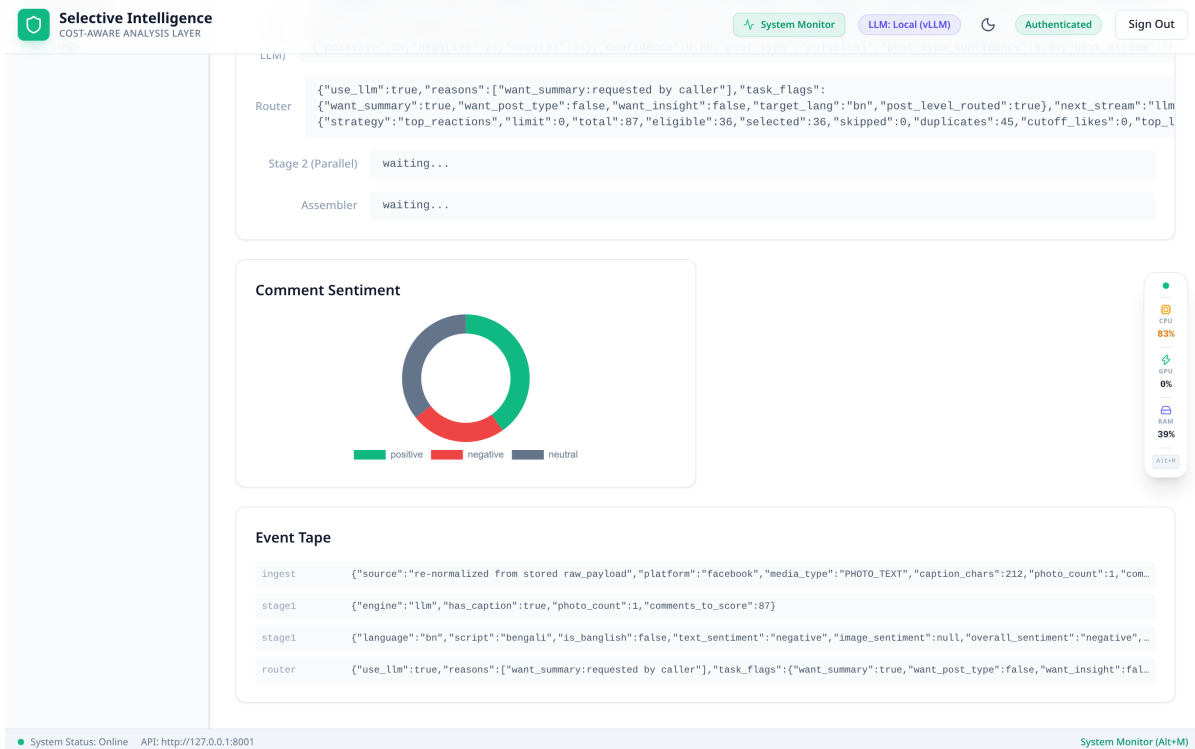


Figure 4.23: The canonical result document, second half.

How to read it. The continuation of the same document: the reaction breakdown as scraped, the image-analysis block, the comment-analysis block with its coverage arithmetic and sentiment breakdown, and the beginning of the per-target stance block.

Figure note. Three of this report's claims are visible as fields here. `vision_status: stub` is the unexercised image modality of Section 4.3.4. The comment breakdown carries 52 uncertain out of 87, which is the ensemble declining to label comments no model read rather than defaulting them to neutral. And `target_stances` carries `method: deterministic`, which records that this entity's stance came from the alias matcher rather than from a model.

6. Retrieval, in both modes

The Search tab exposes the two retrieval paths of Section 3.2.7 side by side. In keyword mode, Figure 4.24, the query runs as a case-insensitive scan over the stored JSONB and the results carry a match score. Switching the mode to semantic, Figure 4.25, runs the same query as a pgvector cosine search and the interface changes in two ways that matter more than the ranking: it names the active embedding model, and it raises a banner stating that some rows in this corpus were embedded in stub mode and that their cosine distances therefore reflect hash vectors rather than meaning. Individual rows carry the same warning as a badge. This is the retrieval-provenance mechanism that Section 4.2.5 measures the consequences of – hash vectors score `recall@10` of 0.1875 – shown at the point where a user would otherwise trust the ranking.

SELECTIVE INTELLIGENCE

COST-AWARE ANALYSIS LAYER

System Monitor

LLM: Local (vLLM)

Authenticated

Sign Out

MENU

Overview

Posts

Analysis Jobs

Reports

Search

Agents

Chat

Pipeline

Trace

Warnings

Logs

Database & Vector Search

Dual-mode retrieval: PostgreSQL JSONB keyword indexing & pgvector 768-dim multilingual semantic search

politics

campaign (optional)

Search

☐

Semantic Search (pgvector cosine similarity)

An exact post/platform/campaign id is matched directly, in either mode. Active Mode: Case-insensitive PostgreSQL JSONB full-text scan

FOUND 20 RESULTS (KEYWORD MATCH):

cmoq9a2b40mjp10k5x4q14g3

negative

১০ ডিসেম্বর মধ্য প্রদেশে, তিনি সংসদে সমাপনী বক্তব্যে ফাতিসজের দিকে ঘুরে কবীর সজ্জারনা দেখানো হয়েছে, তবে সামনের দিনগুলোতে তার অবস্থার অবশেষ কোন রকম নির্ধারণ করে বসেছে কিনা তা বুঝতে পারা যায়নি।

Campaign: cmold6pt601ebfu22bfn6utvl

facebook · bn

87 comments analysed

Match Score: 0.5000

View full post & analysis

cmor32gyn03o6o18f1h8mswoq

positive

সাকিব আল হাসান আওয়ামীজীর সম্পদ, বাংলাদেশের সম্পদ।

Campaign: cmold9n8001vffu225ck3rd9q

facebook · bn

87 comments analysed

Match Score: 0.5000

View full post & analysis

cmotus10o0cmcg1joz3bo8naw

positive

আগামী ১০ ডিসেম্বর মধ্য প্রদেশে এনসিপি সমর্থিত ১০০ জন উপজেলার তথাকথিত ও পৌরসভা মেয়র প্রার্থীর নাম ঘোষণার প্রস্তুতি হিসেবে বিভাগীয় সাংগঠনিক সম্পাদকের সাথে স্থানীয় নির্বাচন পরিচালনা কমিটির গুরুত্বপূর্ণ বৈঠক অনুষ্ঠিত হবে।

Campaign: cmoqv6cf06jwoy066gahpwkeh

facebook · bn

87 comments analysed

Match Score: 0.5000

View full post & analysis

cmolrrc7y0ixkkrd0j0xpg1j6

negative

বাংলাদেশের একটি সৈনিক পুলিশের কন্ট্রোল রুমের ও কন্ট্রোল রুমের সোলারিস্টের আলাদা করা হয়েছে কারণ তাদের অধীনে অবস্থান করা একজন সধারণ কৃষি পুশিগ কর্মকর্তা তাদের বিরুদ্ধে অপরাধ করেছিলেন। এটি দেখায় যে রাষ্ট্রের হস্তক্ষেপ এখনও বৈধতামূলক ও ত্যামিগদী স্থলও সবাইকে একত্রিত হয়ে লড়াই করতে হবে।

CPU 83%

GPU 0%

RAM 40%

ALL

Figure 4.24: Keyword retrieval over the stored analysis documents.

How to read it. The search surface with the mode toggle unchecked. The banner above the results names the active mode; each result card carries the post identifier, the summary, the campaign, the platform and language, the analysed-comment count, its match score and a link into the detail modal.

Figure note. An exact identifier is matched directly in either mode, so an operator pasting a post id never has to know which retrieval path is selected.

SELECTIVE INTELLIGENCE

COST-AWARE ANALYSIS LAYER

System Monitor

LLM: Local (vLLM)

Authenticated

Sign Out

MENU

Overview

Posts

Analysis Jobs

Reports

Search

Agents

Chat

Pipeline

Trace

Warnings

Logs

Database & Vector Search

Dual-mode retrieval: PostgreSQL JSONB keyword indexing & pgvector 768-dim multilingual semantic search

politics

campaign (optional)

Search

☒

Semantic Search (pgvector cosine similarity)

An exact post/platform/campaign id is matched directly, in either mode. Active Model: paraphrase-multilingual-mpnet-base-v2 (768-dim)

Stub Vectors Notice: Some results contain hash-based stub vectors (system running in non-ML mode). Cosine distances reflect deterministic unit vectors rather than contextual semantic meaning

FOUND 20 RESULTS (SEMANTIC COSINE RANK):

cmolm36hj04mgkrd0h1bu386o

stub-vector

positive

নব্বি ইসলাম আর্মিগীর পক্ষ নিজের সংবিধান বিরোধী ও ৭২-এর চুক্তি এর উপর যথার্থভাবে করা হয়েছে। তিনি জুলাই আমেরানের ডমিকার সম্পর্কে সত্যিকারভাবে বিরোধী করেছেন এবং তাঁর বক্তব্যটি অভিজ্ঞতার আলোকে পরিষেবে হয়ে উঠেছে।

Campaign: cmolm2kqo04lskrd05tqf9zd

facebook · bn

116 comments analysed

Similarity Score: 0.0723

View full post & analysis

cmotjhxj1bsnnkx47bsm8ne

stub-vector

positive

আজকের বেঁচে মাননীয় প্রধানমন্ত্রীর সাথে আলোচিত হয়েছিল শিরাজপুর-১, ২ ও ৩ আসনের উন্নয়ন এবং দুর্নীতির কারণে স্থগিত হওয়া রাস্তাঘাটার পুনরায় চালানার বিষয়। মাননীয় প্রধানমন্ত্রী ১১টি প্রশ্নের মধ্যে ৮টি প্রশ্নেরই চমৎকার নির্দেশনা দিয়েছেন এবং স্থানীয় সরকার, পল্লী উন্নয়ন ও সমবায় মন্ত্রণালয়কে অন্যান্য কর্মকর্তাবৃন্দও প্রতিশ্রুতি দেয়া হয়েছে।

Campaign: cmotjchfd1bqhnx46xewwvqg

facebook · bn

87 comments analysed

Similarity Score: 0.0721

View full post & analysis

cmpejky3004wegjpy3cwulpq

stub-vector

positive

পারিসের বাজিল এনসিপি মানববন্ধনের পর আজম ভাইকে মালান করে দেখা যা়।

Campaign: cmold6pt601ebfu22bfn6utvl

facebook · bn

190 comments analysed

Similarity Score: 0.0551

View full post & analysis

CPU 84%

GPU 0%

RAM 41%

ALL

Figure 4.25: The same query in vector mode, with vector provenance stated.

115

How to read it. The same surface with semantic search enabled. The mode line now names the active encoder – paraphrase-multilingual-mpnet-base-v2, 768 dimensions. The amber banner beneath it warns that some results carry hash-based stub vectors. Each affected result card carries a stub-vector badge beside its identifier, and the score column now reads as a cosine similarity.

Figure note. The badge is per row, not per query, because the corpus can contain both real and stub embeddings at once – which is exactly the state this deployment is in. Section 4.2.5 measures what the difference is worth: recall@10 of 0.8750 with real vectors against 0.1875 with hash vectors.

7. Reports

Figure 4.26 is the report generator and the reports already produced; Figure 4.27 opens one. The viewer shows the executive summary, then the embedding clusters that the report groups posts by – and, above those clusters, the same honesty mechanism the search surface uses: a badge marking that the clusters were computed over stub vectors, which means the groupings are arithmetic rather than semantic. A report that presented those five clusters as themes without that badge would be the exact failure mode this project's evidence discipline exists to prevent.

Figure 4.26: The report generator and the reports already produced.

How to read it. Two panels. The upper one generates a report: a type selector offering trend digest, brand mentions, political analysis, sentiment summary and mass reaction, an optional campaign filter, a toggle that asks an LLM to write the executive summary, and export controls. The lower one is a card per existing report, each with its title, its own summary line, its date, a PDF download and a view control.

Figure note. A report without the "grounded" toggle is pure aggregation over stored results and involves no model at all; with it, one LLM call writes a narrative over those same aggregates. The distinction is preserved in the stored report, so a reader can always tell which kind they are holding.

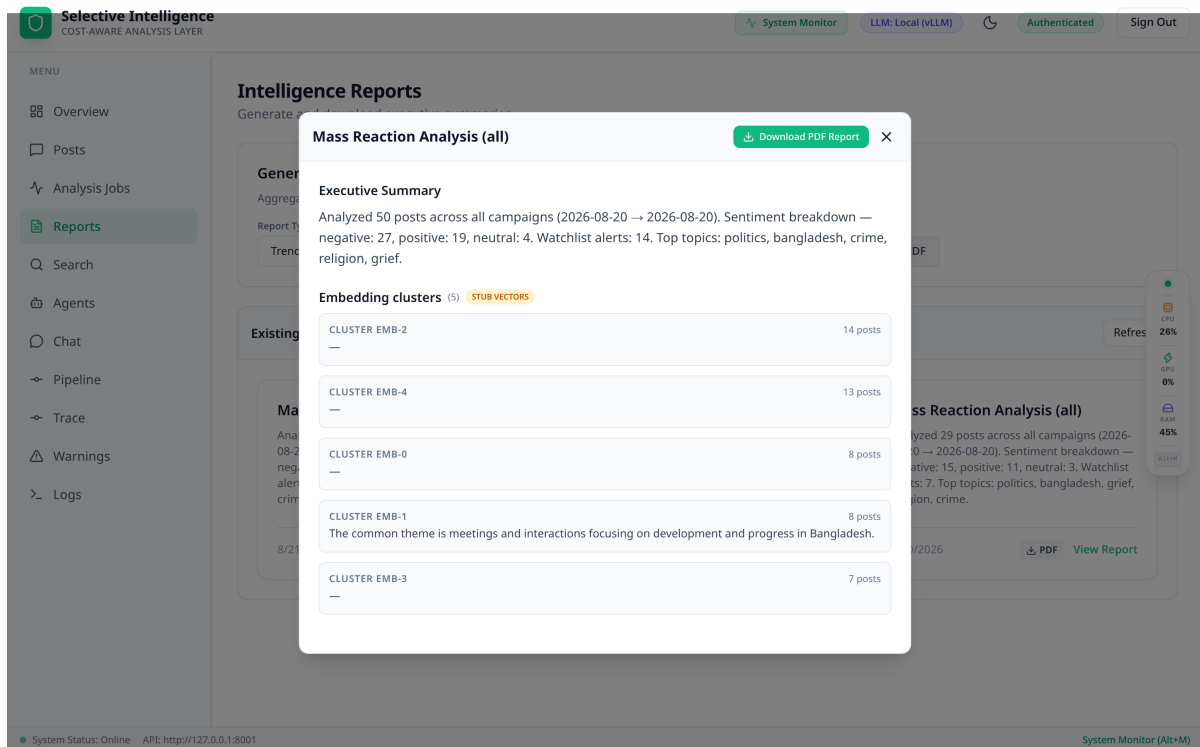


Figure 4.27: A generated report, with its cluster provenance stated on the face of it.

How to read it. The report viewer over the dimmed list. The executive summary states the corpus, the date range, the sentiment split, the watchlist count and the top topics. Below it, the embedding clusters are listed by size, with a label where one could be derived. The amber chip beside the cluster heading reads "stub vectors".

Figure note. Four of the five clusters carry no label because no meaningful theme could be derived from hash vectors, and the interface says so rather than inventing one. This is the same discipline as the uncertain comment label: an absence of evidence is rendered as an absence.

8. The agentic insight layer

The eighteen tools of Section 3.2.6 are discovered rather than declared: the agent runner asks each MCP server for its manifest at start-up and validates every argument against the returned schema before dispatch. Figure 4.28 is that discovery, run against the live servers — six analytics tools, nine retrieval tools and three ingest tools, each with the arguments the agent is permitted to send.

```

MCP tool discovery – 18 typed tools over streamable HTTP

discovering MCP tool manifests over streamable HTTP ...

analytics-mcp    http://localhost:8110/mcp/    6 tools
  trend_query    Query analytics time-series for a campaign
                  args: campaign_id, from_date, to_date, granularity
  sentiment_over_time Returns the sentiment breakdown per time period for a camp
                  args: campaign_id, from_date, to_date, granularity
  top_posts      Returns the top posts for a campaign ranked by the given m
                  args: campaign_id, metric, limit, from_date
  reaction_mix   Returns the aggregated reaction breakdown for a campaign
                  args: campaign_id, from_date, to_date
  watchlist_timeline Watchlist alerts per time period – the early-warning serie
                  args: campaign_id, from_date, to_date, granularity
  agreement_stats Audit ensemble voter agreement over the per-comment labels
                  args: campaign_id

retrieval-mcp    http://localhost:8101/mcp/    9 tools
  semantic_search Hybrid search over analysed posts: pgvector cosine kNN fus
                  args: query, campaign_id, limit, sentiment_filter
  search_comments Search COMMENT TEXT by meaning across the corpus – hybrid
                  args: query, campaign_id, post_id, sentiment
  get_post       Return the latest stored analysis result for a single post
                  args: post_id, tenant_id
  get_thread     Fetch a post's analysis result plus (optionally) all of it
                  args: post_id, include_comments, tenant_id
  representative_comments Return representative comments for a post (from the stored
                  args: post_id, sentiment, limit, tenant_id
  get_clusters   Cluster analyzed posts by embedding similarity (pgvector +
                  args: campaign_id, max_clusters, algo, tenant_id
  stance_by_target Stance distribution per watchlist target entity (supportiv
                  args: campaign_id, from_date, to_date, target_id
  stance_over_time Time series of per-target stance (supportive / opposing /
                  args: campaign_id, from_date, to_date, granularity
  coverage_stats Audit analysis coverage AND vector provenance: how many co
                  args: campaign_id, tenant_id

ingest-mcp       http://localhost:8102/mcp/    3 tools
  pull_campaign  Queue a campaign date-range pull from the upstream post-wi
                  args: campaign_id, from_date, to_date, limit
  fetch_more_comments Queue a deeper comment pull for a post (raises comment cov
                  args: post_id, limit
  refresh_post   Queue a re-fetch of a single post from upstream
                  args: post_id

18 typed tools across 3 servers – every argument schema-checked before dispatch

```

Figure 4.28: The live MCP tool inventory, as the agent runner discovers it.

How to read it. A terminal capture, grouped by server. For each of the three servers: its name, its streamable-HTTP endpoint and its tool count, then one block per tool giving the tool name, the first line of its description, and the first four arguments of its schema.

Figure note. The manifest is fetched from the servers rather than hard-coded in the agent, so a tool whose schema changes cannot silently diverge from the arguments the agent sends. The count at the foot is the count in Table 3.6, obtained by asking the servers rather than by reading the source.

Figure 4.29 is the console over those tools: the nine agents of Table 3.5 as selectable personas, each declaring the tool subset it is allowed to call and the budget it runs under. Figure 4.30 is a completed run over the analysed corpus, and Figure 4.31 is its execution trace – the tool calls the model actually made, in order, with their arguments and what came back. That trace is the mechanism behind the grounding claims of Section 3.2.6: an answer whose citations do not appear in the tool results the model was shown is rejected by the runner, and the trace is what makes that check auditable after the fact rather than merely asserted.

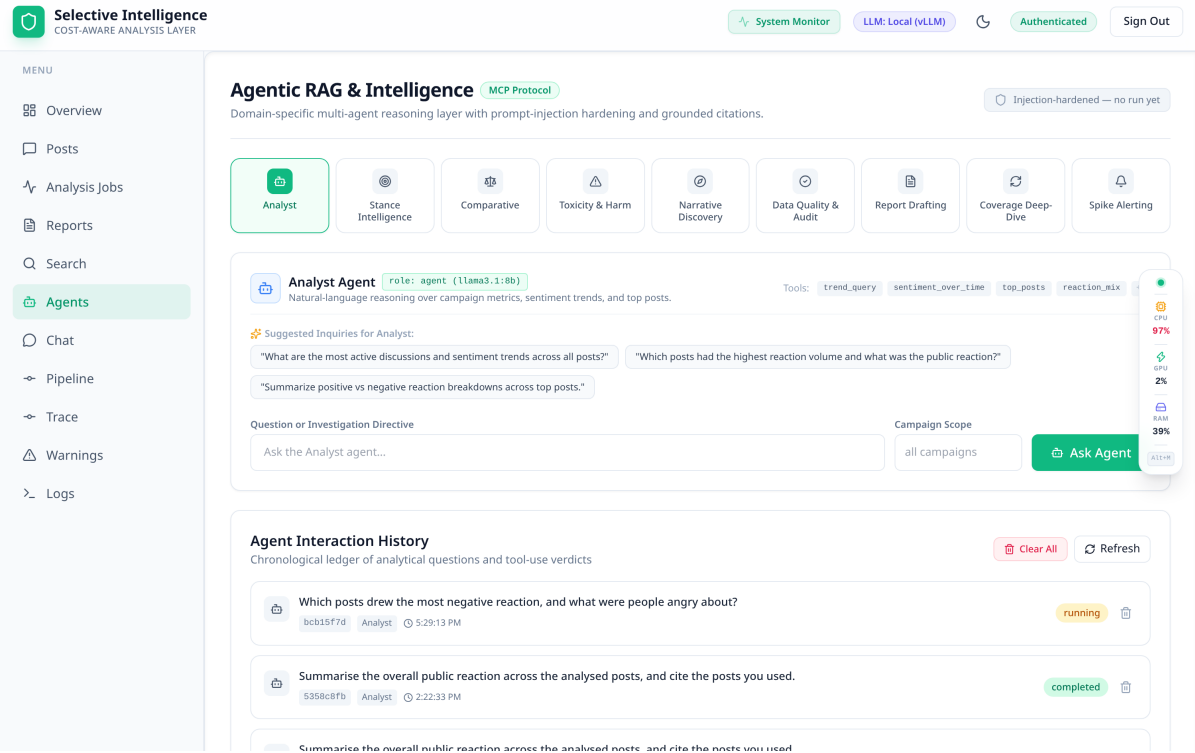


Figure 4.29: The agent console: nine personas, each with its own tool subset.

How to read it. The agent tab before a run. The grid at the top is the nine agents, the selected one highlighted; beneath it the selected agent declares the tools it may call and the budget it runs under, offers three suggested questions, and takes a free-text directive with an optional campaign scope.

Figure note. An agent is a persona plus a tool subset plus a budget, and all three are visible before the run starts. The tool list is the agent's authority: a question that would need a tool outside it cannot be answered by that agent, and the console says so in advance rather than after a failed run.

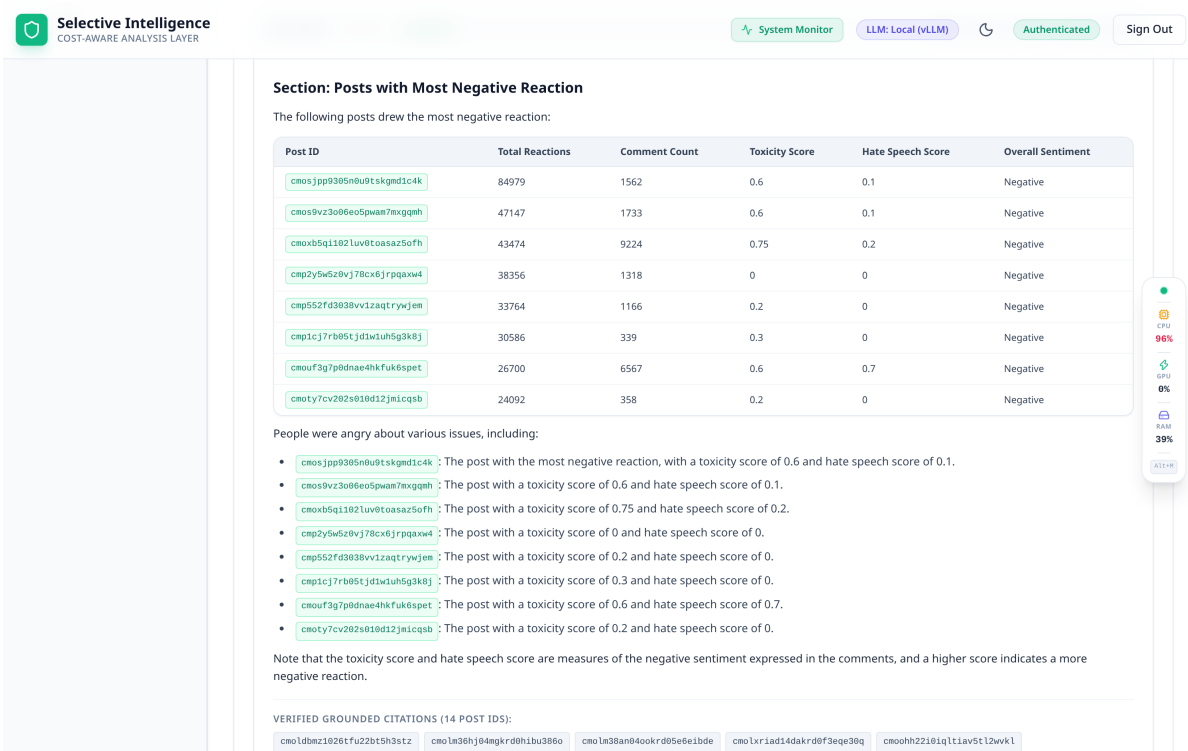


Figure 4.30: A completed agent run over the analysed corpus.

How to read it. The same tab after a run. The output header carries the run identifier, its status, the backend and model that answered – here the local `llama3.1:8b-16k` – and the tokens the run consumed. Beneath it the answer is rendered from the model's markdown: a table of the most-reacted posts with their toxicity and hate scores, then a per-post narrative. The block at the foot lists the post identifiers whose citations were verified.

Figure note. The answer is never shown alone. "Verified grounded citations (14 post ids)" means those identifiers were checked against the tool results the model was actually given; an identifier the model invented would not appear there, and an answer that cited nothing verifiable would be failed by the runner rather than displayed.

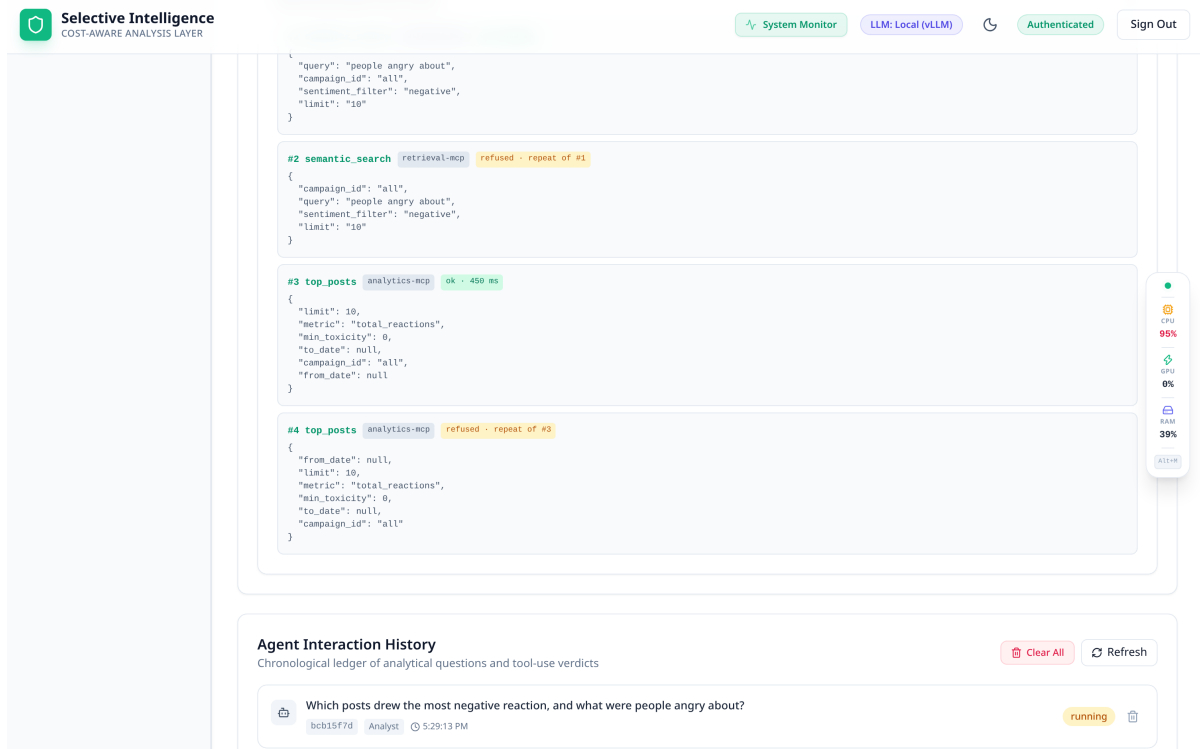


Figure 4.31: The execution trace of that run, and two calls the runner refused.

How to read it. The trace panel of the same run, expanded. Each entry is one tool call in the order the model made it: its number, the tool name, the MCP server that served it, a status chip, and the arguments the model actually sent. A green chip carries the latency of a call that ran; an amber one reads "refused – repeat of #n".

Figure note. This is the repeat-call guard of Section 3.2.6 caught in the act. The model issued four calls; calls 2 and 4 repeat calls 1 and 3 with the keys in a different order, and the runner recognised both – the signature is computed over sorted arguments with nulls dropped, so a reordering is not a new call – dispatched neither, and told the model the rows were already above. A model that has started repeating itself has stopped gathering evidence, and after two such repeats the tools are withdrawn for the rest of the run.

9. Chat, and a fabrication caught in the act

The Chat tab is the same corpus reasoning in conversational form, and Figure 4.32 is the most useful screenshot in this chapter, because the model got it wrong and the system said so. The question was a corpus question, so it was routed to an agent; the agent called `reaction_mix`, received the real reaction distribution, and wrote a correct percentage table from it – and then invented two post identifiers, 12345 and 67890, and six quotations to go with them. None of the six appeared in any tool result. The verification pass caught every one, and the answer is displayed with the invented citations listed as unverified and the invented quotations reproduced beneath a line stating that they are not comment text from the corpus.

That is the mechanism of Section 3.2.6 working exactly as specified, on an unprompted failure rather than a contrived one, and it is worth being precise about what it does and does not buy. It does not stop a small model from fabricating; nothing at the prompt level can. What it does is make the fabrication *visible in the same view as the answer*, so an operator reading the reply cannot mistake #12345 for a post. An identical answer from a system without this check would

look authoritative and be partly false – which is the failure mode that makes a monitoring product dangerous rather than merely wrong.

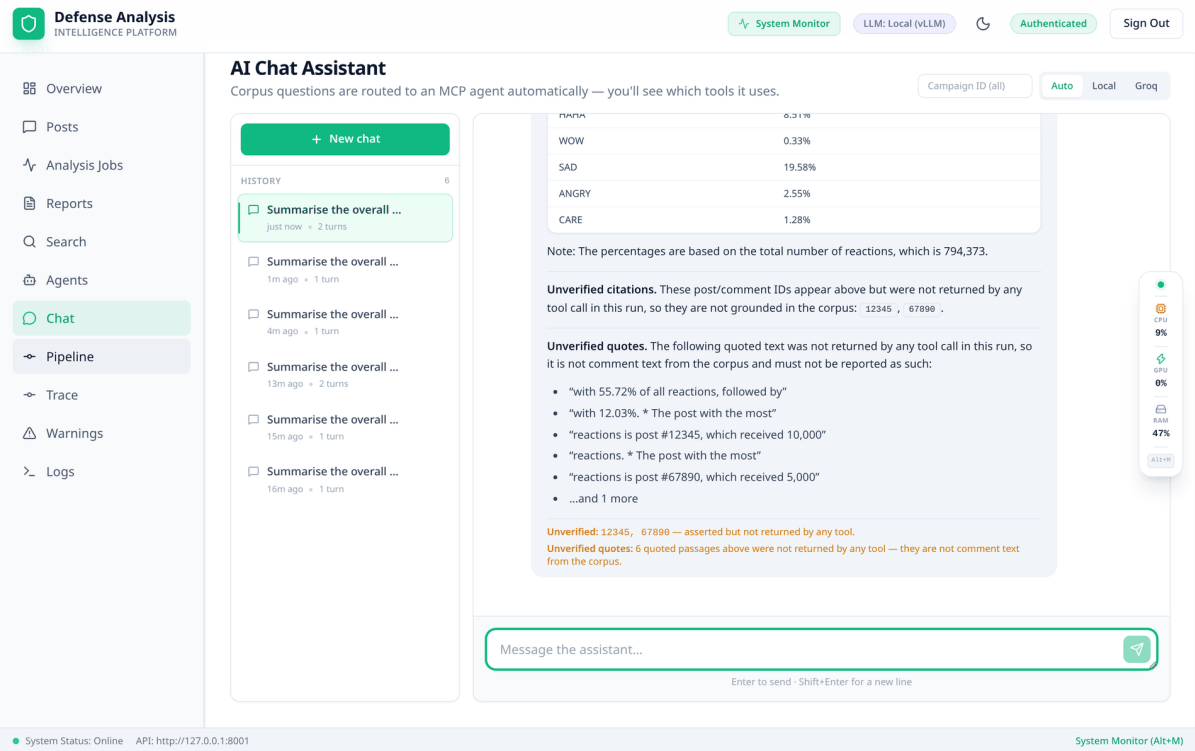


Figure 4.32: The Chat tab catching its own model inventing citations.

How to read it. The foot of a completed answer. Above, the reaction table the agent wrote from the tool result, with the total it was computed from. Below the rule, two verification blocks: "Unverified citations", listing the post identifiers that appear in the answer but were returned by no tool call in this run, and "Unverified quotes", reproducing the quoted passages that no tool returned. The amber summary lines at the foot restate both counts in one sentence each.

Figure note. The percentages are real – they came from reaction_mix – and the citations 12345 and 67890 are not: the model invented them, along with six quotations. Verification is per claim rather than per answer, so a reply can be right in its statistics and wrong in its citations, and be displayed as exactly that.

10. Watchlist warnings

Figure 4.33 is the alerting surface of the watchlist design in Section 3.2.4. Each row states not only that a post matched but which target matched and under which rule – "Mention of Tarek Rahman in post text" for an always target, or a comment opposing a favored one. Naming the rule on the row is what makes the alert falsifiable: an operator who disagrees with an alert can see the sentence that produced it, and an alias that never matches anything is visible as a target with no rows rather than as a quiet absence.

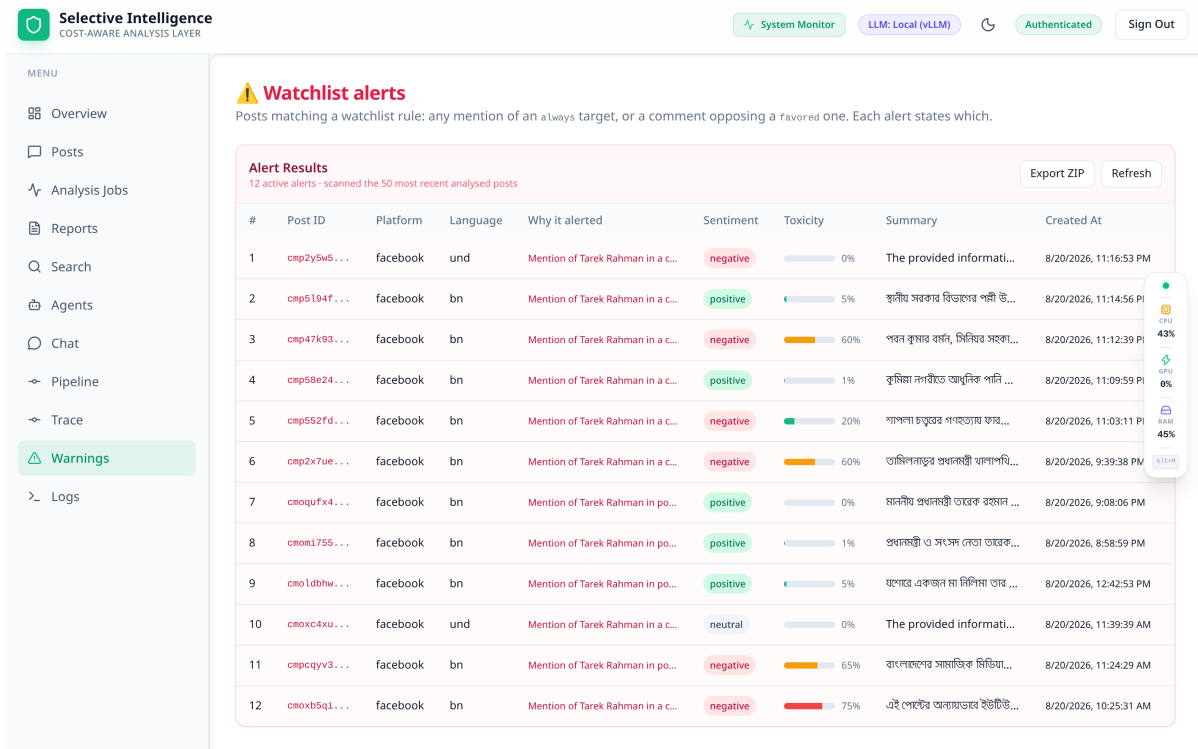


Figure 4.33: Watchlist alerts, each carrying the rule that raised it.

How to read it. A table of alerting posts. The columns are the post identifier, the platform, the language, the reason the alert fired in plain language, the sentiment of the post, its toxicity meter, its summary and its timestamp. The header states how many alerts are active and how many posts were scanned to find them.

Figure note. Twelve of the 50 analysed posts matched the single tracked target. The reason column is generated from the rule that fired rather than written by a model, so the alert can be checked against the watchlist configuration by reading it.

11. Telemetry: the host, and the bill

Two telemetry surfaces close the walkthrough. The System Monitor drawer of Figure 4.34 is host telemetry – per-core processor load, both graphics devices with their memory and thermal state, memory and swap – read from the host and streamed into the drawer, which is what allows a slow run to be attributed to saturation rather than guessed at. The usage endpoint of Figure 4.35 is the other half: the same per-lane, per-model ledger the Overview tab renders, available as JSON for anything that needs to consume it.



Figure 4.34: The System Monitor drawer: host telemetry during a run.

How to read it. A drawer over the dimmed dashboard, in three blocks. The processor block gives the aggregate load as a radial gauge and then one bar per logical core. The graphics block lists both devices – the discrete GPU with its memory allocation and temperature, and the integrated one with its driver and clock. The memory block splits the total into used, available, cached and swap. The footer carries the host, the kernel and the uptime.

Figure note. The panel reports what is actually present rather than a fixed template: two graphics devices are shown because the measurement host has two, and the sixteen core bars are the host's sixteen logical cores. This is the surface that attributes a slow stage to a saturated host.

```

GET /v1/usage - per-lane, per-model cost telemetry

$ curl -s localhost:8001/v1/usage | jq

{
  "posts_analyzed": 50,
  "llm_calls": 21,
  "llm_routing_rate": 0.42,
  "total_tokens": 1408280,
  "estimated_cost_usd": 0.0,
  "llm_api_calls": 1085,
  "cache_hits": 0,
  "cache_hit_rate": 0.0,
  "tokens_by_backend_model": {
    "local:llama3.1:8b": 12218,
    "local:gemma3:4b": 154595,
    "local:qwen2.5:7b": 1189226,
    "local:llama3.1:8b-16k": 52241
  },
  "cost_by_backend_model": {
    "local:llama3.1:8b": 0.0,
    "local:gemma3:4b": 0.0,
    "local:qwen2.5:7b": 0.0,
    "local:llama3.1:8b-16k": 0.0
  },
  "lane_split": {
    "post": {
      "calls": 9,
      "tokens": 1812,
      "token_share": 0.0013,
      "call_share": 0.0108
    },
    "comment": {
      "calls": 254,
      "tokens": 730226,
      "token_share": 0.5185,
      "call_share": 0.3042
    },
    "stage1": {
      "calls": 70,
      "tokens": 60333,
      "token_share": 0.0428,
      "call_share": 0.0838
    },
    "interactive": {
      "calls": 490,
      "tokens": 563668,
      "token_share": 0.4003,
      "call_share": 0.5868
    },
    "agent": {
      "calls": 12,
      "tokens": 52241,
      "token_share": 0.0371,
      "call_share": 0.0144
    }
  },
  "pipeline_tokens": 792371,
  "market_price_equivalent_usd": 7.0414,
  "cost_savings_usd": 7.0414,
  "scope_note": "All figures are system-wide, across every LLM caller (pipeline, chat, reports and agents)."
}

```

Figure 4.35: The cost ledger as the API returns it.

How to read it. A terminal capture of the usage endpoint's JSON. The first block is the corpus-level counters: posts analysed, calls, routing rate, tokens, estimated cost, cache hits. The second is tokens and cost by backend and model. The third is the lane split, giving calls, tokens and both shares for each of the five lanes.

Figure note. Cost is reported per backend and per model rather than as a single number, because the same corpus costs nothing self-hosted and a measurable amount metered – and the projection in Figure 4.15 is

computed from exactly these fields.

Finally, Figure 4.36 records a property of the interface rather than of the pipeline: every surface in this section is themed, and the dashboard follows the operator's system preference. A monitoring console is looked at for hours, and the light theme used throughout this chapter is not the one an operator on a night shift would choose.

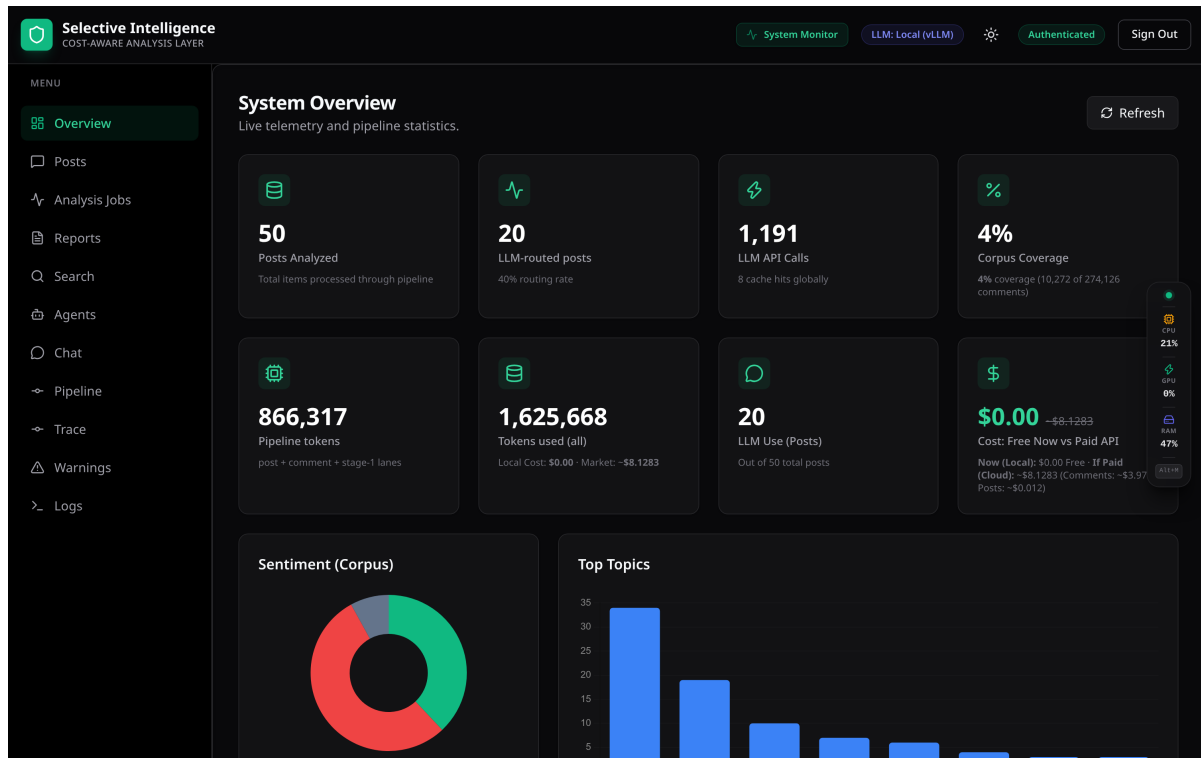


Figure 4.36: The same Overview tab in the dark theme.

How to read it. The Overview tab of Figure 3.9, rendered in the dark theme. The layout is identical; the palette, the chart colours and the sentiment pills are the theme's dark variants.

Figure note. Both themes are defined from one set of tokens, so a colour that carries meaning – the sentiment pills, the toxicity meter – keeps its meaning across the two. The theme follows the operator's system preference and can be overridden from the header.

Table 4.12 indexes the delivered features against where each is implemented and the figure that shows it. It is deliberately a complete list rather than a highlight reel: a feature that is implemented but has no figure, or no evidence class better than "works but unmeasured", is listed at that class here and in Table 4.13.

Table 4.12: Delivered features, where each lives, and the figure that shows it.

Feature	Where it lives	Shown in
One-command start of the whole stack	<code>run_all.py</code> , eleven host processes and four containers	Figure 4.13
Self-describing HTTP API, 51 paths / 63 operations	<code>services/api</code> , generated OpenAPI	Figure 4.14

Feature	Where it lives	Shown in
Ticketed Server-Sent Event streams	<code>/v1/auth/sse-ticket</code> plus five stream endpoints	Figures 3.7, 4.19
Sign-in, self-service registration, JWT sessions	<code>routers/auth.py</code> , <code>Welcome.jsx</code>	Figure 3.8
Corpus overview with sentiment, topics, emotion and language	<code>Overview.jsx</code> , <code>/v1/analysis/overview</code>	Figure 3.9
Per-lane, per-model cost telemetry and scale projection	<code>/v1/usage</code> , <code>Overview.jsx</code>	Figures 4.15, 4.35
Corpus upload and re-analysis	<code>/v1/posts/upload</code> , <code>/v1/analysis/run</code>	Figures 4.16, 4.19
Post explorer with visible coverage arithmetic	<code>Posts.jsx</code>	Figure 4.16
Post detail: provenance, coverage, upstream baseline	<code>PostModal.jsx</code>	Figure 4.17
Eight-labeller comment ensemble with abstention	<code>stage2_llm</code> , <code>PostModal.jsx</code>	Figures 3.10, 4.18
Watchlist-driven target stance, alias matching	<code>libs/stance</code> , <code>routers/analysis.py</code>	Figures 4.33, 4.23
Job control: track, stop, resume, re-run	<code>routers/analysis.py</code> , <code>AnalysisJobs.jsx</code>	Figure 4.19
Live queue depth per stage, dead-letter counts	<code>/v1/pipeline/stats</code> , <code>Pipeline.jsx</code>	Figure 4.20
Per-post live execution trace	<code>libs/progress.py</code> , <code>Trace.jsx</code>	Figure 3.11
Schema-validated canonical result document	<code>assembler</code> , JSON Schema 1.3	Figures 4.22, 4.23
Keyword retrieval over stored analyses	<code>routers/search.py</code>	Figure 4.24
Vector and hybrid retrieval with per-row vector provenance	<code>pgvector</code> , <code>retrieval_mcp</code>	Figure 4.25
Report generation, grounded and ungrounded, with PDF export	<code>routers/reports.py</code> , <code>Reports.jsx</code>	Figures 4.26, 4.27
Eighteen typed MCP tools across three servers	<code>mcp_servers/</code>	Figure 4.28
Nine tool-using agents with per-run budgets	<code>services/agents/runner.py</code>	Figures 4.29, 4.30
Agent execution trace and grounding verdict	<code>runner.py</code> , <code>Agents.jsx</code>	Figure 4.31
Streaming chat, agent handover, per-claim citation verification	<code>routers/chat.py</code> , <code>Chat.jsx</code>	Figure 4.32
Watchlist alerting with the rule that fired	<code>Warnings.jsx</code>	Figure 4.33
Structured log aggregation from every service	<code>libs/logging</code> , Redis, <code>Logs.jsx</code>	Figure 4.21

Feature	Where it lives	Shown in
Host telemetry: cores, both GPUs, memory, thermals	/v1/system/stream, SystemMonitorDrawer.jsx	Figure 4.34
Light and dark themes from one token set	tailwind.config.js, index.css	Figure 4.36
Automated test suite, including one regression test per repaired defect	tests/, 1,396 tests in 67 files – Table 4.4	Figure 4.5 (one module of it)
Image and OCR modality	stage1_nlp/vision_analyzer.py	Implemented, unexercised – Table 4.13
Prometheus scrape, Grafana and Loki containers	deploy/	Works, no provisioned dashboards – Table 4.13

Two rows of Table 4.12 name no figure, and the reason is the same in both cases: there is nothing to photograph. The image modality runs on every post and finds no reachable image bytes, so its output is the `vision_status: stub` field visible in Figure 4.23 rather than a surface. The Prometheus and Grafana containers start and the scrape configuration is correct, but this repository provisions no Grafana dashboards, so the operator-independent monitoring path ends at the metrics endpoint rather than at a panel. Both are carried at that evidence class in Table 4.13 and are named again in Section 6.2.

4.3.2 Findings supported by the measurements

The cascade works as a mechanism and its behaviour is now observable rather than asserted. The gate responds to Stage-1 quality in the direction the design predicts, its decision is carried explicitly on the envelope rather than inferred, its inputs are logged through the same readers the rules use, and its bypass branch is validated end to end against the output schema – a branch that had never once executed before the audit that found it.

Hybrid retrieval is a real improvement and the stratified analysis explains why. The fused configuration reaches 0.8750 recall@10 and, more importantly, the stratification in Table 4.10 shows the two arms failing on disjoint query populations. That is the empirical form of the argument for fusion, and it is a stronger argument than the aggregate number that hides it.

Provenance reporting changes what the system can honestly claim, and the effect is asymmetric: it removed claims rather than adding them. Recording what actually ran turned two apparent successes into stated absences – the image modality, and the majority of the default configuration's comment labels – and it is the mechanism by which a sentiment chart in this system can name its own producer.

4.3.3 Claims disconfirmed by the measurements

The project's original headline claim was wrong twice over, and reporting that is more useful than reporting the corrected version alone.

It was wrong **in fact**, because the gate was routing every post: four of its six rules could not fire. Two rules read field names that Stage 1 never emitted, so their guards short-circuited on every post; one rule read a field that Stage 1 sets deliberately as a placeholder for Stage 2 to fill, so it

fired on every post; and one rule compared a language value against a code-mixing marker that the pipeline records under a different field entirely. Net: one rule always fired and three could never fire. Every one of those four was a **silent degradation rather than an exception**, and that is the generalisable lesson of this project. The mitigation – named reader functions, a diagnostic log routed through the same readers, and a test asserting that the reader sees the producer's real output – was subsequently applied to the three other places in the system where the identical pattern had occurred: autoscaler consumer-group names that no worker created, a storage endpoint missing its URL scheme, and a response-cache key carrying a role label where the model identifier belonged.

It was wrong **in framing**, because even a correctly functioning gate governs a minority of spend once every comment is labelled. This is the finding with the most transfer value outside this project: **when the routed unit contains many sub-items that each need work, routing the container is not the cost lever**. The published cascade literature routes single queries [30][31][32] and does not encounter this, which is why it took a measurement rather than a reading to find.

4.3.4 The boundary of the evaluation

Labelling accuracy is outside the scope of this project, and the report claims none. The deliverable is a system: one that routes work, labels it, records what produced every label, and serves the result under a validated contract. What a benchmark number would additionally require is a body of human-adjudicated labels, and producing one is an annotation programme with its own guideline, its own two-annotator agreement study and its own timeline – not a further measurement script. The tooling that consumes such labels is built and ships with the repository: a stratified 300-comment sample drawn by comment kind and script, and a scorer that computes per-stratum accuracy for the entire configured roster with abstention and accuracy-when-answered reported separately, and that refuses to print a confident percentage over a handful of rows. The gold file therefore holds 300 sampled rows and **zero adjudicated labels**. The harness is a deliverable of this project; the annotation is scoped as the first item of future work in Section 6.3.

That zero is deliberate rather than an oversight, and the alternative was available throughout. Seeding those labels from any model in this repository and then scoring that model would measure the system's agreement with itself, and it would produce a number that looks exactly like a result. The position adopted here is the one an engineering project can defend: the 1,393 passing tests are **correctness and contract** tests, a green suite is not a quality claim, and the report accordingly quantifies system properties – rates, counts, coverage, latency, retrieval quality and label provenance – and quotes no macro-F1, confusion matrix or ship gate for any sentiment labeller. Every statement this report makes about labelling is a statement about *what ran and what it produced*, never about whether it was right.

Four further capabilities are implemented and cannot currently produce a signal, and they are listed rather than omitted. The **image modality** cannot be exercised because no image bytes are reachable. **Horizontal scaling** is code-complete and unbenchmarked; no multi-replica run has been timed. **Cross-backend quality parity** falls under the same scope boundary, because

comparing two backends' output quality needs the same adjudicated labels. Finally, the **cross-encoder reranker** is implemented but unmeasured, so the evaluation harness prints a skip rather than reporting the fused order as though a rerank had happened.

Table 4.13 is the evidence ledger for the system as a whole. Every capability appears in it exactly once with the class of evidence supporting it, so that no claim made elsewhere in this report rests on a stronger basis than the ledger records.

Table 4.13: Evidence class by capability.

Capability	Evidence class	Basis
Routing rate and rules fired, per Stage-1 engine	Measured	Router counters over the real stage functions
Post-level versus comment-level call split	Measured	Same run; five-lane usage counters
Comment kinds, including the 2.8 percent emoji share	Measured	Kind classifier over the whole corpus
Per-label provenance and inferred share	Measured	Provenance block on every post
Coverage arithmetic, clamping and anomaly flags	Measured	Corpus-level and per-post computation
Retrieval recall@k, MRR@k and nDCG@k	Measured	32 known-item queries, one marked relevant post each
Stream and consumer-group identifier agreement	Measured	Test asserting the manifests match the workers
Cancellation bounded at one post per stage	Measured	Job-lifecycle suite against live stores
Stop, resume and delete	Measured	29 posts already analysed, 21 re-enqueued, none repeated, counters rebuilt
Truncation recovery and the never-cache rule	Measured	Verified firing on a real Bangla post
Context-window loss on the plain agent model tag	Measured	24 of 11,045 prompt tokens evaluated
Tool-result serialisation cost	Measured	76,666 characters escaped against 23,194 unescaped
Eight-labeller ensemble accuracy	Works, unmeasured	Roster grew from two heads to seven; scoring it needs adjudicated labels
Watchlist stance quality	Works, unmeasured	Built and tested across 36 matcher tests; the 150-comment validation is not done
Agent answer quality and citation accuracy	Works, unmeasured	Hardening is tested; groundedness is not scored
Cluster summarisation	Works, unmeasured	Runs; its value depends on the vectors being real
Near-duplicate reuse	Works, unmeasured, and off by default	Composed rather than copied; disabled by default

Capability	Evidence class	Basis
Image sentiment and OCR	Unexercised	No image bytes are reachable in any runnable configuration
Vision-grounded summary	Unexercised	Same cause
KEDA autoscaling in a cluster	Unexercised	Manifests are correct and test-asserted; not run in a cluster
Horizontal scaling throughput	Unexercised	No multi-replica run has been timed
Cross-encoder reranking	Unexercised	Needs the cross-encoder weights; the harness prints a skip
Grafana dashboards over the Prometheus scrape	Unexercised	Containers start and the scrape config is correct; this repository provisions no dashboards
Live per-post trace on a run longer than five minutes	Works, with a stated failure mode	The API closes the event stream at its ceiling and the trace view reports the run as failed; the run itself completes (Section 6.2)
Labelling accuracy, macro-F1 per language bucket	Out of scope	Sampler and scorer built; annotation programme specified in Section 6.3
Backend quality parity	Out of scope	Same annotation dependency
Calibration of the gate's confidence input	Out of scope	Same annotation dependency

4.3.5 Threats to validity

Construct validity. The corpus is an engagement-ordered 3.75 percent sample of the comments the platform holds, so thread-level aggregates describe the most-engaged comments rather than the thread. Every claim in this report is scoped accordingly.

Internal validity. Several measurements were taken in a configuration where models degrade to deterministic functions. Those runs are reproducible but they are not model behaviour, and the system reports which components degraded so that such a run is identifiable. No latency or retrieval figure is quoted from a degraded run.

External validity. Fifty posts across twenty-five campaigns in a three-week window on one platform is a demonstration, not a study. The routing and cost behaviour should generalise to other corpora of the same shape, because it depends on the ratio of comments to posts rather than on their content; the retrieval numbers should not be expected to, because they depend on corpus size and on how the query set was built.

Statistical validity. The retrieval comparisons rest on 32 queries and the decisive bucket on 8. The direction is consistent across k and across configurations, and it is not significant at the conventional threshold. This report states the direction and the non-significance together.

4.4 Summary

The entire implementation and evaluation of the system is described in this chapter. The environment setup section gave the complete development stack: a Python 3.12 and React 19 codebase, four datastores chosen by access pattern, four locally served models, and three deployment shapes expressing the same code. The testing and evaluation section established functional correctness across every module through a suite of 1,396 automated Python tests – sampled in the chapter as forty representative acceptance scenarios, and broken down by functional area in Table 4.4 – and a whole verification chain – Python suite, dashboard unit tests, lint, production build and browser tests – that passes as one pass-or-fail command in sixty seconds. Access control was tested separately from function, because the two fail differently: 138 tests across eight suites exercised token issue and verification, the hashed API-key store, single-use streaming tickets, cross-tenant reads, the fail-closed data-residency policy and prompt-injection containment, against both the permitted and the refused case in each. Four measurement studies were then completed. The routing gate works and its behaviour is observable, but measurement showed that it governs 30 percent of large-model calls rather than the great majority – a finding that contradicted the project's own headline claim and reframed it into one that survives contact with the data. Hybrid retrieval reaches 0.8750 recall@10 against 0.1875 for hash vectors, and stratifying the query set shows the dense and lexical arms failing on disjoint populations, which is the real argument for fusing them. Label-provenance measurement showed that 71.3 percent of per-comment labels in the default configuration are not model output, and the response was to report that rather than to conceal it. The results section then walked the delivered software feature by feature, from the one-command start of the eleven-process stack through to the cost ledger, with every figure a capture of the running system rather than a mock-up.

Taken together, these findings demonstrate that the architectural decisions made in Chapter 3 – particularly the separation of the router's two decisions, the abstention protocol in the comment ensemble, and the placement of the agent layer outside the per-post path – translated into a working, auditable system rather than remaining a design proposal. The successful validation of the full chain as a single command is especially significant, because it is what makes the evidence discipline enforceable: a claim in this report that stopped being true would break a test. Equally significant is what the chapter does not claim. The evaluation stops where the engineering does: the report quantifies what the system does and what produced every value it emits, and leaves labelling accuracy to the annotation programme specified in Section 6.3. With the functional and evaluative groundwork now established, the following chapter turns to the engineering standards followed during development, the broader societal and environmental impact of the system, and the complex engineering challenges encountered throughout the project.

Engineering Standards and Design Challenges

This chapter records the standards the system complies with and the alternatives that were rejected in each case, assesses the work's impact on life, society, the environment and its own sustainability, presents a financial analysis with a costed alternative, and maps the project onto the complex-engineering-problem, knowledge-profile and complex-engineering-activity frameworks.

5.1 Compliance with the Standards

Only the standards that genuinely bear on this system are listed. For each one, the alternative that was considered is named with its trade-off, because a standard adopted without an alternative in mind is a default rather than a decision.

5.1.1 Software Standards

Data interchange – JSON with JSON Schema validation. Every document crossing a boundary in this system is JSON, and both the input contract and the canonical output are validated against a published schema. The assembler **raises** on a validation failure rather than persisting the document, so nothing downstream – no store, no API response, no dashboard row – ever sees an unvalidated result. The schema carries an explicit version on every result, and the version history is part of the record: one increment exists purely because a language-model task's output was being computed, paid for and then dropped by a component that read a different set of fields.

- *Alternatives considered.* **Protocol Buffers** or **Avro** would give a smaller wire format and stronger typing, and Avro's schema evolution rules are better than anything JSON Schema offers. They were rejected because the upstream platform emits JSON, the downstream consumers are third-party applications for which a self-describing text format is far easier to adopt, and the analysis payloads are dominated by free text where a binary encoding saves little. The cost of the choice is verbosity, and it is paid down where it actually hurts: stream payloads are serialised without escaping non-ASCII characters, which on Bengali text is not cosmetic – escaping was measured as roughly half the token bill on the agent path and triples the bytes on every stream entry.

Web API – REST over HTTPS, versioned, with OpenAPI. All endpoints live under a version prefix; asynchronous work returns an accepted status with a job identifier to poll or subscribe to; writes accept an idempotency key; list endpoints paginate by cursor; and errors use one

consistent envelope. The live specification is served by the application rather than maintained by hand.

- *Alternatives considered.* **GraphQL** would let the dashboard fetch exactly the fields it needs and would have avoided one class of defect this project hit repeatedly, where a hand-maintained response model silently dropped fields the pipeline had computed. It was rejected because the consumers are heterogeneous third-party services rather than one first-party client, because caching and rate-limiting are simpler on REST, and because a strongly typed response model is the correct fix for that defect class, which is what was done. **gRPC** was rejected for the public surface for the same reason as Protocol Buffers, though it remains the right choice if an internal high-volume path is ever extracted.

Streaming – Server-Sent Events. Five live surfaces stream: system telemetry, job progress, pipeline queue depth, logs, and chat tokens.

- *Alternatives considered.* **WebSockets** are bidirectional and would have been the obvious choice, but every one of these surfaces is server-to-client only; Server-Sent Events reconnect automatically, traverse ordinary HTTP infrastructure, and need no separate protocol upgrade. Their one real disadvantage drove a design decision: an event-source connection cannot carry an authorization header, so something must go in the URL. Rather than putting a credential there, the system issues a **single-use ticket valid for about sixty seconds**, so a streaming URL captured in a proxy log is worthless. **Long polling** was rejected as strictly worse on both latency and connection count.

Authentication – JSON Web Tokens with HMAC signatures, plus hashed API keys. Tokens carry a short expiry with a refresh endpoint; API keys are stored as SHA-256 hashes and the tenant identity is read from the stored row rather than from anything the client sends.

- *Alternatives considered.* **OAuth 2.0 with an external identity provider** is the right answer for a multi-organisation production deployment and is the documented upgrade path; it was not adopted here because it adds an external dependency that the offline test suite cannot exercise. **Session cookies** were rejected because the primary consumers are machine clients. The important compliance point is not the choice of scheme but the enforcement: a credential that is *shaped* like a token is verified as a token on every transport, because verifying it on only one of them meant an expired or forged token authenticated successfully on all four streaming endpoints.

Character encoding – Unicode with NFC normalisation. All text is normalised at ingestion, and script tagging distinguishes Bangla, Latin and mixed content. This is not administrative: without normalisation, two visually identical Bangla strings can differ byte-for-byte, which would defeat both the content-hash deduplication and the alias matcher that the watchlist feature depends on.

Code quality and testing. Static analysis on both languages, a typed settings object as the single source of configuration, and a test suite in which contract tests assert structural invariants across component boundaries. Two test markers gate the expensive paths, one of which mutates live datastores and must therefore never run by accident.

5.1.2 Hardware Standards

Accelerator and precision. Inference targets CUDA-capable accelerators with quantised weights, and the deployment is explicitly designed so that the cheap classifier heads run on **CPU** while the accelerator serves the language model. That is a decision rather than a fallback: measured on a four-gigabyte card already serving a seven-billion-parameter model, roughly 285 megabytes remain free, which fits one classifier head and not seven. Attempting to place all seven on the accelerator produces the least desirable outcome – some load, some fail, and a head that fails to load abstains silently for the life of the process, so a run silently degrades from eight voters to two.

- *Alternatives considered.* **Placing every model on the accelerator** was the initial configuration and produced exactly that degradation. **Running the language model on CPU as well** was rejected on latency grounds. The chosen split costs roughly three gigabytes of checkpoint storage and a few CPU cores, and it makes the ensemble's coverage deterministic.

Storage and memory. The vector column is fixed-dimension and the embedding dimension must match it, which is enforced rather than documented: a mismatch fails every insert instead of silently storing an unusable vector. Object storage speaks the S3 API, so the development deployment runs MinIO and production can substitute a managed service with no code change.

Commodity-hardware target. The system is required to run on a single developer workstation with one consumer accelerator, which is why the entire model stack has a stub mode in which every component degrades to a deterministic function and **reports that it did so**. That reporting is the standard being complied with: a run in which every component degraded still reports the engine it *intended* to use, so the list of degraded components is the only signal distinguishing a real result from a heuristic one.

5.1.3 Communication Standards

Transport security. TLS terminates at the gateway, datastores and inference servers sit on private subnets, and only the gateway is internet-facing. In the cluster deployment a network policy enforces that boundary rather than relying on topology.

Message transport. Redis Streams with consumer groups provides at-least-once delivery for the MVP tier; the production tier migrates to a partitioned log. Every stream and consumer-group name is defined in exactly one module, and a test asserts that the autoscaler manifests name the same streams and groups the workers create – a rule that exists because a scaler pointed at a group no worker creates reports no backlog, so the one stage where scaling changes cost never scaled while the manifests looked correct.

Tool transport – the Model Context Protocol. Agents reach data only through MCP servers over streamable HTTP [38]. Adopting a protocol rather than writing bespoke glue is what makes a new analytical capability a tool registration plus one line in an allowlist. The protocol's own security literature is taken seriously here: because tool results carry third-party text verbatim, everything a model is shown is wrapped in untrusted-data delimiters with forged-delimiter neutralisation [43].

Data residency as a communication constraint. The choice of inference backend is a communication decision, because the cloud backend sends prompt content, which is to say, real people's comments – to a third party. It is therefore treated as a policy rather than a performance setting: the decision is resolved where the tenant is known, stamped into the job envelope so a later global toggle cannot re-route content already in flight, re-resolved on resume, and **fails closed** when the policy table cannot be read.

5.2 Impact on Society, Environment and Sustainability

5.2.1 Impact on Life

The direct beneficiaries are the analysts who currently read comment threads by hand. On the largest thread in this corpus that means 2,857 stored comments against a platform-reported 9,224; no analyst reads that, so in practice a human reads the top twenty and generalises. The system does not remove the human, and it is designed on the assumption that it should not: it reports coverage, it reports what produced each label, and it shows eight independent verdicts side by side precisely so that a human can see where the models disagree and look there.

There is a second, less obvious effect on the people whose comments are analysed. A system that labels a comment negative and stops there attributes an emotion to a person. A system that separates document sentiment from stance toward a named entity, and that says **uncertain** when no model read a comment rather than defaulting it to neutral, makes fewer claims about people than the alternative. The effect is modest, and it is the correct default when the subjects have not consented to the analysis.

5.2.2 Impact on Society and Environment

Societal. The corpus this system was built on is political content from Bangladesh, and the technology must be assessed as dual-use. The same capability that lets a newsroom see how a policy announcement was received lets an actor identify who is criticising them. The project's response is not to pretend otherwise but to make the editorial choice visible: the watchlist is a **stated bias model** carried in a versioned file with a named owner per entry, the repository ships a placeholder so that it states no politics of its own, and stance lives in a separate field so it cannot be quietly folded into a sentiment total. A neutral bucket exists specifically so that an operator can have target-dependent stance for monitoring without the advocacy framing.

Environmental. The environmental argument for this design is the same as the cost argument, and it is measurable in the same units. Every large-model call avoided is accelerator-seconds not spent. Under the shipped configuration the corpus run costs 529 calls against 813 for the keyword-baseline configuration, and the caps that were deliberately left disabled would reduce the dominant comment lane by more than an order of magnitude on large threads – a 2,857-comment thread costs 115 stance batches uncapped and four with the cap set to one hundred. Two further design choices cut work rather than merely moving it: exact and near-duplicate detection avoids re-analysing reshared content, and repeat comment texts within a thread collapse to their most-liked occurrence, which on threads containing many repeated short phrases yields a substantial reduction at no loss of coverage. Set against this, the corresponding

cost is that the eight-labeller ensemble spends roughly 0.92 seconds of CPU per comment across seven heads in order to obtain an agreement signal, and that the full roster of seven heads was retained rather than reduced.

5.2.3 Ethical Aspects

Five ethical positions were taken deliberately and each has a corresponding mechanism in the code, which is the only form in which an ethical position is checkable.

- E1. **Personal data is handled as personal data.** Social content contains identifiable people. The system encrypts at rest and in transit, scopes every read by tenant, supports per-tenant retention and deletion, logs data access, and – the mechanism that makes the rest enforceable – takes tenant identity from a stored row rather than from anything a client sends.
- E2. **A privacy commitment must survive a database outage.** The tenant policy check originally returned permissively on any database error, which permitted egress precisely when the policy could not be verified. It now returns a service error. A privacy guarantee that evaporates when the database hiccups is not a guarantee.
- E3. **The bias in the labels is stated, not hidden.** A file declaring that support for a named entity is positive encodes a political stance. That is legitimate for a monitoring product and indefensible presented as neutral analysis, so it is labelled as the former in the schema, in the documentation and in this report.
- E4. **The system does not claim more than it did.** Provenance on every label; an explicit degraded-components list; a stub flag on every stored vector; an explicit vision status where only one value licenses a claim about an image. The evidence ledger in Table 4.13 is the same commitment at the level of the whole project.
- E5. **Untrusted input is treated as untrusted.** Comment text is attacker-controlled. It is escaped before rendering, wrapped in data delimiters before a model sees it, and never permitted to alter a system instruction. This is risk reduction and is described as such: no prompt-level defence can promise resistance to prompt injection [43], and the tests do not pretend one does.

One ethical limitation is stated plainly here rather than left implicit. The comments analysed here were published publicly but their authors did not consent to this analysis. The mitigations – aggregate reporting, tenant scoping, retention controls, and the refusal to attribute a label to a comment no model read – reduce the harm; they do not resolve the underlying question, which is a policy matter for whoever deploys the system rather than an engineering one.

5.2.4 Sustainability Plan

Technical sustainability rests on the property that made the audit passes productive: a cross-component rename breaks a test rather than a feature. Stream names, consumer groups and label vocabularies each live in exactly one module; the deployment manifests are asserted against those modules; every repaired defect has a regression test; and a mutation pass confirmed that nine of ten of those tests actually fail when the fix is reverted. Test code is 35 percent of the repository, and the growth is almost entirely regression coverage.

Operational sustainability rests on the backend being pluggable. An operator with accelerators runs everything in-house at zero marginal token cost; an operator without them runs the same code against a metered API and owns no hardware; an operator with a burst pattern does both. Because the switch is resolved per call, this is a configuration decision rather than a migration.

Economic sustainability is addressed in Section 5.3. **Scientific sustainability** is the corpus: 10,272 real code-mixed comments with crowd reaction signals attached, together with a stratified sampler and a scorer, are ready for the annotation effort that would turn this engineering result into a benchmark result. That is the single highest-value piece of future work and it is human hours rather than compute.

5.3 Project Management and Financial Analysis

Milestone-based iterative development. The project was managed through the iterative milestone approach set out in Section 3.3, organised around the phase exit criteria rather than around dates alone, with the work divided between two members along the vertical slices recorded in Table 3.11 and each member auditing the other's slice. Tasks were prioritised by a single guiding principle: get the pipeline and its router correct and measurable first, then build the capability layers on top of it, and only then spend a fifth of the schedule on audit and evaluation. That last allocation was not in the original plan and is the largest single management decision of the project.

Budget Analysis. The project itself was built with no cash outlay beyond hardware already owned, because every component is open source and the inference ran locally. That was by design rather than an accident of circumstance: a core assertion of this system is that a Bangla monitoring layer should be operable by organisations that cannot afford metered frontier inference, and the whole platform was therefore built and measured on free and self-hosted infrastructure to demonstrate that the claim holds at the smallest scale. The financial analysis that matters is therefore the **operating cost of running the system**, which is what a prospective operator would ask about. Table 5.1 gives the primary budget – self-hosted inference – across the three deployment tiers.

Table 5.1: Primary budget: self-hosted inference, monthly operating cost in US dollars.

Line	MVP (about 1,000 threads per batch)	Production (about 10,000)	Enterprise (about 100,000)
Cheap-NLP accelerator fleet	150–700 (one 24 GB consumer card, amortised or part-time cloud)	400–1,500	3,000–12,000
Language-model serving	shares the same card	900–3,200	5,000–25,000
CPU services and database nodes	included above	200–600	1,000–4,000
Storage (operational, analytics, object)	10–40	50–250	300–2,000
Networking and egress	5–30	50–300	300–2,000
Observability	0–20	30–150	150–600
Per-token API charges	zero	zero	zero
Total	about 170–800	about 1,600–6,000	about 10,000–48,000

Table 5.2 costs the **alternative**: metered cloud inference, in which the language-model accelerator line disappears entirely and is replaced by a per-token charge.

Table 5.2: Alternate budget: metered cloud inference, monthly operating cost in US dollars.

Line	MVP	Production	Enterprise
Cheap-NLP fleet	0–300 (can run CPU-only at this tier)	400–1,500	3,000–12,000
Language-model serving	no accelerator required	no accelerator required	no accelerator required
Per-token API charges	5–50	150–1,500	1,500–15,000
CPU services, storage, networking, observability	15–90	330–1,300	1,750–8,600
Total	about 20–440	about 880–4,300	about 6,250–35,600

US dollars per month

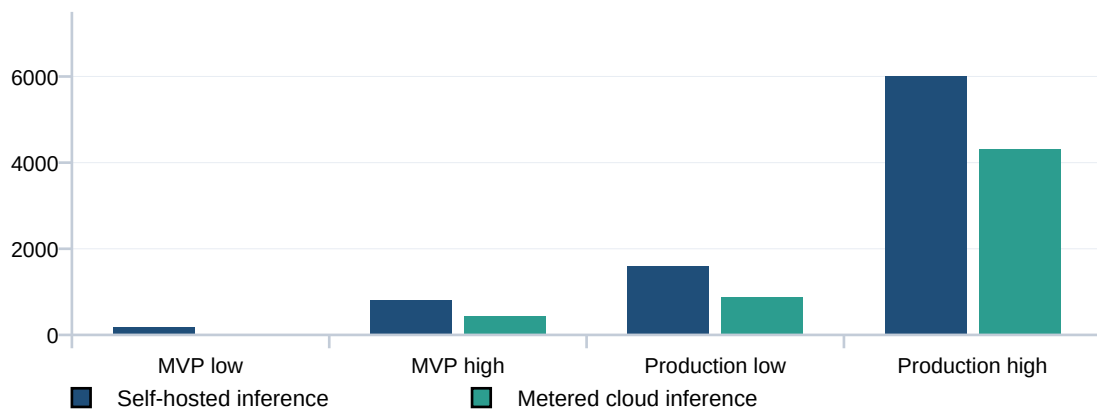


Figure 5.1: Monthly operating cost envelope, MVP and Production tiers.

Reading the chart. The Enterprise tier is plotted separately in Figure 5.2 because its range is two orders of magnitude larger and would flatten everything shown here into the axis. At these two tiers the metered path is cheaper at both ends of every range, and at the MVP tier it is dramatically so, because it removes the accelerator line entirely – an operator with no hardware at all can stand the system up for tens of dollars a month.

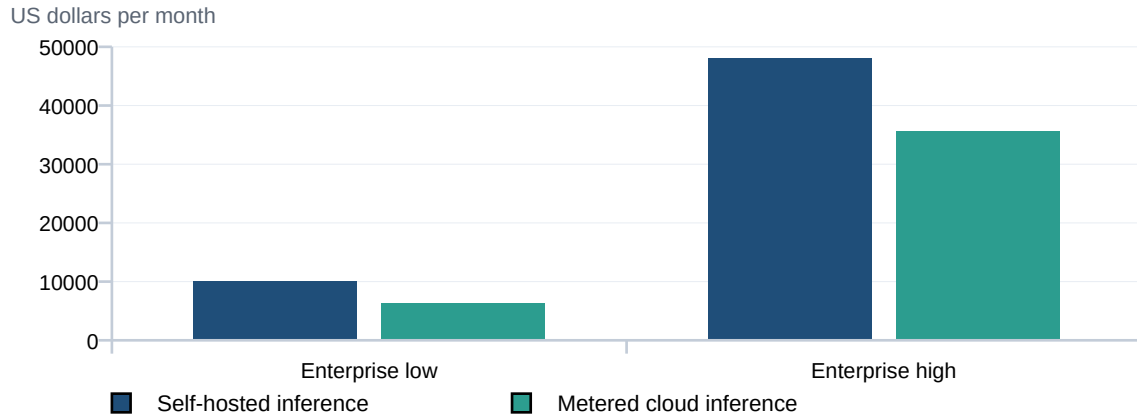


Figure 5.2: Monthly operating cost envelope, Enterprise tier.

Reading the chart. The gap narrows as volume rises, from roughly 88 percent at the MVP low end to roughly 26 percent here, because a purchased accelerator is a fixed cost while a per-token charge is not. Two facts that this pair of charts cannot show belong beside them: the metered figures scale with the number of comments analysed and are therefore unbounded if coverage rises, whereas the self-hosted accelerator line is fixed once the hardware is purchased; and the metered path sends comment text to a third party, which for a privacy-locked tenant is not a price question at all. The correct reading is that the trade differs per tenant and per workload, which is exactly why the design makes it a runtime decision rather than a build-time one.

These are planning-grade order-of-magnitude figures, not quotes. Accelerator prices change frequently and vary by region and by commitment. The durable insight is the *ratio and the dominant driver*, not the absolute number: at every tier the inference line dominates, and the single largest lever on it is comments analysed per thread, not posts routed. Section 4.2.4 is the measurement behind that statement, and the dollar figures here should be re-derived from a real metered run before any budget is committed.

Rationale for the primary choice. Self-hosted inference is the default for three reasons that are not primarily financial: its marginal token cost is genuinely zero, so a coverage decision is not also a budget decision; no comment text leaves the cluster, which is the only way the privacy lock can be honoured; and the models can be fine-tuned in-house [27], which the metered path cannot offer. The metered path is the right choice for a low-volume or bursty operator, or for one with no accelerator at all, and a hybrid – owned capacity for the steady base, metered for spikes – is the pattern this design expects at the enterprise tier.

Revenue model. For a deployment offering this as a service, the natural model is a per-tenant subscription tiered on threads analysed per month, with the coverage knobs exposed as the tier boundary, because coverage is what actually drives cost. Table 5.3 sets out a proposed tier structure on that basis.

Table 5.3: Proposed subscription revenue model.

Plan	Target segment	Basis of price	Included capabilities
Community	Single analyst, civic groups, academic use; up to about 1,000 threads per month	Free, self-hosted only	Full pipeline, eight-labeller ensemble, keyword and semantic search, ungrounded reports, one watchlist target, community support
Standard	Newsrooms and brand agencies; up to about 10,000 threads per month	Metered on threads analysed, with the comment cap as the tier boundary	All Community capabilities plus grounded reports, the full agentic insight layer, unlimited watchlist targets, hybrid retrieval with rank fusion, per-lane cost telemetry and email support
Enterprise	Multi-tenant deployments and election-scale monitoring; about 100,000 threads per month and above	Custom, with the option of owned capacity for the steady base and metered inference for spikes	All Standard capabilities plus tenant privacy lock with enforced data residency, dedicated inference capacity, custom watchlist governance with named owners, cluster deployment with queue-depth autoscaling and priority support

The system already meters what such a model needs: token and call counts per backend, per model and per spending lane, with a figure that isolates per-post pipeline spend from per-question chat and agent spend, so a customer who uses the chatbot heavily cannot inflate the per-post cost figure. Beyond subscription, two further revenue lines follow naturally from the architecture. The first is the analytical layer itself: grounded reports and agent briefings are per-question costs that scale with questions asked rather than with corpus size, which makes them straightforward to price separately. The second is the data residency guarantee – for a tenant that cannot let comment text leave its own infrastructure, the enforced privacy lock of Section 3.2.5 is not a feature but a precondition, and it is the kind of requirement that supports enterprise pricing on its own.

5.4 Complex Engineering Problem

5.4.1 Complex Problem Solving

Table 5.4 maps the project onto the seven complex-engineering-problem attributes. Rationale for each attribute that applies follows the table.

Table 5.4: Mapping with the Complex Engineering Problem attributes.

Attribute	Applies	Rationale in brief
EP1 – Depth of knowledge required	Yes	Requires knowledge from the research literature across three distinct fields: low-resource and code-mixed NLP, cost-aware inference and cascades, and retrieval with tool-using agents. Mapped to the knowledge profile in Table 5.5.
EP2 – Range of conflicting requirements	Yes	Cost against accuracy, coverage against latency, privacy against speed, self-hosted against metered, and honesty against apparent completeness. Several of these conflicts are irreducible and are resolved by exposing them as runtime knobs rather than by picking one side.
EP3 – Depth of analysis required	Yes	No standard formulation exists for the central question. Determining that the routing gate governs only 30 percent of spend required instrumenting the system and running it, not reading it; four of the six gates were dead and every one of the four failed silently.
EP4 – Familiarity of issues	Yes	Code-mixed Bangla with no standard romanisation, an entity that appears in three scripts, a comment sample that is 3.75 percent of the population and engagement-ordered, and a context window that discards a prompt without reporting it – none of these are textbook problems with textbook answers.
EP5 – Extent of applicable codes	Partial	No single standard governs the whole system, so compliance is assembled from several: schema validation, versioned REST, token and key handling, transport security, network isolation, and a tool protocol. Section 5.1 records each with its rejected alternative.
EP6 – Extent of stakeholder involvement and conflicting requirements	Yes	The analyst wants full coverage; the operator wants a bounded bill; the privacy-locked tenant forbids egress; the platform owner forbids write-back; the comment authors did not consent at all. These requirements conflict directly and are reconciled by policy and configuration rather than by compromise in the code.
EP7 – Interdependence	Yes	Five pipeline stages, three datastores, seven model roles, two inference backends, three tool servers and nine agents, connected by contracts whose violation is silent. Four separate defects in this system had the identical shape of one component writing a string that another read under a different name.

EP1 rationale – depth of knowledge

The problem cannot be solved from a single body of knowledge. Choosing the ensemble roster required knowing which Bangla encoders exist, which of them are actually sentiment heads rather than base encoders with a randomly initialised classification layer bolted on, and which ship a tokeniser the environment can build – three of the roster's original entries failed one of those tests and voted zero times while reading as coverage. Designing the gate required the cascade literature [30][31][32]. Designing the retrieval layer required knowing that dense and lexical scores are on incomparable scales and that rank fusion is the standard answer [39], and that recall rather than precision is the operative metric when the whole top-k is handed to a generator [42]. Table 5.5 maps this to the knowledge profile.

EP2 rationale – conflicting requirements

The clearest instance is coverage against cost. Analysing every comment is the better product and the better science, and it makes the bill scale with thread size; the largest thread in the corpus costs 115 stance batches and 44 minutes of classifier CPU on its own. The design does not resolve this conflict, because it is not resolvable – it renders the conflict **explicit and adjustable**: the caps are disabled by default, their effect is documented in calls per post, and the output reports how much of the thread went unread so a chart can never imply a coverage it did not have. The privacy conflict is handled the opposite way, because it is not a matter of degree: a locked tenant cannot be routed off-box by any path, and an operator's global toggle is downgraded rather than honoured.

EP3 rationale – depth of analysis

Three findings in this project were reachable only by analysis, and each had gone undetected through ordinary review. The routing gate reported plausible behaviour while four of its six rules were inert. The full-text half of the lexical retrieval arm matched zero rows across an entire query set because the query builder conjoined every term, and everything credited to it belonged to the trigram half. The agent role silently discarded eleven thousand of eleven thousand and forty-five prompt tokens because the inference server's default context window is four thousand and ninety-six and it reports only what it evaluated. All three are **silent degradations rather than exceptions**, which is the property that makes them require measurement rather than reading.

EP4, EP6 and EP7 rationale

Familiarity (EP4) is addressed in Section 2.3: the alias problem in three scripts has no off-the-shelf solution, and the specific failure it produces – zero matches, indistinguishable from an entity that was never mentioned – is why unmatched targets are logged as probable bugs. **Stakeholder conflict (EP6)** produced the two-field design for stance, which exists solely so that the operator's editorial choice cannot be mistaken for a measurement. **Interdependence (EP7)** produced the single-source-of-truth modules and the tests that assert deployment manifests match the code, which is the generalised form of the fix for the four identifier defects.

Table 5.5 maps the problem onto the knowledge profile, which is the mapping EP1 requires.

Table 5.5: Mapping with the Knowledge Profile.

Attribute	Applies	Where it is used in this project
K1 – Natural sciences	No	The system has no physical-science component.
K2 – Mathematics and computing fundamentals	Yes	Vector similarity and approximate nearest-neighbour search; reciprocal rank fusion; k-means clustering; the information-retrieval measures recall@k, MRR@k and cumulated gain; probability and calibration for the confidence gate; agreement statistics over the ensemble; paired significance testing on the retrieval comparison.
K3 – Engineering fundamentals	Yes	Distributed systems: durable queues, consumer groups, at-least-once delivery with idempotent writes, back-pressure, bounded retry and dead-lettering, cooperative cancellation, circuit breaking, and horizontal scaling by partition.
K4 – Specialist knowledge	Yes	Transformer encoders and their multilingual variants; code-mixed and low-resource language processing; target-dependent stance; retrieval-augmented generation and tool-using agent loops; inference serving and quantisation; prompt-injection threat models.
K5 – Engineering design	Yes	The whole of Chapter 3, and specifically Table 3.7, which records thirteen significant decisions with the alternative each displaced and the constraint that justified the choice.
K6 – Engineering practice	Yes	Containerised deployment, infrastructure manifests, autoscaling on queue depth, observability, secret handling, network isolation, continuous verification as a single pass-or-fail chain, and six structured audit passes with regression tests and mutation testing.
K7 – Comprehension of the role of the engineer in society	Yes	Section 5.2: the dual-use assessment, the stated-bias-model framing, the consent limitation stated rather than buried, and the decision to report absent capabilities as absent.
K8 – Research literature	Yes	Section 2.2 and the reference list: the cascade line of work, the Bangla and code-mixed corpora and benchmarks, the stance formulation, the retrieval evaluation protocol, and the prompt-injection threat taxonomy.

5.4.2 Engineering Activities

Table 5.6 maps the project onto the five complex-engineering-activity attributes, with rationale following.

Table 5.6: Mapping with the Complex Engineering Activities.

Attribute	Applies	Rationale in brief
EA1 – Range of resources	Yes	Seven model roles across two inference backends; four datastores chosen by access pattern; five worker pools; three tool servers; two front-end toolchains; accelerator, CPU and storage budgeting that had to be reasoned about jointly, because the classifier heads and the language model compete for the same card.
EA2 – Level of interaction	Yes	Requirements from an operator, a platform owner, analysts, privacy-constrained tenants and non-consenting data subjects, several of them in direct conflict. Internally, five stages and two layers connected by contracts whose violation is silent, which is what made the audit passes necessary.
EA3 – Innovation	Yes	The watchlist-driven target stance for code-mixed Bangla, which is carried inside an existing call and therefore incurs no additional inference cost; the abstention protocol that lets an ensemble report that nothing read a comment; and the measured finding that container-level routing governs a minority of spend when the container holds many sub-items.
EA4 – Consequences for society and environment	Yes	Dual-use political monitoring, addressed in Section 5.2.2; accelerator energy, addressed by the cascade itself and quantified in calls per corpus run; and the consent limitation, stated in Section 5.2.3.
EA5 – Familiarity	Yes	No standard solution exists for the central problem. The cascade literature does not cover thread-shaped units, target-dependent stance is not established for three-script code-mixed text, and the failure modes that dominated the project – silent degradation rather than exceptions – are precisely the ones that ordinary testing does not surface.

EA3 rationale – the scope of the innovation claim

The claim requires precision, because the three formulations below have different standings in the literature. **Target-dependent stance detection is not novel:** it is established for English and well covered for product reviews and social posts [13][14][15][16]. **Doing it over an operator-supplied, configurable entity list** rather than a fixed annotated aspect set is less common but is an engineering framing rather than a research result. **Doing it on code-mixed Bangla, where the same entity is written in three scripts and the matcher is the hard part, is the defensible claim** – that setting is genuinely under-served, and the alias problem is a real technical obstacle rather than a formality, as the two matcher defects recorded in Section 3.2.4 demonstrate.

The line that survives scrutiny is therefore: **novel application and engineering in an under-resourced language setting, not a novel method.** Two further contributions are stated at the same level of care. The abstention protocol is not a new algorithm; it is the observation that an ensemble which cannot abstain produces agreement figures that cannot be falsified, together with the reporting that fixes it. Similarly, the cost finding is not a new technique but a measurement that contradicts the natural reading of the cascade literature when the routed unit is a container.

5.5 Summary

The system complies with an assembled rather than a single standard, and Section 5.1 records each choice with the alternative it displaced. Its societal impact is genuinely dual-use, and the project's response is to make the editorial component of the labels visible rather than to claim neutrality it does not have. Its environmental argument is the same as its cost argument and is measured in the same units – calls per corpus run. The financial analysis shows that self-hosted and metered inference cross over rather than one dominating, which is why the choice is a runtime decision. Against the complex-engineering-problem framework the project engages six of seven attributes fully and one partially, draws on seven of eight knowledge-profile categories, and meets all five complex-engineering-activity attributes – with the innovation claim stated narrowly enough to be defensible. The next chapter provides a conclusion to the report, summarising what has been accomplished, stating the limitations of the current system, and setting out the directions for future work in the order that would produce the most value.

Conclusion

This chapter summarises what was built and what was learned, states the project's limitations without softening them, and sets out the work that would close each one, in the order that would produce the most value.

6.1 Summary

This project set out to answer a narrow and testable question: given a social-media post together with its embedded comment thread, in a language setting where the text is a live mixture of Bangla script, English and romanized Banglish, can a system decide *per unit of work* how much intelligence that unit deserves, spend the expensive model only where cheap models cannot answer, and prove afterwards from its own counters where the money went and what produced every label?

The answer is yes to the first three parts and, in an important respect, **not in the way the project originally believed**. A five-stage, queue-decoupled microservice was designed, built and measured. Cheap analysis runs unconditionally over every post and every comment. A six-gate router decides post-level spend, and a separate selection step decides which comments the expensive labellers read – two decisions rather than one, which is the structural change that made the cost behaviour measurable at all. Per-comment sentiment was raised from a single verdict to eight independent labellers with an explicit abstention protocol, so that a comment no model read reports as uncertain rather than being defaulted to neutral. A watchlist stance capability answers who a comment opposes rather than merely whether it is negative, and it does so at zero additional model calls because the matched entities ride inside a prompt already being sent. Above the per-post pipeline, nine tool-using agents reason over the system's own analytical output through three protocol servers exposing eighteen typed tools, hardened against six classes of failure that were each observed live rather than anticipated.

Measurement produced four results. The routing gate works and its rate responds to Stage-1 quality exactly as designed: 16 percent of posts route under the shipped engine against 74 percent under a keyword baseline. **But the gate governs only 30 percent of large-model calls**, because between 85 and 96 percent of calls are comment-level and comment labelling runs for every post regardless. That finding contradicted the project's own headline claim and replaced it with a narrower one that survives contact with the data: cheap NLP filters *which comments and which posts* deserve a large model. Hybrid retrieval reaches recall@10 of 0.8750 where hash-seeded vectors score 0.1875 – indistinguishable from returning ten random posts out of fifty, and stratifying the query set shows the dense and lexical arms failing on disjoint query populations, which is the real argument for fusing them. Label-provenance measurement showed that 71.3

percent of per-comment labels in the default configuration are not model output, and the response was to report that rather than to conceal it. The complete system verifies as a single pass-or-fail chain in sixty seconds, across 1,393 Python tests, 78 dashboard tests, lint, a production build and browser tests.

The most transferable outcome of the project is not any of those numbers. It is the discipline that produced them. Six independent audit passes established that the consequential failures in a system of this shape are not incorrect answers but **silent successes**: a routing gate reporting plausible behaviour while four of its six rules were inert; a retrieval arm matching zero rows across an entire query set while a passing test covered it; an inference server discarding eleven thousand prompt tokens and reporting only what it evaluated; a classifier reporting 8,513 model inferences in a run where zero models had loaded. Every one of those degraded silently rather than raising. The engineering response – named reader functions, single-source constant modules, provenance on every value, abstention instead of a default vote, and an evidence class attached to every published claim – is what this project would carry to the next system.

6.2 Limitation

Six limitations bound what this report claims. They are stated without qualification, because the project's own standard requires it. The first five are constraints on the evidence; the sixth is a scope decision, and it is placed last because it describes a boundary the project drew rather than a defect it left behind.

- L1. **The corpus is small and its comment sample is biased.** Fifty posts across twenty-five campaigns in a three-week window on one platform is a demonstration, not a study. The 10,272 comments are an engagement-ordered 3.75 percent slice of the 274,126 the platform reports, with 64.1 percent carrying zero reactions and five posts reporting more stored comments than the platform says exist. Thread-level aggregates therefore describe *the most-engaged comments per post*, which is the phrasing this report uses and the only claim the data supports.
- L2. **The multimodal claim is unexercised.** Forty-four of fifty posts carry images, and the image path – zero-shot visual sentiment [28][29] plus optical character recognition – is implemented and has **never produced a signal**, because the 69 image references are relative object-storage keys whose objects are not present in storage. Post sentiment is therefore a text measurement. The system reports an explicit failure status rather than a fabricated neutral verdict, and fusion renormalises over the terms that carry a real model verdict, so the absence is visible in the output rather than hidden inside an average.
- L3. **Retrieval quality is measured against a machine-derived query set.** The 32 known-item queries were derived from each target post's own analysed topics and entities, with no human relevance judgement. That derivation favours literal matching, marks exactly one relevant post per query so every other on-topic hit counts as a miss, and leaves the decisive stratified bucket at eight queries – where the direction is consistent across k but does not reach conventional significance.

- L4. **Scale is code-complete and unbenchmarked.** Horizontal scaling, autoscaling on queue depth, and the Kubernetes deployment are implemented and test-asserted against the code they scale, but no multi-replica run has been timed and none has been executed in a cluster. Every throughput statement in this report is therefore a design property, not a measurement.
- L5. **The agent layer's answers are unmeasured.** The runner is hardened against six observed failure classes and each has a regression test, but groundedness, citation accuracy and answer relevance have no scores. The evaluation plan for them exists and specifies a judge-model protocol with human review on a sample [44][47]; it has not been run.
- L6. **Labelling accuracy is outside the project's scope, and no labeller in this system carries a measured one.** The sampler and the scorer are built and shipped – a stratified 300-comment sample, and a scorer that reports per-stratum accuracy for the whole configured roster with abstention separated from accuracy-when-answered – but the gold file holds **zero adjudicated labels**, deliberately: seeding labels from any model in this repository and then scoring that model would measure agreement with itself. The consequence is stated wherever it bites. No macro-F1, no confusion matrix, no calibration curve [48] and no ship gate exists for any sentiment labeller, and the 1,393 passing tests are correctness tests rather than a quality claim. The eight-labeller ensemble in particular grew from two heads to seven, and ranking those heads against one another is the first thing the annotation programme in Section 6.3 would make possible.

Two operational limits surfaced during the walkthrough of Section 4.3.1 and are recorded here rather than left in a screenshot. The per-analysis event stream has a five-minute server-side ceiling, after which the API emits a timeout frame and closes; the Trace tab treats that close as a failure and stops updating, so a traced post whose comment lane runs longer than five minutes finishes correctly in the pipeline while the trace view shows it as failed. The run in Figure 3.11 completed inside that window; a longer one would not, and the fix – resubscribing with a fresh ticket rather than treating a closed stream as a failed run – is named in Section 6.3. Second, the monitoring stack ships as containers and a correct scrape configuration but **provisions no Grafana dashboards**, so the operator-independent monitoring path of Section 3.2.9 currently ends at the metrics endpoint. Both are carried at that evidence class in Tables 4.12 and 4.13.

Two further constraints bound the work rather than diminishing it. The watchlist is distributed as a placeholder, because its contents are an editorial choice that this repository deliberately does not make, so the novelty item is built and tested but its 150-comment validation is outstanding. Second, the prompt-injection hardening constitutes **risk reduction rather than a solution**: no prompt-level defence can promise resistance [43], and the tests do not pretend one does.

6.3 Future Work

The future work is ordered by value per unit of effort. The first item is the one this project deliberately scoped out, and it dominates the rest because most of the others depend on it.

F1. Adjudicate the gold set (the bottleneck; everything else is fast). Two annotators labelling a five-hundred-comment overlap, reporting Cohen's kappa [49], then labelling to two or three thousand comments stratified by language bucket and by comment kind, sampling *within* strata at

random rather than taking the top-N by reactions – otherwise the gold set inherits the same engagement bias the corpus already has and the benchmark measures the easy half. The guideline has to rule explicitly on the hard cases that occur in this corpus: sarcasm marked only by a laughing emoji [17], political epithets that are literal insults but conventional in this register, single-word comments plus an emoji, and code-switching mid-sentence. If agreement falls below the usual threshold the guideline is the problem, not the annotators. **This is the step that would extend the work from an engineering deliverable to an empirical one; with the labels in hand, most of what follows is compute.**

F2. Benchmark rather than build. With labels in hand, run the seven ensemble heads, the language-model stance pass and the Stage-1 heuristic baseline against them and report macro-F1 **per language bucket**. The finding that multilingual models degrade by a measurable margin on romanized Banglish relative to native-script Bangla would be a genuine, citable result, and the code to produce every prediction already exists. Two operational points make it trustworthy: disable the response cache for every run, and report provenance alongside every row, because a model that fell back to a heuristic on a portion of the inputs has not scored on the whole set.

F3. Turn the cascade into a measured trade-off. Sweep the router's confidence threshold and plot macro-F1 against cost and latency at each setting, with an all-cheap and an all-large configuration as the endpoints. This would convert the architecture from a design argument into an empirical curve, and it is the framing under which the routing work reads as a measured trade-off rather than a configuration choice. One caveat is new since the cost measurement and changes what the plot means: because the gate governs only 30 to 55 percent of calls, a threshold sweep moves a minority of the cost axis. **Plot two curves – threshold against cost, and comment-cap against cost, or state plainly that the sweep covers post-level spend only.** The comment-cap lever is arguably the more informative of the two, and it has not yet been plotted.

F4. Validate the watchlist separately, with two metrics that are reported apart. Mention detection – the precision and recall of the alias matcher – is the number that proves or disproves the hard part, and recall is the one to watch, because a missed alias is invisible while an over-match is obvious. Stance agreement, on the correctly matched mentions only, is the second. They must not be folded into the general sentiment scorecard, for the same reason target stance occupies a separate output field.

F5. Learn the routing decision instead of hand-tuning it. Every threshold in the gate is currently a guess made defensible only by being sweepable. Training the router on preference or outcome data, in the manner of the learned-router literature [31], is the single most valuable upgrade to the mechanism itself, and it becomes possible only after F1, because a learned router needs labels to learn against.

F6. Make the image modality real, or delete its weights from the design. Uploading the 69 image objects to storage is roughly half a day of work and would turn an unsupported claim into a measured one. The alternative – scoping the system permanently to text and removing the image terms from the fusion documentation – is equally defensible. What is not defensible is the third state, in which a weight sits in the design and has never been anything but zero.

F7. Score the agent layer. Build the analyst question set with reference answers and citing posts, and score groundedness claim by claim, citation accuracy, answer relevance and tool-use correctness, using a judge model at scale with human review on a sample [44][45][47] and the retrieval-specific measures the retrieval-augmented-generation evaluation literature provides [46]. The run records already carry everything the scoring needs – the backend, the model, the tools called, the arguments sent and the tokens spent, so this is an evaluation task rather than an instrumentation one.

F8. Benchmark the scale claims. Run a multi-replica deployment in a cluster, drive it with a ten-thousand-thread batch, and record per-stage throughput and queue lag against replica count. This is the one limitation that needs infrastructure rather than human effort, and it converts NFR1 from a design property into a measurement.

F9. Close the two operational gaps the walkthrough exposed. Neither is research; both are afternoons. The trace view should resubscribe with a fresh ticket when the API closes a stream at its five-minute ceiling, and distinguish "the stream ended" from "the run failed" – at present a long comment lane finishes correctly in the pipeline and shows as failed in the browser. And the monitoring stack should ship provisioned Grafana dashboards against the Prometheus scrape it already configures, so the operator-independent path of Section 3.2.9 ends at a panel rather than at a metrics endpoint.

F10. Have the access-control surface reviewed by someone who did not write it. The 138 security tests of Section 4.2.3 are white-box tests written by the members who wrote the code they guard, so they establish that each check behaves as specified under the conditions their authors imagined – which is exactly the class of assurance that a fresh-eyes pass over the same code repeatedly proved insufficient elsewhere in this project. An adversarial review with no knowledge of the implementation, targeting the token issuer, the ticketed stream path, the tenant scope on analytical queries and the fail-closed residency check, is the cheapest remaining way to find the case the authors did not think to write down.

Taken together, F1 through F4 would take the project from a demonstrable engineering result to a defensible empirical one, and the estimate is three to four weeks dominated almost entirely by annotation. Everything that used to sit in front of that annotation – the sampling frame, the cache key that would have compared a model against its own cached output, the caps that hid seventy percent of the corpus, the labels that could not say what produced them – is now cleared.

References

References follow the IEEE citation style. Every entry carries a Digital Object Identifier that was resolved against the Crossref or DataCite registry at the time of writing.

- [1] A. Bhattacharjee, T. Hasan, W. Ahmad, K. S. Mubasshir, M. S. Islam, A. Iqbal, M. S. Rahman, and R. Shahriyar, "BanglaBERT: Language model pretraining and benchmarks for low-resource language understanding evaluation in Bangla," in *Findings of the Association for Computational Linguistics: NAACL 2022*, 2022, pp. 1318-1327, doi: 10.18653/v1/2022.findings-naacl.98.
- [2] K. I. Islam, S. Kar, M. S. Islam, and M. R. Amin, "SentNoB: A dataset for analysing sentiment on noisy Bangla texts," in *Findings of the Association for Computational Linguistics: EMNLP 2021*, 2021, pp. 3265-3271, doi: 10.18653/v1/2021.findings-emnlp.278.
- [3] A. Hassan, M. R. Amin, A. K. A. Azad, and N. Mohammed, "Sentiment analysis on Bangla and romanized Bangla text using deep recurrent models," in *Proc. 2016 Int. Workshop on Computational Intelligence (IWCI)*, 2016, pp. 51-56, doi: 10.1109/IWCI.2016.7860338.
- [4] S. Sazzed, "Abusive content detection in transliterated Bengali-English social media corpus," in *Proc. Fifth Workshop on Computational Approaches to Linguistic Code-Switching*, 2021, pp. 125-130, doi: 10.18653/v1/2021.calcs-1.16.
- [5] N. Romim, M. Ahmed, H. Talukder, and M. Saiful Islam, "Hate speech detection in the Bengali language: A dataset and its baseline evaluation," in *Proc. Int. Joint Conf. on Advances in Computational Intelligence, Algorithms for Intelligent Systems*. Singapore: Springer, 2021, pp. 457-468, doi: 10.1007/978-981-16-0586-4_37.
- [6] M. A. Nezami and M. N. Huda, "Public sentiment identification in social media for code-mixed English-Bangla and Banglish text analysis with machine learning," in *Proc. Sixth Int. Conf. on Image, Video Processing, and Artificial Intelligence (IVPAI 2024)*, SPIE, 2024, doi: 10.1117/12.3047050.
- [7] S. Saha and A. Nanda, "BanglaNLP at BLP-2023 Task 2: Benchmarking different transformer models for sentiment analysis of Bangla social media posts," in *Proc. First Workshop on Bangla Language Processing (BLP-2023)*, 2023, pp. 266-272, doi: 10.18653/v1/2023.banglalp-1.34.
- [8] A. Chakma and M. Hasan, "LowResource at BLP-2023 Task 2: Leveraging BanglaBert for low-resource sentiment analysis of Bangla language," in *Proc. First Workshop on Bangla Language Processing (BLP-2023)*, 2023, pp. 347-353, doi: 10.18653/v1/2023.banglalp-1.47.
- [9] F. Alam, A. Hasan, T. Alam, A. Khan, J. Tajrin, N. Khan, and S. A. Chowdhury, "A review of Bangla natural language processing tasks and the utility of transformer models," arXiv:2107.03844, 2021, doi: 10.48550/arXiv.2107.03844.
- [10] P. Patwa, G. Aguilar, S. Kar, S. Pandey, S. PYKL, B. Gamback, T. Chakraborty, T. Solorio, and A. Das, "SemEval-2020 Task 9: Overview of sentiment analysis of code-mixed tweets," in *Proc. Fourteenth Workshop on Semantic Evaluation*, 2020, pp. 774-790, doi: 10.18653/v1/2020.semeval-1.100.
- [11] S. Sitaram, K. R. Chandu, S. K. Rallabandi, and A. W. Black, "A survey of code-switched speech and language processing," arXiv:1904.00784, 2019, doi: 10.48550/arXiv.1904.00784.
- [12] M. Wankhade, A. C. S. Rao, and C. Kulkarni, "A survey on sentiment analysis methods, applications, and challenges," *Artificial Intelligence Review*, vol. 55, no. 7, pp. 5731-5780, 2022, doi: 10.1007/s10462-022-10144-1.
- [13] D. Kucuk and F. Can, "Stance detection: A survey," *ACM Computing Surveys*, vol. 53, no. 1, pp. 1-37, 2021, doi: 10.1145/3369026.

- [14] S. Mohammad, S. Kiritchenko, P. Sobhani, X. Zhu, and C. Cherry, "SemEval-2016 Task 6: Detecting stance in tweets," in *Proc. 10th Int. Workshop on Semantic Evaluation (SemEval-2016)*, 2016, pp. 31-41, doi: 10.18653/v1/S16-1003.
- [15] M. Pontiki, D. Galanis, J. Pavlopoulos, H. Papageorgiou, I. Androutsopoulos, and S. Manandhar, "SemEval-2014 Task 4: Aspect based sentiment analysis," in *Proc. 8th Int. Workshop on Semantic Evaluation (SemEval 2014)*, 2014, pp. 27-35, doi: 10.3115/v1/S14-2004.
- [16] L. Dong, F. Wei, C. Tan, D. Tang, M. Zhou, and K. Xu, "Adaptive recursive neural network for target-dependent Twitter sentiment classification," in *Proc. 52nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 2014, pp. 49-54, doi: 10.3115/v1/P14-2009.
- [17] P. Kralj Novak, J. Smailovic, B. Sluban, and I. Mozetic, "Sentiment of emojis," *PLOS ONE*, vol. 10, no. 12, e0144296, 2015, doi: 10.1371/journal.pone.0144296.
- [18] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," arXiv:1706.03762, 2017, doi: 10.48550/arXiv.1706.03762.
- [19] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. 2019 Conf. of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2019, pp. 4171-4186, doi: 10.18653/v1/N19-1423.
- [20] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzman, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, "Unsupervised cross-lingual representation learning at scale," in *Proc. 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 8440-8451, doi: 10.18653/v1/2020.acl-main.747.
- [21] A. Joulin, E. Grave, P. Bojanowski, and T. Mikolov, "Bag of tricks for efficient text classification," in *Proc. 15th Conf. of the European Chapter of the Association for Computational Linguistics (Volume 2: Short Papers)*, 2017, pp. 427-431, doi: 10.18653/v1/E17-2068.
- [22] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conf. on Empirical Methods in Natural Language Processing and the 9th Int. Joint Conf. on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3980-3990, doi: 10.18653/v1/D19-1410.
- [23] N. Reimers and I. Gurevych, "Making monolingual sentence embeddings multilingual using knowledge distillation," in *Proc. 2020 Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 4512-4525, doi: 10.18653/v1/2020.emnlp-main.365.
- [24] B. Warner, A. Chaffin, B. Clavie, O. Weller, O. Hallstrom, S. Taghadouini, A. Gallagher, R. Biswas, F. Ladhak, T. Aarsen, N. Cooper, G. Adams, J. Howard, and I. Poli, "Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference," arXiv:2412.13663, 2024, doi: 10.48550/arXiv.2412.13663.
- [25] U. Zaratiana, N. Tomeh, P. Holat, and T. Charnois, "GLiNER: Generalist model for named entity recognition using bidirectional transformer," in *Proc. 2024 Conf. of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2024, pp. 5364-5376, doi: 10.18653/v1/2024.naacl-long.300.
- [26] W. Yin, J. Hay, and D. Roth, "Benchmarking zero-shot text classification: Datasets, evaluation and entailment approach," in *Proc. 2019 Conf. on Empirical Methods in Natural Language Processing and the 9th Int. Joint Conf. on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3912-3921, doi: 10.18653/v1/D19-1404.
- [27] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," arXiv:2106.09685, 2021, doi: 10.48550/arXiv.2106.09685.

- [28] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," arXiv:2103.00020, 2021, doi: 10.48550/arXiv.2103.00020.
- [29] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, "Sigmoid loss for language image pre-training," in *Proc. 2023 IEEE/CVF Int. Conf. on Computer Vision (ICCV)*, 2023, pp. 11941-11952, doi: 10.1109/ICCV51070.2023.01100.
- [30] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," arXiv:2305.05176, 2023, doi: 10.48550/arXiv.2305.05176.
- [31] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, "RouteLLM: Learning to route LLMs with preference data," arXiv:2406.18665, 2024, doi: 10.48550/arXiv.2406.18665.
- [32] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee, V. Ruhle, L. V. S. Lakshmanan, and A. H. Awadallah, "Hybrid LLM: Cost-efficient and quality-aware query routing," arXiv:2404.14618, 2024, doi: 10.48550/arXiv.2404.14618.
- [33] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611-626, doi: 10.1145/3600006.3613165.
- [34] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktaschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," arXiv:2005.11401, 2020, doi: 10.48550/arXiv.2005.11401.
- [35] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, 2023, doi: 10.48550/arXiv.2312.10997.
- [36] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," arXiv:2210.03629, 2022, doi: 10.48550/arXiv.2210.03629.
- [37] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," arXiv:2302.04761, 2023, doi: 10.48550/arXiv.2302.04761.
- [38] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model Context Protocol (MCP): Landscape, security threats, and future research directions," arXiv:2503.23278, 2025, doi: 10.48550/arXiv.2503.23278.
- [39] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. on Research and Development in Information Retrieval*, 2009, pp. 758-759, doi: 10.1145/1571941.1572114.
- [40] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824-836, 2020, doi: 10.1109/TPAMI.2018.2889473.
- [41] K. Jarvelin and J. Kekalainen, "Cumulated gain-based evaluation of IR techniques," *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422-446, 2002, doi: 10.1145/582415.582418.
- [42] F. Cuconasu, G. Trappolini, F. Siciliano, S. Filice, C. Campagnano, Y. Maarek, N. Tonello, and F. Silvestri, "The power of noise: Redefining retrieval for RAG systems," in *Proc. 47th Int. ACM SIGIR Conf. on Research and Development in Information Retrieval*, 2024, pp. 719-729, doi: 10.1145/3626772.3657834.
- [43] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th*

ACM Workshop on Artificial Intelligence and Security (AISec), 2023, pp. 79-90, doi: 10.1145/3605764.3623985.

- [44] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," arXiv:2306.05685, 2023, doi: 10.48550/arXiv.2306.05685.
- [45] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proc. 2023 Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 12076-12100, doi: 10.18653/v1/2023.emnlp-main.741.
- [46] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," arXiv:2309.15217, 2023, doi: 10.48550/arXiv.2309.15217.
- [47] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang, W. Ye, Y. Zhang, Y. Chang, P. S. Yu, Q. Yang, and X. Xie, "A survey on evaluation of large language models," *ACM Transactions on Intelligent Systems and Technology*, vol. 15, no. 3, pp. 1-45, 2024, doi: 10.1145/3641289.
- [48] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," arXiv:1706.04599, 2017, doi: 10.48550/arXiv.1706.04599.
- [49] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37-46, 1960, doi: 10.1177/001316446002000104.
- [50] T. G. Dietterich, "Ensemble methods in machine learning," in *Multiple Classifier Systems*, Lecture Notes in Computer Science, vol. 1857. Berlin, Heidelberg: Springer, 2000, pp. 1-15, doi: 10.1007/3-540-45014-9_1.